opentext™
Information reimagined

OpenText Content Management for SAP Solutions 26.4

SDK and API Documentation

Contents

1 Introduction 6
1.1 About OpenText Content Management for SAP Solutions 6
1.2 About this document 6
1.2.1 Target audience 6
1.2.2 Related documentation 6
1.2.3 SDK and API guide revision history 7
1.2.4 OpenText CM Release and Packages 7
2 Concepts 8
2.1 What is OpenText Content Management for SAP Solutions 8
2.2 Business workspace 8
2.3 Business attachment 8
2.4 Records management for SAP 9
3 Requirements for SAP Applications 9
3.1 List of Requirements 9
4 Basic enhancements 11
4.1 Introduction 11
4.2 Document property providers 11
4.2.1 Best practices 15
4.2.2 Generic property provider 16
4.2.3 Support of Additional Attributes in GOS Attachment List 17
4.2.4 Support of ArchiveLink PLUS attributes 18
4.3 Business object property providers 18
4.3.1 Cookbook for workspace property providers 20
4.3.2 Properties of workspace property providers 27
4.3.3 Best practices 36
4.3.4 BOR API interface 38
4.3.5 Using the asynchronous methods 39
4.4 Custom service provider for Cloud for Customer integration 40
4.5 Other OpenText Content Management-specific web services (APIs) 41
4.5.1 OpenText Content Management for SAP Solutions Library Service 42
4.5.2 Deprecated services and interfaces 44
4.6 Deletion of ArchiveLink records 44
4.6.1 Standard behavior 44
4.6.2 Creating a program which cleans up orphaned records on Content Server 46
5 Business Attachment enhancements 47
5.1 Enhancements on SAP 47
5.1.1 Introduction 47
5.1.2 SAP BAdIs 47
5.2 Enhancements on Content Server 48
5.2.1 Introduction 48
5.2.2 Callbacks on Content Server 49
6 Business workspace enhancements 51

6.1 Enhancements on SAP 51
6.1.1 Additional business workspace tabs for SAP objects 51
6.1.2 Example for SAP object type “Customer” 52
6.1.3 Example for SAP object type “Vendor” 62
6.1.4 Authentication 62
6.1.5 Generic Web Dynpro for Workspaces 66
6.1.6 Additional context-menu options in Business Content Window 74

6.2 Enhancements on Content Server 75
6.2.1 Look & feel sidebar 75
6.2.2 Callbacks on Content Server 77
6.2.3 Configuration API on Content Server 78

7 Enhancements in business scenarios 85

7.1 Enhancements on SAP 85
7.1.1 Introduction 85
7.1.2 General approach 85
7.1.3 Use case: add, change, or remove business workspace attributes 87
7.1.4 Use case: add or remove business workspace relationships . 93
7.1.5 Use case: add or remove role members 95

7.2 Enhancements on SAP for Enterprise Asset Management 95
7.2.1 Filling up roles for Team widget 95
7.2.2 Impersonating users outside OpenText Content Management 98

7.3 Enhancements on SAP for Procurement 98
7.3.1 Enhancing invoice declaration in Business Partner workspace 99

8 Side-by-Side extension: Enabling Custom API to Synchronize SAP roles with CS groups 101

8.1 Prerequisites 101

8.2 Goal 101

8.3 Steps to create custom API (tier 2) 102
8.3.1 Source codes and properties 103

8.4 Maintaining Content Server Customizing 105

9 Fiori (SAPUI5) Integration 107

9.1 Overview 107
9.1.1 Required SAPUI5 version 109
9.1.2 Deprecation of Web IDE and other changes (Visual Editor) . 109
9.1.3 Overview of integration examples in screenshots 110

9.2 SAPUI5 integration components 116
9.2.1 Restrictions 116
9.2.2 SAPUI5 app for Business Workspace 117

9.3 Integrating with cross-app (intent-based) navigation 124
9.3.1 Important things to note 125

9.4 Embedding Content (Object Pages) with Key User Adaptation 126
9.4.1 General Information and Requirements 126

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.4.2 Quick procedure 128
9.4.3 Enabling Key User Adaptation 128
9.4.4 Only S/4HANA On Premise: Defining Target mapping(s) for integration 129
9.4.5 Creating key user adaptation (embed the URL of the target mapping) 129
9.4.6 Drafting, Versioning, Publishing 131
9.4.7 URL examples 132

9.5 Extending SAP Fiori Elements-based app with Adaptation 138
9.5.1 General Information and Requirements 138
9.5.2 Quick procedure 140
9.5.3 Creating a new Adaptation project 141
9.5.4 Integrating the OpenText control for Business Workspace 142
9.5.5 Making Original files or attachment sections invisible 154
9.5.6 Deploying the project to the SAP Gateway on premise 154
9.5.7 Testing display of business workspaces 155
9.5.8 Accessing the adapted Fiori app in on-premise Fiori Launchpad 157
9.5.9 Reusable Code snippets 157

9.6 Extending own SAP Fiori Elements-based app with app extensions during creation 162
9.6.1 General Information and Requirements 162
9.6.2 Quick procedure 164
9.6.3 Creating a new custom app based on Fiori Elements 165
9.6.4 Adding relative path to ui5_ui5 node into neo-app.json file 170
9.6.5 Integrating the OpenText controls 171
9.6.6 Deploying the project to the SAP Gateway on premise 176
9.6.7 Testing display of business workspaces 181
9.6.8 Accessing the custom app in on-premise Fiori Launchpad 183
9.6.9 Reusable Code snippets 184

9.7 Extending freestyle SAPUI5 apps with Adaptation 193
9.7.1 General Information and Requirements 193
9.7.2 Quick procedure 194
9.7.3 Creating a new Adaptation project 195
9.7.4 Using the SAPUI5 Visual Editor to add changes 198
9.7.5 Modifying the XML fragment 205
9.7.6 Modifying the custom controller 206
9.7.7 Defining localized text labels 209
9.7.8 Deploying the project to the SAP Gateway on premise 209
9.7.9 Testing display of business workspaces 209
9.7.10 Accessing the adapted Fiori app in on-premise Fiori Launchpad 213
9.7.11 Reusable Code Snippet 215

9.8 Integrating own freestyle SAPUI5 apps during creation 216
9.8.1 General Information and Requirements 216

9.9 Integrating into SAP Cloud Platform with external access to on-premise landscape 218
9.9.1 General Information and requirements 218
9.9.2 Access to other Content Resources 219
9.9.3 Implementation hints 219

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.10 Integrating into SAP S/4HANA Cloud Essential Edition 222
9.10.1 General Information and Requirements 222
9.10.2 Examples for the UI integration 223
9.10.3 Implementation steps 223
9.11 Resolving cross-origin resource sharing scenarios with Content Server 229
9.11.1 Example for scenario 1: Using SAP Web Dispatcher in SAP Fiori scenarios 230
9.11.2 Scenario 2 - SAP Cloud platform with external access to on-promise landscape 230
9.11.3 Scenario 3 – no reverse proxy 231
9.12 Basic Steps using the SAP Cloud Connector and the SAP Web IDE on the SAP Cloud Platform 231
9.13 Behavior on mobile phones 236
9.13.1 Usage of Startup parameter mobileOpenDirectly=true 236
9.13.2 Other 236
10 OpenText™ Activator for SAP Solutions 238
10.1 About OpenText Activator for SAP Solutions 238
10.2 Components 238
10.3 SAP OSpace 239
10.3.1 RFC/BAPI API 240
10.3.2 Configuration API 245
10.4 Tutorials 250
10.4.1 Using SAP OSpace 250
10.4.2 Calling RFC RFC_READ_TABLE 250
About OpenText 257

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

1 Introduction

1.1 About OpenText Content Management for SAP Solutions

The purpose of OpenText Content Management for SAP Solutions is to integrate the SAP world into the ECM world. In general, SAP S/4HANA manages transactional processes. OpenText Content Management for SAP Solutions extends these processes with ECM capabilities such as document management and records management.

1.2 About this document

This manual describes the OpenText Content Management for SAP Solutions API and its abilities, and how to integrate it into your own scenario. As this manual is accompanied by several sample programs, you do not only get a reference of the available functionality, but also an introduction to the practical use of the OpenText Content Management for SAP Solutions API.
In addition, for SAPUI5/Fiori related scenarios this document also includes the Document Access for SAP Solutions API and its functions, and how to integrate it into your own scenario.

1.2.1 Target audience

The manual is written for developers who want to enrich their scenarios with custom functionality. To use it, you should have programming experience in either OScript or ABAP. You should also be familiar with at least the Content Server Builder or Oclipse and should be familiar with the Content Server's basic architectural concepts. Furthermore, you should have basic SAP experience. For integrations with SAP Fiori or SAPUI5 you should have knowledge of the UI development toolkit for HTML5 (SAPUI5) or at least basic understanding of HTML5 and JavaScript. Some of the extension techniques of SAPUI5 apps may require the use of the SAP Web IDE and the knowledge how to use it.

1.2.2 Related documentation

For further information, see the following sources:

- OScript Builder Fundamentals (comes with Content Server Builder installation)
- Content Server Administration Manual:
- OScript Tutorial
- OpenText Content Management for SAP Solutions Documentation in OpenText My Support (https://knowledge.opentext.com/)
- SAP documentation at help.sap.com

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

1.2.3 SDK and API guide revision history

Revision	Modification Date	Section modified	Modifications
1.0	Jul 2026	All	Initial draft
1.1	Sep 2026	All	Release version

1.2.4 OpenText CM Release and Packages

Please read the OpenText Content Management for SAP Solutions – SAP Add-on Packages - Additional Release Information for a more detailed list of component versions included in each release.

This is a brief overview.

OpenText CM Release	Included ABAP Packages
26.4	Archiving and Document Access Foundation OTEXFND 2640_758 Content Management Foundation: OTCMFND 2640_758 Applications (CRM, ERP): OTCMS4A 2640_758 Business Scenarios: OTCMBSC 2640_758

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

2 Concepts

2.1 What is OpenText Content Management for SAP Solutions

With OpenText Content Management for SAP Solutions, you can link SAP Business Suite with its structured content and unstructured content. Basically, SAP consists of so called “business objects” with data, roles, transactions, and events, whereas the ECM world, here represented by the Content Server, structures the content with versions, categories, and records management.
A tight connection publishes aspects of SAP like data, roles, and events to the unstructured content in ECM, and vice versa business object in SAP pay attention to the ECM facets.
In principle OpenText Content Management for SAP Solutions has three main concepts: Business Workspaces, Business Attachments, and Records Management for SAP which are explained in detail below.

2.2 Business workspace

A *business workspace* is a document management area in Content Server, which represents an SAP business object in a one-to-one relationship. An SAP business object can be uniquely identified in SAP by its business object type and business object identifier.
The business workspace knows the business object type, business object identifier, the SAP system, and SAP client. Thus, the SAP business object is tightly bundled with the business workspace and the user can smoothly switch from the business workspace in Content Server to the transaction in SAP. Furthermore, by means of Content Server categories, the business workspace can display the most important data of the SAP business object. Thus, the search engine in Content Server can clearly relate the corresponding business Workspace to an SAP business object. SAP business object events keep this metadata of the SAP business object in the categories of business workspace up to date in real-time. On the SAP side, the business workspace can either be displayed in the GOS menu, it can be integrated in the transaction of the SAP business object as an additional “Tab” folder, or it can be called in a workspace node in DocuLink. Furthermore, the business workspace is displayed in an SAPUI5 (Fiori) app, which can be integrated into other SAPUI5 (Fiori) apps. Business policies control the access of Content Server users to SAP business objects.

2.3 Business attachment

A *business attachment* links an individual Content Server item to an SAP business object.
In fact the business attachment is more a shortcut or reference to the SAP business object.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

In contrast to business workspaces, you do not have a special Content Server item connected to an SAP business object and you can have several business attachments from a Content Server item to different SAP business objects. Nevertheless, the tight integration between SAP and Content Server enables starting the corresponding SAP transaction of the SAP business object from a business attachment and it permits *metadata mapping* the same way as it is possible with business workspaces.
In consideration of the fact that users have different views depending on their working environment (SAP or Content Server), the name business attachment does not occur in Content Server. In SAP environment we use the term business attachment because a Content Server item is attached to the SAP business object. In Content Server environment we talk about *Add Business Object*, because a business object is added to the Content Server item.

2.4 Records management for SAP

The OpenText Content Management for SAP Solutions records management functionality enables putting SAP ArchiveLink documents under Content Server records management. Document property providers define and evaluate business-object-related ArchiveLink properties. OpenText Content Management for SAP Solutions maps such properties to corresponding Content Server records category attributes.

3 Requirements for SAP Applications

OpenText Content Management for SAP Solutions is built on SAP standard technologies that are used in most of the SAP applications. This chapter lists the required SAP standard technologies. This list is useful for all who want to check whether an SAP application or a specific SAP transaction can be used in combination with OpenText Content Management for SAP Solutions.

3.1 List of Requirements

WSP = Relevant in the context of the business workspace concept
REF = Relevant in the context of the business reference concept
RM = Relevant in the context of the records management concept

- SAP Business Objects [WSP, REF, RM]: OpenText™ Content Management for SAP® Solutions relies strongly on the concept of SAP business objects. For each entity, e.g., a material, a business object type maintained in the SAP business object repository is required (transaction SWO1).

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- Requirements for the business object type:
  - Application runs on ABAP stack
  - IFSAP interface implemented
  - IFARCH21 interface implemented [RM only]
  - Triggers SAP business events for creation and change
    - The events need to be called after commit call
    - Alternative: usage of change document (see transaction SCDO or table TCDOBT)
  - Reasonable key definition of the business object [WSP, REF]

- Data Access [WSP, REF, RM]: Document property providers and/or workspace property providers must be written for each business object type/document type as part of OpenText™ Content Management for SAP® Solutions implementation. Document property providers and workspace property providers need access to the data and the relationships of a business object. BAPIs for this data access with reliable performance make the implementation easier.
  - Alternative: Usage of ABAP SQL for access to the data
- User Interface Integration [WSP, REF]:
  - SAP GUI for Windows: The general way of UI integration into SAP transactions is based on the GOS (Generic Object Services). Therefore, GOS needs to be available in the UI of the transaction and the business object needs to be published to the GOS.
  - CRM UI: OpenText™ Content Management for SAP® Solutions provides for CRM 7 an UI integration via CRM assignment blocks and a tab integration into the CRM interaction center
  - Web Dynpro: OpenText™ Content Management for SAP® Solutions provides an UI integration via Web Dynpro components.
    - It can be leveraged in SRM7.
    - OpenText™ Content Management for SAP® Solutions provides a Web UI and an ABAP Web Dynpro control for a business workspace and even an ABAP API. Therefore, specific UI integrations can be programmatically achieved in case the SAP transaction provides suitable enhancement capabilities or is a custom transaction that can be modified.
  - SAP Fiori:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

  - OpenText™ Content Management for SAP® Solutions provides an SAPUI5 based SAP Fiori app for business workspaces. The app can be integrated in different ways (for example, by intent-based navigation, key user adaptation or by developer adaptation).

  - SAP Fiori apps of application type SAP GUI are rendered using SAP GUI for HTML. If GOS menu is available, the integration is available, too.(See SAP GUI for Windows)

- Search Help [WSP, REF]: Manual selection of business objects relies on SAP search helps. For these scenarios it is necessary to define SAP search helps.

4 Basic enhancements

4.1 Introduction

In general, OpenText Content Management for SAP Solutions property providers extract the metadata of SAP business objects to make them available for record declaration, Business Workspaces, and Business Attachments. A single metadata attribute is called “property” in the following sections.

4.2 Document property providers

Document property providers define and evaluate properties either related to print lists or to ArchiveLink links between SAP business objects and archived documents. The process of declaring records involves such properties in several ways: computing the record title, evaluating the record path, filling category attributes, etc.

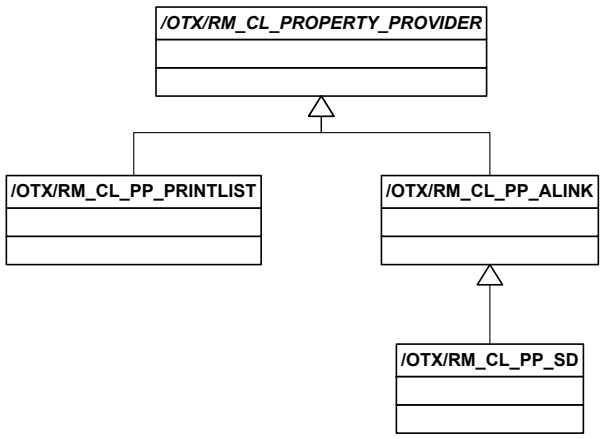


Figure 1: Document property providers

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Figure 1 shows that the base class for all document property providers is /OTX/RM_CL_PROPERTY_PROVIDER. /OTX/RM_CL_PP_PRINTLIST is the print list document property providers and /OTX/RM_CL_PP_ALINK is the basic ArchiveLink document property providers. /OTX/RM_CL_PP_SD is a sample property provider for links involving Sales-and-Deliveries business objects. Any property provider for an ArchiveLink link should be a child of /OTX/RM_CL_PP_ALINK because the ArchiveLink property provider defines the following mandatory system properties:

- ArchiveLink.Archive ID
- ArchiveLink.Assignment Count
- ArchiveLink.Business Object Type
- ArchiveLink.Classification
- ArchiveLink.Client
- ArchiveLink.Creator‡
- ArchiveLink.Description‡
- ArchiveLink.Document ID
- ArchiveLink.Document Type
- ArchiveLink.Expiry Date
- ArchiveLink.Filename‡
- ArchiveLink.Object ID
- ArchiveLink.Other Linkages
- ArchiveLink.Record Date
- ArchiveLink.Reserved
- ArchiveLink.Storage Date‡

Each record declaration configuration involves a document property provider (see transaction SPRO, **OpenText Content Management for SAP Solutions > Business Configuration > ArchiveLink Document and Print List Declarations > Maintain Document Declaration**) as shown in Figure 2.

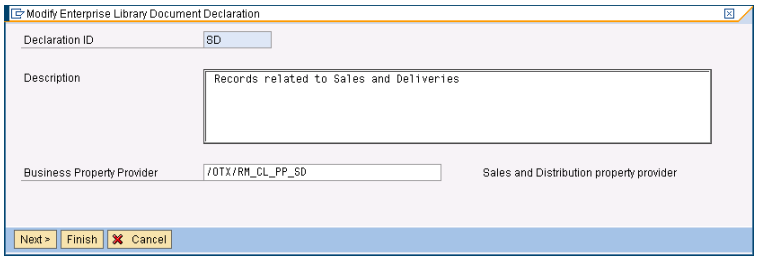


Figure 2: Associating a document property provider to a record declaration

- A document property provider must define the three following methods:
- GET_DESCRIPTION: Returns a description of the property provider
- GET_PROPERTY_DEFS: Returns a list of property definition structures with the following elements:
- NAME: Property name
- DATA_TYPE: Property data type (Boolean, Date, DateTime, Integer, Set, String, or LongString) – *See note about Date and DateTime format*
- REQUIRED: Whether the property must have a value of not
- SEARCHABLE: Whether the property is searchable or not
- MULTIVALUE: Whether the property is multi-value or single-value

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- VARIANT_LEVEL: Whether the property is at the variant or at the item level
  (Item level is deprecated. Always use variant level..)
- DESC: Property description
- SUBPROPS[1]: Table of sub-property definition structure (for property groups) with the following elements:
  - NAME: Sub-property name
  - DATA_TYPE: Sub-property data type (Boolean, Date, DateTime, Integer, String, or LongString) – See note about Date and DateTime format
  - REQUIRED: Whether the sub-property must have a value of not
  - SEARCHABLE: Whether the sub-property is searchable or not
  - MULTIVALUE: Whether the sub-property is multi-value or single-value
  - VARIANT_LEVEL[2]: Whether the sub-property is at the variant or item level
  - DESC: Sub-property description
- GET_PROPERTIES: Takes a list of link descriptions and returns, for each description, a list of property values or an exception.
- The property value structure has the following attribute:
- DEF: Property definition
- VALUES: Table of string values (1 entry for single-value properties, one or more entries for multiple-value properties)
- VALUES_GROUPS: Table of string table for group properties (to map Content Server set attributes)
- LANG_VALUES: Replaces VALUES for language-specific single-value properties
- This exception can be one of two types:
  - /OTX/CX_RM_PROPERTY_PROVIDER: The document declaration fails, and the corresponding document gets both an SAP Log entry and an Error Queue entry.
  - /OTX/CX_RM_PP_NO_DECLARING (child class of /OTX/CX_RM_PROPERTY_PROVIDER): The property provider does not want to declare a record for this document. In such cases, the document declaration does not occur, but the processing is successful (i.e., no SAP Log entry, no Error Queue entry).

Table 1 lists all properties that /OTX/RM_CL_PP_SD defines, including the properties of its parent class OTX/RM_CL_PP_ALINK.

[1] SUBPROPS stands for sub-properties and is only defined when the property data type is a Set. A sub-property is part of a property group and is the SAP equivalent to a Content Server set attribute.
[2] Deprecated: Always set to TRUE.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

NAME	DATA_TYPE	R	S	M	VL	DESC	SP
Customer Purchase Order Date	DateTime		X		X	The customer purchase order date	0x7
Customer Purchase Order Number	String		X		X	The customer purchase order number	0x7
Document Date	DateTime	X	X		X	The document date (date received/sent)	0x7
Purchase Value	String	X	X		X	The purchase value	0x7
Requested Delivery Date	DateTime	X	X		X	The requested delivery date	0x7
Line Items	Set	X	X	X	X	The line items	5x7
Sales Document Item	Integer		X		X	The sales document item	N/A
Material Number	String		X		X	The material number	N/A
Description	String		X		X	The description of the sales order item	N/A
Value	String		X		X	The purchase value	N/A
Currency	String		X		X	The purchase currency	N/A
ArchiveLink.Archive ID	String	X	X		X	Content repository ID	0x7
ArchiveLink.Business Object Type	String	X	X		X	Business object type	0x7
ArchiveLink.Client	String	X	X		X	Client	0x7
ArchiveLink.Creator‡	String	X	X		X	Creator name	N/A
ArchiveLink.Description‡	String		X		X	Document type description	0x7
ArchiveLink.Document ID	String	X	X		X	Archived document ID	0x7
ArchiveLink.Document Type	String	X	X		X	Document type	0x7
ArchiveLink.Expiry Date	DateTime		X		X	Expiration date	0x7
ArchiveLink.Filename‡	String	X	X		X	Filename	N/A
ArchiveLink.Object ID	String	X	X		X	Business object ID	0x7
ArchiveLink.Reserved	String		X		X	Reserved data	0x7
ArchiveLink.Storage Date‡	DateTime	X	X		X	Storage date	0x7

Legend: **R** = Required, **S** = Searchable, **M** = Multi-value, **VL** = Variant level, **SP** = Sub Props

Table 1: Sample property definitions (/OTX/RM_CL_PP_SD)

Note: OpenText recommends that you always use DateTime and never use Date. Both Date and DateTime values must be in UTC and must have the following format: 2008-01-22T09:01:21.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

4.2.1 Best practices

Here are seven best practices for implementing document property providers:

1. Always use /OTX/RM_CL_PP_ALINK as base class for ArchiveLink property providers and /OTX/RM_CL_PP_PRINTLIST as base class for print list property providers
2. Define constant class attributes to keep property names:

```
public section.
  constants C_PROP_NAME_EQUI_NUM type STRING
    value 'Equipment Number'.
  constants C_PROP_NAME_EQUI_DESC type STRING
    value 'Equipment Description'.
  constants C_PROP_NAME_MANU_ASSET type STRING
    value 'Manufacturer of Asset'.
  constants C_PROP_NAME_ACQUI_VALUE type STRING
    value 'Acquisition Value'.
  constants C_PROP_NAME_CURRENCY type STRING
    value 'Currency'.
  constants C_PROP_NAME_MAINT_PLANT type STRING
    value 'Maintenance Plant'.
```

3. Begin the constructor method by calling the super constructor:

```
CALL METHOD super->constructor.
```

4. Begin the GET_PROPERTIES method by calling its super method:

```
  CALL METHOD super->get_properties
    EXPORTING
      pit_content_ref = pit_content_ref
    IMPORTING
      pot_properties  = lt_content_ref_props.
```

5. If possible, use BAPIs instead of directly querying database tables:

```
    CALL FUNCTION 'BAPI_EQUI_GETDETAIL'
      EXPORTING
        equipment        = ls_equi-equipment
      IMPORTING
        data_general_exp = ls_equi-data_general_exp
        return           = ls_return.
```

6. Handle all errors, for example, by always verifying BAPI return codes:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
    IF ls_return-type IS NOT INITIAL AND ls_return-type <> 'S'.
      l_error_message =
        'Could not read data of Equipment ' &
        'with number = &1. &2'.                                  "#EC
NOTEXT
      REPLACE '&1' IN l_error_message WITH ls_equi-equipment.
      REPLACE '&2' IN l_error_message WITH ls_return-message.
      CREATE OBJECT ls_equi-exception
        EXPORTING
          m_sap_object    = ls_toav0-sap_object
          m_object_id     = ls_toav0-object_id
          m_archiv_id     = ls_toav0-archiv_id
          m_arc_doc_id    = ls_toav0-arc_doc_id
          m_ar_object     = ls_toav0-ar_object
          m_ar_date       = ls_toav0-ar_date
          m_error_message = l_error_message.
    ENDIF.
```

7. To perform date-time format conversion, always use the inherited utility methods:
    - CONVERT_DATS_TO_STRING
    - CONVERT_DATSTIMSTZ_TO_STRING

4.2.2 Generic property provider

Note:
Generic Property Provider /OTX/RM_GEN_PP_CL_ALINK_DOC replaces /OTX/RM_GEN_PP_ALINK_DOC, which is obsolete with version 16 EP4 (16.2.4.)

/OTX/RM_GEN_PP_CL_ALINK_DOC is a generic property provider based on /OTX/RM_CL_PP_ALINK. Generic property providers do not require any programming. Instead, they detect the persistence database table of the associated business object type and then use the field names and values of this table as property names and values. Figure 3 shows that opting for a generic property provider requires specifying the business object type

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

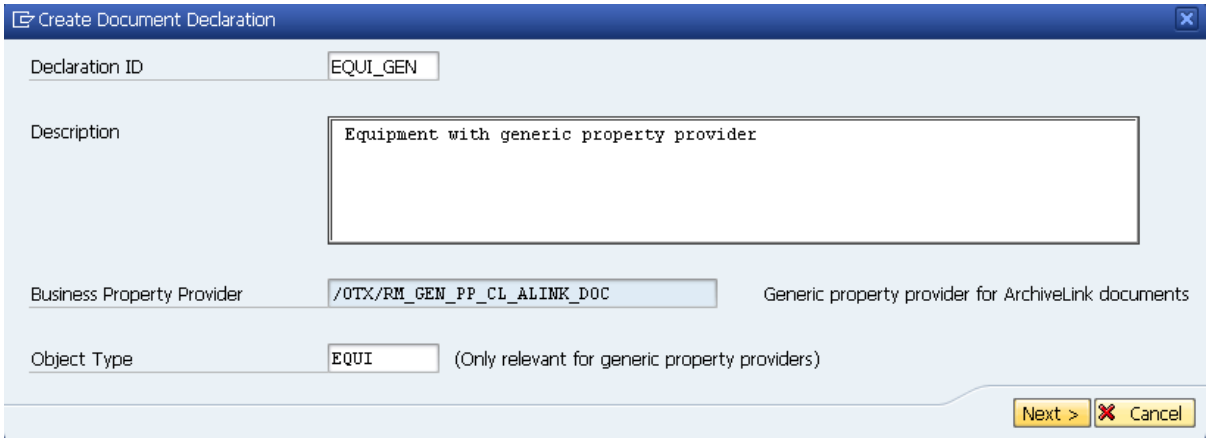


Figure 3: Generic property provider for documentation declaration

4.2.3 Support of Additional Attributes in GOS Attachment List

With SAP Notes 1451769 and 1560955 SAP enhances the GOS Attachment List with additional attributes filename, creator, descr and creatime stored in database table TOAAT.

Starting with 16.2 EP4 (2018-03; that is OTEXRL 1620_700 support package 0001), document declaration can use those additional attributes.

For this an SAP_BASIS release 740 and the application of SAP Note 2515351 are required.

Furthermore, to use this functionality, one needs to check the “Reserve 1” checkbox (sometimes also called “Save the File Attributes”).

If one fulfills all these requirements, the functionality of the document declaration changes in the following way:

- When a new ArchiveLink connection is created, the implementation of OA_BADI_LINK also takes over the values of the additional attributes FILENAME, CREATOR and DESCR. The value of CREATIME is set with value sy-uzeit. In the case of an update of a connection, the values stored from table TOAAT are retrieved.
- The property provider /OTX/RM_CL_PP_ALINK and all its sub classes add the new properties
  - ArchiveLink.Creator - login name of the creator as also stored in table TOAAT.
  - ArchiveLink.Filename - filename as also stored in table TOAAT.
- Computing the following properties is different:
  - ArchiveLink.Description – if the value of DESCR in table TOAAT is not empty, then its value fills ArchiveLink.Description (this is done in the same way in the GOS Attachment List)

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- ArchiveLink.Storage Date – the creation time CREATIME replaces the default of “120000” inside the datetime string of storage date.

4.2.4 Support of ArchiveLink PLUS attributes

Starting with 16.2 EP4 (2018-03) the functionality of ArchiveLink PLUS and OpenText Content Management for SAP Solutions was optimized: changes to ArchiveLink PLUS storage tables are now committed before an ArchiveLink connection is inserted.
This way property providers can be created and used, which simply read from ArchiveLink PLUS storage tables.

Currently, OpenText Content Management for SAP Solutions does not deliver property providers, which already convey ArchiveLink PLUS attributes to properties.

4.3 Business object property providers

Workspace property providers are like document property providers. They also extract the metadata of the SAP business object but define and evaluate properties related to SAP business object. Each creation or update of a business workspace involves a workspace property provider. You also need workspace property providers for business attachments.

Workspace property providers are used in the business object declaration which assigns a workspace property provider to a business object type.

Business Object Declaration

Object Type	Conn. ID	Property Provider	Search Help Name	Use Widgets for UI	Active
BKPF	ECMLINK19D	/OTX/RM_GEN_PP_CL_BO	VF_BKPF	☐	☑
BUS0010	ECMLINK19D	/OTX/RM_GEN_PP_CL_BO	IFLMH	☑	☑
BUS1001006	ECMLINK19D	/OTX/RM_WSCI_CL_WSPP_MATERIAL	MAT1J	☑	☑
BUS1006	ECMLINK19D	/OTX/RM_WSCI_CL_WSPP_BUPA	BUPAP	☐	☑
BUS1505	ECMLINK19D	/OTX/RM_WSCI_CL_WSPROV_DEFAULT	RECNCNA	☐	☑

Figure 4: Business Object Declarations

Figure 5 shows the workspace property provides that are provided with software component OTEXRL.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

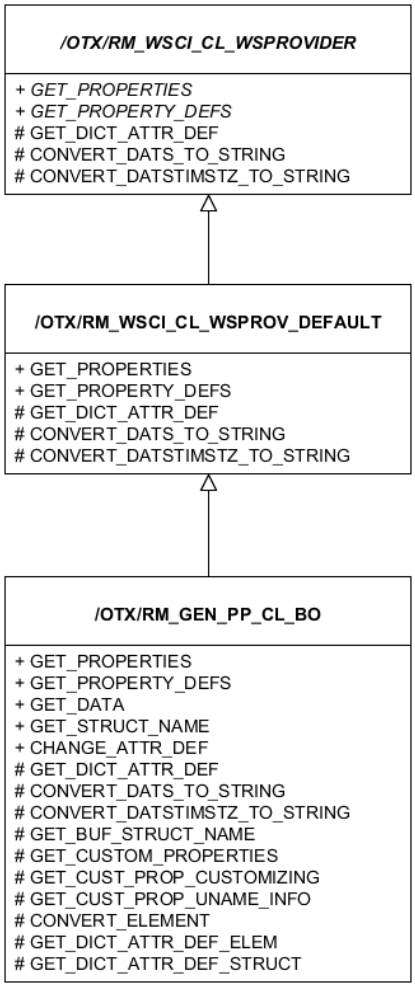


Figure 5: Workspace property providers (Abstract class/methods are written italic)

The abstract class /OTX/RM_WSCI_CL_WSPROVIDER is the base class for all workspace property provider. In contains three protected methods with implementation and two public abstract methods. Implementation for these abstract methods had to be supplied by a descendant of the class. This class cannot be instantiated.

The class /OTX/RM_WSCI_CL_WSPROV_DEFAULT inherits from class /OTX/RM_WSCI_CL_WSPROVIDER and provides the implementation for the most important business properties of a property provider: the business object type and the business object ID. This is the simplest workspace property provider you can use.

The class /OTX/RM_GEN_PP_CL_BO inherits from class /OTX/RM_WSCI_CL_WSPROV_DEFAULT. This class is also called generic property provider. It provides metadata of the business object from the business object repository without any programming. For that purpose, it

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

detects the persistence database table of the business object type for which they are being used and then use the field names and values of this table as property names and values.

Note:
Generic Property Provider /OTX/RM_GEN_PP_CL_BO replaces /OTX/RM_GEN_PP_BO, which is obsolete with version 16 EP6 (16.2.4.).

4.3.1 Cookbook for workspace property providers

The generic property provider aims at helping quickly configuring base scenario. The followings are examples of constructions that are over the scope of generic property providers:

- Complex data selection or data from different tables
- Multiline attributes, sets or multiline sets in Content Server
- Relations between the business workspaces, cross-application business workspaces, composite workspaces, or users (or groups) to roles feature

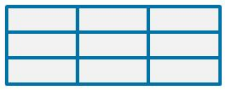
The generic property provider comes with an implementation interface that makes it easy to create property providers for such complex scenarios. To create your custom workspace property provider with additional business properties and relations, you need to follow these steps:

1. Create a DDIC structure
2. Create your custom property provider class
3. Redefine method GET_STRUCT_NAME
4. Redefine method GET_PROPERTY_DEFS (optional)
5. Redefine method GET_DATA
6. Redefine method CHANGE_ATTR_DEF (optional)

4.3.1.1 Create a DDIC structure

Generic property provider offers implementation interface that supports out-of-the-box special Content Server attribute types. ABAP developers only need to maintain a DDIC structure that fits to the desired category attributes at Content Server side and fill the DDIC structure with business object related data within their custom property provider. The following figure visualizes how to design the DDIC structure for getting the desired attribute type at Content Server side.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

	Attribute Type	Included Components Type In DDIC Structure
	Single value attribute	Data element
	Attribute set	Structure or several data elements*
	Multiline attribute	Table with line type data element
	Multiline attribute set	Table with line type structure

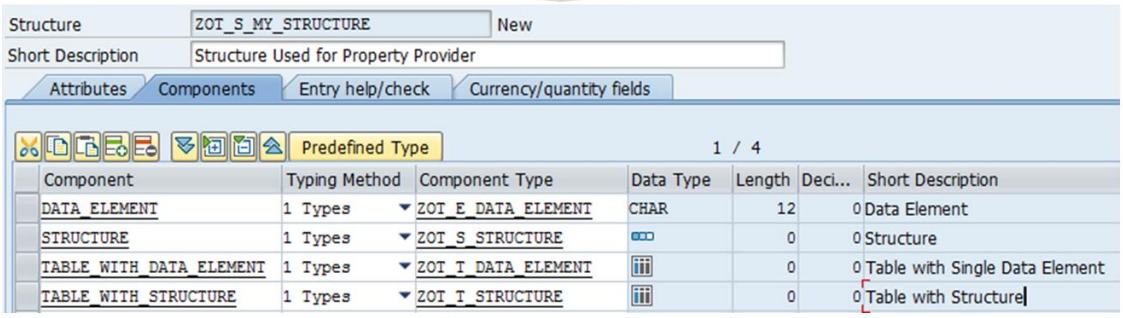
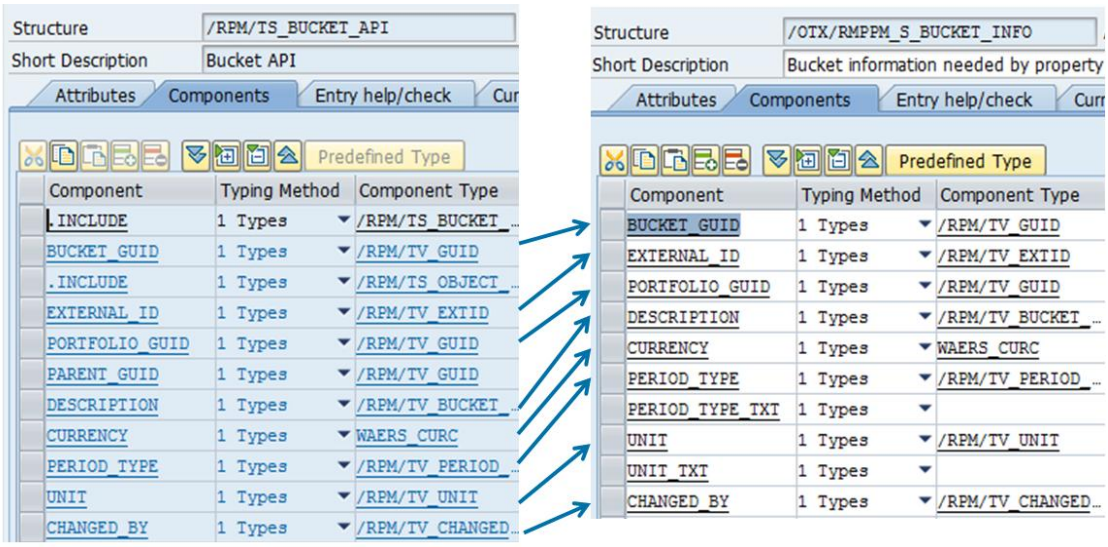


Figure 6: Overview attribute types to DDIC components

If you model a single value attribute in a category, you need to add a data element into you DDIC structure. To fill an attribute set, you would add a structure to your DDIC structure. For using a multiline attribute, you need a table with a line type of one data element. For a multiline attribute set, you would add a table type with line type structure. If you have a set with just one line (for instance for a nicer layout), you can process it also like single value attributes.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Name your components in your DDIC structure like in the fields used in the called methods or functional modules that you use to retrieve the business object data.




```
MOVE-CORRESPONDING ls_api_attribute TO my_ddic_structure.
```

Figure 7: Naming of components in DDIC structure

For instance, if the functional module provides a returning structure ls_api_attribute of type /rpm/ts_bucket_api, you add the same component to your DDIC structure. If the component name is identical, you can move the business object data easily to your DDIC structure by using the statement move-corresponding.

Attribute Type Conversion

The generic property provider will automatically convert the ABAP attribute value to the corresponding Content Server value if necessary (for example a date). For this the Data Elements in the DDIC structures are evaluated.
All ABAP character, string, integer, and date Data Elements are automatically converted to the corresponding Content Server types.
For Boolean, timestamps and long strings special Data Elements / Domains need to be used in the DDIC structure to define an attribute. If the generic Property Provider encounters this Data Elements/Domains, it will convert to the corresponding Content Server type.

See Table 2 for the details of the attribute type conversion and special Data Elements for generic property provider.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Data type	ABAP type	Content Server type
Character, String	All Character and String Data Elements	String
Integer	All Integer Data Elements	Integer
Date	All Date Data Elements	Date
Boolean	Data Element /OTX/RM_GEN_PP_DE_BOOLEAN or a Data Element with Domain /OTX/RM_GEN_PP_DO_BOOLEAN	Boolean
Timestamp	Data Element TIMESTAMP or a Data Element with Domain TZNTSTMPS	DateTime
Long string	Data Element /OTX/RM_GEN_PP_DE_LONG_STRING or a Data Element with Domain /OTX/RM_GEN_PP_DO_LONG_STRING	LongString
Other	All other Data Elements	String

Table 2: Attribute type conversion in generic property provider

Multilingual attributes

The generic property provider supports multilingual attributes. These are used in multilingual workspace names (document names in document declarations).

To define a multilingual attribute in the DDIC structure, instead of defining an attribute with a Data Element, you define the attribute with the special structure /OTX/RM_GEN_PP_S_LANG_VALUE (/OTX/RM_GEN_PP_S_AL_LANG_ VALUE for document Property Provider).

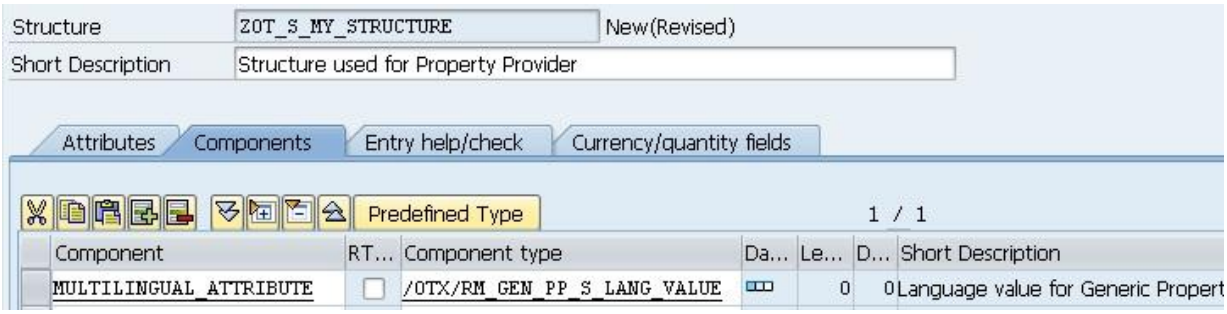


Figure 8: Multilingual attribute in DDIC structure

In the multilingual attribute structure, you add the language code / value pairs to the LANG_VALUES table. The VALUE component can be filled with a default value.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

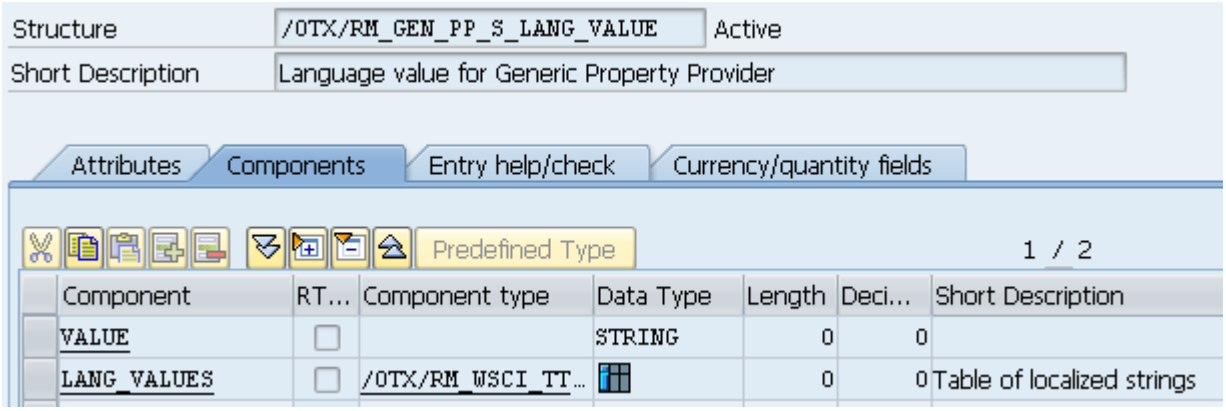


Figure 9: Multilingual attribute structure

Please note that multilingual attributes can only be used for single attributes, not for multiline or set attributes.
Reason for this is, that Category Attributes in Content Server are not multilingual, therefore multilingual attributes can only be used for the workspace names (document names in document declaration).

4.3.1.2 Create your custom property provider class

In transaction **SE24**, create your new class and inherit from class /OTX/RM_GEN_PP_CL_BO to use the implementation interface. By doing it this way, you will inherit all attributes and methods.

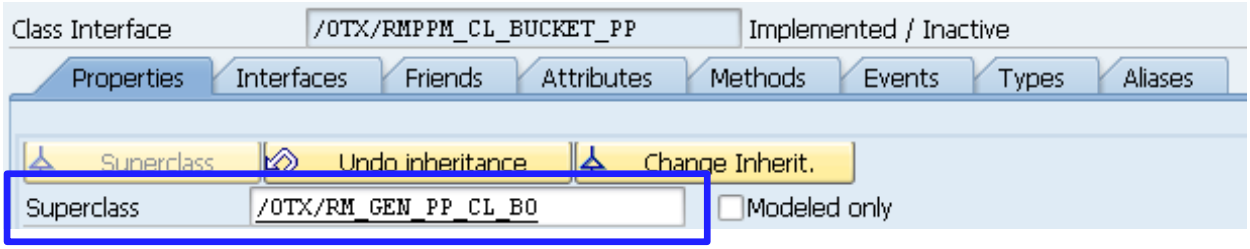


Figure 10: Super class of custom workspace property provider

4.3.1.3 Redefine method GET_STRUCT_NAME

This method must return your DDIC structure.

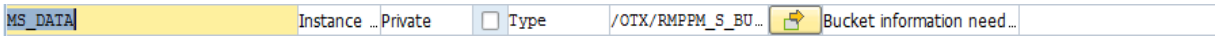
```
METHOD get_struct_name.

  pr_struct_name = '/OTX/RMPPM_S_BUCKET_INFO'.

ENDMETHOD.
```

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Alternatively, if you want to fill the DDIC structure by different methods in your class, define an instance attribute in your class.



Take this code snipped to dynamically return DDIC structure of the attribute me->ms_data that makes it easy to exchange the DDIC structure.

```
  METHOD get_struct_name.

    DATA: lo_str_desc  TYPE REF TO cl_abap_structdescr,
          lv_type_name TYPE        ddobjname.

    lo_str_desc ?= cl_abap_structdescr=>describe_by_data
( me->ms_data ).
    lv_type_name = lo_str_desc->get_relative_name( ).

    pr_struct_name = lv_type_name.

  ENDMETHOD.
```

4.3.1.4 Redefine method GET_PROPERTY_DEFS (optional)

The method GET_PROPERTY_DEFS provides automatically the table with business property definitions according the defined DDIC structure and in addition allowed parent and child relations for all business object types with a business object declaration.

If your property provider provides any relation to a business object that is not yet customized, the execution of the property provider is aborted. Therefore, our recommendation would be to redefine the method. If you want to use cross application business workspaces - you always need to redefine this method.

Call the super method to get the table with business property definitions and redefine the other exporting parameters according to your needs.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
METHOD get_property_defs.

      " Call super method
      CALL METHOD super->get_property_defs
        EXPORTING
          pi_disp_lang                  = pi_disp_lang
          pi_bo_type                    = pi_bo_type
        IMPORTING
          pet_attr_defs                 = pet_attr_defs
          pet_allowed_parent_types_ext  = pet_allowed_p
arent_types_ext
          pet_allowed_child_types_ext   = pet_allowed_c
hild_types_ext.

    " Provide my allowed parent and child relations in
same system
    ls_relation-type = 'KNA1'.
" Customer
    APPEND ls_relation TO pet_allowed_parent_types_ext.
    APPEND ls_relation TO pet_allowed_child_types_ext.

    ls_relation-type = 'BUS2032'.
" Sales Order
    APPEND ls_relation TO pet_allowed_child_types_ext.

    DELETE ADJACENT DUPLICATES FROM pet_allowed_child_ty
pes_ext.
    DELETE ADJACENT DUPLICATES FROM pet_allowed_parent_t
ypes_ext.

    " Provide my allowed cross application relations in
another system
    ls_iden_bus_obj-type = 'BUS1006'.
" Bus. Partner
    ls_iden_bus_obj-external_system_id = me->mv_sysid.
    APPEND ls_iden_bus_obj
TO pet_allowed_ident_busobj_types.

  ENDMETHOD.
```

4.3.1.5 Redefine method GET_DATA

The GET_DATA method is used for business logic. Here you provide metadata as business property, users/roles, groups/roles, parent-child relations, cross-application references, and composite workspaces relations. Properties of property providers and their implementation are described in *4.3.2 Properties of workspace property providers* on page 27.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

4.3.1.6 Redefine method CHANGE_ATTR_DEF (optional)

The CHANGE_ATTR_DEF method can be used to overwrite business property definitions manually, which are generated by the framework. This hook method is called by GET_PROPERTIES after getting the property provider definition.

In the code snipped, the type of business property CHANGED_ON is set to type *String*.

```
  METHOD change_attr_def.

    FIELD-
SYMBOLS: <ls_attr_def>  TYPE REF TO /otx/rm_wsci_s_attr_
def.

    " Convert date to string for business property
CHANGED_ON
    READ TABLE pct_attr_def  ASSIGNING <ls_attr_def>
      WITH KEY KEY = 'CHANGED_ON'.
    IF sy-subrc = '0'.
      <ls_attr_def>-data_type = 'String'.
    ENDIF.

  ENDMETHOD.
```

4.3.2 Properties of workspace property providers

In this section, all properties of workspace property providers are described and explained how to implement them. All code samples are based on the integration interface of the generic property provider.

4.3.2.1 Business property

If you want to visualize or use the metadata of business objects in the business workspace, you can transfer the metadata as business property of the SAP system to the category attributes in the Content Server.

Content server divides between four types of attributes to visualize the metadata. These types could also be combined, for instance you can include a multiline attribute in a set.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

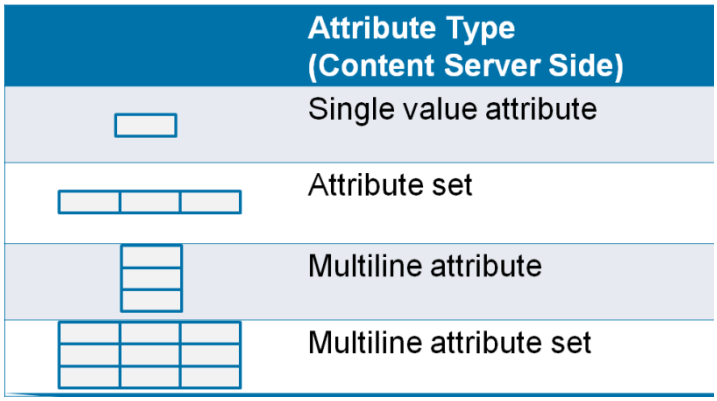


Figure 11: Overview attribute types in Content Server

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Attributes can be filled with the business property delivered by the workspace property provider that represents metadata of the SAP business object. For that purpose, the workspace property provider delivers a list of business properties of structure type /OTX/RM_WSCI_S_PROPERTY. It contains four components:

Component	Typing Method	Component Type	Data Type	Length	Deci...	Short Description
DEF	1 Types	/OTX/RM_WSCI_S_ATTR_DEF		0	0	Workspace attribute definition
VALUES	1 Types	/OTX/RM_WSCI_TT_STRINGTAB		0	0	Table with Strings
VALUES_GROUP	1 Types	/OTX/RM_WSCI_TT_3DSTRINGMATRIX		0	0	3D string matrix (row * prop * values)
LANG_VALUES	1 Types	/OTX/RM_WSCI_TT_LANG_STRINGTAB		0	0	Table of localized strings

Figure 12: Structure of business property

Component DEF contains the definition of a business property. Here the key, data type, length, multi value indicator and sub-business properties for a set are defined.

Component	RT...	Component type	Data Type	Length	Deci...	Short Description
KEY	☐	/OTX/RM_WSCI_DE_ATTR_KEY	CHAR	32	0	Workspace attribute key
DATA_TYPE	☐	/OTX/RM_WSCI_DE_ATTR_DATA_TYPE	CHAR	14	0	Workspace attribute data type
LENGTH	☐	/OTX/RM_WSCI_DE_ATTR_DATA_LEN	INT4	10	0	Workspace attribute data length
MULTIVALUE	☐	/OTX/RM_WSCI_DE_ATTR_MVALUE	CHAR	1	0	Workspace attribute multivalue capability
DISP_NAMES	☐	/OTX/RM_WSCI_TT_ATTR_DISP_NAME		0	0	Table of language-specific attribute display names
SUBATTRS	☐	/OTX/RM_WSCI_TT_SUBATTR_DEF		0	0	Table of workspace sub-attribute definitions

Figure 13: Structure of business property definition

The component VALUES, LANG_VALUES and VALUES_GROUP contains the value(s) of the business property. Component VALUES contains value for single value attributes or multiline attributes. Component VALUES_GROUP contains values when you work with a set. It spans up a 3D string matrix: The first dimension is the row; the second dimension is business property, and the third dimension is the value that could be one value for a single attribute or several values for a multiline attribute in the table.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Figure 14 illustrates the data transformation into a 3D matrix: A table that consists of three rows and two business properties should be transferred into the 3D string matrix table. In this case the component VALUES_GROUP would contain three rows. Each row contains two business properties, and the business property contains one value because it contains just a single value attribute in each cell.

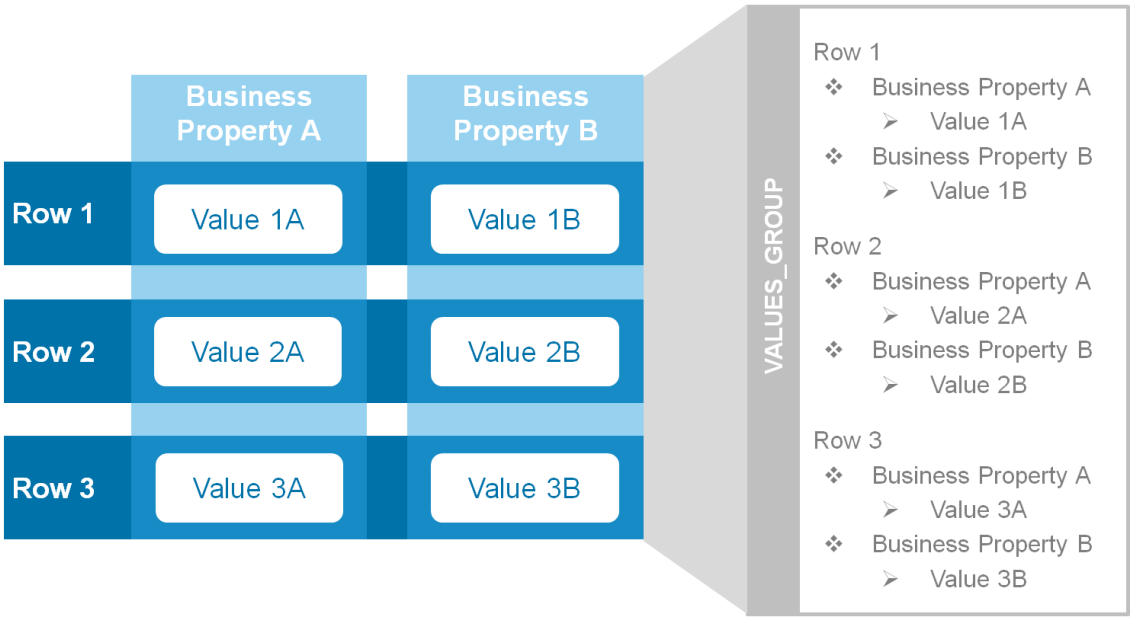


Figure 14: Transformation of table content in a 3D string matrix

If this is too complicated, you can use the integration interface of the generic property provider that fills the structure automatically.

The exporting parameter PE_DATA is defined with the DDIC structure provided by method GET_STRUCT_NAME. You fill the DDIC structure with all data that should be provided as business property to Content Server.

```
METHOD get_data.
  ...

  CLEAR ls_sales_document.
  ls_sales_document-vbeln = pis_bor-id.
  APPEND ls_sales_document TO lt_sales_document.

  " Gets sales document details
   CALL FUNCTION 'BAPISDORDER_GETDETAILEDLIST'
     EXPORTING
       i_bapi_view                  = ls_sales_view
     TABLES
       sales_documents              = lt_sales_document
       order_headers_out            = lt_header.
```

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
    " Move data
    READ TABLE lt_header ASSIGNING <ls_header>
      WITH KEY doc_number = pis_bor-id.
    IF sy-subrc = 0.
      MOVE-CORRESPONDING <ls_header> TO pe_data.
    ENDIF.
  ...
ENDMETHOD.
```

4.3.2.2 Users/Groups and roles

If you want to design access control to business workspaces based on information in SAP system, you can use the users (or groups) to role concept. Here you map SAP users and Content Server groups to business workspace roles.

For instance, you could maintain team members to a project.

The information about the users or groups is provided as property in the workspace property provider. In the method that provides the properties, you need to provide information about the users or groups.

```
METHOD get_data.
  ...
  " Provide users and its roles for the project
  CLEAR: ls_roles_users.
  LOOP AT lt_staffing  ASSIGNING <fs_staffing>.
    CLEAR: lv_user_id.

    " Get user or group by a function module
    ...

    APPEND lv_user_id TO ls_roles_users-users.
  ENDLOOP.


   " Get role name by a function module
    ...

  ls_roles_users-role = lv_role_name.
  APPEND ls_roles_users TO pet_roles_users.
  ...
ENDMETHOD.
```

To automatically add the user or group to the business workspace role, the role needs to exist in the business workspace. The best way would be to add the role to the template that is used for the business workspace creation.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Notice: Via the API, define each role only once, using the respective property provider to specify all role users/groups.

4.3.2.3 Relations

If a business object is related to another business object, you can create a parent-child relation between these two business workspaces that allows users to easily navigate to the documents in the related business workspace. A parent-child relation is bidirectional association between two business workspaces.

For instance, a customer could have several sales orders; so, the customer's business workspace is handled as parent and the sales order's business workspace as child.

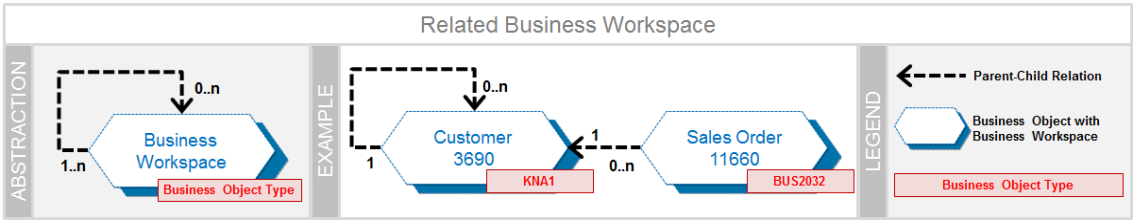


Figure 15: Relations between business workspaces

The information about the relations is provided as property in the workspace property provider. Only one property provider needs to report the relation: either the property provider of the parent or of the child business object.

To use relations, the child relation and parent relation to the desired business object type needs to be defined as allowed. You need to set the external system ID, if the related business workspace references a business object in another system; otherwise, you can leave it empty because it is set automatically to the system ID of the current system.

```
  METHOD get_property_defs.
    ...
    " Provide allowed parent and child relations for
customer
    ls_relation-type = 'KNA1'.
" Customer
    APPEND ls_relation TO pet_allowed_parent_types_ext.
    APPEND ls_relation TO pet_allowed_child_types_ext.

    ls_relation-type = 'BUS2032'.
    APPEND ls_relation-
type TO pet_allowed_child_types_ext. " Sales Order
    ...
  ENDMETHOD.
```

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

In the method that provides the properties, you need to provide information about the relation like business object type, ID, and optionally external system. Use the same external system ID as you used in in the method that defines the property.

```
METHOD get_data.
  ...
  " Provide relation to sales order
  CLEAR: ls_relation.
  ls_relation-type = 'BUS2032'.
  ls_relation-id = lv_sales_order_id.
  ls_relation-external_system_id = sy-sysid.
  APPEND ls_relation TO pet_children_ext.
  ...
ENDMETHOD.
```

Please use the extended variant with appendix '_EXT' because it could handle relations across different systems; the other one is obsolete.

NOTE: To avoid count issues in Content Server GUIs, one should define the relationship only once, either in the property provider of the parent object, or in the property provider of the child object.

4.3.2.4 Cross-application business workspace

If you have equivalent business objects - even if they are not linked - they could have a shared business workspace. These business objects could be at the same or at different systems.

A scenario could be a system landscape with two SAP systems, for instance an SAP S/4HANA system working with customers (KNA1) and an SAP TM system working with business partners (BUS1006). In this business scenario, the debtor is created in SAP S/4HANA as customer and in SAP TM as business partner. Since both business objects represent the same debtor, you like to have just one business workspace for both business objects; therefore, you use cross-application business workspace concept. The customer and the business partner share a business workspace.

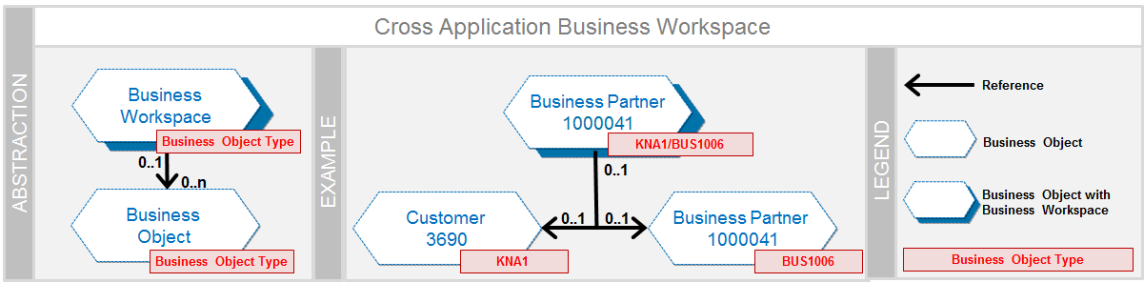


Figure 16: Cross-application business workspaces

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

The information about cross-application business workspace for multiple business objects is provided as property in the workspace property provider. All property providers that participate at a shared business workspace need to provide the information.

To use a cross-application business workspace, you need to define the allowed identical business object. You need to set the external system ID, if the related business workspace references a business object in another system; otherwise, you can leave it empty because it is set automatically to the system ID of the current system.

```
  METHOD get_property_defs.
    ...
    " Provide cross application business object for
customer
    ls_ident_bus_obj-type                = 'BUS1006'.
" Bus. Partner
    ls_ident_bus_obj-external_system_id = me->mv_sysid.
    APPEND ls_ident_bus_obj
TO pet_allowed_ident_busobj_types.
    ...
  ENDMETHOD.
```

In the method that provides the properties, you need to provide information about the identical business object like business object type, id and optionally external system. Use the same external system ID as you used in in the method that defines the property.

```
  METHOD get_data.
    ...
    " Provide identical business object
    CLEAR: ls_iden_bus_obj.
    ls_iden_bus_obj-type                = 'BUS1006'.
" Bus. Partner
    ls_iden_bus_obj-external_system_id = me->mv_sysid.
    ls_iden_bus_obj-id                  = lv_partner_id.
    APPEND ls_iden_bus_obj
TO pet_identical_business_objects.
    ...
  ENDMETHOD.
```

4.3.2.5 Composite workspace

If you have business object types that depends on each other and cannot stand alone, you could use the composite workspace concept to represent this dependency for business workspaces. Composite workspaces are like nested business workspace but include additional functionality, for more information about the functionality, see the customizing guide.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

A common example could be SAP EAM, where you have the business objects maintenance order and maintenance order operation. The maintenance order operations have a part-of relation to the maintenance order; they cannot stand alone.

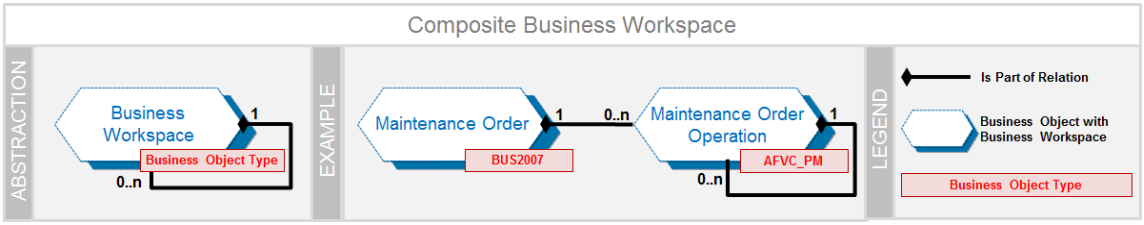


Figure 17: Composite business workspaces

To use composite business workspaces, you need to provide information about the composite relation like business object type, id and external system. You need to set the external system ID, if the composite business workspace references a business object in another system; otherwise, you can leave it empty because it is set automatically to the system ID of the current system.

Both property providers that are involved in the composite business workspace need to provide the information about the composition.

In the example, you see how to provide information in the parent workspace property provider about the children.

```
METHOD get_data.
  ...
  " Provide composite children
  LOOP AT lt_operation INTO ls_operation.
    CLEAR: ls_child.
    ls_child-type = 'AFVC_PM'.
    ls_child-id   = ls_operation.
    " ls_child-external_system_id =
    APPEND ls_child TO pet_composite_children.
  ENDLOOP.
  ...
ENDMETHOD.
```

In the example, you see how to provide information in the child workspace property provider about the parent.

```
METHOD get_data.
  ...
  " Provide composite parent
  CLEAR ls_parent.
  IF ls_afvgd-sumnr IS INITIAL.
   " operation of a task list
    pes_composite_parent-type = 'BUS2007'.
    pes_composite_parent-id   = lv_order.
    " pes_composite_parent-external_system_id =
```

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
  ELSE.
    " sub-operation of an operation
    pes_composite_parent-type = 'AFVC_PM'.
    pes_composite_parent-id = lv_operation.
    " pes_composite_parent-external_system_id =
  ENDIF.
  ...
ENDMETHOD.
```

The code sample shows that you can construct deep compositions; the business workspace of the operation could be either located inside the maintenance order or inside another operations business workspace.

4.3.2.6 Exceptions

If an exceptional situation occurs in the property provider, you can control via the thrown exception the behavior of the OpenText Content Management for SAP Solutions. Either handle it like an unwanted situation or like an accepted situation. Nevertheless, the current process is stopped, so the business workspace is not created/updated, or the business object is not attached.

Use /OTX/CX_RM_WSCI_WSPROVIDER for unwanted situations to write a log for the corresponding business object and add an entry in the error queue for processing it later.

Use /OTX/CX_RM_WSCI_NO_DECLARING (child class of /OTX/CX_RM_WSCI_WSPROVIDER) to handle the situation as expected: The property provider does not want to declare a workspace for this business object, so the processing is considered successful (i.e., no SAP Log entry, no Error Queue entry).

4.3.3 Best practices

Here are some best practices for implementing workspace property providers:

1. Always use /OTX/RM_WSCI_CL_WSPROV_DEFAULT or /OTX/RM_GEN_PP_CL_BO as base class.

2. If you are not working with a DDIC structure and need to define all properties by implementation, define constant class attributes to keep property names:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
public section.
  constants C_ATTR_NAME_CATEGORY type STRING value 'EQUICATGRY
'.
  constants C_ATTR_NAME_SERIAL_NO type STRING value 'SERIALNO'
.
  constants C_ATTR_NAME_CREATION_DATE type STRING value 'READ_C
RDAT'.
  constants C_ATTR_NAME_LOCATION_CITY type STRING value 'CITY1'
.
  constants C_ATTR_NAME_LOCATION_COUNTRY type STRING value 'COU
NTRY'.
  constants C_ATTR_NAME_LOCATION_NAME type STRING value 'NAME1'
.
  constants C_ATTR_NAME_PLAN_PLANT type STRING value 'PLANPLAN
T'.
```

3. Begin redefined method by calling its super method:

```
  CALL METHOD super->get_properties
    EXPORTING
      pit_bor            = pit_bor
    IMPORTING
      pet_bor_properties = pet_bor_properties.
```

4. If possible, use BAPIs instead of directly querying database tables:

```
    CALL FUNCTION 'BAPI_EQUI_GETDETAIL'
      EXPORTING
        equipment         = l_id
      IMPORTING
        data_general_exp  = ls_equip_general
        data_specific_exp = ls_equip_specific
        return            = ls_equip_return.
```

5. Handle all errors, for example, by always verifying BAPI return codes:

```
    IF ls_equip_return-type = c_bapi_err.
      l_error_msg = ls_equip_return-message.
      CREATE OBJECT <ls_bor_properties>-exception
        EXPORTING
          m_error_text = l_error_msg.
      CONTINUE.
    ENDIF.
```

6. To perform date-time format conversion, always use the inherited utility methods:

    - CONVERT_DATS_TO_STRING
    - CONVERT_DATSTIMSTZ_TO_STRING

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Note: OpenText recommends that you always use DateTime and never user Date. Both Date and DateTime values must be in UTC and must have the following format: 2008-01-22T09:01:21.

7. Check before setting a relation that the ID is not initial

```
IF lv_sales_order_id is not initial.
  CLEAR: ls_relation.
  ls_relation-type = 'BUS2032'.
  ls_relation-id = lv_sales_order_id.
  APPEND ls_relation TO pet_children_ext.
ENDIF.
```

8. Only define relations in either the parent business objects, or the child business objects, but not both. The recommendation is to opt for the business object type with the fewest relationships.

   Doing so improves declaration and update performance. It also prevents the related item widget from showing duplicate entries.

4.3.4 BOR API interface

/OTX/RM_WSC_IF_ECMLINK_BOR_API is an OpenText Content Management for SAP Solutions public API that provides BOR related functionality to manage Business Workspaces. The API offers the following methods:

- **CREATE_UPD_WORKSPACES**[3]: Creates or updates Business Workspaces
- **GET_WORKSPACE_NODE_IDS**: Gets the node IDs of the given Business Workspaces
- **UPDATE_BUS_REFS**: Updates Business Attachments
- **UPDATE_WORKSPACES**: Updates already existing Business Workspaces
- **CREATE_UPD_WORKSPACE_ASYNC**: Creates or updates Business Workspaces asynchronously
- **GET_WORKSPACE_NODE_IDS_ASYNC**: Gets the node IDs/state of the given Business Workspaces

[3] Note: To transfer users (or groups) and roles with the CREATE_UPD_WORKSPACES method, the *action_type* attribute of the /OTX/RM_WSC_IROLE_USRS_LIST structure must be set to "Replace".

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- **UPDATE_WORKSPACES_ASYNC**: Updates already existing Business Workspaces asynchronously

For more information, see the interface itself and the demonstration report (/OTX/RM_WSC_ECMLINK_BOR_API).

4.3.5 Using the asynchronous methods

With the asynchronous methods one can detach the workspace creation/update process from the caller process. The requests are placed in a queue and are processed separately by another process in the background while the control immediately gets back to the caller process.

The methods use SAP's standard batch infrastructure, therefore please make sure that the user context who calls these API methods has enough privileges to schedule background job. In the opposite case you can also manually start the queue processing by using the /OTX/RM_WSC_Q_CONSUMER_BGJOB report.

The methods simulate transactional behavior. The API supports placing more than one workspace creation/update request at the same time. In that case the requests are bundled together and handled in a transactional manner meaning if one creation fails in a bundle, the other eventually created workspaces are also not visible until the failure is corrected. Please note that this behavior is only valid if you use the corresponding asynchronous read method to get information about a workspace.

4.3.5.1 Changing priority for queue processing

The API methods CREATE_UPD_WORKSPACE_ASYNC and UPDATE_WORKSPACES_ASYNC of the BOR API interface (/OTX/RM_WSC_IF_ECMLINK_BOR_API) offer the import parameter **PI_PRIORITY**. Valid values of this parameter are:

- /OTX/RM_WSC_CL_Q_ITEM=>C_PRIO_LOW (15)
- /OTX/RM_WSC_CL_Q_ITEM=>C_PRIO_MEDIUM (10)
- /OTX/RM_WSC_CL_Q_ITEM=>C_PRIO_HIGH (5)

The default value of PI_PRIORITY is /OTX/RM_WSC_CL_Q_ITEM=>C_PRIO_LOW (5).

If an invalid value is used, the methods raise an exception.

4.3.5.2 Using receiver function modules

You are also allowed to use the receiver function modules for asynchronous creation or update, /OTX/RM_WSC_CRTUPD_ASYNC, /OTX/RM_WSC_UPD_ASYNC, /OTX/RM_WSC_UPD_EXISTING_ASYNC, /OTX/RM_WSC_UPD_SUPER_ASYNC, and /OTX/RM_WSC_UPD_SUPER_EXST_ASY in your custom code.

- They are wrapping around the API methods.

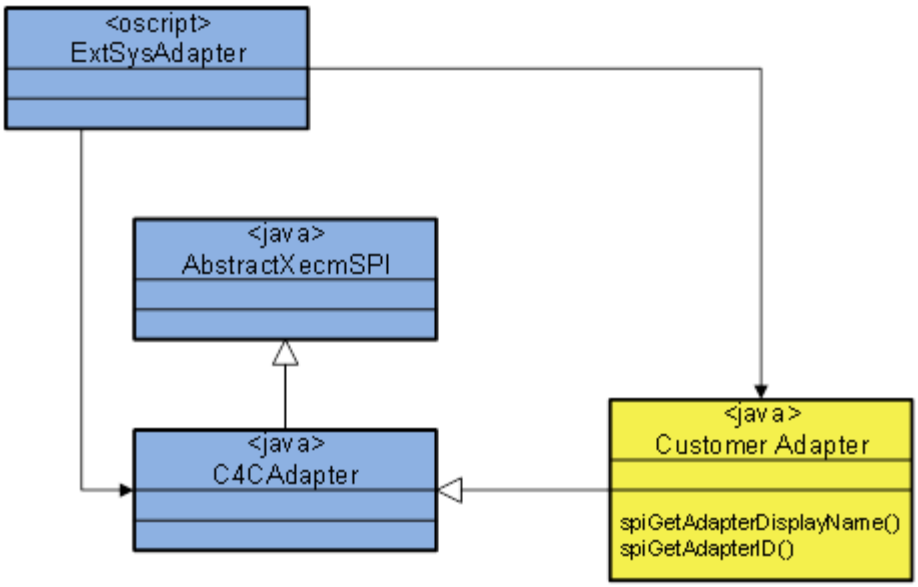
Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- Since version 24.4, each of these function modules has the optional parameter PI_PRIORITY which accepts the same values listed previously.
- But if PI_PRIORITY is not defined when calling one of the function modules, then the default priority is /OTX/RM_WSC_CL_Q_ITEM=>C_PRIO_MEDIUM (10).
- This way, priority /OTX/RM_WSC_CL_Q_ITEM=>C_PRIO_MEDIUM (10) continues to be the priority if these function modules are used as receiver function modules as described in the customizing guides.

4.4 Custom service provider for Cloud for Customer integration

From version 16.2 on, the SPI Framework has been enhanced with a Custom Java Adapter. For more technical details, please use the OpenText Content Management Platform SDK and API Documentation.

The Cloud for Customer (CX) integration already leverages the new SPI implementation technique. The standard implementation could be enhanced or if necessary, completely replaced by customers.



To re-use the standard implementation, customers need to inherit their adapter from the ***com.opentext.xecmsap.c4c.SpiAdapterV2*** class. The class is deployed as part of the OpenText Content Management module in the ***spi.jar***.

OpenText may provide support in the future for additional business objects. Please keep in mind while building on the standard adapter class.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Another solution would be a side-by-side approach, where one external application definition uses the standard CX adapter and a second external application which logically equals to the prior one uses a custom adapter for the customer supported objects. OpenText recommends going this way.

Please note that OpenText reserves the right to change the implementation logic of the published adapter while keeping the interface stable.

4.5 Other OpenText Content Management-specific web services (APIs)

Regarding OpenText Content Management for SAP Solutions, we distinguish between application programming interfaces (APIs) and service provider interfaces (SPIs). APIs are used to access OpenText Content Management for SAP Solutions, functionality from a leading application like SAP® S/4HANA. The SPI is required to access data from the leading application for business workspaces on Content Server.

OpenText Content Management uses the following API web services:

- OpenText Content Management for SAP Solutions Library Service

The factory class /OTX/RM_CL_XECM_FACTORY provides an instance of the API (see Table 3). The instance implements a specific interface, offering a subset of the Web Service functionality.

The API interface is available for customer-specific implementation.

The above API has the following interfaces listed in Table 3.

Web Service	Factory Class	Factory Class Method	Interface
OpenText Content Management for SAP Solutions Library Service	/OTX/RM_CL_XECM_FACTORY	get_xecm_srv()	/OTX/RM_IF_XECM_SRV

Table 3: API factories

The following section describes the most important web service methods.
Note: The section *Deprecated services and interfaces* lists legacy services and interfaces which shall not be used anymore.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

4.5.1 OpenText Content Management for SAP Solutions Library Service

The method get_xecm_srv() of the factory class returns the instance of class /OTX/RM_CL_XECM_SRV, the default implementation of /OTX/RM_IF_XECM_SRV. The implementation uses REST based web services from the OpenText Content Server. Several new methods were introduced in version 25.4 to replace deprecated interfaces.

This web service contains many useful methods which execute many kinds of operations on the Content Server, for example:

- crt_node()
  Creates a Content Server node
- crt_records()
  Declares archived documents as records on Content Server
- cpy_node()
  Copies a Content Server node to a parent Content Server node. With optional changes of name and description.
- del_bus_attachments()
  Deletes business attachments.
- del_records()
  Deletes the records of archived documents from Content Server. It can be checked with method get_use_xecm_del_records( ) of the factory class if this feature is already supported on the Content Server. Whether you are allowed to delete a record depends on the user permissions and records management related settings for the record on Content Server. If the record is the last record, this will only delete the record. The archived document is not deleted.
- get_bus_attachments()
  Gets business attachments.
- get_node_child_list()
  Gets subordinate nodes of a Content Server node. If parameter PI_FILTER is used, it recursively gets all subordinates which match the filter. Otherwise, it gets all direct subordinates using pagination. Then for parameter PI_PAGE-SIZE a negative value can be defined to get all direct subordinates.
- get_node()
  Get information about a Content Server node. Exporting parameter PE_STATE returns current states with metadata_token.

  Exporting parameter PE_VERSIONS returns version related data.
  The exporting parameter PE_PROPERTIES returns expanded user login names create_user, modify_user, and reserved_user, if their node ID's are defined.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- get_records()
  Gets information of specified records from Content Server
- get_unique_names()
  Gets information of all unique names defined on the Content Server. The call can be filtered by lists of unique names or subtype node ID's which are combined in a logical OR condition.
- mov_node()
  Moves a Content Server node to a parent Content Server node. With optional changes of name, description or existing categories. See upd_node() for more details.
- upd_node()
  Update name, description or categories of an existing Content Server node. Changing parameter pc_state supports use of metadata_token which can be retrieved beforehand by using get_node(). If pc_state is initial or the specific metadata_token values are initial, then the updates are not using metadata_token.

Member related methods

This web service includes methods related to users, groups and restricted privileges.These methods make it possible to assign users to generated Content Server groups. For more information, see **Using generated groups to define permissions for workspace folders** in OpenText Content Management for SAP Solutions 26.4 User Management and Access Control Scenarios (ERLK-CUM-EN).

- add_members()
  Adds a list of member ID's to a group or restricted privilege.
- del_members()
  Removes a list of member ID's from a group or restricted privilege.
- get_member()
  Search a user, group or restricted privilege. Structured constant C_MEMBER_INFO_TYPE contains the relevant types. The method may return more than one match. Then please narrow down the results accordingly.
- get_member_info()
  Gets information about users, groups or restricted privileges.
- get_member_list()
  Get the list of members from a group or restricted privilege.

This web service also provides structures of constants, like:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- C_CS_PERMISSION
  A list of hardcoded types of permission on a Content Server node
- C_NODE_TYPE
  A list of the most relevant Content Server node types

4.5.2 Deprecated services and interfaces

Starting with version 25.4 the following SOAP based services and interfaces are deprecated.

- Class /OTX/RM_CL_WS_FACTORY
- Content Server Member Web Service (interface: /otx/rm_if_csmember_service)
  Use the member related methods of the OpenText Content Management for SAP Solutions Library Service instead.
- Unique Names Service (interface /otx/rm_una_if_xecm_service)
  Use the unique names method of the OpenText Content Management for SAP solutions Library Service instead.
- Directory Services Authentication (interface /otx/rm_if_elauth_service)

4.6 Deletion of ArchiveLink records

Important note: If you work with declarations always configure the connection in SAP transaction OAC0 to ArchiveLink Filter.

4.6.1 Standard behavior

This is the behavior since version 22.4.

4.6.1.1 Deleting declared ArchiveLink links

The implementation of BAdI OA_BADI_LINK uses the method del_records() of /OTX/RM_IF_XECM_SRV.

This only deletes the record on the Content Server but not the content of the archived document if this was the last record.

Whether method del_records() is executed successfully depends on the permissions of the Content Server user assigned to your SAP user. Holds or enabled disposition management will block any deletion.

The BAdI implementation is restoring the link in the TOAxx table if the record exists and cannot be deleted.

The deletion of the archived document via ArchiveLink Filter got stricter but also more straightforward. If there is any record left on the Content Server, the ?delete command to ArchiveLink Filter will fail.

Business Content Window and Fiori App for Business Documents

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

The UIs only allow to delete links if the records have no Records Management (RM) classification. This is still stricter than the BAdI implementation would allow.

ArchiveLink Technical Search (SAP transaction OAAD) and other solutions using function module ARCHIV_DELETE_META

If the last link could be deleted successfully, the follow-up step to delete the archived document is expected to work as well.

Employee File Management

Please check documentation of Employee File Management.

Troubleshooting

We successfully deleted every link on SAP but function modules ARCHIV_DELETE_META and ARCHIVOBJECT_DELETE are not allowed to delete the archived document.

Check on Content Server, if there are still any records or documents referring to the archived document.

We are not allowed to delete records with ARCHIV_DELETE_META.

Check application log (SLG1) for the detailed message from the Content Server. Check if the Content Server user assigned to your SAP user has sufficient permissions.

4.6.1.2 Moving declared ArchiveLink links

A move of an ArchiveLink link entry is an insert of a new link followed by the deletion of the source link.

Here is a list of scenarios.

Using action Reassign document in the ArchiveLink technical search

In SAP transaction OAAD you can execute the action `Choose detail` of a selected link entry. In the details screen you can execute the action `Reassign document`. Please note, that this does not roll back the reassign action if the deletion of the source link failed. In this case you still have the source link preserved and the new link is also created.

Employee File Management

Please check documentation of Employee File Management.

Troubleshooting

We have successfully assigned a new ArchiveLink entry to the document but after action reassign the source entry is still there.

The `Reassign document` action in the ArchiveLink technical search does no rollback. If a link is restored because the corresponding record on Content Server cannot be deleted, this is as designed.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

4.6.2 Creating a program which cleans up orphaned records on Content Server

Prerequisites:

- You had a Content Server with version 22.4 or higher.
- ABAP Add-On package OTEXRL on the SAP system was version 2140_700 or lower.
- You have an ArchiveLink log of deleted entries which still have undeleted records and undeleted archived documents.
- OTEXRL on the SAP system is now version 2240_700 or higher.

Solution:

Contact OpenText Customer support to get a cleanup program from OpenText development.

Alternatively implement a program which fits to your needs. Here are some implementation hints.

- Use function module ALINK_READ_LOG to get log entries. The delete entries have oalogfct = 'D'.
- For each result
    - Use method cl_alink_connection=>find_by_doc_key() to find out if there are still link entries in SAP. If yes, then do not delete.
    - Use function module ARCHIVOBJECT_STATUS to find out if the archived document still exists. Skip, if no document exists anymore.
    - Use method /otx/rm_if_xecm_srv->del_records( ) to delete the record related to the link. If the delete was successful or if no record was found, then do not skip.
    - Use function module ARCHIVOBJECT_DELETE to delete the archived document via ArchiveLink filter. If there is still some record left on the Content Server, the deletion will fail.

You may use transaction oa_log_view_doc to delete log entries which you do not require anymore.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

5 Business Attachment enhancements

5.1 Enhancements on SAP

5.1.1 Introduction

OpenText Content Management for SAP Solutions provides the possibility to integrate custom enhancements via BAdI during business attachment creation.

5.1.2 SAP BAdIs

5.1.2.1 Introduction to SAP BAdIs

Business Add-Ins (BAdIs) is an SAP enhancement technique based on ABAP objects. They can be inserted into the SAP system to accommodate user requirements that are too specific to be included in the standard implementation. In OpenText Content Management for SAP Solutions, we created a BAdI definition for the add-in.

5.1.2.2 OpenText Content Management for SAP Solutions BAdIs

The Business Add-Ins available for OpenText Content Management for SAP Solutions can be found in the SAP package /OTX/RM_DOL. The definition name is /OTX/RM_DOL_REF_LL.
You can use transaction **SE18**, enter BAdI Name /OTX/RM_DOL_REF_LL -> Display.
Here you can find two interface methods: execute_add() and execute_del(). execute_add() is executed before a Business Attachment is created and execute_del() is performed before a Business Attachment is deleted.
You can transfer the following parameters to execute_add()/execute_del():

```
PI_OBJ_TYPE:     SAP object type (e.g., KNA1, EQUI)
PI_OBJ_KEY:      SAP object identifier
LL_NODE_ID:      Content Server node id
PT_CAT_ATTR:     category and attributes of the Content Server node id
PT_CAT_ATT_SET: category and attributes of the Content Server node id
grouped by Content Server attribute type “set”
FLT_VAL: SAP     SAP object type (for BAdI filter)
Exceptions:
/OTX/CX_RM_DOL:              Exception in case of failure
/OTX/CX_RM_DOL_NOT_ALLOWED: Exception in case it is not allowed to
add/delete a business attachment in CS
```

In addition, the OpenText Content Management for SAP Solutions BAdI has implemented a filter value. Thus, it is possible to have different

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

implementations for different SAP object types. The filter mechanism is done by SAP.

5.1.2.3 How to create a BAdI implementation

Starting from the name and the documentation of the OpenText Content Management for SAP Solutions enhancement (/OTX/RM_DOL_REF_LL), you can create a BAdI implementation. During the implementation, the system creates a class that implements the enhancement's interface. The implementation has a separate transport object and lies within the namespace of the person or organization implementing it.

For the example given here, you must create an implementation for the SAP object type KNA1.

To create an implementation:

1. Use transaction **`SE19`** (Business Add_Ins: Implementations).
2. Enter in **Create Implementation** section the name of the BAdI definition (OTX/RM_DOL_REF_LL) and then click **Create Impl.**.
3. Enter the implementation name of the add-in.
4. Enter an Implementation short text describing your implementation.
5. Choose **Attributes** tab and define an object type as filter (here KNA1)
6. Choose the **Interface** tab.
7. Navigate to the **Class Builder** by double-clicking the method execute_add. You must first enter a package before the "Class Builder" is displayed.
8. Insert the desired source code for the implementation between the **`method if_/OTX/RM_DOL_IF_EX_REF_LL~execute_add.`** and **`endmethod.`**
9. Save and check your entries and return to the **Change Implementation** screen.
10. Do the same steps 7. – 9. for the execute_del method.
11. Click Activate.
12. Go to the Content Server via http browser and activate the execution of BAdI for the SAP object type KNA1.

5.2 Enhancements on Content Server

5.2.1 Introduction

The Content Server knows the concept of callbacks, i.e., defining a piece of code that is passed as a reference to existing code. Thereby it is possible to influence the existing code by such pieces of code.
In the context of Business Attachment, different callbacks are defined. All

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Business Attachment specific callbacks return an Assoc with Boolean ok and String errMsg as it is standard in Content Server.

You can find examples for the callbacks in the additional module otsapxecmsample (execute ….func = otsapxecmsample.SampleBusinessReference). For successful execution of the samples, you need a fully configured Content Server notification.

(CS Administration -> Notification Administration -> Configure Notification)

5.2.2 Callbacks on Content Server

5.2.2.1 Callbacks during automatic creation of a Business Attachment

Use the standard Content Server callbacks “CBCategoriesUpdatePre” and “CBCategoriesUpdatePost”, i.e., make an orphan of $LLIAPI -> LLIApi Root -> NodeCallbacks into your OSpace and implement CBCategoriesUpdatePre/-Post accordingly. Do not forget to set .fEnabled of the orphaned object to “true”.

This allows you to write for example your own bottom-up search.
In the global $OTSAPLINK.AutomaticBusinessReferences you can find some useful functions to use for your own automatic creation of a Business Attachment.

For example:

IsAttrChanged():	Checks whether a trigger attribute has changed
GetKeyDefs():	Gets the key values from categories of the Content Server item

5.2.2.2 Callbacks during creation of a Business Attachment

Use the Business Workspaces callbacks “CBCreateBusinessReferencePre” and “CBCreateBusinessReferencePost”, i.e., make an orphan of $OTSAPLINK -> Otsaplink Root -> BusinessReferenceCallBacks into your OSpace and implement CBCreateBusinessReferencePre/-Post accordingly. Do not forget to set .fEnabled of the orphaned object to “true”.
Run BuildOspace() in your OSpace globals, stop and start your Content Server.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
function Assoc CBCreateBusinessReferencePre(
   Object     dapiCtx,        // DAPI context
   DAPINODE   node,           // node for which the business
attachment is created for
   Assoc         ref_config,     // business object configuration
   String     sapObjectKey,   // unique sap object identifier
   String     comment         // comment for the business attachment
)
function Assoc CBCreateBusinessReferencePost(
   Object     dapiCtx,        // DAPI context
   DAPINODE   node,           // node for which the business
attachment is created for
   Assoc         ref_config,     // business object configuration
   String     sapObjectKey,   // unique SAP object identifier
   String     comment         // comment for the business attachment
)
```

5.2.2.3 Callbacks during update of a Business Attachment

The Business Attachment is updated if an SAP object is changed (name or metadata of this SAP object).
Basically, beside some checks whether update is allowed, the following sequence is executed:

- Execution of callback CBUpdateBusinessReferencePre
- Update of meta data in the categories of the Content Server item containing the Business Attachment
- Update of the SAP object name of the Business Attachment
- Execution of callback CBUpdateBusinessReferencePost

Make the callbacks as it is described in the “callbacks during creation of the Business Attachment”.

```
function Assoc CBUpdateBusinessReferencePre(
   Object     dapiCtx,        // DAPI context
   DAPINODE   node,           // node for which the business
attachment is created for
   Assoc      ref_config,     // business object configuration
   Assoc         linkInfo,       // see database table
OTSAPXECM_REF_LINKS
   List       categories,     // which categories to update
   String     newName         // new name of the sap object
 )
function Assoc CBUpdateBusinessReferencePost(
   Object     dapiCtx,        // DAPI context
   DAPINODE   node,           // node for which the business
attachment is created for
   Assoc      ref_config,     // business object configuration
   Assoc      linkInfo,       // see database table
OTSAPXECM_REF_LINKS
   List       categories,     // which categories to update
   String     newName         // new name of the sap object
 )
```

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

5.2.2.4 Callbacks during deletion of a Business Attachment

Use the Business Workspaces callbacks “CBDeleteBusinessReferencePre” and “CBDeleteBusinessReferencePost”.
Make the callbacks as it is described in the “Callbacks during creation of a Business Attachment”.

```
function Assoc CBDeleteBusinessReferencePre(
   Object     dapiCtx,,       // DAPI context
   DAPINODE   node,           // node for which the business
attachment is created for
   Assoc      ref_config,     // business object configuration
   String     sapObjectKey )  // unique sap object identifier
)
function Assoc CBDeleteBusinessReferencePost(
   Object     dapiCtx,,       // DAPI context
   DAPINODE   node,           // node for which the business
attachment is created for
   Assoc      ref_config,     // business object configuration
   String     sapObjectKey )  // unique sap object identifier
)
```

6 Business workspace enhancements

6.1 Enhancements on SAP

6.1.1 Additional business workspace tabs for SAP objects

6.1.1.1 Introduction

You can display the Business Workspace of Content Server in SAP by the GOS menu. Sometimes it is desired to display the Business Workspace in the “Tabs” of an SAP object. As each SAP object type has its own infrastructure to implement an additional tab, we provide here only 2 examples with their specific implementation. For other SAP object types refer to the SAP documentation.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

6.1.2 Example for SAP object type “Customer”

6.1.2.1 Example screen

In customer (object type KNA1, transaction **XD03**) you want to get a screen like in Figure 17.

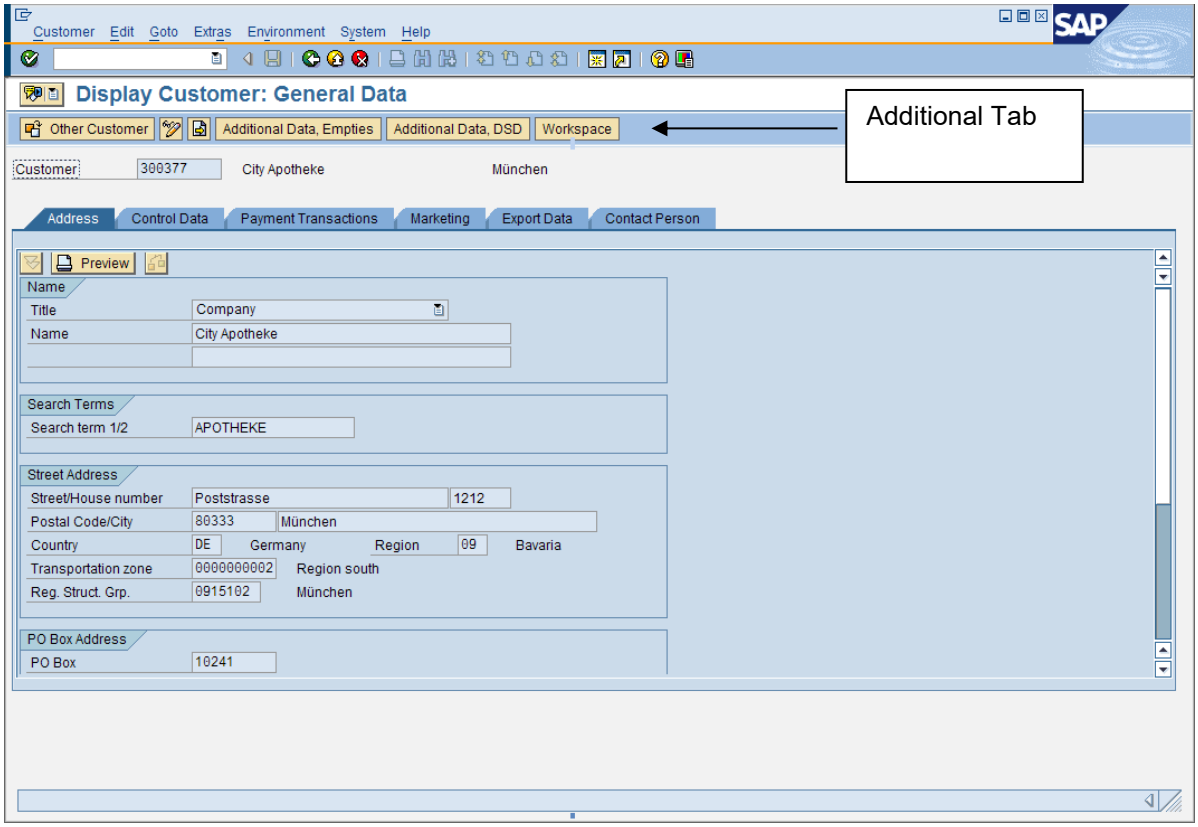


Figure 18: XD03 transaction

Figure 19 shows the tab integration when clicking on the Workspace button (Business Workspace for customer 300377).

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

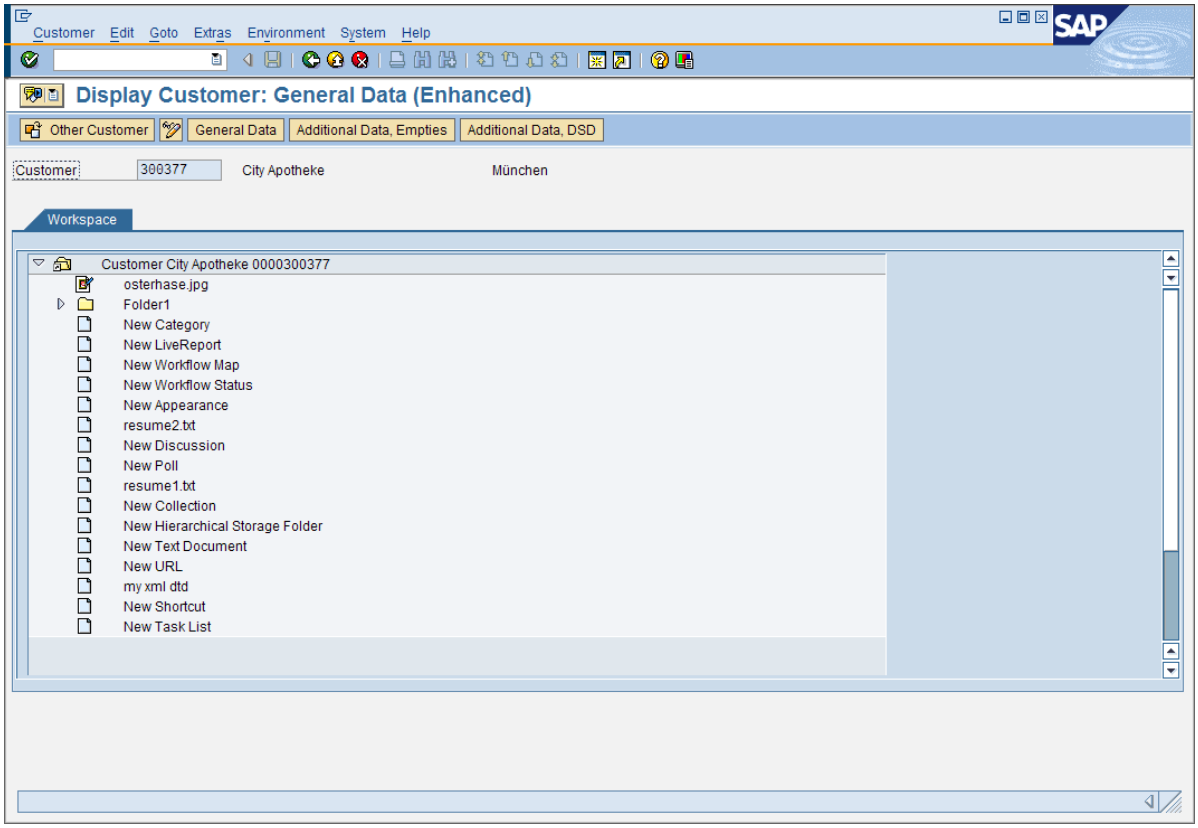


Figure 19: Business Workspace tab integration

6.1.2.2 Implementing the Business Workspace tab

1. Go to SPRO activity **Logistics – General > Business Partner > Customers > Control > Adoption of Customer's own Master Data fields > Prepare Modification - Free enhancement of Customer Master Record.**

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

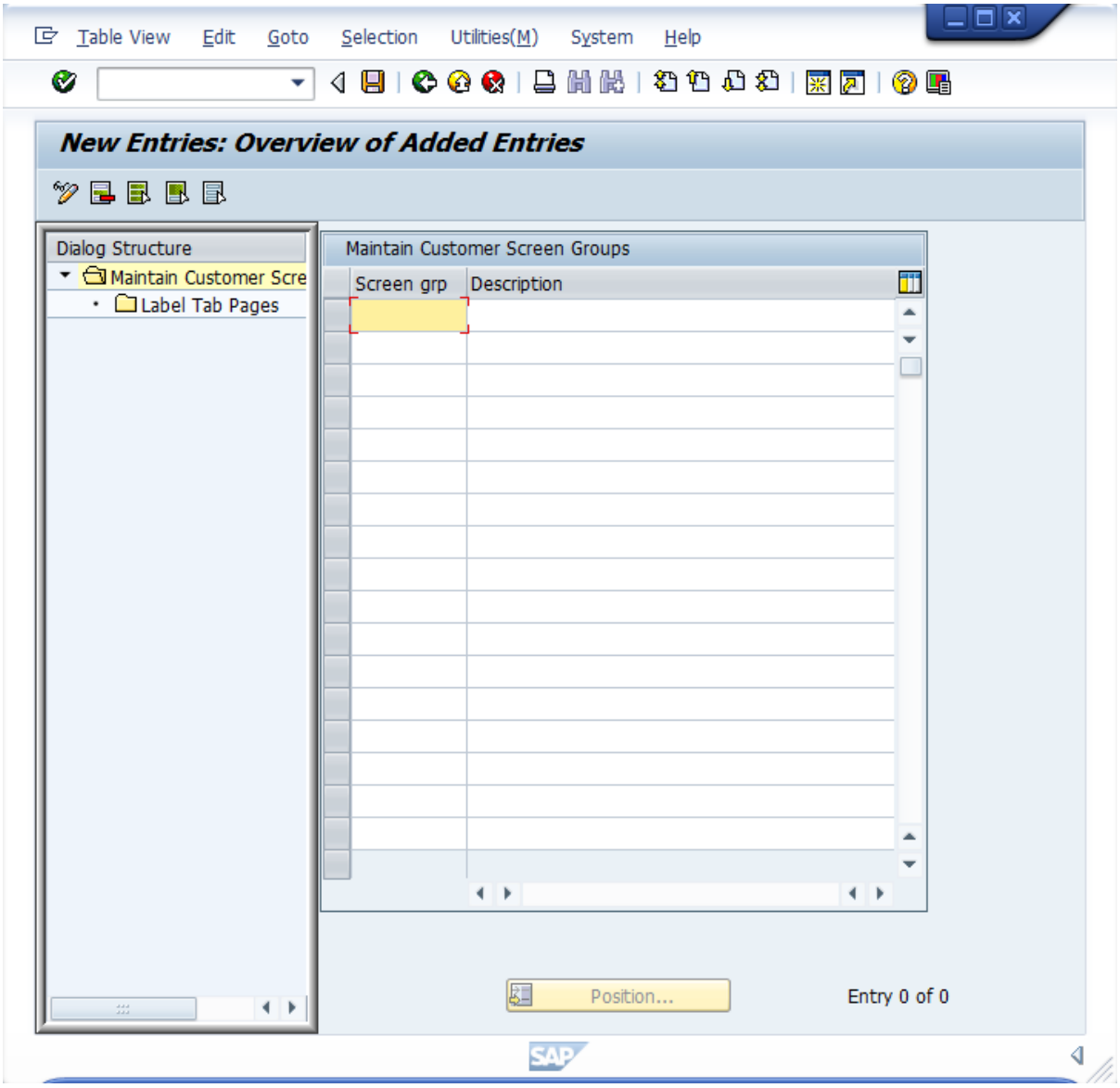


Figure 20: How to integrate a business workspace tab (1)

2. Go to change mode and click **New Entries**. Then add a Screen Group (SK here) and a meaningful description. Save the entry, and then select that entry and click **Label Tab Pages**.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

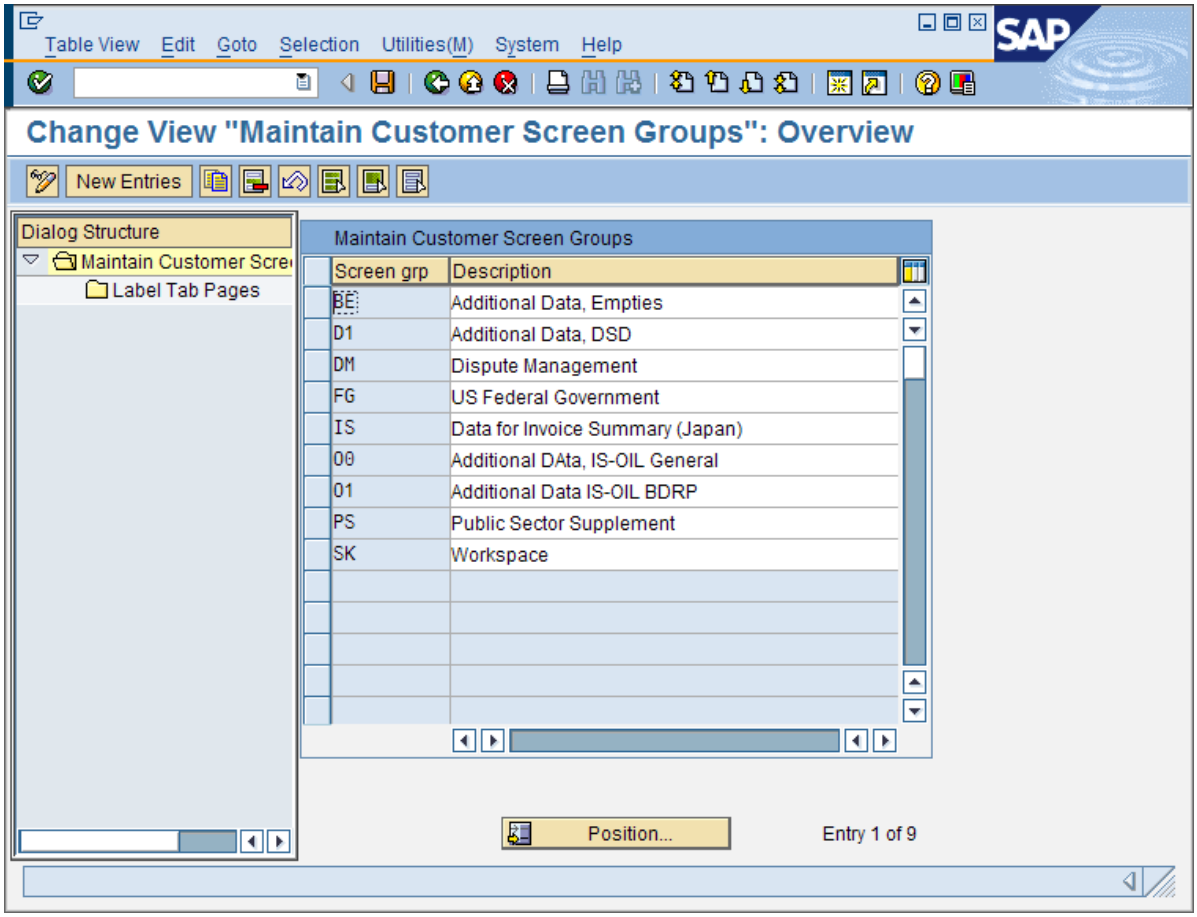


Figure 21: How to integrate a business workspace tab (2)

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

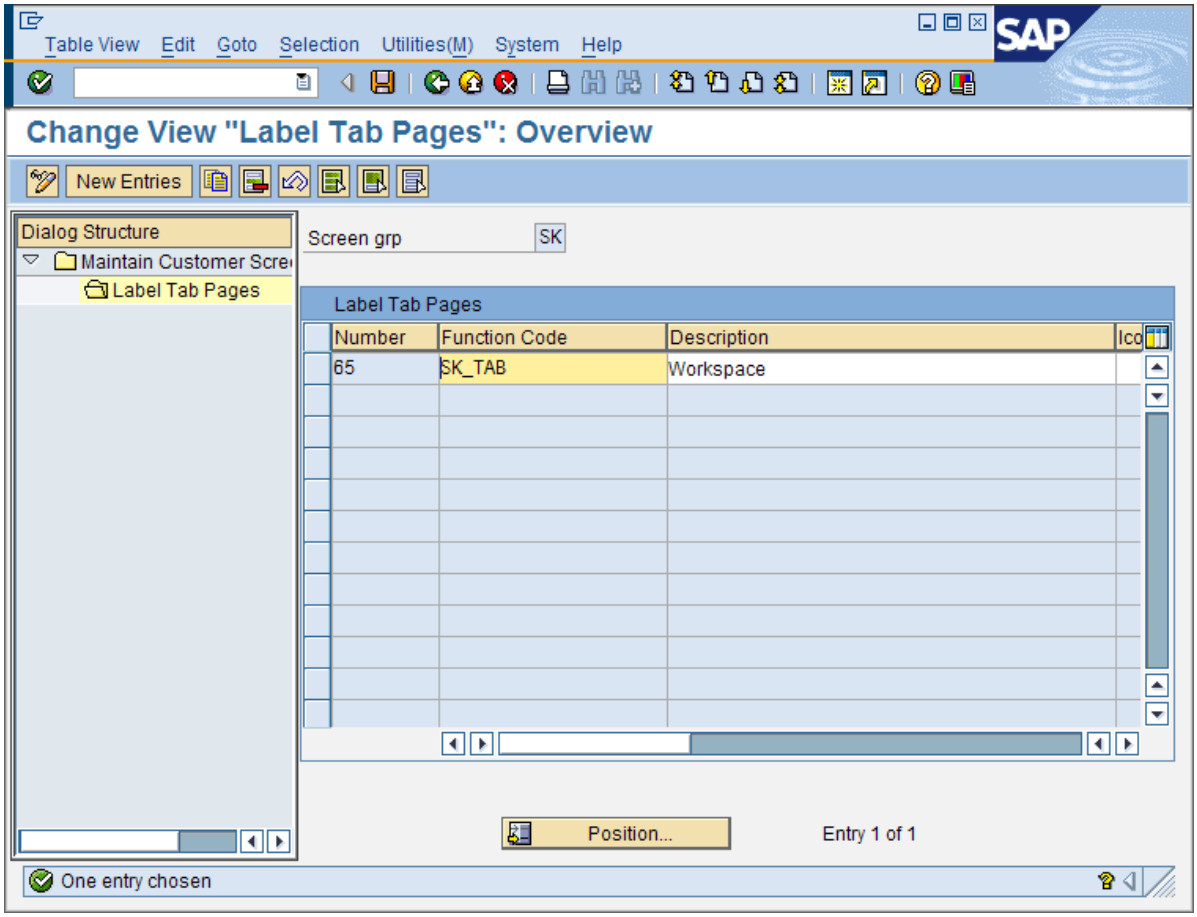


Figure 22: How to integrate a business workspace tab (3)

3. Add a function code **SK_TAB** and a description. Save the entry and go back to the SPRO screen. Click **Business Add-in > Processing Master Data Enhancements**.

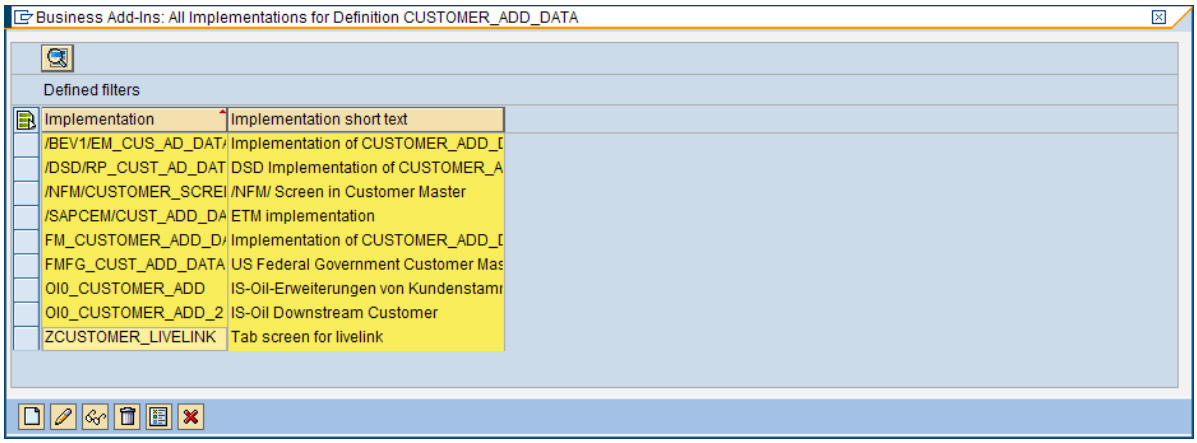


Figure 23: How to integrate a business workspace tab (4)

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

4. Create one implementation by giving implementation name (must start with Z. ZCUSTOMER_LIVELINK here). Enter a meaningful description and press ENTER. You see the following screen:

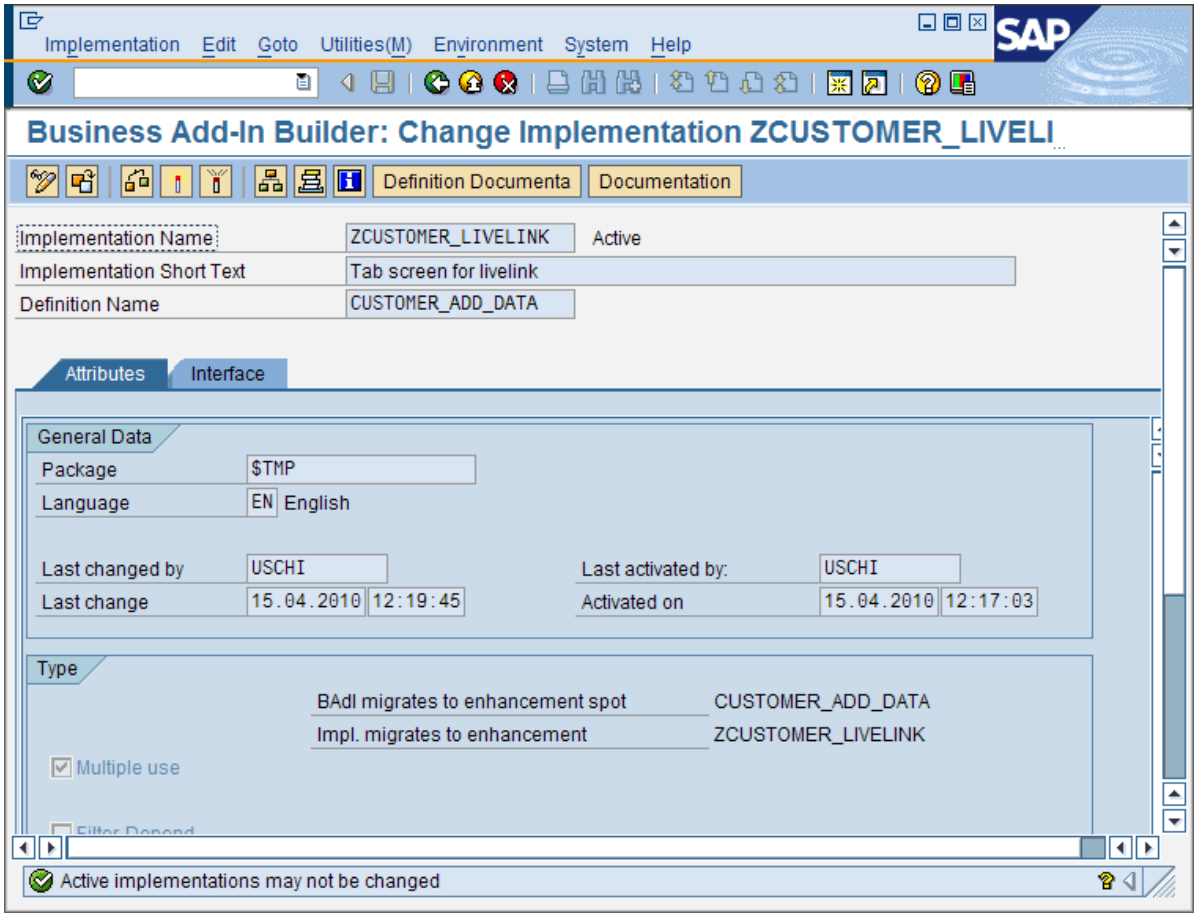


Figure 24: How to integrate a business workspace tab (5)

5. Go to the **Interface** tab and click CHECK_ADD_ON_ACTIVE method. Add the following code and save.

```
method IF_EX_CUSTOMER_ADD_DATA~CHECK_ADD_ON_ACTIVE.
data: l_flg_active type BOOLE-BOOLE.
if i_screen_group = 'SK'.
      e_add_on_active = 'X'.
  endif.
endmethod.
```

6. Activate that and go back to the SPRO initial screen. Click **Business Add-in > Customer Subscreens**.
Create one implementation (ZCUSTOMER_LIVELINKCS here).

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

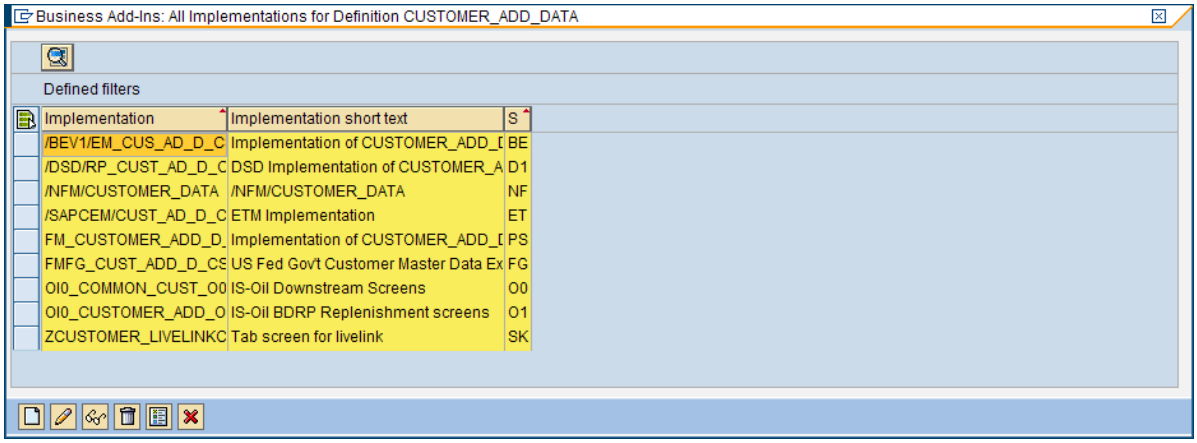


Figure 25: How to integrate a business workspace tab (6)

7. Press ENTER. The following screen is displayed. Enter a meaningful description and screen group (which we created earlier) as shown below.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

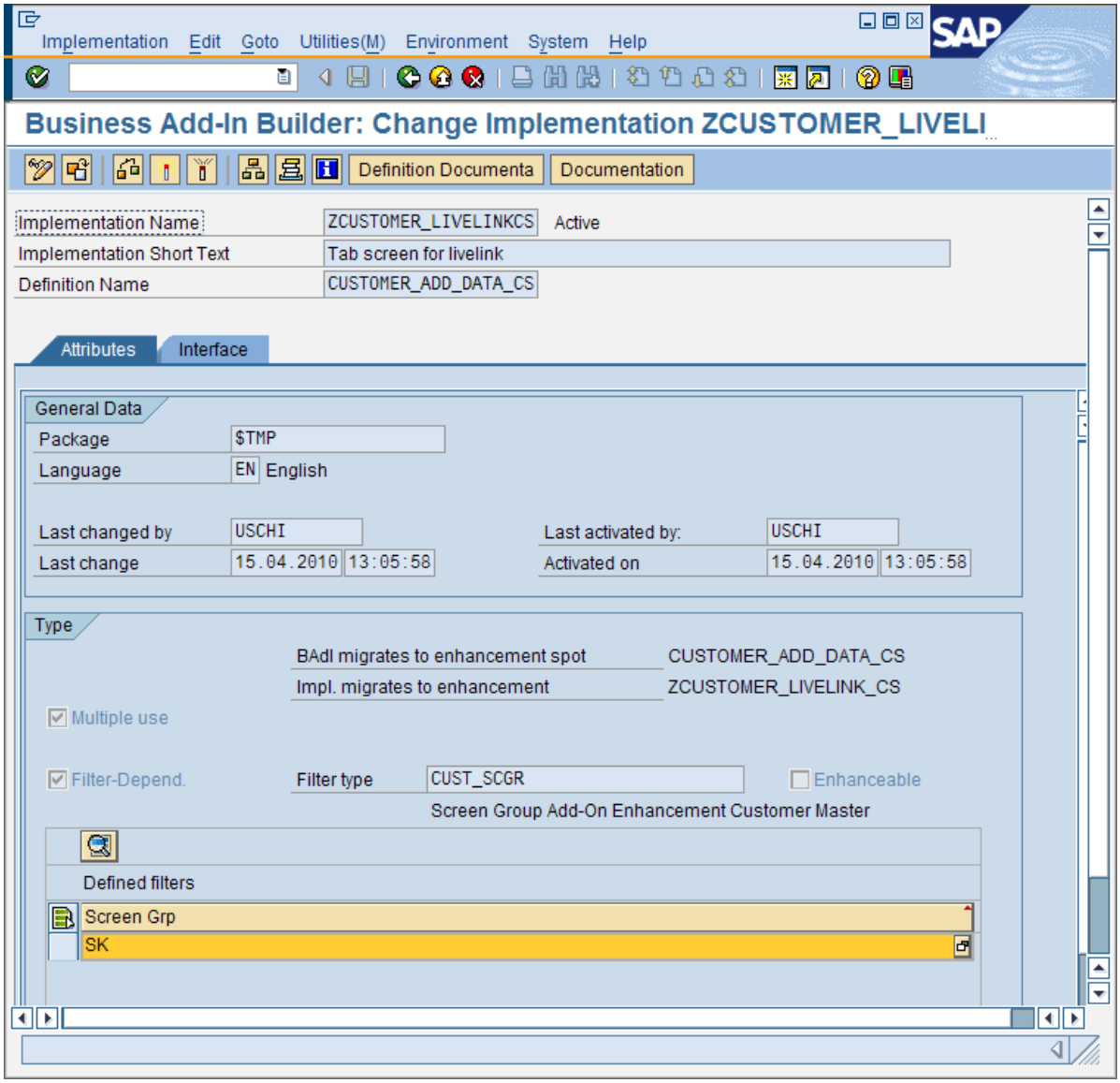
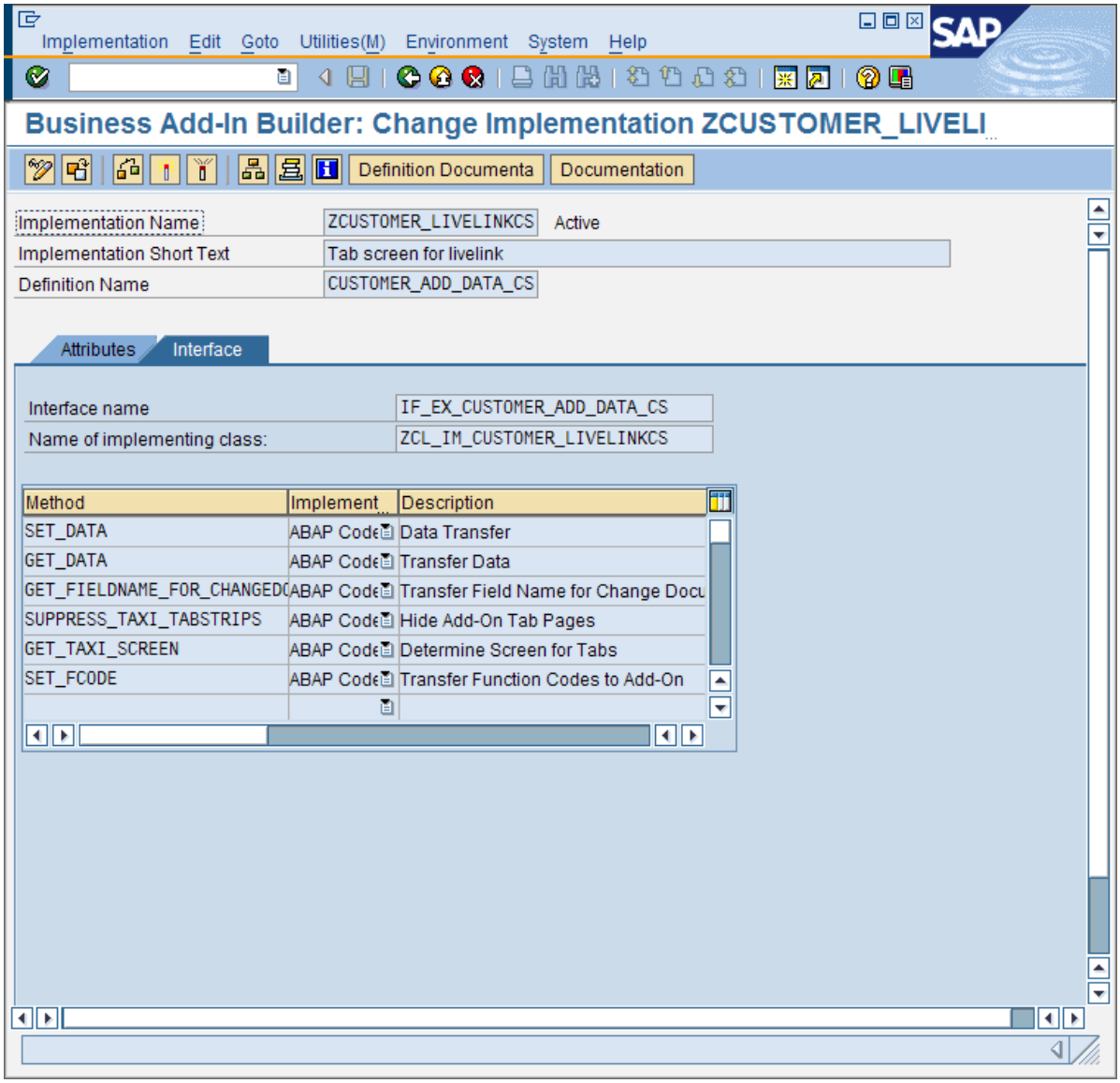


Figure 26: How to integrate a business workspace tab (7)

8. Click the **Interface** tab.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.



9. Create a PROGRAM ZLIVELINKSCREEN_PROG. Create a Screen ‘1111’ with PBO:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
module STATUS_1111 output.
* SET PF-STATUS 'xxxxxxxx'.
* SET TITLEBAR 'xxx'.
  DATA:init    TYPE c,
       l_kun   TYPE KUN16,
       obj_key TYPE  SWEINSTCOU-OBJKEY.

DATA container TYPE REF TO cl_gui_custom_container.

IF init IS INITIAL.
    CREATE OBJECT container
      EXPORTING
         container_name = 'CUSTOM'.
    IF sy-subrc <> 0.
    MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
      WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
    ENDIF.
ENDIF.

GET PARAMETER ID 'KUN' FIELD l_kun.
obj_key = l_kun.
CALL FUNCTION '/OTX/RM_WSC_TREE_POPUP'
    EXPORTING
      pi_obj_type   = 'KNA1'
*     pi_obj_key    = '0000300377'
*     pi_obj_key    = '0000000176'
      pi_obj_key    = obj_key
      pih_container = container.

endmodule.
```

10. Click on GET_TAXI_SCREEN method and add the following code.

```
method IF_EX_CUSTOMER_ADD_DATA_CS~GET_TAXI_SCREEN.
  case i_taxi_fcode.

    when 'SK_TAB'.

      e_screen  = '1111'.

      e_program = 'ZLIVELINKSCREEN_PROG'.

      e_headerscreen_layout = ' '.

  endcase.
endmethod.
```

11. Go to **XD03** transaction.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

6.1.3 Example for SAP object type “Vendor”

6.1.3.1 Implementing the tab

12. Analog to Customer, but in the **SPRO** transaction go to: **Financial Accounting > Accounts Receivable and Accounts Payable > Vendor Accounts > Master Records > Preparations for Creating Vendor Master Records > Adoption of Customer's Own Master Data Fields**
13. Implement VENDOR_ADD_DATA and VENDOR_ADD_DATA_CS BAdIs and a program with a screen:

```
module STATUS_1112 output.
*  SET PF-STATUS 'xxxxxxxx'.
*  SET TITLEBAR 'xxx'.
  DATA: init TYPE c.
  data: l_vend type LIF16.
  DATA: obj_key TYPE  SWEINSTCOU-OBJKEY.

DATA container TYPE REF TO cl_gui_custom_container.

IF init IS INITIAL.
    CREATE OBJECT container
      EXPORTING
         container_name = 'CUSTOM'.
    IF sy-subrc <> 0.
    MESSAGE ID sy-msgid TYPE sy-msgty NUMBER sy-msgno
      WITH sy-msgv1 sy-msgv2 sy-msgv3 sy-msgv4.
    ENDIF.
ENDIF.

GET PARAMETER ID 'LIF' FIELD l_vend.
obj_key = l_vend.
CALL FUNCTION '/OTX/RM_WSC_TREE_POPUP'
    EXPORTING
      pi_obj_type   = 'LFA1'
*     pi_obj_key    = '0000000424'
*     pi_obj_key    = '0000000176'
      pi_obj_key    = obj_key
      pih_container = container.
endmodule.
```

6.1.4 Authentication

The Content Server proceeds with user authentication, regardless of whether the SAP server communicates with the Content Server via Web Services. OpenText Content Management for SAP Solutions offers the two following authentication scenarios:

- Single Sign-on (SSO) authentication scenario
- BAdI-based impersonation

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

6.1.4.1 Single sign-on

Single sign-on is the default authentication mechanism. In this scenario, SAP users must have identical log-on names in the Content Server. The OTDS Services grant access to the Content Server by ensuring the authentication of SAP tokens (see Install Guide).

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

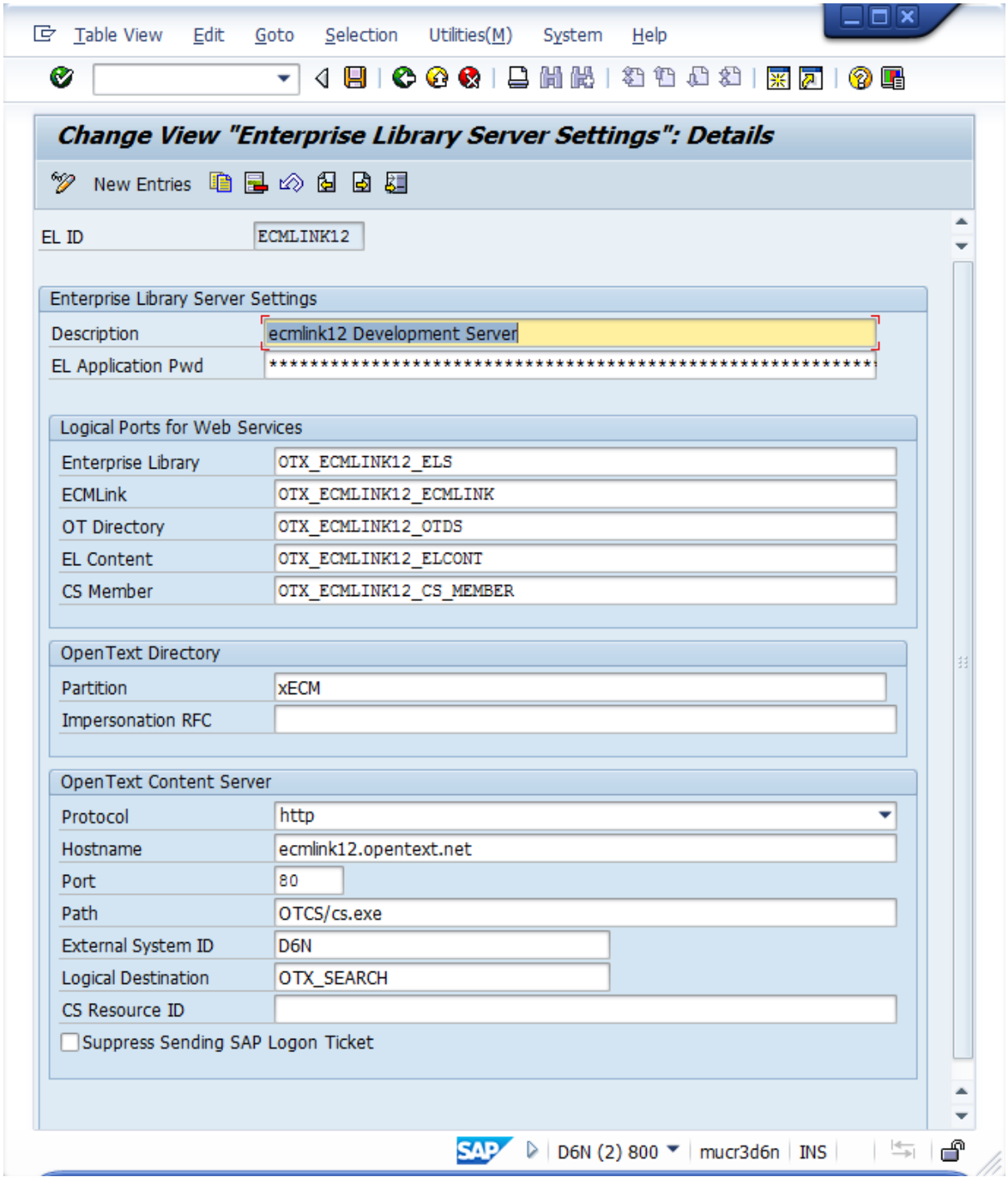


Figure 27: Single sign-on

6.1.4.2 BAdI-based impersonation

BAdI-based impersonation enables impersonating each SAP user with a corresponding Content Server user (i.e., OTDS user). To achieve this scenario, an SAP developer (e.g., Open Text Global Services consultant) must

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

implement and activate the /OTX/RM_IMPERSONATE BAdI. This BAdI has the following two methods:

- GET_IMPER_USER substitutes the current user (i.e., *sy-uname*) with another Content Server user. OpenText Content Management for SAP Solutions will use this substitute user for communicating with the Content Server.
- GET_IMPER_USERS_POLICIES substitutes the given table of SAP policy users with a table of their respective Content Server users. OpenText Content Management for SAP Solutions will use the substitute users to assign user policies in the Content Server. Notice that the BAdI-based impersonation only requires this method if policies are used.

For both methods, the substitute username must exactly match in upper and lower cases the corresponding Content Server username.

The BAdI-based impersonation scenario is active if all the following conditions apply:

- There is an active implementation of the /OTX/RM_IMPERSONATE BAdI
- The target Content Server settings has an “Impersonation RFC” destination (see Installation Guide [ERLK-IGD,EESPCD-IGD])
- For Web Service:
  The user defined in the “Impersonation RFC” destination must be assigned to an OTDS administrator group e.g., otadmins.
- CS Resource ID must be defined in OpenText Content Management connection settings
- Roles must be assigned according to the Customizing Guide [ERLK-CGD,EESPCD-IGD]

For more information on how to implement the impersonation BAdI methods, please look at the /OTX/RM_IF_IMPERSONATE interface documentation.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

6.1.5 Generic Web Dynpro for Workspaces

6.1.5.1 Introduction

For some SAP systems, for example SRM, it is necessary to display a business workspace in the portal by a Web Dynpro for ABAP component. An SAP-specific Web Dynpro embeds a generic Web Dynpro component:

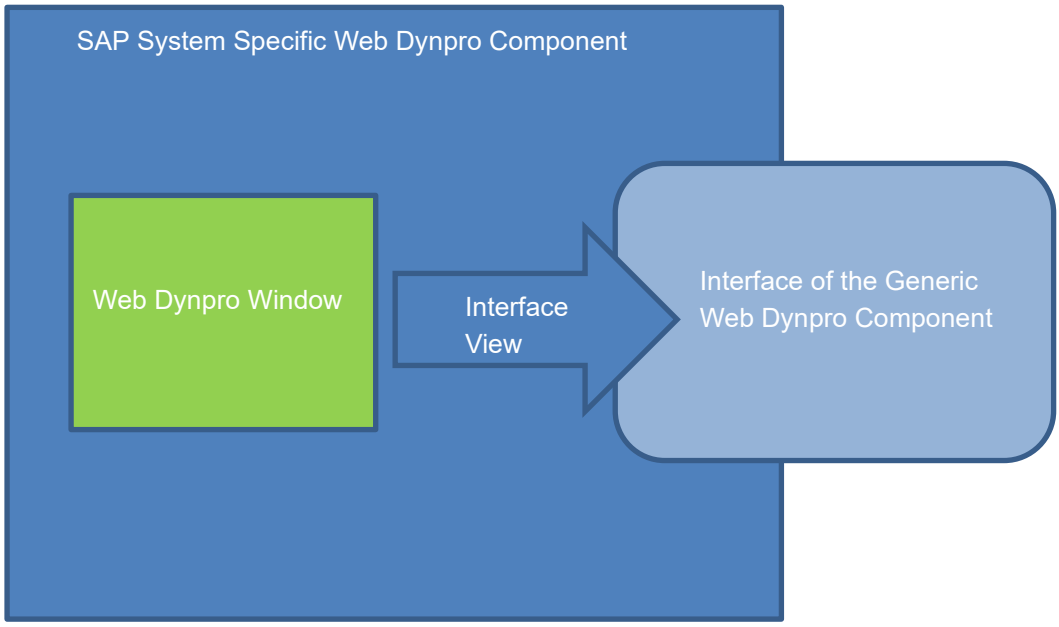


Figure 28: Generic Web Dynpro introduction

6.1.5.2 The interface of the Generic Web Dynpro Component

The interface of the Generic Web Dynpro Component consists of the method *set_business_object* which provides the following input parameters:

Input parameter	Type of input parameter	Optional	Description
PI_CC_VIEW	STRING	yes	Mode of xecmpf Create and Complete integration widget: 'FB' for folderBrowse or 'FP' for fullPage, if not set the csui folder widget is the used
PI_IS_KEY_INITIAL	WDY_BOOLEAN		Flag whether business workspace is already created or not, true or false
PI_IS_KEY_INITIAL_MSG	STRING	yes	Message shown in case business workspace is not yet created
PI_OBJID	SWO_OBJID		Business object id
PI_OBJTYP	SWO_OBJTYP		business object type
PI_THEME_ID	STRING	yes	Theme ID, e.g., sap_tradeshow; starting with version 22.4 the value of this parameter will be ignored

Table 4: Interface of the Generic Web Dynpro Component

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

6.1.5.3 How to embed the generic Web Dynpro in your SAP-specific Web Dynpro

Here we describe the steps from scratch how to embed the generic Web Dynpro in your specific Web Dynpro.

- Create your specific Web Dynpro component
- Add the generic Web Dynpro component to your created Web Dynpro component:

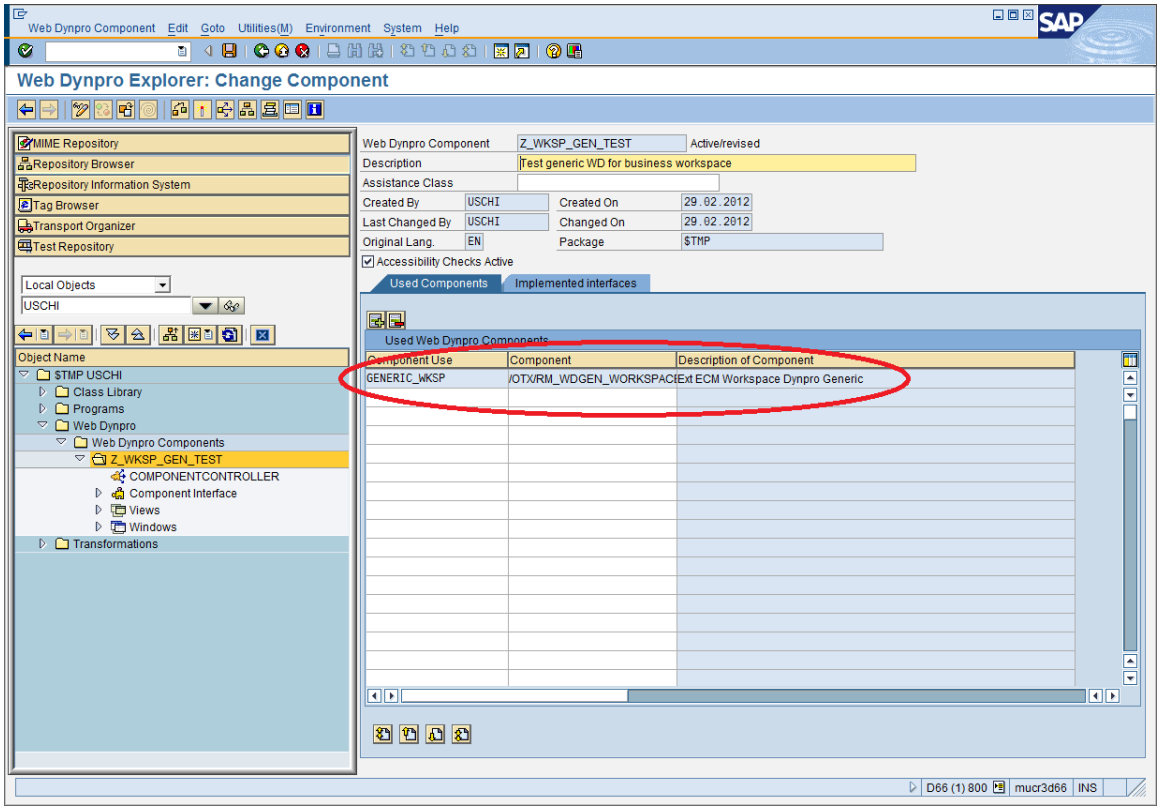


Figure 29: Add generic Web Dynpro (1)

- Add the generic Web Dynpro component to the component controller of your Web Dynpro:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

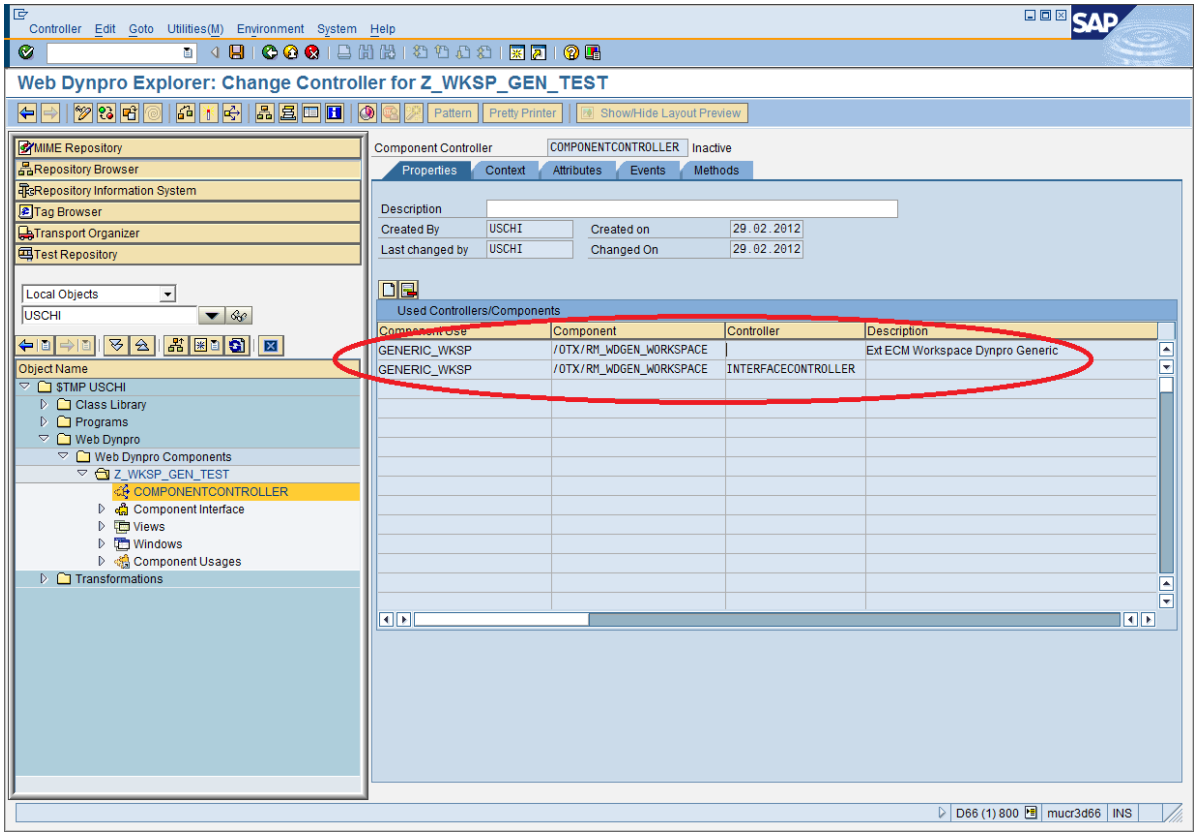


Figure 30: Add generic Web Dynpro (2)

- Create a view in your Web Dynpro component
- Select from the Standard Simple UI library the ViewContainerUIelement and drag it to the layout of the view. A new ViewContainerUIelement will be placed at the layout.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

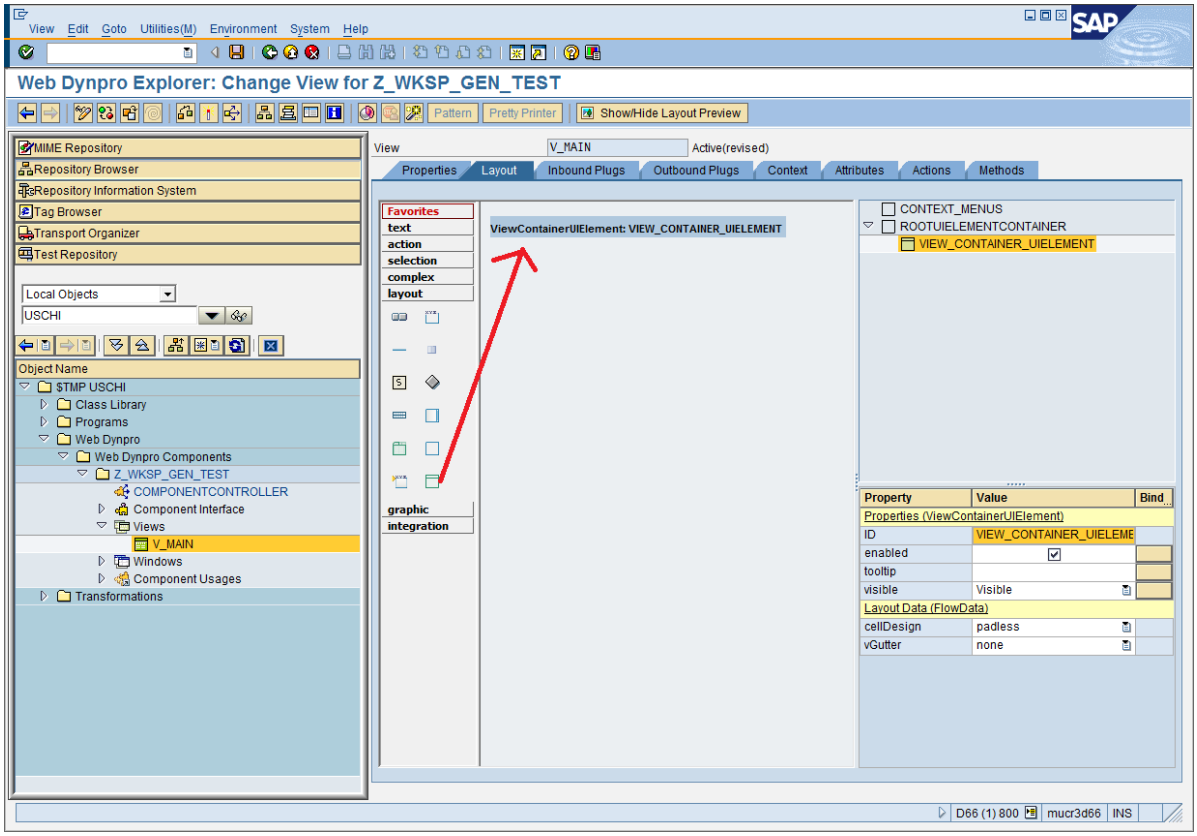


Figure 31: Add ViewContainerUIelement

- Create a window in your Web Dynpro component, if it does not yet exist
- Embed the view (containing the ViewContainerUIelement element) of your Web Dynpro component in the created window
- Embed the view of the generic Web Dynpro component in the “VIEW_CONTAINER_UIELEMENT” of you window

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

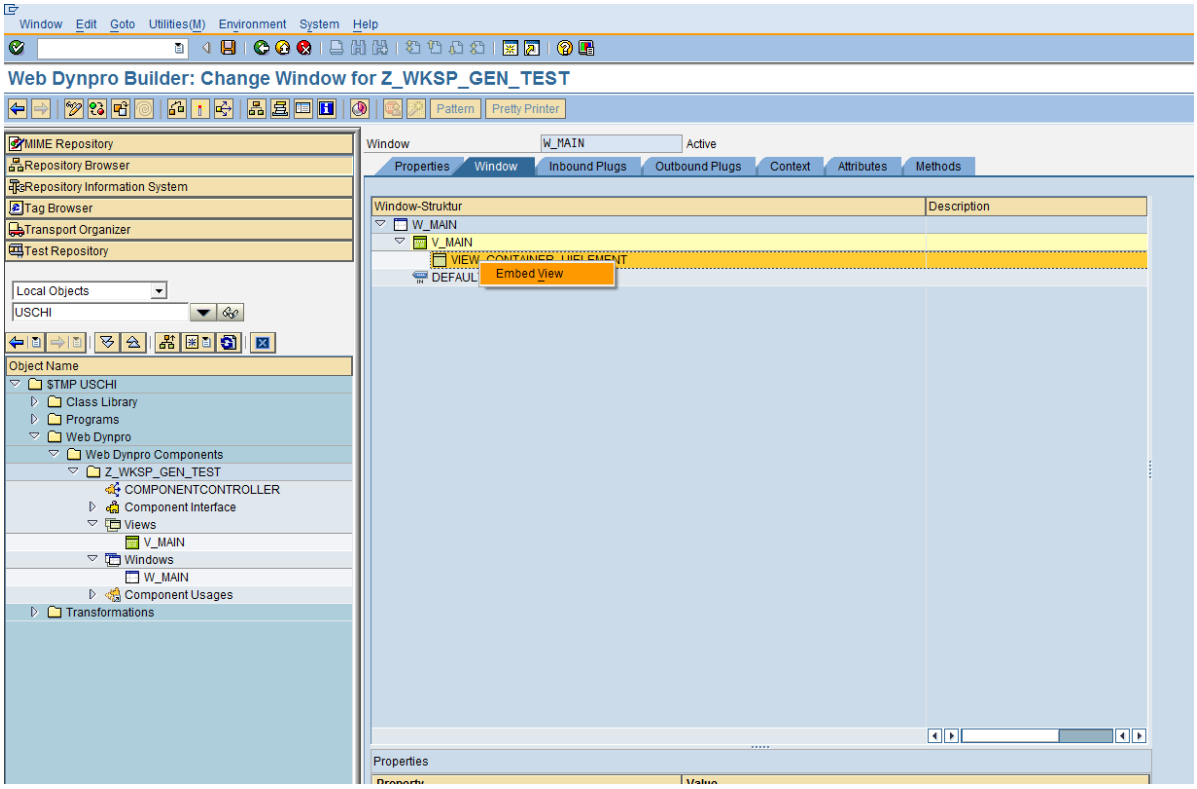


Figure 32: Add generic Web dynpro view (1)

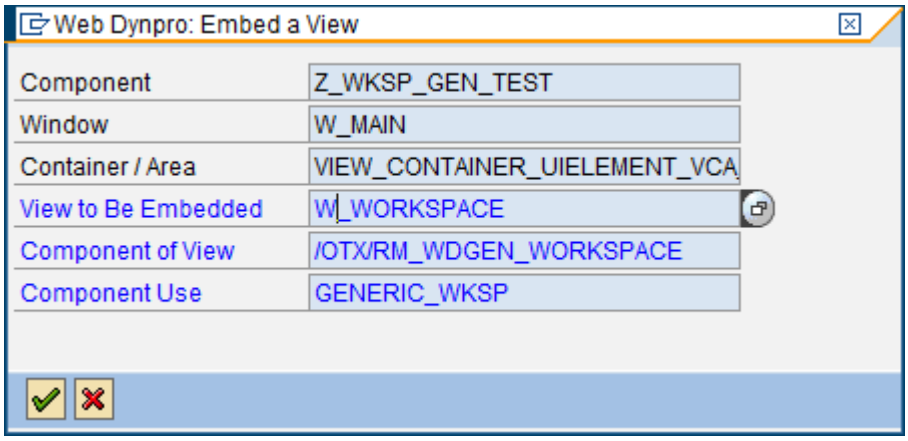


Figure 33: Add generic Web Dynpro view (2)

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Result:

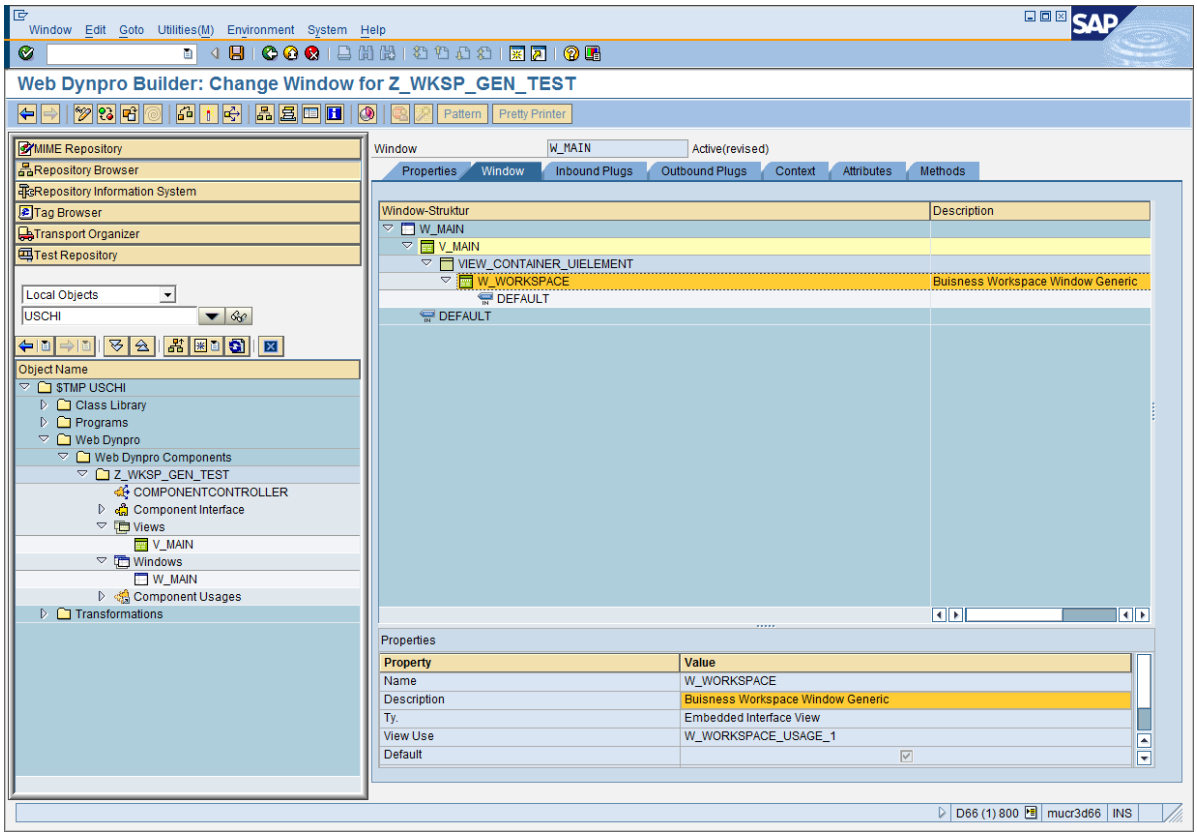


Figure 34: Add generic Web Dynpro view (3)

- Add SAP system specific interfaces to your Web Dynpro.
  Here we added SRM-specific interfaces as an example:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

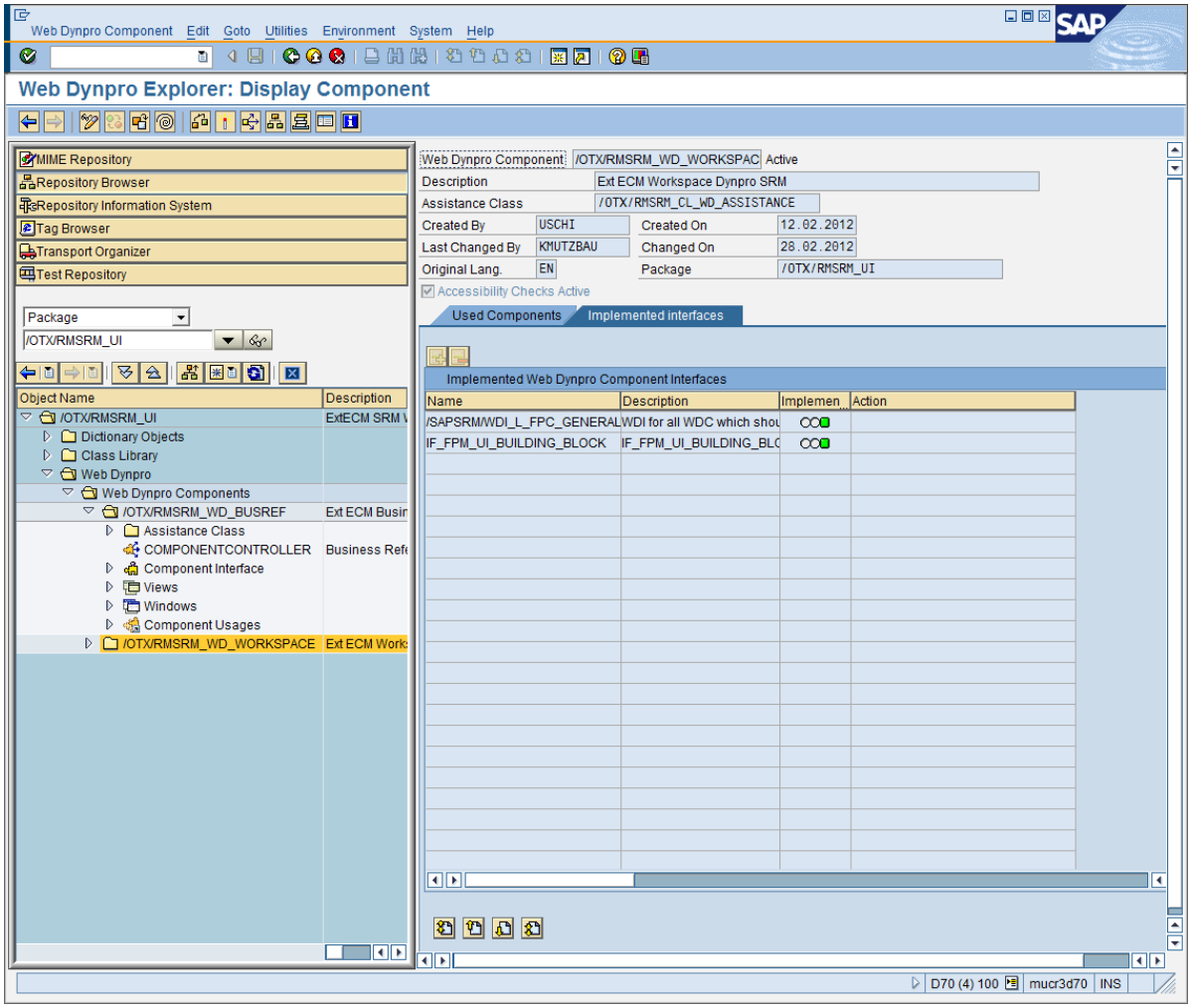


Figure 35: Add system specific interfaces

- Implement the controller functions (e.g., wddoinit() etc.) in your Web Dynpro.
  Example for wddoinit() for an SRM system:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
  DATA:
    lo_interface_generic   TYPE REF TO /otx/iwci_rm_wdgen_workspace,
    lo_interfacecontroller TYPE REF TO if_wd_component_usage,
    lh_task_factory        TYPE REF TO
/sapsrm/if_cll_taskcon_factory,
    l_bo_type              TYPE crmt_subobject_category_db,
    l_bo_guid              TYPE bbp_guid,
    l_bor_type             TYPE swo_objtyp,
    l_bor_id               TYPE swo_objid,
    l_is_key_initial       TYPE wdy_boolean,
    l_theme_id             TYPE string,
    l_message              TYPE string.

  " Create used component
  lo_interfacecontroller = wd_this->wd_cpuse_generic_wksp( ).
  IF lo_interfacecontroller->has_active_component( ) IS INITIAL.
    lo_interfacecontroller->create_component( ).
  ENDIF.

  " Get task container to determine active BO type and ID
  " Generic approach
  lh_task_factory = /sapsrm/cl_ch_wd_taskcont_fact=>get_instance( ).
  wd_this->mh_task_container = lh_task_factory->get_task_container(
).
  l_bo_type = wd_this->mh_task_container->get_bo_type( ).
  l_bo_guid = wd_this->mh_task_container->get_bo_guid( ).

  IF l_bo_guid IS NOT INITIAL.
    TRY.
        CALL METHOD /otx/rmsrm_cl_wd_assistance=>get_bor_key_from_bo
          EXPORTING
            pi_bo_type  = l_bo_type
            pi_bo_id    = l_bo_guid
          IMPORTING
            pe_bor_type = l_bor_type
            pe_bor_id   = l_bor_id.
      CATCH /otx/cx_rmsrm.
        l_message = wd_assistif_wd_component_assistance~get_text( key
= '001.
        report_error_message( l_message ).
    ENDTRY.
  ENDIF.

  IF l_bor_id IS INITIAL.
    l_is_key_initial = abap_true.
  ELSE.
    l_is_key_initial = abap_false.
  ENDIF.

  " Get theme ID
  l_theme_id = get_theme_id( ).

  " Get business object
  lo_interface_generic = wd_this->wd_cpifc_generic_wksp( ).
  lo_interface_generic->set_business_object(
```

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
      pi_objid              = l_bor_id
      pi_objtyp             = l_bor_type
      pi_is_key_initial     = l_is_key_initial
      pi_cc_view            = 'FB'
*      pi_theme_id           = l_theme_id
*        pi_is_key_initial_msg = l_is_key_illegal_msg
  ).
```

6.1.6 Additional context-menu options in Business Content Window

There are two possibilities to add items to the Business Content Window context menu. First, you might want to show items which are already available at the Content Server context menu but not appearing in the Business Content context menu. Secondly, you might want to add functionality which is applicable at the SAP side only.

For the first approach there is the “Maintain Business Content Window menu” IMG entry. With the following entries you can manage to show or remove items which are present in the Content Server context menu.

- Code ID: Specifies the web node command name coming from the Content Server context menu.
- Add/Remove: Specifies whether the item should be removed or added to the existing Business Content Window context menu.
- Description: Describes the context-menu entry. This text is shown at the content menu. This field supports translation. If there is no text provided, the Code Id is shown instead.



Figure 36: Change View Maintain Business Content Window Menu

The other possibility is to create menu items which are available only on SAP side. To do so you need to implement the /OTX/RM_GOS_CTX_MNU BAdI. This BAdI has two methods. One to add menu items to the context menu (PROCESS_CTX_MENU_CONSTRUCTION) and another one to react to the triggered function codes (PROCESS_WKSP_FCODE). For further reference please look at Sample Code provided with the BAdI definition.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

SE18 → BAdI Name → /OTX/RM_GOS_CTX_MNU

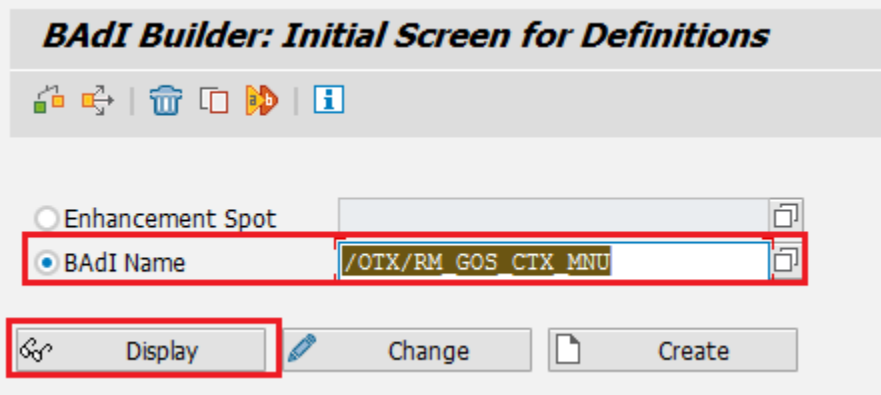


Figure 37: /OTX/RM_GOS_CTX_MNU BAdI Definition

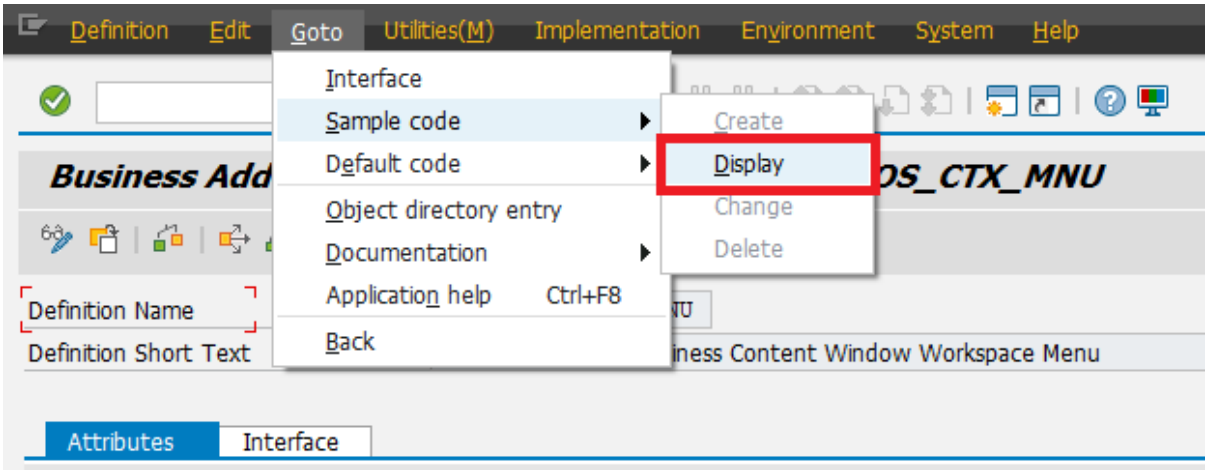


Figure 38: Display Sample Code for /OTX/RM_GOS_CTX_MNU BAdI

Click on “Display” and then navigate to the menu Goto→Sample Code→Display and Click on it. Now you can see the sample code for this BAdI.

6.2 Enhancements on Content Server

6.2.1 Look & feel sidebar

The look & feel of the sidebar is defined by CSS (Cascading Style Sheets) definitions in the file sidebar.css in the support directory of otsapxecm. This section explains how to change the look & feel of the sidebar for specific

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

reference types or for all reference types with a custom module. In this way the changes are pertained when upgrading the standard software.

6.2.1.1 Reference type specific changes

The first example shows how to change the look & feel of the sidebar for a specific reference type. This is done by defining your own widget as described in the previous section. In the example module this is implemented by the AddCSS widget.

The specific changes made are:

1. DisplayWidget: This function always returns true, because no specific configuration is required.
2. The WebLingo file addcss.html is specified as fHTMLContent. The content of the file is:
   <LINK REL="stylesheet" TYPE="text/css"
   HREF="`.IMG()`otsapxecmsample/addcss.css">

When the Custom CSS widget is enabled the background color of the widget is changed. See below for the differences.

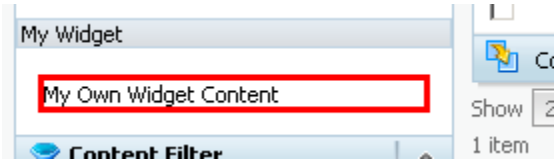


Figure 39: Original display

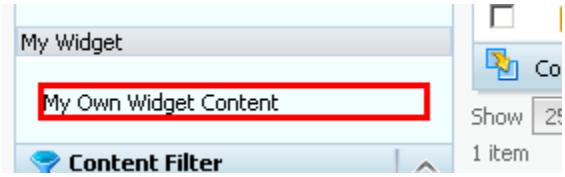


Figure 40: Display with new CSS

6.2.1.2 Global changes

The previous changes can be made global by setting the feature fHidden to TRUE.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

6.2.2 Callbacks on Content Server

For the Business Workspaces no new callbacks are implemented. The standard callbacks provided by Content Server can be leveraged to implement customer specific actions. In the table below an overview is given which standard callbacks can be used to implement similar functionality as the Business Attachment callbacks. The standard Content Server callbacks are implemented by orphaning LLIAPI.LLIApi Root.NodeCallBacks and implementing the specific callbacks. See the documentation feature for details.

Business Attachment callback	Standard Content Server callback
CBCreateBusinessReferencePre	CBCreate, CBCategoriesUpdatePre
CBCreateBusinessReferencePost	CBCreate, CBCategoriesUpdatePost
CBUpdateBusinessReferencePre	CBUpdate, CBCategoriesUpdatePre
CBUpdateBusinessReferencePost	CBUpdate, CBCategoriesUpdatePost
CBDeleteBusinessReferencePre	CBDeletePre, CBDelete, CBMovePre, CBMove
CBDeleteBusinessReferencePost	CBDelete, CBMovePost, CBMove

The CBCategories* callbacks are listed to handle attribute updates from SAP. The CBMove* callbacks are listed because a deletion can also be a move to the recycle bin.

The following scheme shows the call flow when manually creating a business workspace. The essential part is inside the NodeCreate function and is the same for all ways (manual, automatic, LAPI) a Business Workspace is created.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

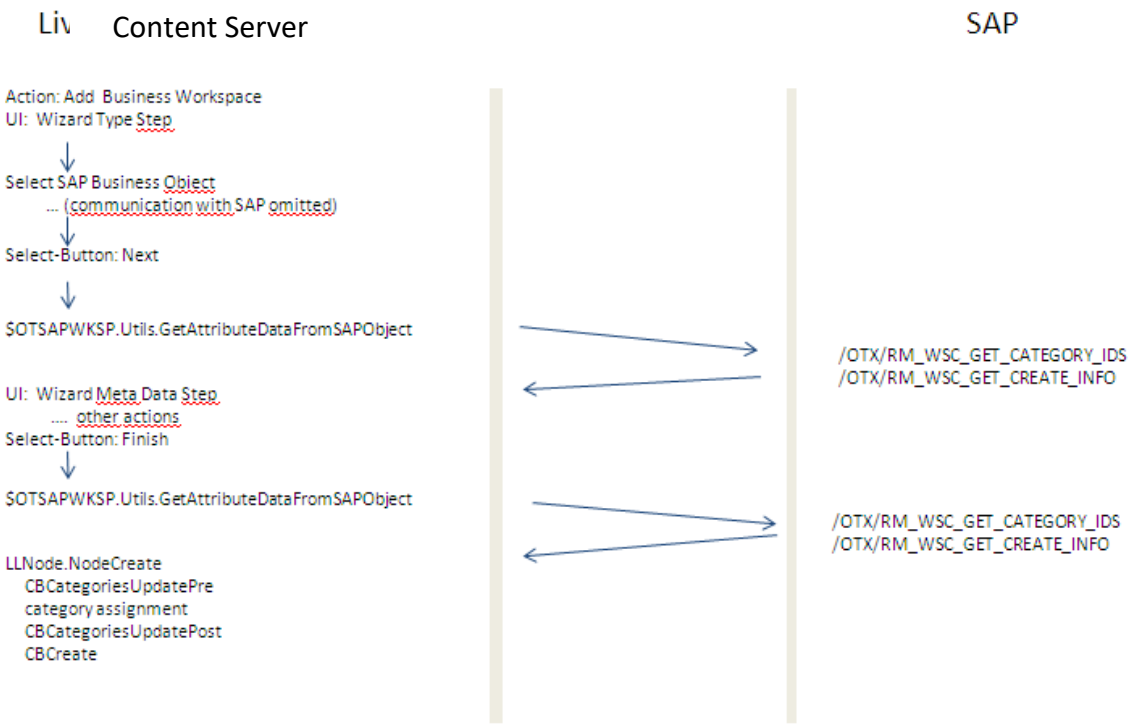


Figure 41: Code flow for manual creation of business workspaces

6.2.3 Configuration API on Content Server

You can extend the configuration for:

Business object types

- Mapping of business properties to category attributes
- Workspace template

Workspace types

- Workspace location
- Workspace sub-location path
- Workspace classification

As an administrator you typically configure those parameters in two steps:

- First you select a configuration method.
- Second you specify the configuration parameters(s) via input control.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Depending on the chosen configuration the input control changes.

Example: Workspace location

Open the Workspace Type configuration page and locate the Workspace Creation Settings section.

In a first step you can choose “From Content Server Item”:



Figure 42: Configuration methods for location

In a second step you browse the Enterprise Volume and select a root folder:



Figure 43: Location configured via Content Server item

6.2.3.1 How to extend the configuration - Overview

The configuration delivered with Business Workspaces 16 already provides the following configuration methods you may derive your own class from:

Select a Content Server item

Usage: workspace location, workspace classification, workspace template.

Select a Content Server category attribute

Usage: workspace location, workspace classification, property mapping.

Select a Content Server category set

Usage: property group mapping.

Specify a business property

Usage: workspace template.

Specify a pattern

Usage: location sub path.

Bound to each configuration method is one the configuration controls you may reuse for your purpose:

Content Server node selector using target browse

Web control: otsapxecm_wsconfig_browsenode_control.html

You can specify e.g., to which subtypes the selection is restricted and where to start browsing.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Content Server category attribute selector

Web Control: otsapxecm_wsconfig_browseattribute_control.html

Consists of a CS node selector (see above) and a combo box to select the attribute.

Content Server category set selector

Web control: otsapxecm_wsconfig_browseset_control.html

Consists of a CS node selector (see above) and a combo box to select the set.

Simple text edit control

Web control: otsapxecm_wsconfig_editfield_control.html

Multiple fields' text edit control

Web control: otsapxecm_wsconfig_multipleeditfield_control.html

When you need to extend the configuration there are two steps to do:

- To implement the configuration method, create a new OScript class derived from *$OTSAPWKSP.ConfigurationCallback* or derive from one of the classes listed above.
- To implement an input control, create a new web script with a defined list of input parameters.

The second part is optional because:

- Typically, you will reuse an existing control.
- In special cases you may not need a control at all if your configuration method doesn't require any input parameter.

6.2.3.2 Implement a configuration method without UI

Create a new OScript class derived from $OTSAPWKSP.ConfigurationCallback and implement the below features. You will find detailed information in the method comments header. Those details are important for implementing, but not included in this documentation.

- *GetCBTypes*:
  Specify for which configuration setting(s) the configuration method should apply.
- *fDisplayName:*
  Specify a display name which will be shown in the method selector of the UI.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- *fId:*
  Specify a unique ID for the configuration method. Choose the name carefully as this ID will be stored in the configuration DB when saving a workspace type or business object type configuration.

- *Execute:*
  Specify how the configuration setting will be determined from the incoming business properties when a workspace is created. For some settings this method simply returns an integer specifying the CS node ID to use.

Finally set *fEnabled* to true, rebuild your OSpace, and restart Content Server (or Builder), otherwise your plug-in will not become active.

6.2.3.3 Implement a configuration method with UI, reusing an existing input control

Perform the additional steps listed below. It is recommended to look at one of the existing configuration methods first to better understand the implementation details.

- *fControlName:*
  Specify the name of the control as html page; choose one of the controls mentioned above.

- *GetControlParameters:*
  Check that all required parameters are correctly passed when the control is instantiated. Refer to the description of the corresponding control. Most parameters have default values.

- If you use a control that supports target browse, override also the methods *_GetBrowseNodeTypes*, *_GetBrowseRootNode*, and *_GetBrowseTitle*.

- *fHelpUrl:*
  Optionally specify a link to a help page if you want to provide editing quick help.

6.2.3.4 Implementing a new configuration input control

Implementing a new configuration control is an advanced task requiring deeper knowledge of OScript, HTML, and Weblingo. To achive this create a new web script file providing the following input parameters:

- Dynamic prgCtx

- *String name:*
  The name of the control which is generated by the calling framework. The framework uses this name to hide or show the control depending

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

on user selection. Make sure that your edit control's name and ID is set to this parameter. If your control contains multiple HTML controls enclose them into a div tag with this name.

- *Boolean visible:*
  Initial visible/invisible state. Typically, you will set display:none for your HTML control or DIV element if initially invisible.

- *String value:*
  Initial control value as string. You must define your own logic how complex user input is mapped to a string and the reverse mapping too.

- *String resultFieldName:*
  Name of the hidden field where the user input must be stored. User input must be stored as string. You must use the same logic to create this string as described above. You should write the hidden field whenever input is changed; otherwise, you cannot be sure that the input is correctly saved. Work with the appropriate HTML control events and provide JavaScript functions.

- *Assoc config:*
  Additional control configuration data as passed by method *ConfigurationCallback:SetControlParameters*. You can e.g., specify here parameters controlling target browse behavior.

6.2.3.5 Example: Create a technical folder structure for workspaces

This chapter explains step-by-step how you can provide a workspace type configuration, which stores newly created workspaces in a technical folder structure, using folders and sub folders named by year, month, day, etc.

The configuration method will be integrated into the sub-location path setting and provide an input control where you can specify the details of the folder structure.

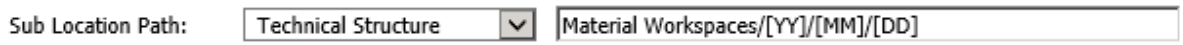


Figure 44: Custom selection method for sub-location path

Step 1:

Start Content Server Builder and create a new OSpace. Save it to folder *c:\opentext\module\xecmextensions_1_0_0\ospace* and name it *TechnicalStructure.oll*.

Step 2:

Create an INI file named *xecmextensions.ini* for your OScript module and store it in folder *c:\opentext\module\xecmextensions_1_0_0*. It should contain the following content:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
[dependencies]
requires_1={'otsapxecm',10,0}

[description]
moduleName=xecmextensions
Name=ECMLink Extensions
Version=1.0 r 0

[OSpaces]
ospace_1=technicalstructure
```

Step 3:

Create a new entry for your new module in opentext.ini, so the OSpace will be loaded automatically in the future, e.g.:

```
[Modules]
…
module_79=xecmextensions
ospaces_xecmextensions={'technicalstructure'}
xecmextensions=_1_0_0
```

Step 4:

Open Content Server Builder again, unlock the technicalstructure OSpace, navigate to $OTSAPWKSP.ConfigurationCallback and create a new orphan of that class.

Step 5:

Override the following features as given below:

fId: The unique ID that will be stored in the configuration DB.

```
TechnicalStructure
```

fDisplayName: The name of the configuration method shown in the UI.

```
Technical Structure
```

fControlName: The name of the editing control – reuse the simple edit control.

```
otsapxecm_wsconfig_editfield_control.html
```

GetCBUsages: Which setting will be configured – use sub-location path.

```
Function List GetCBUsages()
    return { $OTSAPXECM.fCBUsageLocationPath }
End
```

Execute: How the sub-location path must be calculated from the configured pattern.

```
Function Dynamic Execute( Object prgCtx, String pattern, String
langCode, String boType, String boId, List boProperties, List
categories )
       String retVal = pattern
       Date now = Date.Now()
       List replacements = { \
```

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
                { "[YYYY]", Str.ValueToString( Date.Year(now) ) },
\
                { "[YY]",   Str.ValueToString( Date.Year(now)
)[3:4] }, \
                { "[MM]",   ConvertAndFillTrailingZeros(
Date.Month(now), 2 ) }, \
                { "[DD]",   ConvertAndFillTrailingZeros(
Date.Day(now), 2 ) },\
                { "[hh]",   ConvertAndFillTrailingZeros(
Date.Hour(now), 2 ) }, \
                { "[mm]",   ConvertAndFillTrailingZeros(
Date.Minute(now), 2 ) }, \
                { "[ss]",   ConvertAndFillTrailingZeros(
Date.Second(now), 2 ) } \
        }
        List replacement
        for replacement in replacements
                retVal = Str.ReplaceAll( retVal, replacement[1],
replacement[2] )
        end
        return retVal
End

Function String ConvertAndFillTrailingZeros( Integer i, Integer
num )
        String str = Str.ValueToString( i )
        while Length(str) < num
                str = "0" + str
        end
        return str
End
```

fEnabled

```
        true
```

Final step:

Do not forget to rebuild your OSpace.

If everything was done correctly, the new configuration setting will be available upon next Content Server restart.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

7 Enhancements in business scenarios

7.1 Enhancements on SAP

7.1.1 Introduction

The business scenarios of OpenText Content Management for SAP Solutions (ABAP Add-On OTEXRLSA) deliver ready-to-use business object property providers.

The following sections give advice if you need to make enhancements to meet your specific business requirements.

7.1.2 General approach

The business object property provider classes of the business scenarios are sub classes inherited from the generic property provider class /OTX/RM_GEN_PP_CL_BO.

If you want to change the behavior for a specific SAP business object type do the following.

1. Create a new object class in your customer namespace.
2. In the properties of the new object class define the original business object property provider class as superclass.
3. Redefine methods as suggested by the descriptions of the following chapters.
4. Additional artifacts (attributes, methods) may be added to the new object class. See more guidelines in the following chapters.
5. Create append structures to extend existing structures with new components. See more guidelines in the following chapters.
6. Replace the original business object property provider class in the business object declaration customizing.

7.1.2.1 General guidelines for append structures

The original business object property provider class uses the structure defined in public method GET_STRUCT_DATA. Each component of that structure is returned by the property provider as a property.
You can add additional components to such a structure with the function append structure in ABAP data dictionary. You also can append new components to the structures of table types included in the original main

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

structure. Learn more about the DDIC structure and behavior in **4.3 BUSINESS OBJECT PROPERTY PROVIDERS**.

Furthermore, follow these guidelines.

- Use components names which start with Y, Z, or your own customer namespace. **This way naming conflicts are avoided.**
- Components which are table types or structures are allowed to be appended to the main structure of the property provider (as returned by method GET_STRUCT_NAME).
    - The structure of the table type must not contain components which are structures or table types
    - Names of the components inside the structure of the table type do not need to start with Y or Z.
    - If the structure of the table type is a structure defined by SAP or Open Text and you append to that structure, then use Y,Z or customer namespace for the names
- In the case you want to append to a structure which has enhancement category unclassified or does not allow the type of components you want to add, please contact OpenText customer support.

7.1.2.2 General guidelines for inherited property provider classes

You can inherit the original property provider classes of the business scenarios to your own custom sub classes.

Learn more about the basic behavior in **4.3 BUSINESS OBJECT PROPERTY PROVIDERS**.
In addition, follow these guidelines for property provider classes inherited from the property provider classes of business scenarios.

- In a redefined method, call the original method of the same name from the super class by using the pseudo reference super before you implement your own code.
- Use names which start with Y, Z, or your own customer namespace for new artifacts in your new class (like methods and class attributes) to avoid naming conflicts.
- Only redefine public methods, specifically GET_DATA or GET_PROPERTY_DEFS as described in the use case related sections after this chapter.
- Avoid redefinition of protected methods. They are used by other methods and changing their behavior may have wider impact.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

However, exemptions of this rule may still be given by OpenText customer support or development for the use of bug fixes.

7.1.3 Use case: add, change, or remove business workspace attributes

7.1.3.1 Add a new attribute

1. Select the original structure used by the property provider (for example in the object browser, transaction SE80).
2. Use the function to append a structure. The new append structure is created in the customer namespace.
3. Add a new component for each attribute. See also **4.3.1.1 CREATE A DDIC STRUCTURE.** See *Figure 45 Example of Append Structure to /OTX/RLSA_PM_S_BO_EQUIPMENT* below.
4. Create a new object class (transaction SE24) in the customer namespace. Define the original property provider class of the business scenarios as the super class.
5. In the new property provider class redefine method GET_DATA
    a. Implement call of original method of super class
    b. Implement own logic. The new component is a column/component in exporting parameter PE_DATA
    c. See *Code Snippet 1: Example of constant for new attribute name* and *Code Snippet 2: Example of redefined method GET_DATA to add a new attribute* below.
6. Activate the new property provider and use it as replacement of the original property provider class in the customizing of the business object declaration.
7. The new component of your append structure is usable on the Content Server as a new property which can be mapped to category attributes on the OpenText Content Server.

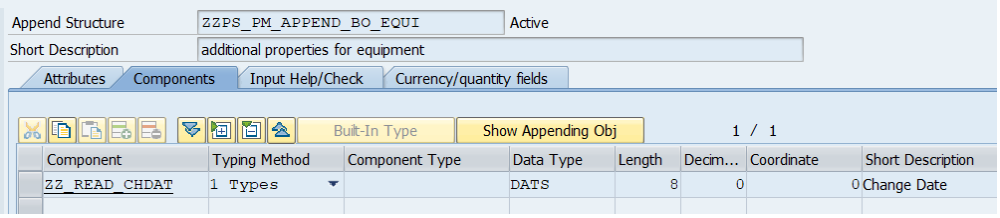


Figure 45 Example of Append Structure to /OTX/RLSA_PM_S_BO_EQUIPMENT

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
public section.

  constants ZC_ATTR_NAME_ZZ_READ_CHDAT type STRING value 'ZZ_READ_CHDAT' ##NO_TEXT.
```

Code Snippet 1: Example of constant for new attribute name

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
  METHOD get_data.

* Example: to add new attribute (property) ZZ_READ_CHDAT for date of last change
    DATA: l_equipment      TYPE equnr,
          l_message        TYPE string,
          ls_bapi_general  TYPE bapi_itob,
          ls_bapi_return   TYPE bapiret2,
          l_bapi_error_msg TYPE string.
    FIELD-SYMBOLS: <l_zz_read_chdat> TYPE dats.

    l_equipment = pis_bor-id.

    CALL METHOD super->get_data
      EXPORTING
        pis_bor                         = pis_bor
        pi_struct_name                  = pi_struct_name
      IMPORTING
        pet_composite_children          = pet_composite_children
        pes_composite_parent            = pes_composite_parent
        pet_roles_users                 = pet_roles_users
        pet_children_ext                = pet_children_ext
        pet_parents_ext                 = pet_parents_ext
        pet_identical_business_objects = pet_identical_business_objects
        pe_data                         = pe_data
        pet_parents                     = pet_parents
        pet_children                    = pet_children
        peh_exception                   = peh_exception.

* BAPI Equipment
    CALL FUNCTION 'BAPI_EQUI_GETDETAIL'
      EXPORTING
        equipment        = l_equipment
      IMPORTING
        data_general_exp = ls_bapi_general
        return           = ls_bapi_return.

    IF ls_bapi_return-type = c_bapi_err.
      l_bapi_error_msg = ls_bapi_return-message.
      RAISE EXCEPTION TYPE /otx/cx_rm_wsci_wsprovider
        EXPORTING
          m_error_text = l_bapi_error_msg.
    ENDIF.

    ASSIGN COMPONENT me-
>zc_attr_name_zz_read_chdat OF STRUCTURE pe_data TO <l_zz_read_chdat>.
    IF sy-subrc EQ 0.
      <l_zz_read_chdat> = ls_bapi_general-read_chdat.
    ENDIF.

  ENDMETHOD.
```

Code Snippet 2: Example of redefined method GET_DATA to add a new attribute

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

7.1.3.2 Change an attribute

If you want to completely change an attribute (not only the value, but also the type), just add a new component (see previous chapter) and map it to the respective category attribute on the OpenText Content Server.

Here are the steps to change the value of an existing attribute.

1. On the OpenText Content Server the category attribute is mapped to a property.
2. The property is corresponding to a component in the main structure used by the property provider.
3. Create a new object class (transaction SE24) in the customer namespace. Define the original property provider class of the business scenarios as the super class.
4. In the new property provider class redefine method GET_DATA
    a. Implement call of original method of super class
    b. Implement own logic. The component is a column/component in exporting parameter PE_DATA.
    c. See *Code Snippet 3: Example of constant for existing attribute name* and *Code Snippet 4: Example of redefined method GET_DATA to change value of existing attribute EQUICATGRY_TXT* below.
5. Activate the new property provider and use it as replacement of the original property provider class in the customizing of the business object declaration.

```
public section.

  constants ZC_ATTR_NAME_EQUICATGRY_TXT type STRING value 'EQUICATGRY_TXT' ##NO_TEX
T.
```

Code Snippet 3: Example of constant for existing attribute name

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
  METHOD get_data.

* Example: change value of attribute (property) EQUICATGRY_TXT
* if default value is not English language value,
* then add English value enclosed by round brackets
    FIELD-SYMBOLS: <l_equicatgry_txt> TYPE /otx/rm_gen_pp_s_lang_value,
                   <ls_lang_values>   TYPE /otx/rm_wsci_s_lang_string.

    CALL METHOD super->get_data
      EXPORTING
        pis_bor                         = pis_bor
        pi_struct_name                  = pi_struct_name
      IMPORTING
        pet_composite_children          = pet_composite_children
        pes_composite_parent            = pes_composite_parent
        pet_roles_users                 = pet_roles_users
        pet_children_ext                = pet_children_ext
        pet_parents_ext                 = pet_parents_ext
        pet_identical_business_objects = pet_identical_business_objects
        pe_data                         = pe_data
        pet_parents                     = pet_parents
        pet_children                    = pet_children
        peh_exception                   = peh_exception.

    ASSIGN COMPONENT me-
>zc_attr_name_equicatgry_txt OF STRUCTURE pe_data TO <l_equicatgry_txt>.
    IF sy-subrc EQ 0.
      READ TABLE <l_equicatgry_txt>-
lang_values WITH KEY lang = 'en' ASSIGNING <ls_lang_values>.
      IF sy-subrc EQ 0 AND
         <ls_lang_values>-value IS NOT INITIAL AND
         <ls_lang_values>-value NE <l_equicatgry_txt>-value.
        CONCATENATE <l_equicatgry_txt>-value '(English:' <ls_lang_values>-value ')'
         INTO <l_equicatgry_txt>-value.
      ENDIF.
    ENDIF.

  ENDMETHOD.
```

Code Snippet 4: Example of redefined method GET_DATA to change value of existing attribute EQUICATGRY_TXT

7.1.3.3 Remove attribute

If you do not need an attribute, first remove the property mapping and/or attribute on the side of the Content Server.

The benefit to remove the corresponding property on side of SAP may be to reduce traffic between SAP and Content Server.

Here are the steps to remove properties on ABAP side.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

1. Make sure, that on the OpenText Content Server no category attribute is mapped to the property.
2. Create a new object class (transaction SE24) in the customer namespace. Define the original property provider class of the business scenarios as the super class.
3. In the new property provider class redefine method GET_PROPERTY_DEFS
    a. Implement call of original method of super class
    b. Remove the row of the attribute from the exporting parameter PET_ATTR_DEFS.
    c. See *Code Snippet 5: Example of constant for attribute to remove* and *Code Snippet 6: Example of redefined method GET_PROPERTY_DEFS to remove a property.*
4. Activate the new property provider and use it as replacement of the original property provider class in the customizing of the business object declaration.

```
public section.


  constants ZC_ATTR_NAME_PLANPLANT type STRING value 'PLANPLANT' ##NO_TEXT.
```

Code Snippet 5: Example of constant for attribute to remove

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
  METHOD get_property_defs.
* Example to remove property PLANPLANT
    DATA: lt_attr_defs TYPE /otx/rm_wsci_tt_attr_def,
          ls_attr_defs TYPE /otx/rm_wsci_s_attr_def.

    CLEAR: pet_attr_defs[].
    CALL METHOD super->get_property_defs
      EXPORTING
        pi_disp_lang                  = pi_disp_lang
        pi_bo_type                    = pi_bo_type
      IMPORTING
        pet_attr_defs                 = lt_attr_defs
        pet_allowed_parent_types      = pet_allowed_parent_types
        pet_allowed_child_types       = pet_allowed_child_types
        pet_allowed_ident_busobj_types = pet_allowed_ident_busobj_types
        pet_allowed_parent_types_ext  = pet_allowed_parent_types_ext
        pet_allowed_child_types_ext   = pet_allowed_child_types_ext.

    LOOP AT lt_attr_defs INTO ls_attr_defs.
      IF ls_attr_defs-key NE me->zc_attr_name_planplant.
        APPEND ls_attr_defs TO pet_attr_defs.
      ENDIF.
    ENDLOOP.
  ENDMETHOD.
```

Code Snippet 6: Example of redefined method GET_PROPERTY_DEFS to remove a property

7.1.4 Use case: add or remove business workspace relationships

Please read chapter *4.3 Business object property providers* about definition of business workspace relations. Specifically, see the descriptions of the following chapters for more details. *4.3.1.4 Redefine method GET_PROPERTY_DEFS (optional)*, *4.3.2.3 Relations*, and *4.3.2.4 Cross-application business workspace*.

7.1.4.1 Add new business workspace relations

1. Create a new object class (transaction SE24) in the customer namespace. Define the original property provider class of the business scenarios as the super class.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

2. Redefine method GET_PROPERTY_DEFS in the new property provider class.

    a. Fill the exporting parameters PET_ALLOWED_IDENT_BUSOBJ_TYPES, PET_ALLOWED_PARENT_TYPES_EXT and PET_ALLOWED_CHILD_TYPES_EXT with the business object types according to your needs. You can leave the value for EXTERNAL_SYSTEM_ID empty if the relation is on business workspaces of the same system.

3. In the new property provider class redefine method GET_DATA.

    a. Append new entries to exporting parameters PET_IDENTICAL_BUSINESS_OBJECTS, PET_CHILDREN_EXT, and PET_PARENTS_EXT according to your needs. You can leave the value for EXTERNAL_SYSTEM_ID empty if the relation is between business workspaces of the same system.

4. Activate the new property provider and use it as replacement of the original property provider class in the customizing of the business object declaration.

Even if the original property provider class populates PET_ALLOWED_PARENT_TYPES, PET_ALLOWED_CHILD_TYPES or PET_PARENTS or PET_CHILDREN, please use the “_EXT” parameters for additional relations.

7.1.4.2 Remove business workspace relations

1. Create a new object class (transaction SE24) in the customer namespace. Define the original property provider class of the business scenarios as the super class.

2. If you want to specifically disallow a business object type in a relation, then redefine method GET_PROPERTY_DEFS. Implement code which removes entries from PET_ALLOWED_IDENT_BUSOBJ_TYPES, PET_ALLOWED_PARENT_TYPES_EXT, PET_ALLOWED_CHILD_TYPES_EXT, PET_ALLOWED_PARENT_TYPES, and PET_CHILDREN

3. Redefine method GET_DATA in the new property provider class. Implement code which removes entries from exporting parameters PET_IDENTICAL_BUSINESS_OBJECTS, PET_CHILDREN_EXT, PET_PARENTS_EXT, PET_PARENTS, and PET_CHILDREN.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

4. Activate the new property provider and use it as replacement of the original property provider class in the customizing of the business object declaration.

7.1.5 Use case: add or remove role members

Read chapter *4.3.2.2 Users/Groups and roles* for details.

7.1.5.1 Add role members

1. Create a new object class (transaction SE24) in the customer namespace. Define the original property provider class of the business scenarios as the super class.
2. Redefine method GET_DATA in the new property provider class. Implement code which adds a new entry to exporting parameter PET_ROLES_USERS.
3. Activate the new property provider and use it as replacement of the original property provider class in the customizing of the business object declaration.

7.1.5.2 Remove role members

1. Create a new object class (transaction SE24) in the customer namespace. Define the original property provider class of the business scenarios as the super class.
2. Redefine method GET_DATA in the new property provider class. Implement code which removes unwanted entries from exporting parameter PET_ROLES_USERS.
3. Activate the new property provider and use it as replacement of the original property provider class in the customizing of the business object declaration.

7.2 Enhancements on SAP for Enterprise Asset Management

This chapter presents additional hints and code samples for the Enterprise Asset Management Business Scenario.

7.2.1 Filling up roles for Team widget

The Enterprise Asset Management Business Scenario Team widget shows Content Server Team roles for Maintenance Technician, Maintenance Planner, and Reporter. See the chapter View and manage team members in the

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Business Scenarios Business Administration Guide (EEBS-UGD) for more information.

To fill up the Enterprise Asset Management team roles, a custom property provider must be implemented and customized. The property provider implementations presented here will map the following:

- Content server group `OT Maintenance Planner` to role `Maintenance Planner`
- Content server group `OT Maintenance Technician` to role `Maintenance Technician`
- Only Maintenance Notification: user that has created the maintenance notification to role `Reporter`

To implement and customize custom property provider:

Business Object Type	Delivered Property Class ➔ Example Name of Sub Class
Equipment	/OTX/RLSA_PM_CL_PP_EQUIPMENT ➔ Z_RLSA_PM_CL_PP_EQUIPMENT
Functional Location	/OTX/RLSA_PM_CL_PP_FUNC_LOC ➔ Z_RLSA_PM_CL_PP_FUNC_LOC
Maintenance Order	/OTX/RLSA_PM_CL_PP_ORDER ➔ Z_RLSA_PM_CL_PP_ORDER.
Maintenance Order Operation	/OTX/RLSA_PM_CL_PP_ORDER_OP ➔ Z_RLSA_PM_CL_PP_ORDER_OP
Task List	/OTX/RLSA_PM_CL_PP_TASK_LST ➔ Z_RLSA_PM_CL_PP_TASK_LST
Task List Operation	/OTX/RLSA_PM_CL_PP_TASK_LST_OP ➔ Z_RLSA_PM_CL_PP_TASK_LST_OP
Maintenance Notification	/OTX/RLSA_PM_CL_PP_NOTIFICAT ➔ Z_RLSA_PM_CL_PP_NOTIFICAT

Table 5 Enterprise Asset Management Property Provider Classes

1. Create a subclass in customer namespace (Z, Y) for each of the property provider classes listed in Table 5 Enterprise Asset Management Property Provider Classes.
2. For **Equipment, Functional Location, Maintenance Order, Maintenance Order Operation, Task List, or Task List Operation**: Redefine the `DETERMINE_ROLES_USERS` method and implement it with **CODE SNIPPET 7 METHOD DETERMINE_ROLES_USERS FOR ENTERPRISE ASSET MANAGEMENT**.
3. For **Maintenance Notification**: Redefine the `DETERMINE_ROLES_USERS` method and implement it with **CODE SNIPPET 8 METHOD DETERMINE_ROLES_USERS FOR MAINTENANCE NOTIFICATION**.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

4. Configure, the sub class (for example: Z_RLSA_PM_CL_PP_EQUIPMENT) as property provider for the respective business object type (for example: Equipment).

Code Snippets:

```
METHOD determine_roles_users.

  DATA: ls_role_users TYPE /otx/rm_wsci_s_role_users.

  ls_role_users-role = 'Maintenance Planner'.
  APPEND 'OT Maintenance Planner' TO ls_role_users-users.
  APPEND ls_role_users TO pet_roles_users.

  CLEAR ls_role_users.
  ls_role_users-role = 'Maintenance Technician'.
  APPEND 'OT Maintenance Technician' TO ls_role_users-users.
  APPEND ls_role_users TO pet_roles_users.

ENDMETHOD.
```

Code Snippet 7 Method DETERMINE_ROLES_USERS for Enterprise Asset Management

```
METHOD determine_roles_users.

  DATA: ls_role_users TYPE /otx/rm_wsci_s_role_users.

  ls_role_users-role = 'Maintenance Planner'.
  APPEND 'OT Maintenance Planner' TO ls_role_users-users.
  APPEND ls_role_users TO pet_roles_users.

  CLEAR ls_role_users.
  ls_role_users-role = 'Maintenance Technician'.
  APPEND 'OT Maintenance Technician' TO ls_role_users-users.
  APPEND ls_role_users TO pet_roles_users.

  CLEAR ls_role_users.
  ls_role_users-role = 'Reporter'.
  APPEND pis_data_external-reportedby TO ls_role_users-users.
  APPEND ls_role_users TO pet_roles_users.

ENDMETHOD.
```

Code Snippet 8 Method DETERMINE_ROLES_USERS for Maintenance Notification

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

7.2.2 Impersonating users outside OpenText Content Management

Generally, OpenText Content Management for SAP Enterprise Asset Management Business Scenario can be used by users without a license for OpenText Content Management. For example, a user might want to add SAP attachments to the SAP Fiori App Request Maintenance (App ID: F1511) that also shall be declared as records in Content Server.

See sample code in **CODE SNIPPET 9 SAMPLE CODE FOR IMPERSONATION OF A USER ASSIGNED TO A GROUP**.

- If the user is assigned to a a special group like SAP_BR_EMPLOYEE_MAINTENANCE you can impersonate the user to a technical user. For more information, see *BAdI-based impersonation*.
- All other users are impersonated to their initial name.

```
METHOD /otx/rm_if_impersonate~get_imper_user.
  DATA:
    ls_agr_user TYPE AGR_USERS,
    l_role      TYPE AGR_NAME VALUE 'SAP_BR_EMPLOYEE_MAINTENANCE'.

  SELECT SINGLE * FROM agr_users INTO ls_agr_user WHERE agr_name = l_role AND uname = sy-uname.
  IF sy-subrc EQ 0. " user belongs to this role
    pe_imper_user_name = 'xecmtech01'.
  ELSE. " all other users are impersonated to their initial name
    pe_imper_user_name = sy-uname.
  ENDIF.

ENDMETHOD.
```

Code Snippet 9 Sample code for impersonation of a user assigned to a group

7.3 Enhancements on SAP for Procurement

This chapter presents additional hints and code samples for the Procurement Business Scenario.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

7.3.1 Enhancing invoice declaration in Business Partner workspace

The Business Scenarios Business Administration Guide (EEBS-UGD) describes the option how to declare invoices in the Business Partner workspace. For this, the property provider class /OTX/RLSA_0_S4_CL_PP_AL_FIACC is delivered. It has the following features:

- It inherits from the generic property provider /OTX/RM_GEN_PP_CL_ALINK_DOC
- The only supported business object type which also must be defined in the declaration is BKPF.
- It does not expose all kinds of properties of table BKPF. It exposes basic ArchiveLink related properties plus the properties defined in method GET_STRUCT_NAME.
- Declarations are only processed if the financial account type (KOART) is K and if a supplier is defined which can be mapped to the business partner using CVI (Customer Vendor Integration)
- Data is retrieved from CDS entities which are based on tables BKPF and BSEG, because they are still populated with relevant data.

If the delivered property provider class does not match your requirements, you have the option to define your own custom property provider.

The following options make sense and have different advantages and disadvantages.

Option 1: Using copy of /OTX/RLSA_0_S4_CL_PP_AL_FIACC

1. Copy property provider class /OTX/RLSA_0_S4_CL_PP_AL_FIACC.
2. Modify and redefine to your needs. But it still has /OTX/RM_GEN_PP_CL_ALINK_DOC as its super class. Follow the recommendations mentioned in the previous chapters of this document.

Main advantage: There are more degrees of freedom even if OpenText changes /OTX/RLSA_0_S4_CL_PP_AL_FIACC.

Main disadvantage: You do not automatically participate in any enhancements done to /OTX/RLSA_0_S4_CL_PP_AL_FIACC in the future.

Option 2: Creating a subclass of /OTX/RLSA_0_S4_CL_PP_AL_FIACC

1. Create a subclass of Property Provider Class /OTX/RLSA_0_S4_CL_PP_AL_FIACC.
2. Note that the subsets of structure /OTX/RLSA_0_S4_S_AL_FIACC may still be subject to change in the future.
3. If you need to enhance with new properties, then here are two possible choices.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

   a. Redefine method CHANGE_ATTR_DEF. Redefine method GET_PROPERTIES which inherits from super class and then modify property values to your needs.
   b. Enhance structure /OTX/RLSA_0_S4_S_AL_FIACC but be careful with names of columns! (For example, use Y or Z as prefixes) and then redefine methods GET_DATA.
4. Follow the recommendations in the previous chapters of this document.

Main advantage: Less effort if you just need smaller modifications.

Main disadvantage: There is a bigger chance of breaking when updating Add-On OTEXRLSA.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

8 Side-by-Side extension: Enabling Custom API to Synchronize SAP roles with CS groups

8.1 Prerequisites

1. You are using SAP S/4HANA (on-premise) or SAP S/4HANA Cloud Private Edition.
2. You are using OpenText Content Server in side-by-side approach, that is you are using connection type S/4HANA SPI Adapter in Business Applications.
3. You want to use the scheduled bot **Synchronize SAP roles with CS groups** of product component **SAP Solutions Integration**.
   For more details, please read the chapter Synchronize SAP roles with CS Groups in a recent version of Smart View User Guide: OpenText Content Management (CSSUI-UGD-EN).
4. Your developer or consultant requires ABAP Development Tools (ADT) on Eclipse. See https://help.sap.com/docs/ABAP_DEVELOPMENT_TOOLS_FOR_ECLIPSE?locale=en-US

8.2 Goal

For SAP S/4HANA Cloud Public Edition for ABAP Platform, SAP has released the API **Business Role -Read** , **Communication Scenario: BusinessRole Read Integration (SAP_COM_0A04)**.

Therefore, the OpenText Content Management for SAP Solutions scheduled bot of type **Synchronize SAP roles with CS groups** has set the **OData V4 Endpoint** default value to **aps_iam_api_brole_read/srvd_a2x/sap/businessrole/0001**.
(Go to https://api.sap.com and search for **SAP_COM_0A04**)

SAP S/4HANA (on-premise) or SAP S/4HANA Cloud Private Edition do not have a corresponding API.

With the steps described in the next chapter you will create and enable a custom API on your SAP system, which returns data corresponding to SAP_COM_0A04 or to be more precise which returns the data required by the scheduled bot.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

8.3 Steps to create custom API (tier 2)

Important: Before creating any artifacts, make sure that the package in which you create all the artifacts has **Standard ABAP Language Version** defined as **Standard ABAP**.

Use ADT (ABAP Development Tools) in Eclipse

- Create 2 CDS data definitions (view entities) to be exposed as entities
- Optional/Recommended: Create 2 CDS access controls for authorization checks
- Create a service definition to expose the 2 view entities as **BusinessRoles** and **BusinessUsers**
- Create a service binding of type OData V4 – Web API and publish a local service endpoint

The code examples are created in a package ZOTCM_CUSTOM_API and start with ZOTCM in the name. Feel free to adapt the name patterns in the Z-Namespace or your own custom namespace.

1. Create package (ZOTCM_CUSTOM_API). Define **Standard ABAP** value in Default ABAP Language Version.
   Create the following artifacts under this package.
2. Create and activate view entity ZOTCM_I_BUSINESS_USERS_R.
   See *Code Snippet 10 View Entity ZOTCM_I_BUSINESS_USERS_R*.
3. Create and activate view entity ZOTCM_I_BUSINESS_ROLES_R.
   See *Code Snippet 11 View Entity ZOTCM_I_BUSINESS_ROLES_R*.
4. Create and activate access control ZOTCM_I_BUSINESS_USERS_R.
   See *Code Snippet 12 Access Control ZOTCM_I_BUSINESS_USERS_R*.
5. Create and activate access control ZOTCM_I_BUSINESS_ROLES_R.
   See *Code Snippet 13 Access Control ZOTCM_I_BUSINESS_ROLES_R*.
6. Create and activate business service definition ZOTCM_BUSINESS_ROLES_R.
   See *Code Snippet 14 Service Definition ZOTCM_BUSINESS_ROLES_R*.
7. Create and publish service binding ZOTCM_BUSINESS_ROLES_R_V4.
   See *Table 6 Details of Service Binding ZOTCM_BUSINESS_ROLES_R_V4*.

The following chapter lists the source codes and the properties of the service binding.

The business service definition (ZOTCM_BUSINESS_ROLES_R) defines the names under which the view entities are exposed. They must be exposed to the names **BusinessRoles** and **BusinessUsers**.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

8.3.1 Source codes and properties

```
@AbapCatalog.viewEnhancementCategory: [#NONE]
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Role members exposed as BusinessUsers'
@Metadata.allowExtensions: true
@Metadata.ignorePropagatedAnnotations: true
@ObjectModel.usageType:{
    serviceQuality: #X,
    sizeCategory: #S,
    dataClass: #ORGANIZATIONAL
}
define view entity ZOTCM_I_BUSINESS_USERS_R
  as select from agr_users
{
      // agr_name has length 30 but here we go for a length like in the public cloud
  key cast(agr_name as abap.char(40)) as BusinessRoleID,
  key uname                           as UserID
}

where
     from_dat <= $session.system_date
  and to_dat   >= $session.system_date
```

Code Snippet 10 View Entity ZOTCM_I_BUSINESS_USERS_R

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
@AbapCatalog.viewEnhancementCategory: [#NONE]
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Changed roles exposed as BusinessRoles'
@Metadata.allowExtensions: true
@Metadata.ignorePropagatedAnnotations: true
@ObjectModel.usageType:{
   serviceQuality: #X,
   sizeCategory: #M,
   dataClass: #ORGANIZATIONAL
}
define root view entity ZOTCM_I_BUSINESS_ROLES_R
  as select from cdhdr
    inner join   cdpos on  cdpos.objectclas = cdhdr.objectclas
                       and cdpos.objectid   = cdhdr.objectid
                       and cdpos.changenr   = cdhdr.changenr
  association of one to many ZOTCM_I_BUSINESS_USERS_R as BusinessUsers on $projection.BusinessRoleID =
BusinessUsers.BusinessRoleID
{
     // agr_name has length 30 but here we go for a length like in the public cloud
  key cast( cdhdr.objectid as abap.char(40) ) as BusinessRoleID,
     // max( tstmpl_to_utcl(cast(concat(concat(cdhdr.udate,cdhdr.utime),'.0000000') as abap.dec(21,7)),
     max( tstmpl_to_utcl(cast(concat(cdhdr.udate,cdhdr.utime) as abap.dec(21,7)),
         'FAIL',
         'INITIAL'
       )
      )                              as LastChangedDateTime,
     BusinessUsers // Make association public
}
where
     cdhdr.objectclas = 'PFCG'
  and cdpos.tabname   = 'AGR_USERS'
group by
  cdhdr.objectid
```

Code Snippet 11 View Entity ZOTCM_I_BUSINESS_ROLES_R

```
@EndUserText.label: 'Authorization to read role members'
@MappingRole: true
define role ZOTCM_I_BUSINESS_USERS_R {
   grant
      select
         on
            ZOTCM_I_BUSINESS_USERS_R
               where
                  (BusinessRoleID) =
                  aspect pfcg_auth(S_USER_AGR,
                                   ACT_GROUP,
                                   actvt = '03');

}
```

Code Snippet 12 Access Control ZOTCM_I_BUSINESS_USERS_R

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
@EndUserText.label: 'Authorization to read role data'
@MappingRole: true
define role ZOTCM_I_BUSINESS_ROLES_R {
   grant
      select
         on
            ZOTCM_I_BUSINESS_ROLES_R
               where
                  (BusinessRoleID) =
                  aspect pfcg_auth(S_USER_AGR,
                                   ACT_GROUP,
                                   actvt = '03');

}
```

Code Snippet 13 Access Control ZOTCM_I_BUSINESS_ROLES_R

```
@EndUserText.label: 'Service Definition for BusinessRoles'
define service ZOTCM_BUSINESS_ROLES_R {
  expose ZOTCM_I_BUSINESS_ROLES_R as BusinessRoles;
  expose ZOTCM_I_BUSINESS_USERS_R as BusinessUsers;
}
```

Code Snippet 14 Service Definition ZOTCM_BUSINESS_ROLES_R

Define and publish the Service Binding ZOTCM_BUSINESS_ROLES_R_V4 with the following details:

Property, Detail	Value
Service Name	ZOTCM_BUSINESS_ROLES_R_V4
Description	Web API Business Rolese OData v4
Bsinding Type	OData V4 – Web API
Service Definition	ZOTCM

Table 6 Details of Service Binding ZOTCM_BUSINESS_ROLES_R_V4

Publish the Service Binding for each client you want to synchronize.

8.4 Maintaining Content Server Customizing

After you published the Service Binding its details display a Service URL. Based on the example names of the previous chapters the Service URL is expected to be:

/sap/opu/odata4/sap/zotcm_business_roles_r_v4/srvd_a2x/sap/zotcm_business_roles_r/0001/

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

On the Content Server you must maintain customizing according to the steps in chapter **Synchronize SAP roles with CS Groups** in a recent version of **Smart View User Guide: OpenText Content Management (CSSUI-UGD-EN)**.
Suppose your leading S/4HANA system has the HTTPS base address
https://your_s4_host:44300/
your_s4_host is the full host name and
44300 is the HTTPS port. And in our example your client is 100.

This leads to the following corresponding example values in the customizing on your Content Server.

Business Application

Connection Type	S/4HANA Adapter
Application Server Endpoint: OData API V2	https://your_s4_host:44300/sap/opu/odata/sap
Application Server Endpoint: OData API V4	https://your_s4_host:44300/sap/opu/odata4/sap
Client	100

(Please maintain the other parameters of the business application connection according to product documentation and matching your requirements.)

If you are synchronizing to the SAP client which is the default client to log on (profile parameter login/system_client), then you can leave the value of Client empty.

Scheduled Bots > Scheduled Bot Type Synchronize SAP roles with CS groups

Create and maintain a scheduled bot of type Synchronize SAP roles with CS groups like:

Business Application	The business application of type S/4HANA Adapter you maintained as described above.
OData V4 Endpoint	zotcm_business_roles_r_v4/srvd_a2x/sap/zotcm_business_roles_r/0001

(Please maintain the other parameters of the scheduled bot according to product documentation and matching your requirements.)

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9 Fiori (SAPUI5) Integration

In this chapter the following terms are used.

- **S/4HANA On Premise** refers to an S/4HANA system hosted in the customer's local system landscape.
- **S4HANA Cloud** refers to SAP S/4HANA Cloud, Public Edition or Essential Edition which is the SaaS ERP solution by SAP.
- **S/4HANA Cloud, private edition** refers to an S/4HANA system hosted in a private cloud hosted within SAP's own data center or hosted by one of the major IaaS providers.

9.1 Overview

For the SAPUI5 components of Archiving and Document Access for SAP Solutions (or Core Archive for SAP Solutions), please refer to the SDK and API documentation of Archiving and Document Access for SAP Solutions.

There is a SAPUI5 component for business workspaces (section **9.2.2**).

It can be called like a standalone app in the SAP Fiori Launchpad.

- This makes sense, if called as a target of cross-app navigation from other apps.
  [Go to 9.3 INTEGRATING WITH CROSS-APP (INTENT-BASED) NAVIGATION]

Since S/4HANA 2020 key users can embed content in object pages without program code.

- This is done via a URL in an iframe.
  [Go to 9.4 EMBEDDING CONTENT (OBJECT PAGES) WITH KEY USER ADAPTATION]

It can be integrated programmatically with component containers into other SAPUI5 apps, that can be

- apps delivered by SAP as listed in SAP Fiori Apps Library implemented using SAPUI5
  - SAP Fiori Elements based apps; SAP Fiori elements (formerly known as smart templates) is a framework (see Introduction to SAP Fiori Elements or Developing Apps with SAP Fiori Elements)

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

        - **[Go to 9.5 Extending SAP Fiori Elements-based app with Adaptation]**

    - freestyle apps, plain SAPUI5 based apps
        - **[Go to 9.7 Extending freestyle SAPUI5 apps with Adaptation]**

- apps created by customers or partners implemented using SAPUI5
    - SAP Fiori Elements based apps (see again Introduction to SAP Fiori Elements or Developing Apps with SAP Fiori Elements)
        - **[Go to 9.5 Extending SAP Fiori Elements-based app with Adaptation]**
        - **[Go to 9.6 Extending own SAP Fiori Elements-based app with app extensions during creation]**
    - freestyle apps, plain SAPUI5 based apps
        - **[Go to 9.7 Extending freestyle SAPUI5 apps with Adaptation]**
        - **[Go to 9.8 Integrating own freestyle SAPUI5 apps during creation]**

Section **9.1.1** gives you an overview in screenshots how integrations of these components look like.

Note: In recent presentations SAP discerns between 3 categories of SAPUI5 apps:

- SAP Fiori Elements
    - Using the templates and controls of the SAP Fiori Elements framework
- Mixed
    - Using Smart controls
    - Using regular controls
- Freestyle
    - Using regular controls

Unfortunately, the details in the SAP Fiori Apps Library do not always give you details, whether an app belongs to these categories.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Hint: If you are in doubt, whether an app from the SAP Fiori Apps Library can be extended in one or the other way, then use the SAP Web IDE to create an Extension project or an Adaptation project based on this app. The respective wizard will tell you whether the original app supports this option or not.

Another aspect is, that for security reasons the (on-premise) Fiori Launchpad shell changes X-XHR-Logon headers to enforce, that the right response headers are sent.
One option is to use a reverse proxy like the SAP Web Dispatcher. Starting with version 22.3 the OpenText Content Server adds the required response headers if the integration widget is used.
[Go to 9.11 RESOLVING CROSS-ORIGIN RESOURCE SHARING SCENARIOS WITH Content Server]

9.1.1 Required SAPUI5 version

Please check the latest release notes and product documentation on any updates.

But the minimum required SAPUI5 version is 1.84.

9.1.2 Deprecation of Web IDE and other changes (Visual Editor)

Please note, that the described developer integrations using the deprecated Web IDE can also be executed with corresponding steps using Business Application Studio (BAS) or SAP Fiori Tools in Visual Studio Code (VS Code).

Specifically, the code snippets and code examples do not change.

More recently, SAP Business Application Studio and SAP Fiori Tools in Visual Studio Code introduced the Adaptation Editor as a replacement of the SAPUI5 Visual Editor. If descriptions still refer to SAPUI5 Visual Editor, the Adaptation Editor can be used instead.

The older references like to Web IDE will be gradually replaced by corresponding descriptions for Business Application Studio (BAS) / Visual Studio Code (VS Code).

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.1.3 Overview of integration examples in screenshots

9.1.3.1 Integrating with cross-app navigation

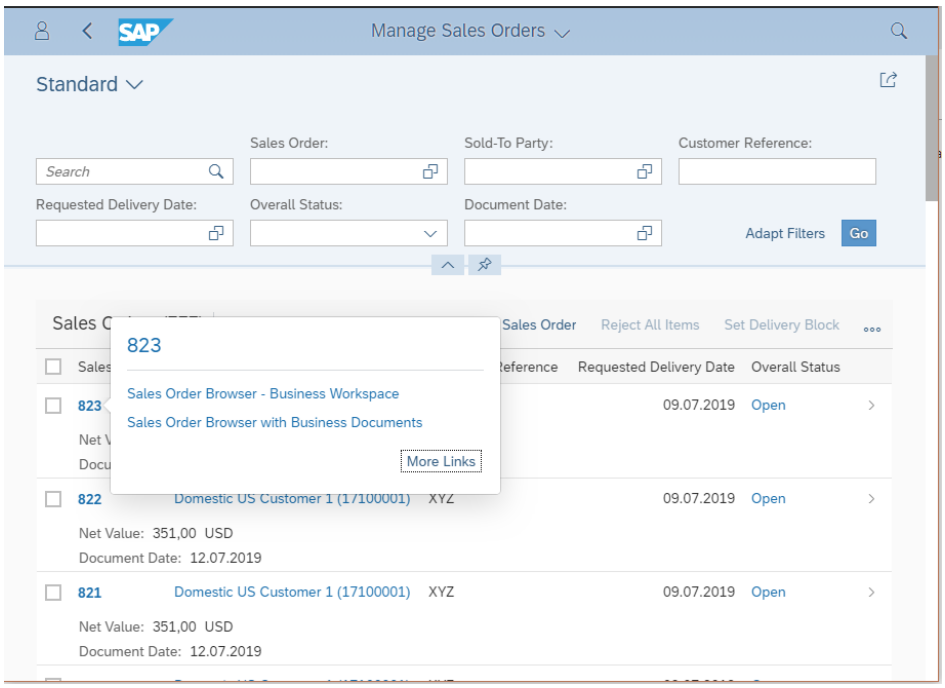


Figure 46 Example of a SmartLink

Figure 46 Example of a SmartLink shows that the standard SAP app Manage Sales Orders has the feature of a quick card or SmartLink which lists link targets of the semantic object SalesOrder. You are free to add your own target mappings in the customizing of the Fiori Launchpad Content.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

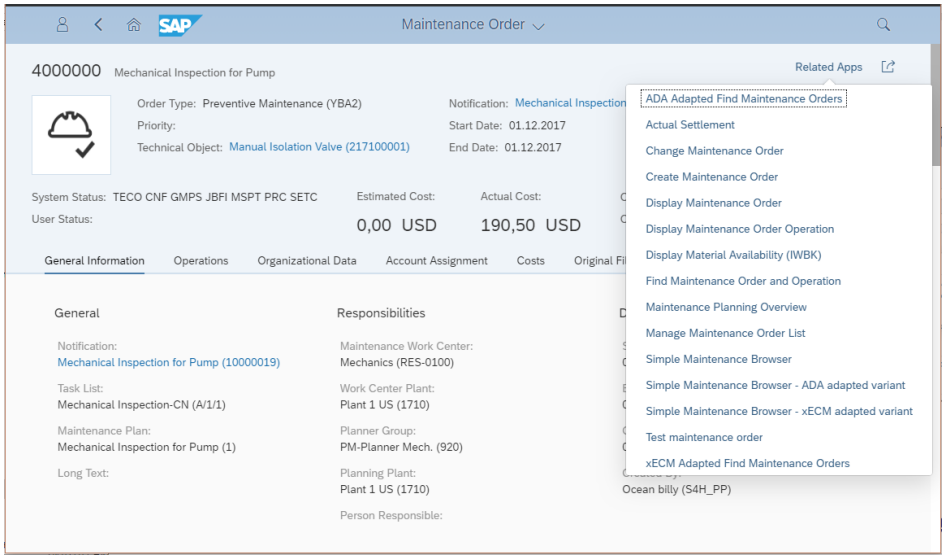


Figure 47 Example of Related Apps

Another feature often seen in standard SAP apps is the Related Apps link as seen in *Figure 47 Example of Related Apps*. Again, the available links to navigation targets of a specific semantic object are offered, in this case, for MaintenanceOrder.

This is extremely nice because you can integrate also with extended apps.

See the related descriptions in the customizing guides of OpenText Extended ECM for SAP Solutions and OpenText Archiving and Document Access for SAP Solutions and in section **INTEGRATING WITH CROSS-APP (INTENT-BASED) NAVIGATION**.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.1.3.2 Embedding content (Object Pages) with Key User Adaptation

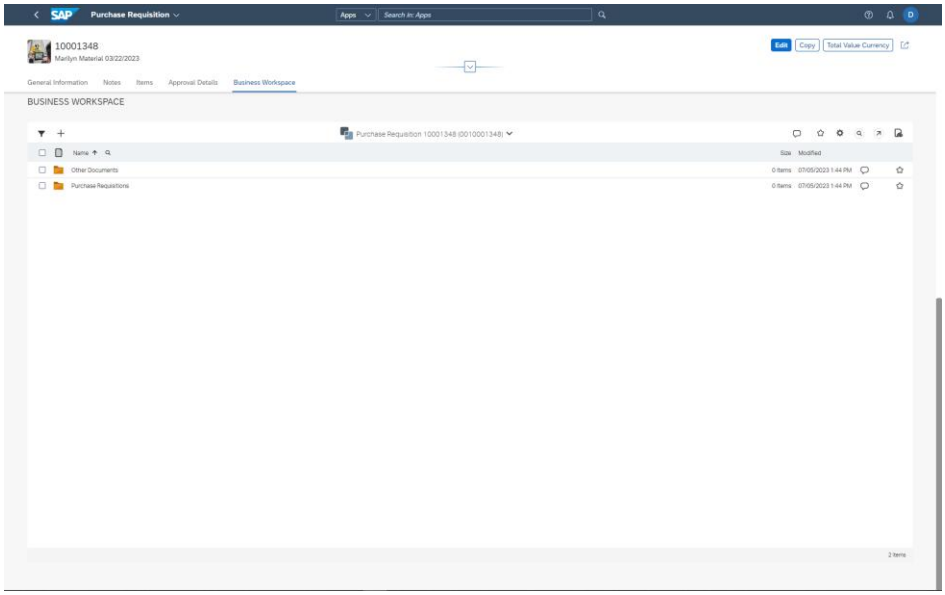


Figure 48 Example of embedding content with key user adaptation

Figure 48 Example of embedding content with key user adaptation displays the Manage Purchase Requisitions – Professional app which was extended with key user adaptation on S/4HANA On Premise**.** The key user adaptation on S/4HANA Cloud, private edition is done in the same way.
Key user adaptation can be used to extend Fiori apps on S4HANA Cloud, too.

At runtime this looks like apps extended with developer adaptation. But it embeds a URL in an iframe.

- In the case of S/4HANA On Premise and S/4HANA Cloud, private edition, the URL is from a target mapping which calls the OpenText integration app for business workspace .
- In the case of S4HANA Cloud the embedded URL calls a request handler on OpenText Content Server for business workspace.

See **EMBEDDING CONTENT (OBJECT PAGES) WITH KEY USER ADAPTATION.**

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.1.3.3 SAP Fiori Elements based app extended with Adaptation Project

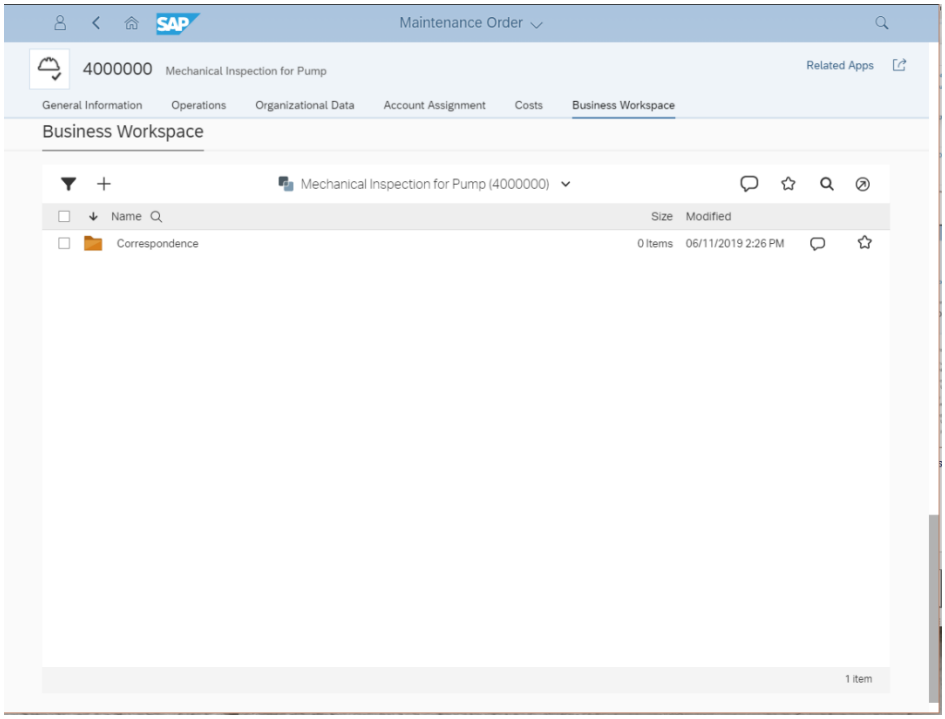


Figure 49 Example of SAP Fiori Elements app extended with Adaptation Project (SAPUI5 Flexibility)

Figure 49 Example of SAP Fiori Elements app extended with Adaptation Project (SAPUI5 Flexibility) shows how the SAP standard app to find Maintenance Orders looks like if extended with an OpenText Business Workspace.

Use this technique to extend SAP Fiori Elements based apps delivered by SAP and partners or for your own SAP Fiori Elements based apps.

See **EXTENDING SAP FIORI ELEMENTS-BASED APP WITH ADAPTATION**.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.1.3.4 Own SAP Fiori Elements based app extended at creation time

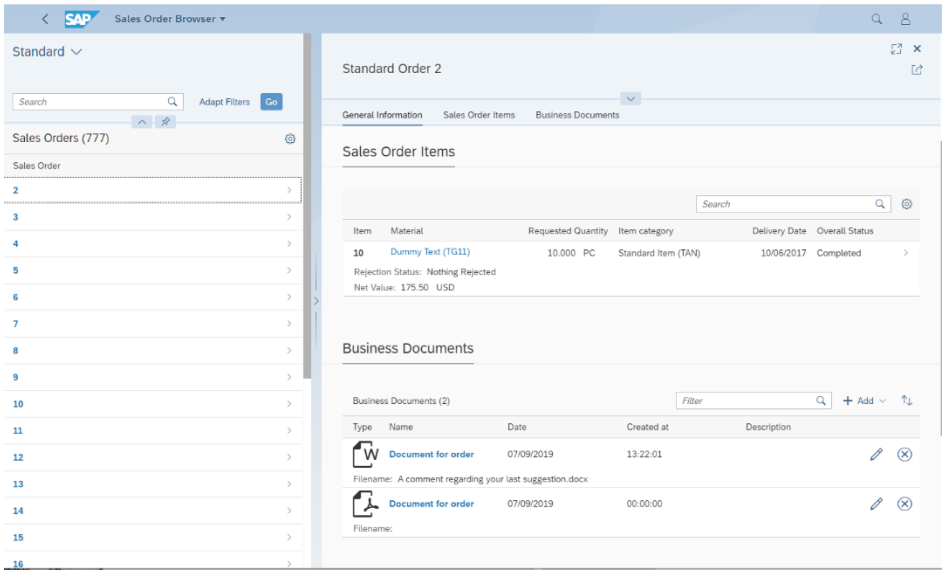


Figure 50 Example of own SAP Fiori Elements app extended at creation time

Figure 50 Example of own SAP Fiori Elements app extended at creation time shows a custom app created from the list report template for SAP Fiori Elements based apps in the SAP Web IDE.
Then it was extended with controls for OpenText Business Documents. The extension with OpenText Business Workspace works the same way.

Read **EXTENDING OWN SAP FIORI ELEMENTS-BASED APP WITH APP EXTENSIONS DURING CREATION** for more details.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.1.3.5 SAPUI5 freestyle app extended with Adaptation Project

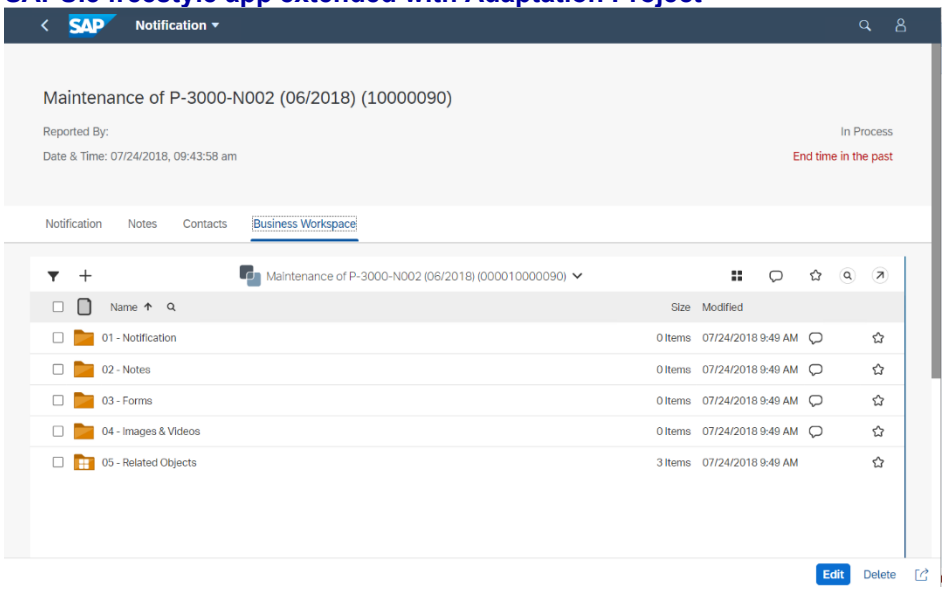


Figure 51 Example of freestyle app extended with Adaptation Project (SAPUI5 Flexibility)

Adaptation projects can now also be used to extend freestyle SAPUI5 apps. SAP supports to add fragments to an extension point with SAPUI5 version 1.78 or above.

Figure 51 Example of freestyle app extended with Adaptation Project (SAPUI5 Flexibility) is the extended SAP app Request Maintenance - for Employee - Maintenance Info. It is a typical freestyle app with extension points.

See **EXTENDING FREESTYLE SAPUI5 APPS WITH ADAPTATION.**

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.1.3.6 Using OpenText controls in freestyle SAPUI5 app

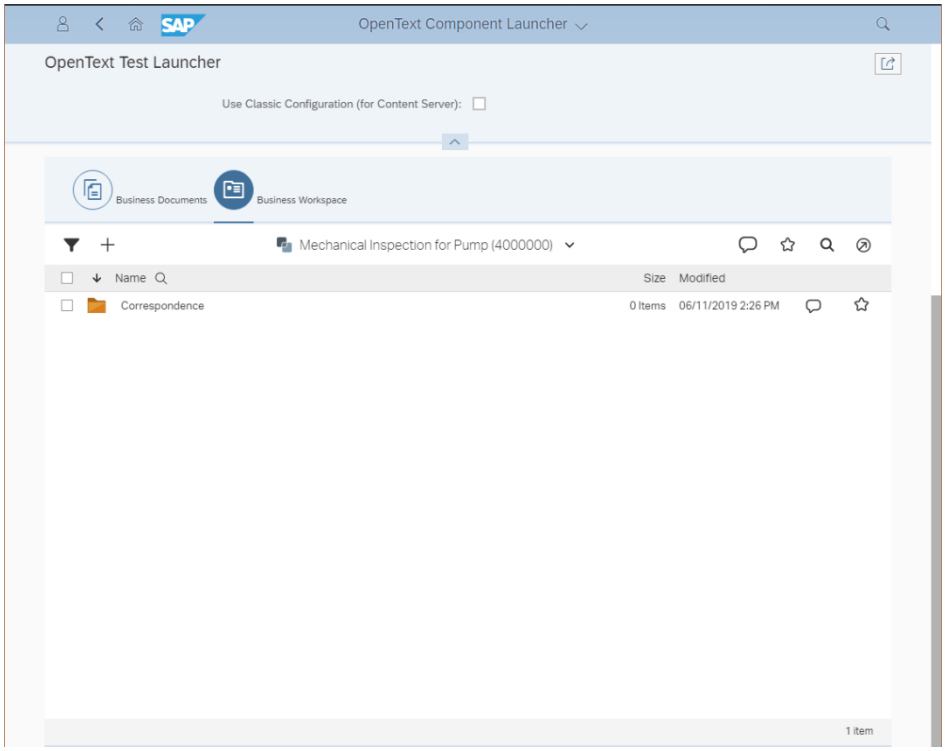


Figure 52 Example of a freestyle app using integration - OpenText Test Launcher

The OpenText Test Launcher app is a freestyle SAPUI5 app using the OpenText business documents and OpenText business workspace components in component container controls, as can be seen in *Figure 52 Example of a freestyle app using integration - OpenText Test Launcher*.

Section **INTEGRATING OWN FREESTYLE SAPUI5 APPS DURING CREATION CONTAINS THE DISCUSSION**.

9.2 SAPUI5 integration components

9.2.1 Restrictions

Only one business workspace control for each SAPUI5 app is supported. (Please keep this specifically in mind in the descriptions of chapter 9.6.)

Any usage of two business workspace controls seen in examples of previous versions cannot be supported.

You can have a business documents control together with a business workspace control at the same time. (Please refer to the SDK and API documentation of Archiving and Document Access for SAP Solutions for the business documents app).

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.2.2 SAPUI5 app for Business Workspace

9.2.2.1 Business Workspace Component with Create and Complete integration widget

9.2.2.1.1 Properties

Available since version 20.2.

BSP Application	/OTCMFND/CSBWS
Relative URL (ICF node)	/sap/bc/ui5_ui5/otx/otcmfnd/csbws
Component Name	otx.ecmlink.businessworkspace_cc
Required OData service	/OTX/RM_WSC_ODATA_SRV;v=2
Startup Parameters	SapObject, ObjectId, SystemOrigin, kv(integer), kp, UseClassicConfig, WkspDisplayMode, PrefixPath, mobileOpenDirectly, DraftUUID
Component Properties	SapObject, ObjectId, SystemOrigin, AppMode, UseClassicConfig, WidgetStyleClass, WkspDisplayMode, PrefixPath
Public Component Functions	Refresh

Table 7 Properties of SAPUI5 Business Workspace Component with Create and Complete integration widget

Please note that the Startup Parameters are case sensitive.

9.2.2.1.2 Purpose

Starting with version 26.4 the Create and Complete integration widget is located under ICF node /sap/bc/ui5_ui5/otcmfnd/csbws.
The component name hasn't been changed since the app's first release.

The business workspace is assigned to an SAP business object instance defined by parameters `SapObject` and `ObjectId`. The control shown is rendered by an Integration Widget on Content Server.

You can display the business workspace as 'folderBrowse' or as 'fullPage' by setting the parameter WkspDisplayMode to 'folderBrowse' or to 'fullPage'.
In addition, you can create the business workspace from an early workspace, specify value of category attributes etc. After deletion of the business workspace the Create and Complete integration widget is displayed for creation again.
This app requires OData service /OTX/RM_WSC_ODATA_SRV version 2 to be active.

9.2.2.1.3 Startup parameters

The following startup parameters are supported in a target mapping URL of intent-based navigation.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

`SapObject`: Enter the exact name of an SAP business object type (BOR type) that exists on the SAP backend system. For example, BUS2032 for sales orders or EQUI for equipment. Together with the parameter `ObjectId` this defines the instance of an SAP BOR object for which the business workspace is displayed.

`ObjectId`: Enter the exact ID of the SAP business object instance for which the business workspace shall be displayed. This needs to be the exact matching ID as stored in the reference property **Number** of the business workspace stored on the related Content Server. This corresponds to the pattern how an object ID is stored in the ArchiveLink link tables TOAxx (TOA01, TOA02, ...).
For example, 0000011660 would be a correct value to identify sales order 11660 in combination with SAP business object type BUS2032. Together with parameter `SapObject` this defines the instance of an SAP BOR object for which the business workspace is displayed.

`SystemOrigin`: Enter either an SAP system alias or a system ID and client. If the value does not contain a dot (**.**), it is evaluated as a system alias that is defined in the SAP Gateway server.
If the value is left empty or undefined, then the default system alias is used, which is assigned to the OData service (external ID = /OTX/RM_WSC_ODATA_SRV) on the SAP Gateway server.
If it contains a dot, then the text before the dot is the system ID (SID) and the text after the dot is the client of the SAP system getting the data from. For example, a value of MYSAP means that the data of the OData service is retrieved from an SAP backend which is defined with system alias MYSAP. (As defined in transaction **`/UI2/GW_SYS_ALIAS`** **or IMG hierarchy SAP NetWeaver > UI Technologies > SAP Fiori > Initial Setup > Connections Settings > Define SAP System Alias**).
A value of DW3.800 means, that the data of the OData service is retrieved from an SAP backend with system ID DW3 and SAP client 800. Furthermore, this must be defined in an SAP system alias on the SAP Gateway server and must be assigned to the OData service with external ID /OTX/RM_WSC_ODATA_SRV.

`kv(integer)`: Available since version 16 EP6 (16.2.8). The specifier (integer) is an integer number greater equal 0. That means, that parameters like kv0, kv1, kv2, … are valid. The kv(integer) parameters are ignored, if parameter ObjectId contains a value.The kv(integer) values are sorted ascending by the integer number.The values are formatted in sort order and concatenated as defined in parameter **`kp`**. The concatenated value is used as BOR object ID. There is no server-side validation, whether a BOR object with this ID exists.

`kp`: Available since version 16 EP6 (16.2.8). A list of format patterns separated by an underscore (_). A format pattern can be DT, DA or TI followed by a pattern in LDML format, if the corresponding kv(integer) value is a datetime, date or time entry. Or a format pattern can be a format specifier **[flags][width]specifier** inspired by the C++ function printf. Starting with

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

version 20.4 there is also a format pattern CH followed by a small integer number, if the corresponding kv(integer) values is either a number or a text.

Datetime, Date and Time patterns:

- DT followed by an URL encoded datetime pattern in LDML format. It is not verified whether the pattern represents a full datetime. The key value must be a **valid datetime** corresponding to date format of the log on user profile.
- DA followed by an URL encoded date pattern in LDML format. It is not verified whether the pattern represents a full date. The key value must be a **valid date** corresponding to date format of the log on user profile.
- DT followed by an URL encoded time pattern in LDML format. It is not verified whether the pattern represents a full time. The key value must be a **valid time** corresponding to time format of the log on user profile.

The format specifier follows this prototype:

- [flags][width]specifier
- This does not correspond to the full capability of printf, but it is a simplified pattern. (JavaScript does not have a function like printf)

The specifier character at the end is the most significant component,since it defines the type and the interpretation of its corresponding argument:

specifier	Output
d or i	Integer
c	Character
s	String of characters

The flags sub-specifier is optional and follows these specifications:

flags	Description
-	Left-justify within the given field width; Right justification is the default (see width sub-specifier).
0	Left-pads the number with zeroes (0) instead of spaces when padding is specified (see width sub-specifier).

The width sub-specifier is optional and follows these specifications:

width	Description
(number)	Minimum number of characters to be printed. If the value to be printed is shorter than this number, the result is padded with blank spaces. The value is not truncated even if the result is larger.

The CH/Character pattern:

CH followed by a small integer corresponds to SAP character types with a specific length/width of the character string. If the corresponding value is a

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

number is if filled with leading zeroes if the width of the number is shorter. If the value is a text, it is filled up with spaces on the right side if the width is shorter. Any value with a larger width is cut off when the number of characters for width/length is reached. Example: kp=CH10 and kv1=1000 results in a BOR object ID=00000001000; kp=CH10 and kv1=USCU_L01 results in a BOR object ID=USCU_L01 ; kp=CH10 and kv1=Abcdefghijkl results in a BOR OBJECT ID=Abcdefghij

`UseClassicConfig`: Available since version 16 EP7 (16.2.10). Boolean parameter. This is a fallback, which should only be needed for certain test or development environments. If set to true, the full URLs for scripts and CSS files are used as defined in the Extended ECM Connection settings in transaction SPRO. If false or empty, the app uses origin of the current web page as the base URL for Content Server calls. The value is false by default. This way the same origin policy is followed.

Since version 20.4:

- If UseClassicConfig is true
  - PrefixPath will be ignored
  - Script parameter crossOrigin=false
- If UseClassicConfig is false
  - PrefixPath is evaluated, if not empty
  - Script parameter crossOrigin=false (we have same origin!)

`WkspDisplayMode:` (only for Create and Complete integration widget):

Available since version 20.2. String parameter. The default value is `folderBrowse`.

`folderBrowse`: display an existing workspace only as nodes table (documents and folders table) with the csui folderBrowse widget.

`fullPage`: display an existing workspace with perspective view.

`PrefixPath:` Available since version 20.4. String parameter. The default value is an empty string. Useful in the case of integration in custom apps deployed on the SAP Cloud Platform. This parameter as a startup parameter is intended for testing purposes. See more details in the next section.

`PrefixPath:` Available since version 22.4. Boolean parameter. The default value is false (or empty). If set to true, the app checks, whether it runs on a mobile device with operating system Android or iOS.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

The following behavior can be discerned if mobileOpenDirectly is true.

- If it is not a phone: The Create and Complete integration widget is called as usual. That means the behavior on Desktop and Tablet devices is transparent.
- If it is not an Android phone or iPhone: The message line “No Business Workspace available” is displayed instead of a widget. An error message is sent to browser console, that no supported phone can be detected.
- If it is an Android phone
    - If the business workspace does not exist: The message line “No Business Workspace available” is displayed instead of a widget.
    - If the business workspace exists, the app redirects to
        - Login page of the installed Content Server Mobile app. After successful login, the content of the business workspace is display in the app
        - Google Play to get the Content Server Mobile app
    - The original Fiori app is shown from which the target URL was triggered
- If it is an iPhone
    - If the business workspace does not exist: The message line “No Business Workspace available” is displayed instead of a widget.
    - If the business workspace exists, the app redirects to
        - Login page of the installed Content Server Mobile app. After successful login, the content of the business workspace is display in the app
        - App Store to download the Content Server Mobile app
    - The original Fiori app is shown from which the target URL was triggered

`DraftUUID:` Available since version 24.4. String parameter. The default value is an empty string. Useful if you embed the business workspace via URL using key user adapatation and the leading app work with draft mode. See *Creating key user adaptation (embed the URL of the target mapping)* for details.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.2.2.1.4 Component properties

`SapObject` corresponds to startup parameter of the same name. Can be used to initially define the SAP business object type when creating the component object. Initially it also can be left empty and defined later when calling function refresh.

`ObjectId` corresponds to the startup parameter of the same name. Can be used to initially define the SAP business object id when creating the component object. Initially it also can be left empty and defined later when calling function refresh.

`SystemOrigin` corresponds to the startup parameter of the same name. Can be used to initially define the SAP system alias from which the OData service shall read its data. This value cannot be changed later when calling function refresh.

`AppMode`: Boolean value to initially define, whether the component runs in application mode (true) or not (false). The default value is true.

`True`: The business workspace control is wrapped in a SAPUI5 page object. This makes sense if the control shall run as a standalone app.

`False`: The business workspace control is directly rendered in the component container. This makes sense if you create a component object (with function `sap.ui.getCore().createComponent()`) that is integrated into a leading Fiori app within a component container.

`UseClassicConfig`: corresponds to the startup parameter of the same name. Available since version 16 EP7 (16.2.10). Boolean parameter. This is a fallback, which should only be needed for certain test or development environments. If set to true, the full URLs for scripts and CSS files are used as defined in the Extended ECM Connection settings in transaction SPRO. If false or empty, the app uses origin of the current web page as the base URL for Content Server calls. The value is false by default. This way the same origin policy is followed.
Since version 20.4:

- If UseClassicConfig is true
  - PrefixPath will be ignored
  - Script parameter crossOrigin=true
- If UseClassicConfig is false
  - PrefixPath is evaluated, if not empty
  - Script parameter crossOrigin=false (we have same origin!)

`WidgetStyleClass`: String value to initially define the name of a CSS style class. If it is not empty, the CSS style class of this name is used in the DIV element, in which the integration widget is rendered. It is meant to change

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

settings like the height of the DIV element. This is then taken over as the height of the integration widget.

See also the code snippet *Code Snippet 27 Function _setDivCssHeight()*, *Code Snippet 35 Function _setDivCssHeight()* .

`WidgetId`: String value with default value “otx_folder_browser”. This ID is merged with a GUID to form the ID of the DIV element into which the integration widget is rendered.

`WkspDisplayMode:` (only for Create and Complete integration widget): corresponds to the startup parameter of the same name. Available since version 20.2. String parameter. The default value is `folderBrowse`.

`folderBrowse`: display an existing workspace only as nodes table (documents and folders table) with the csui folderBrowse widget.

`fullPage`: display an existing workspace with perspective view.

`PrefixPath:` corresponds to the startup parameter of the same name. Available since version 20.4. String parameter. The default value is an empty string. Useful in the case of integration in custom apps deployed on the SAP Cloud Platform using external access type. The app inserts this prefix before the original OpenText Content Server paths. This way the calls to the Content Server are redirected accordingly.

9.2.2.1.5 Public component functions

`refresh (sSapObject, sObjectId, oOptional)`

In the case the widget already has been initiated (rendered) it fires a `REFRESH_WIDGET` event to refresh the display of the business workspace. In the case the widget is not initiated yet, it keeps the stated parameters for later rendition.

The parameters `sSapObject` and `sObjectId` are set to define the instance of the SAP BOR object (SAP business object) for which business workspace is displayed.

Parameters:

`{string} sSapObject` defines the SAP business object type (BOR type) of the object instance. For example, BUS2032 for a sales order.

`{string} sObjectId` defines the ID of the SAP business object (BOR object) instance. Proper format is required for matching. For example, 0000011660 matches sales order 11660, if SAP business object type is BUS2032.

`{object} oOptional` object, which may contain parameter
oOptional.SystemOrigin - corresponds to startup parameter of the same name

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

oOptional.UseClassicConfig - since version 16 EP7 (16.2.10); corresponds to the startup parameter of the same name

Important: If the values of SystemOrigin and UseClassicConfig are undefined, they do not change the current values.

Code example

```
var oCompBws = sap.ui.getCore().createComponent({
                                name: "otx.ecmlink.businessworkspace",
                                id: "OtBwsComp",
                                settings: {
                                        AppMode: false
                                }
                        });
oComponentContainerBws = new sap.ui.core.ComponentContainer({
                                        component : oCompBws
                                });
```

Code Snippet 15 MyView.view.js - refresh business workspace

```
var oComp = {};

/*
 * ...
 * then depending on your MVC model you get the component object,
 * which was defined in your view
 * ...
*/

var sSapObject = 'BUS2032';
var sObjectId = '0000011660';
var oOptional = {};
oOptional.SystemOrigin = 'ABC';
oComp.refresh(sSapObject, sObjectId, oOptional);
```

Code Snippet 16 MyView.controller.js - refresh business workspace

9.3 Integrating with cross-app (intent-based) navigation

Please read the sections about “Using intent-based navigation” in the OpenText Content Management for SAP Solutions: Customizing Guide (ERLK-CGD or EESPCD-CGD) for a description of general steps and examples.

Intent-based navigation allows you to decouple the navigation triggers from the actual navigation targets, which can be flexibly configured for different roles and device types.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Instead of encoding the name of the target app into the URL fragment, you can provide a navigation intent. An intent expresses what you want to do next (rather than how to do it).

For more information, see also Intent-based Navigation in SAP documentation.

The **Business Workspace** app supports this concept.

The SAPUI5 functions for cross application navigation can be implemented in your own custom app.

Furthermore, the SAPUI5 functions sap.ui.comp.navpopover.SmartLink and sap.ui.comp.navpopover.NavigationPopoverHandler make use of target mappings and semantic navigation.

This functionality can be leveraged in SAP Fiori Elements-based apps:

- Some SAP Fiori Elements apps have the Related Apps button enabled (then only new configuration in the Fiori Launchpad Content is required)
- Many SAP Fiori Elements apps use smart fields with UI annotations or the @Consumption.semanticObject which automatically create SmartLink controls at runtime (then only new configuration in the Fiori Launchpad Content is required)
- Even if metadata extensions are not allowed, new attributes may be appended to an existing CDS (Core Data Services) view, this way allowing annotations.

See also Guided Answers - How to extend Fiori elements apps

9.3.1 Important things to note

Application parameter length

The application parameter length (including SemanticObject/Action) shall not exceed 512 bytes when serialized as UTF-8.

Security

The transfer of the SAP business object type and SAP business object ID are not security critical: The OData service checks the user's authorization to see contents of the business workspaces or call services, which do such an authorization check. The business object ID does not reveal any specific data.

The application does no sanity check of the values entered for SAP business object type and object ID.

What happens with mistakenly added or used data?

For example, `?SapObject=BUS2032&ObjectId=11660`

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- If the user can do so on the OpenText Content Server, a business workspace with this reference can be created. Any data or files added to this business workspace are not lost because they are stored on the Content Server. In the case any business data shall be transferred, like category attributes filled by a property provider, the mistake can be discovered.
- If the user can add documents with the wrong object ID to ArchiveLink, then data is not lost. In the case such wrongly assigned data is discovered, it can be still moved to link entries with the correct object ID.
- Since 16 EP6 (16.2.8) the kv1, kv2, … and kp parameter can be used to map to more than one key value.

Convenience pattern:

Since 20.4 the kp parameter has the pattern CH<integer> to support mixed text and/or plain number keys.

9.4 Embedding Content (Object Pages) with Key User Adaptation

9.4.1 General Information and Requirements

Key user adaptation allows you to change the user interface of SAP Fiori apps directly in the launchpad without having to write new code.

The description for S/4HANA On Premise and S/4HANA Cloud, private edition is identical. Therefore, if only S/4HANA On Premise is mentioned it is valid for S/4HANA Cloud, private edition, too.

See SAP online documentation, for example:

- S/4HANA On Premise
  Adapting SAP Fiori UIs at Runtime - Key User Adaptation
- S4HANA Cloud
  Adapting SAP Fiori UIs at Runtime - Key User Adaptation

This option is available on object page apps enabled for key user adaptation.

Note the following:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- S/4HANA On Premise 2020 was the first On Premise version which introduced the option of embedding content (object page).
- Follow the documentation of SAP relevant for the specific S/4HANA version you are using.

Type of S/4HANA system	Type of URL supported
S/4HANA On Premise, S/4HANA Cloud, private edition	Intent to target mapping for business workspace app (recommended) See *URL example - using intent to purchase requisition business workspace*
	Request Handler on Content Server for business workspace
S4HANA Cloud	Request Handler on Content Server See *URL example - using request handler for business workspace*

Here is a list of advantages and disadvantages of the procedure.

Advantages:

+ The original app has not changed.
+ Can be used on S/4HANA On Premise, S/4HANA Cloud, private edition and S4HANA Cloud.
+ No UI developers required. No custom code needs to be maintained in an adaptation project.
+ At design time the SAP framework offers you all available variables which you may use in the URL
+ Standard deployment as described by SAP online documentation.
    - You may publish and deploy the UI changes directly for the original app.
    - Or you can also publish the UI changes as app variants.

Disadvantages:

- URL is called in iframe, overlay of fulltext view is only in iframe
- Less flexible than a developer adaptation; for example: you cannot workaround issues at runtime. (It is recommended to use the latest patches for SAP_UI on your S/4HANA and the latest patches of the original app).

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.4.2 Quick procedure

Follow the steps in this section for a quick procedure.

If you are reading this for the first time, skip this section and continue reading with the next section.

1. Enable key user adaptation.
2. Only S/4HANA On Premise**:** Define target mapping(s) for integration.
3. Create key user adaptation (embed the URL of the target mapping).
4. Publish UI changes or App Variants to your productive system.

9.4.3 Enabling Key User Adaptation

This section and the following sections give a detailed description of how to extend an SAP Fiori app with key user adaptation.

As an example, the following original app is used:

Manage Purchase Requisition Professional - App ID: F2229

Technical Name of SAPUI5 Application: MM_PPR_MAINTS1

As an administrator, enable your key users to change the user interface (UI) of apps in the SAP Fiori launchpad.

1. Authorization has been assigned accordingly,
    a. S/4HANA On Premise: Assign key users to authorization role SAP_UI_FLEX_KEY_USER.
    b. S4HANA Cloud: Assign key users to business role which is authorized for the business catalog SAP_CORE_BC_EXT_FLEX
2. Ensure that the key user uses the Fiori launchpad page.
3. Ensure that the key user has the required authorization for transport.
4. S/4HANA On Premise: Ensure that you have activated the required SICF nodes.

Again, read for details at

- S/4HANA On Premise
  Adapting SAP Fiori UIs at Runtime - Key User Adaptation
- S4HANA Cloud
  Adapting SAP Fiori UIs at Runtime - Key User Adaptation

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Note for S4HANA Cloud: If the business catalog SAP_CORE_BC_EXT_FLEX is not available or if the menu action Adapt UI is not displayed in the key user's menu, then check if you are on the customizing client of the development system.

9.4.4 Only S/4HANA On Premise: Defining Target mapping(s) for integration

Follow the steps detailed in 9.3 Integrating with cross-app (intent-based) navigation.

Using a target mapping has the advantage that one or more specific key values can be mapped to get the right business object key.

You can define target mappings like that for the business workspace integration app.

For business workspace integration this is the recommended way for

S/4HANA On Premise or S/4HANA Cloud, private edition, because you do not require any SAP Business Object (SBO) related customizing, and this is also available on S/4HANA releases older than 2022.

9.4.5 Creating key user adaptation (embed the URL of the target mapping)

Read SAP online documentation for your S/4HANA release

- S/4HANA On Premise
  Adapting SAP Fiori UIs at Runtime - Key User Adaptation
- S4HANA Cloud
  Adapting SAP Fiori UIs at Runtime - Key User Adaptation

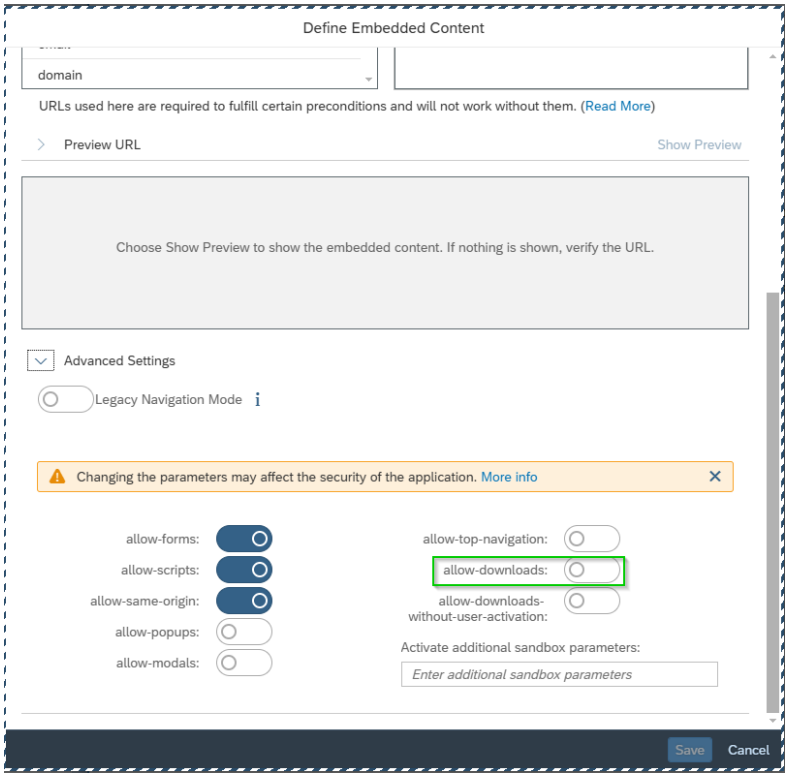
Follow these steps.

1. Start the app in the browser (for example F2229).
2. Search for a specific object, for example a purchase requisition.
3. Go to the object page (of purchase requisition).
4. Start the UI adaptation mode by selecting the action **Adapt UI** in the user actions menu.
5. Right-click the area where the sections of the object page are listed.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

6. If you can add external content there, you will see an **Embed Content…** context menu item.
7. Choose the action **Embed Content: as section**.
8. In the **Define Embedded Content** dialog, specify the size of the new iFrame either as a percentage of the space that's available, in pixels, or in rem units.
9. Click on the URL field and enter manually.
    a. To the left of URL, you see an area with title **Available Values** (click to add to the URL). The available values depend on the context of the area where you want to embed the content. For example: In app F2229, in the object page on header level, there is an available value for PurchaseRequisition. On item level, there is an available value for PurchaseRequisitionItem, too.
    b. See **URL EXAMPLES** related to app F2229.
    c. Business workspaces are currently not available when the Business Object is not created and is still in draft mode. In this situation, you can show a message to the user as "The business workspace can be created once you have created the object.". To achieve this, include the field DraftUUID in the URL during key user adaptation as ...&DraftUUID={DraftUUID}. See more in the URL examples in section 8.4.7.
10. **Advanced Settings**: The integration widget is embedded as iFrame in the SAP Fiori application. Some browsers prevent the download of content from an embedded component. In such cases, the users are not able to download documents from the business workspace. To allow the download of content, you can use the **Advanced Settings** section in the '*Define/Update Embedded Content*' dialog. These settings apply the restrictions to the embedded content. **By default, the download of embedded content is blocked**. You can enable the downloads by enabling the "**allow-downloads**" switch.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.



11. After you are done, choose Save.
12. Leave the UI Adaptation mode by clicking Save & Exit.

9.4.6 Drafting, Versioning, Publishing

After you create a new UI change or changed an existing UI, the changes are in Draft mode. Click Activate New Version to make changes available. Then the new version is available to users calling the app on the same client.

Follow the steps under chapter **Publishing UI Changes** in the respective SAP online documentation **Adapting SAP Fiori UIs at Runtime – Key User Adaptation** to publish the UI changes to the productive system.

Follow the steps under chapter **App Variants: All You Need to Know** in the in the respective SAP online documentation **Adapting SAP Fiori UIs at Runtime – Key User Adaptation** to publish the UI changes in an app variant.

Here are the links to the SAP online documentation referenced above.

- S/4HANA On Premise
  Adapting SAP Fiori UIs at Runtime - Key User Adaptation

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- S4HANA Cloud
  Adapting SAP Fiori UIs at Runtime - Key User Adaptation

9.4.7 URL examples

9.4.7.1 S/4HANA On Premise or S/4HANA Cloud, private edition

This URL example is related to the header level object page of Fiori app F2229 as code snippet with explanation.

```
?sap-ushell-config=headerless#PurchaseRequisition-
openOTXECMPurchaseRequisition?
PurchaseRequisition={PurchaseRequisition}&DraftUUID={DraftUUID}&
WkspDisplayMode=folderBrowse
```

Code Snippet 17 URL example - using intent to purchase requisition business workspace

Code Snippet 17 URL example - using intent to purchase requisition business workspace calls the intent of a target mapping which specifically calls the OpenText Fiori app for business workspace integration. This is the recommended approach for S/4HANA On Premise or S/4HANA Cloud, private edition.

Here is a description of the URL parts:

- /sap/bc/ui2/flp
  - This opens the Fiori Launchpad under the same base address of the leading app.
- sap-client=100
  - defines the SAP client; may also be omitted if you only have one default client.
- sap-ushell-config=headerless
  - with this URL parameter and value the Fiori app is rendered without the Fiori launchpad header
- #PurchaseRequisition-openOTXECMPurchaseRequisition
  - In this case it is the intent of the target mapping which calls the business workspace app (ID: otx.ecmlink.businessworkspace_cc).
  - Replace this with the intent of your target mapping
- PurchaseRequisition={PurchaseRequisition}

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- The target mapping in this example maps the URL parameter PurchaseRequisition to target name kv1. In addition the URL parameter kp was maintained with default value CH10. And the parameter SapObject has default value BUS2105.
- {PurchaseRequisition} is the place holder for the value which is filled by SAP at runtime. Please only use available values offered by SAP at design time.

- DraftUUID={DraftUUID}
  - Here, {DraftUUID} is the placeholder for the draft id for the business object. When the Business object is not yet created and still in draft mode, the field DraftUUID contains the draft id, that is used to derive the message for users.

- WkspDisplayMode
  - The WkspDisplayMode is a startup parameter as mentioned in section 8.2.2.1. This parameter can be overwritten, to either use 'fullPage' or 'folderBrowse' as per your preference.

9.4.7.2 S/4HANA Cloud, public edition

The next two examples can be applied to Fiori app F2229.

In the object page on header level (purchase requisition), you can embed the first example URL. It accesses the business workspace of the purchase requisition.

```
https://yourhost.yourdomain.com/otcs/cs.exe?func=xecmsap.GetWspIntegration&
SAPObjectType=PurchaseRequisition&SAPObjectNodeType=&SourceLogicalSystem=AB
C&SemanticObjectKey=%7B%22PurchaseRequisition%22%3A%22{PurchaseRequisition}
%22%7D
```

Code Snippet 18 URL example - using request handler for business workspace of SAP Object Type (SOT)

Code Snippet 18 URL example - using request handler for business workspace calls the request handler xecmsap.GetWspIntegration on the OpenText Content Server. The business workspace is related to the SAP Object Type (SOT) PurchaseRequisition.

Here is the description of the URL parts:

- https://yourhost.yourdomain.com

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

  - Base URL to the OpenText Content Server
- /otcs/cs.exe
  - Relative path to the main executable for Content Server
- func=xecmsap.GetWspIntegration
  - Content Server request handler to call business workspace integration widget.
  - This request handler is designed to support SAP's Harmonized Document Management (HDM), but it can also be used here
- SAPObjectType=PurchaseRequisition
  - Define here the SAP Object Type (SOT) following the SOT/SONT model for HDM.
  - This corresponds to the Business Object Type your Content Server Administrator defines. See chapter **Configuring the business object type in Content Server** in the respective **OpenText Extended ECM for SAP Solutions Cloud: Customizing Guide (EESPCD-CGD)**.
- SAPObjectNodeType=
  - Define here the SAP Object Node Type (SONT) following the SOT/SONT model for HDM.
  - In this example, we refer to the SAP Object Type. Therefore, this value is empty.
- SourceLogicalSystem=ABC
  - The business application ID or logical system name as defined on the Content Server at **Enterprise > Extended ECM > Connections to Business Applications (External Systems)**
- SemanticObjectKey=%7B%22PurchaseRequisition%22%3A%22{PurchaseRequisition}%22%7D
  - Key to identify the instance of the business object, in this example, of the purchase requisition
  - The value contains encoded special characters. The decoded value of this example is: {"PurchaseRequisition":"{PurchaseRequisition}"}
  - The value must always be a JSON type definition in curly brackets. The key is defined by one or more key-value pairs. Use comma to separate multiple key-value pairs.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- The value for the purchase requisition here is {PurchaseRequisition}. It is a placeholder for the available value of the purchase requisition number.
- See chapter **Configuring the business object type in Content Server** in the respective **OpenText Extended ECM for SAP Solutions Cloud: Customizing Guide (EESPCD-CGD)**. Your Content Server administrator defined properties of Service Name and Collection of the S/4HANA API webservice as detailed on api.sap.com. The API Reference page of the used collection shows the key parameters/fields.

In the object page on item level (purchase requisition item), you can embed the second example URL. It accesses the business workspace of the purchase requisition item.

```
https://yourhost.yourdomain.com/otcs/cs.exe?func=xecmsap.GetWspIntegration&
SAPObjectType=PurchaseRequisition&SAPObjectNodeType=PurchaseRequisitionItem
&SourceLogicalSystem=ABC&SemanticObjectKey=%7B%22PurchaseRequisition%22%3A%
22{PurchaseRequisition}%22%2C%22PurchaseRequisitionItem%22%3A%22{PurchaseRe
quisitionItem}%22%7D
```

Code Snippet 19 URL example - using request handler for business workspace of SAP Object Node Type (SONT)

Code Snippet 19 URL example - using request handler for business workspace of SAP Object Node Type (SONT) calls the request handler xecmsap.GetWspIntegration, too. The business workspace is related to the SAP Object Node Type (SONT) PurchaseRequisitionItem with SAP Object Type (SOT) PurchaseRequisition.

Here is the description of the URL parts:

- https://yourhost.yourdomain.com
  - Base URL to the OpenText Content Server
- /otcs/cs.exe
  - Relative path to the main executable for Content Server
- func=xecmsap.GetWspIntegration
  - Content Server request handler to call business workspace integration widget.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

  - This request handler is designed to support SAP's Harmonized Document Management (HDM), but it can also be used here

- SAPObjectType=PurchaseRequisition
  - Define here the SAP Object Type (SOT) following the SOT/SONT model for HDM.
  - In this example it is the parent SAP Object Type of the SAP Object Node Type.
  - This probably corresponds to the Business Object Type your Content Server Administrator defines. See chapter **Configuring the business object type in Content Server** in the respective **OpenText Extended ECM for SAP Solutions Cloud: Customizing Guide (EESPCD-CGD)**.
- SAPObjectNodeType=PurchaseRequisitionItem
  - Define here the SAP Object Node Type (SONT) following the SOT/SONT model for HDM.
  - Like in this example, it usually is an item or position related type below the main object type defined in parameter SAPObjectType
- SourceLogicalSystem=ABC
  - The business application ID or logical system name as defined on the Content Server at **Enterprise > Extended ECM > Connections to Business Applications (External Systems)**
- SemanticObjectKey=%7B%22PurchaseRequisition%22%3A%22{PurchaseRequisition}%22%2C%22PurchaseRequisitionItem%22%3A%22{PurchaseRequisitionItem}%22%7D
  - Key to identify the instance of the business object or object node type, in this example, of the purchase requisition item
  - The value contains encoded special characters. The decoded value of this example is: {"PurchaseRequisition":"{PurchaseRequisition}","PurchaseRequisitionItem":"{PurchaseRequisitionItem}"}
  - The value must always be a JSON type definition in curly brackets. The key is defined by one or more key-value pairs. Use comma to separate multiple key-value pairs.
  - The value for the purchase requisition here is {PurchaseRequisition}. It is a placeholder for the available value of the purchase requisition number.
  - The value of the purchase requisition item here is {PurchaseRequisitionItem}. It is a place holder for the available

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

value of the purchase requisition item.

See chapter **Configuring the business object type in Content Server** in the respective **OpenText Extended ECM for SAP Solutions Cloud: Customizing Guide (EESPCD-CGD)**. Your Content Server administrator defined properties of Service Name and Collection of the S/4HANA API webservice as detailed on api.sap.com. The API Reference page of the used collection shows the key parameters/fields.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.5 Extending SAP Fiori Elements-based app with Adaptation

9.5.1 General Information and Requirements

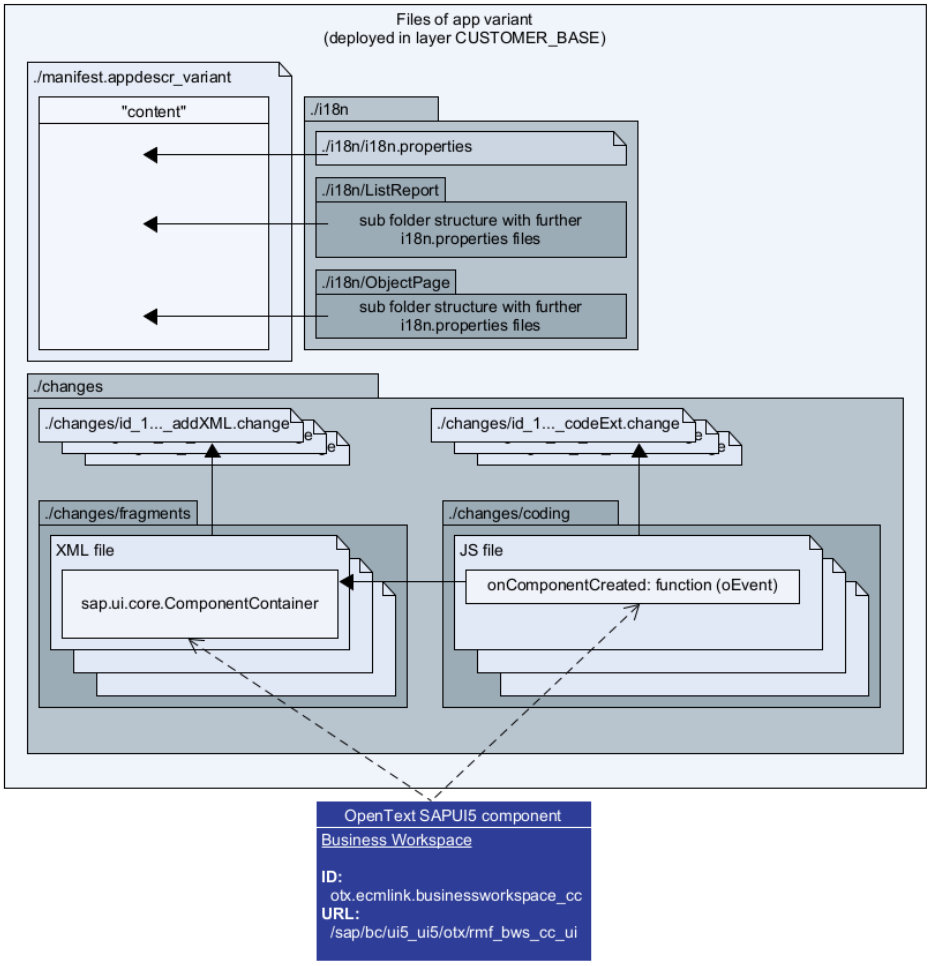


Figure 53 App variant resulting from extending with Adaptation Project

SAP Fiori Elements based apps are SAPUI5 apps, which are created with the SAP Fiori Elements framework. These are not only SAP-delivered standard apps but can also be your own custom apps created with the SAP Fiori elements templates of the SAP Web IDE.

If you already have practice, you then just follow the steps of section **9.5.2 QUICK PROCEDURE** using the code snippets of section **9.5.9 REUSABLE CODE SNIPPETS** to easily extend such an app with the OpenText UI control for business workspace.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

See SAP online documentation

> Introduction to SAP Fiori Elements
> or Developing Apps with SAP Fiori Elements

Adaptation extensions are one of two options to extend SAP Fiori Elements-based apps. You can introduce your own functionality as part of an adaptation project.

OpenText recommends using this option, if you want to integrate an OpenText control into an SAP Fiori Elements-based app.

Note the following:

- You require Business Application Studio on SAP BTP.
- Each adaptation project is deployed to the SAP Gateway server as an app variant in the customer layer of the SAPUI5 Flexibility framework
  - The variant ID starts with prefix `customer.`
  - The flexibility layer is `CUSTOMER_BASE`
  - The app variant has no own BSP node
  - The original app is not changed
- Currently the procedure is only relevant for list reports, object pages, overview pages, and analytical list pages.
- The flexible column layout is not supported.
- In SAP Business Application Studio the SAPUI5 Visual Editor lets you add changes

Important: Please refer to the most recent SAP online documentation like the SAPUI5 SDK regarding restrictions.

Here is a list of advantages and disadvantages of the procedure.

Advantages:

+ The original app is not changed
+ For the same kind of integration, you can reuse the same implementation steps and code snippets
+ The SAPUI5 Visual Editor restricts the areas in which extensions are allowed
+ No code replacement or overrides are required to insert new controls
+ Controller code to hook into the original app is short and no invasive code is required

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

+ The changes are added as single change files and can be monitored quite easy in the adaptation project
+ Calling the deployed app variant in the SAP Fiori Launchpad is straightforward and easy

Disadvantages:

- After deployment the code of the app variant is hidden in the database tables of the SAPUI5 Flexibility framework
- You need an original project in Business Application Studio for further changes.
- On more recent S/4HANA releases (e.g. S/4HANA 2022 and upwards), the embed content feature of key user adaptation is easier and quicker to use and you do not need any custom development and source code.

9.5.2 Quick procedure

Follow the steps in this section for a quick procedure.
If you are reading this the first time, skip this section and continue reading with the next section, **9.5.3 CREATING A NEW ADAPTATION PROJECT**.

1. Create a new adaptation project and modify in the next steps.
2. With the SAPUI5 Visual Editor add a new XML fragment named “otx” (otx.fragment.xml) at the end of the `section` aggregation of the ObjectPage
3. With the SAPUI5 Visual Editor extend the details view of with a new custom controller named “otx” (**otx.js**).
4. Insert new XML code in the XML fragment: **CODE SNIPPET 23 XML CODE OF BUSINESS WORKSPACE FOR OTX.FRAGMENT.XML**.

Make sure, that you set the right name of the function for event componentCreated.

5. Insert new JavaScript code in the JS controller: see code snippets of section **9.5.9.2 JAVASCRIPT CODE SNIPPETS FOR CUSTOM CONTROLLER**.
   But modify the SAP BOR object type and key calculation according to the runtime data of your original leading app. See next steps.
6. In the custom controller in function onComponentCreated() make sure, that function _setDivCssHeight() sets the right height and that the name of the HTML control and CSS style class are ok.
7. In function _getSettingsObject() make sure, that the SystemOrigin (that is the system alias) is ok.
8. Adjust the code in function _getObjectWithKey() according to your original app:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

a. Set the right SAP BOR object type for value oObject.SapObject

b. Make sure, that oObject.ObjectId is filled with the correct key/object ID.

9. Deploy the Adaptation project to your on-premise SAP system

10. Define a target mapping and tile, which call the app variant of your original app.

9.5.3 Creating a new Adaptation project

This section and the following sections give a detailed description of how to extend an SAP Fiori Elements-based app with an Adaptation project. As an example for the original app this is used:

Find Maintenance Order – for a Maintenance Technician - App ID: F2175 (see also SAP Fiori Apps Library)
Technical Name of SAPUI5 Application: EAM_ORD_MANS1

If you or an administrator has done the setup for your subaccount and services in the subaccount, you may start in your browser from

http://hana.ondemand.com

Then click on the navigation node for subaccounts and click on the tile of your subaccount. Then click on the navigation node Services > Instances and Subscriptions. In the list Application click on the entry SAP Business Application Studio. In the page Dev Spaces start and enter your dev space or create a new one.

1. On the page Get Started with SAP Business Application Studio, click **New Project from Template**.

2. Click on the tile **Adaptation Project**.

3. Click **Start**.

4. Select environment: **ABAP**. Click **Next**.

5. Fill in the basic information.

   a. In field Project Name, define a previously unused project name in the customer name space, for example, zxecm.eam.ordmans1

   b. Enter a title of your choice in the field Application Title.

   c. Fill in field Namespace. Although the project name and the namespace after “customer.” do not need to be identical, we keep it that way. Then it is easier to map the project to the app.

6. Select System.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

7. After the page has refreshed, select **Application**, for example: i2d.eam.order.manages1
8. Select the system version for SAP UI5 version.
9. Select radio button option Safe Mode = Yes.
10. Click **Finish**.

After these steps you will find the new project in your workspace.

The Web IDE has created all initial files and folders, for example the file webapp/manifest.appdescr_variant.

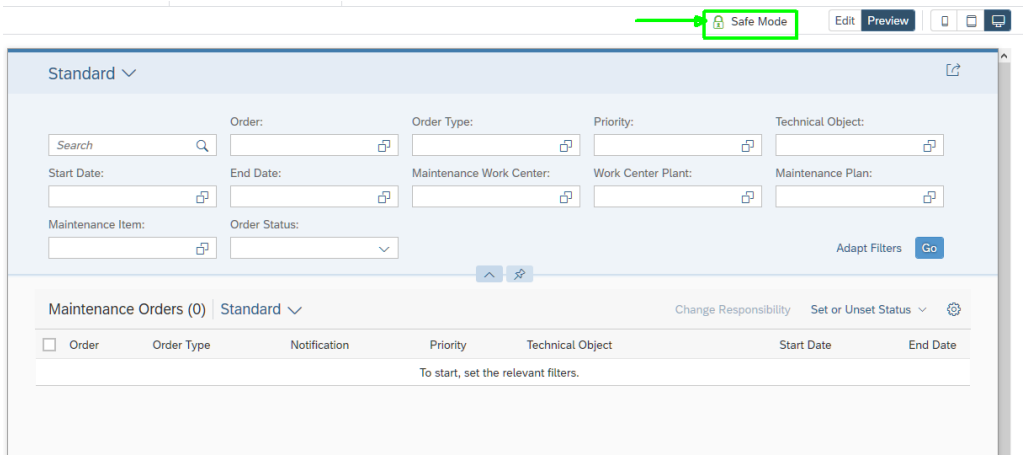
9.5.4 Integrating the OpenText control for Business Workspace

9.5.4.1 Using the SAPUI5 Visual Editor to add a new XML fragment

Now right-click the name of your project and select **SAPUI5 Visual Editor**. Enter login name and password to the on-premise SAP system, if necessary.

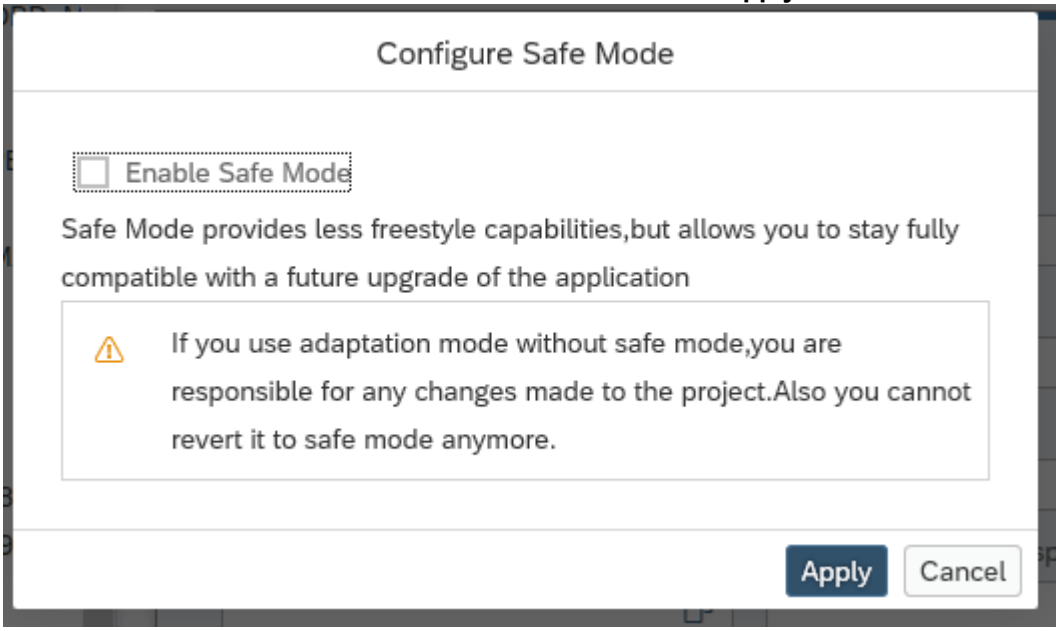
In this situation or other steps, where a connection to the SAP Gateway is required, a popup to log on to the system will be displayed. Then enter the username and password to log on to the SAP Gateway.

1. This opens the visual editor in the safe mode, like.

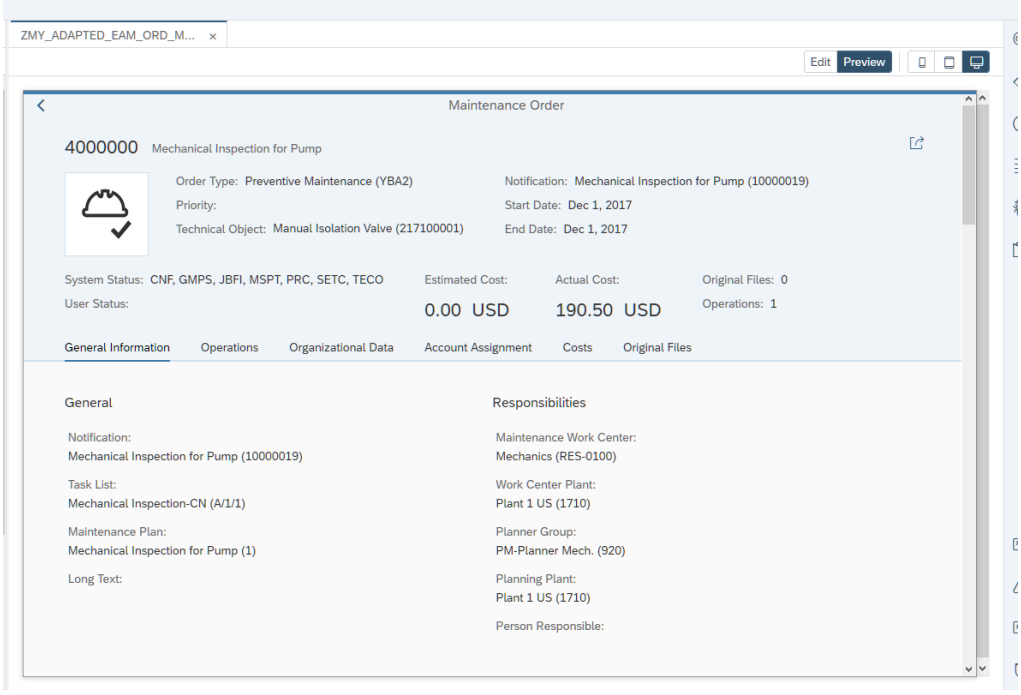


Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

2. Click **Safe Mode** and uncheck “Enable Safe Mode” and click **Apply**.



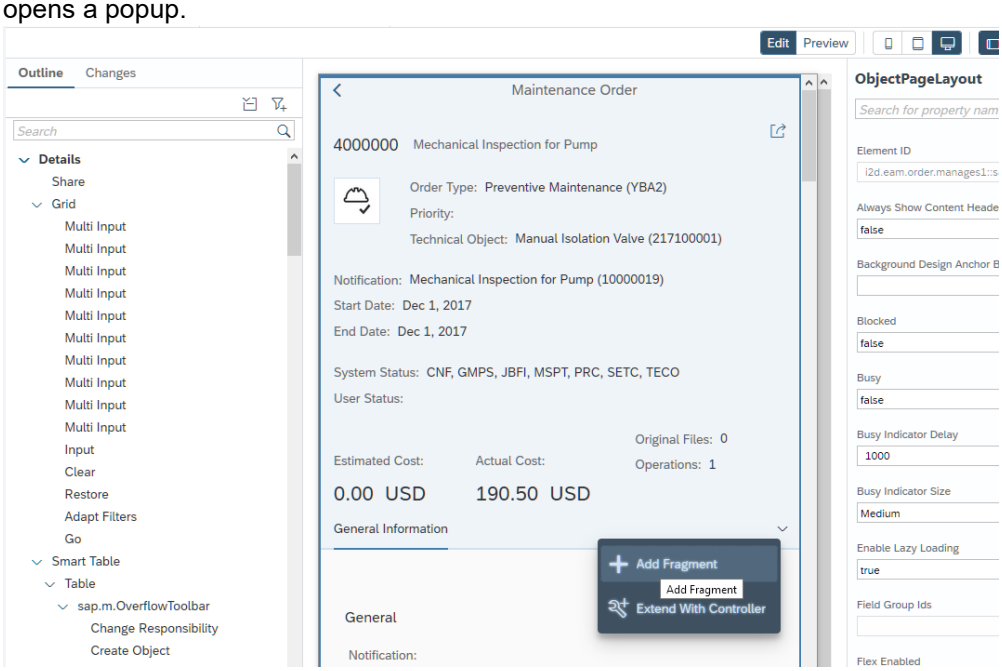
3. In the preview without safe mode, we change to a details page, which is the object page of the app. Click **Go**.
4. In the result list select and click on an item.
5. This opens the details view of the item, which is an object page.



6. Click **Edit** of the SAPUI5 visual editor.

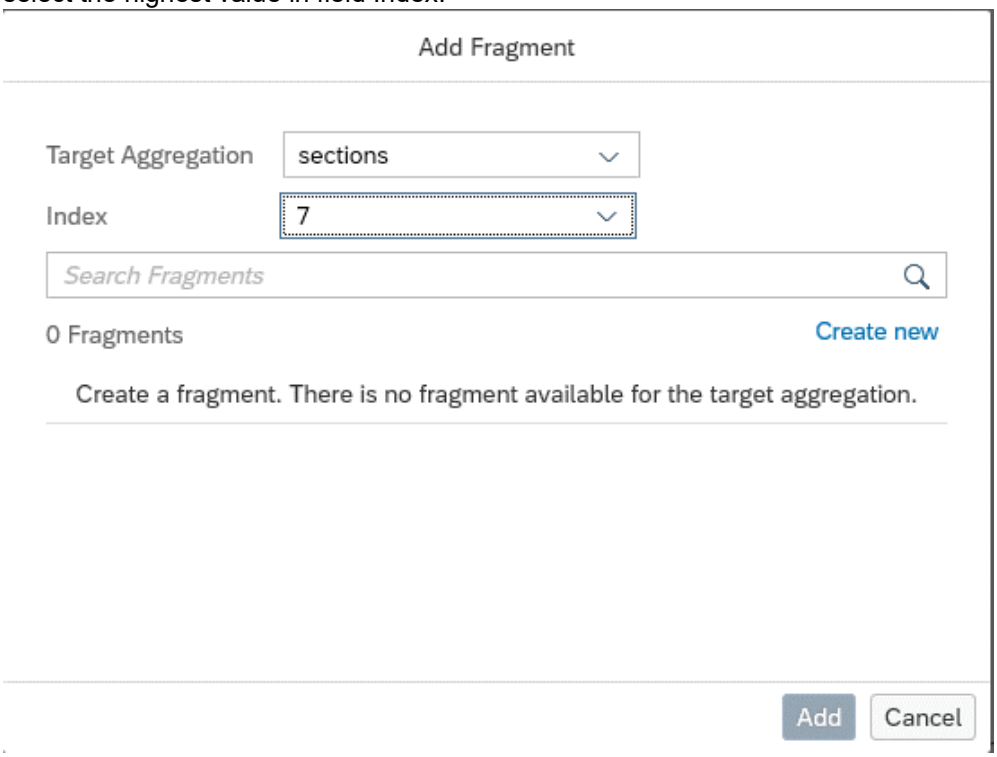
Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

7. This reloads the page. Resize the area of the loaded page in edit mode to get better access to the tabs of the app. Right-click in the tabs. This opens a popup.



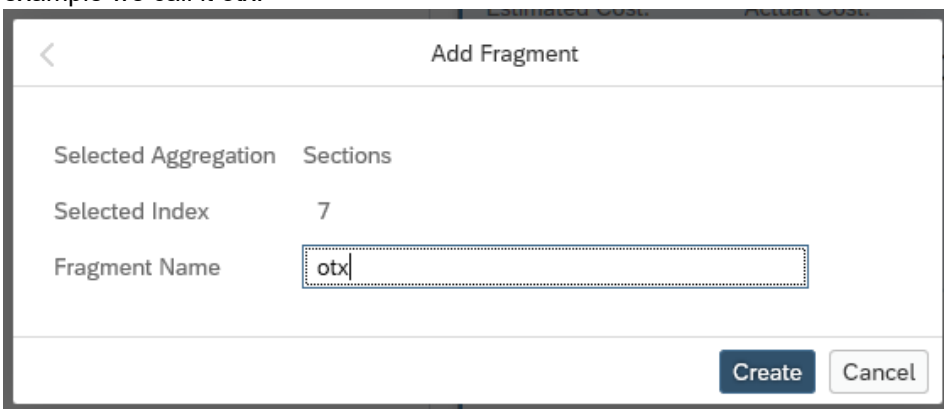
8. Click on **Add fragment**. This opens another popup.

9. In this popup select value sections in field Target Aggregation and select the highest value in field Index.



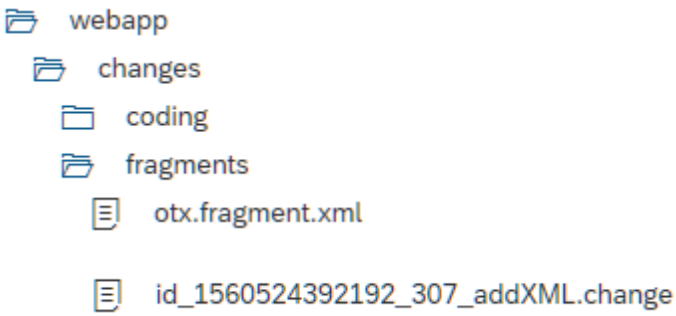
Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

10. Click **Create new**.
11. In the next popup, define a name of the fragment. It should be a recognizable name, which relates to the purpose of your change. In our example we call it otx.



12. Click **Create**.
13. Click on menu **File > Save All** to save the changes so far.

In the project we have now two new files:



./webapp/changes/id…_addXML.change

./webapp/changes/fragments/otx.fragment.xml

The first one is a text file "**id…**" contains all change information of the fragment file. **Do not change the contents of this file manually.**

Then there is a file named **otx.fragment.xml**:

- Do not change the name of this file. If you like to have the same content of this file under a different name, then the suggested steps are the following:
    - Remove the fragment file and the related "id…" file
    - Add a new fragment with the visual editor
    - Then change the content of the new fragment file

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

The next section describes how to change the fragment file in the editor.

9.5.4.2 Hints to avoid or work with coding mistakes

- Start with a simple change and check in the preview, whether it has a visible effect
- Make sure, that every SAPUI5 control has a unique id
- Use i18n keys for language dependent properties like titles
- If you have less experience with XML files for SAPUI5 or if you are adding a new kind of control, then keep a text copy of the content before you make the change. This way you may return to the state of the file before you tried your change

The next sections add more details to the code of file otx.fragment.xml

9.5.4.3 Initial XML fragment

Code Snippet 20 Initial XML fragment in adaptation project is the initial state of the XML fragment. You may return to this state whenever you are not happy with the content changes. But this initial fragment also does not show up any visible change in your app:

```
<!-- Use stable and unique id's!-->
<core:FragmentDefinition xmlns:core='sap.ui.core'></core:FragmentDefinition>
```

Code Snippet 20 Initial XML fragment in adaptation project

9.5.4.4 Adding a new object page section and a sub section for business workspace

```
<!-- Use stable and unique id's!-->
<core:FragmentDefinition xmlns:core='sap.ui.core'>
	<ObjectPageSection xmlns="sap.uxap" id="otx_xecm_sec" titleUppercase="false"
title="{i18n>OTX_WSP_TITLE}">
		<ObjectPageSubSection xmlns="sap.uxap" id="otx_xecm_subsec">
			<Text xmlns="sap.m" id="otx_xecm_placeholder" text="Replace this text"></Text>
		</ObjectPageSubSection>
	</ObjectPageSection>
</core:FragmentDefinition>
```

Code Snippet 21 Section and subsection with simple placeholder text

The XML in Code Snippet 21 Section and subsection with simple placeholder text defines a sap.uxap.ObjectPageSection with 1 subordinate sap.uxap.ObjectPageSubSection.

To be rendered, the subsection also needs some content in its block aggregation. Therefore, a simple text control is added as a placeholder, see also https://sapui5.hana.ondemand.com/#/api/sap.m.Text

Both items have a property *id* which must be defined and must result in a unique in the whole web page. In our case the id starts with the arbitrary “otx_xecm_”. You may also use something like “my_” to make sure, that the

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

ID's do not overlap with something else in the app. See also IDs of Fragments in Views.

Save the change (CTRL+S).

9.5.4.5 Defining localized texts in i18n.properties file

If you need localized texts, you need to define them with text keys in the i18n.properties file(s).

But how to map to the right file?

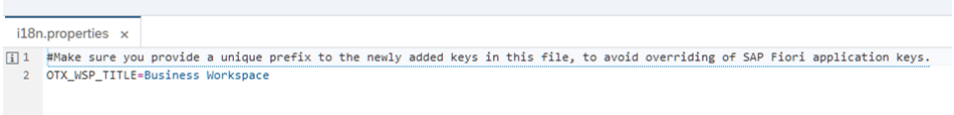
The convention is the following. The generated file manifest.appdescr_variant contains the sections

```
"content": [
    {
        "changeType": "appdescr_app_setTitle",
        "content": {},
        "texts": {
            "i18n": "i18n/i18n.properties"
        }
    },
    {
        "changeType": "appdescr_ui5_addNewModelEnhanceWith",
        "content": {
            "modelId": "i18n|sap.suite.ui.generic.template.ListReport|C_ObjPgMaintOrder"
        },
        "texts": {
            "i18n": "i18n/ListReport/C_ObjPgMaintOrder/i18n.properties"
        }
    },
    {
        "changeType": "appdescr_ui5_addNewModelEnhanceWith",
        "content": {
            "modelId": "i18n|sap.suite.ui.generic.template.ObjectPage|C_ObjPgMaintOrder"
        },
        "texts": {
            "i18n": "i18n/ObjectPage/C_ObjPgMaintOrder/i18n.properties"
        }
    }
```

These tell you the subfolders and file names for the localization. These sub folders and files were also generated.

As the XML fragment was added to the sections of the ObjectPage, you must define the text keys in the file i18n/ObjectPage/C_ObjPgMainteOrder/i18n.properties.

1. Open the file.
2. Add the new text key OTX_WSP_TITLE.



3. Click on **Save**.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Look back into the code at section **9.5.4.4 ADDING** a new object page section and a sub section for business .

You need to use a specific pattern telling the framework, that the localized text is found in a specific binding path.
For example, instead of untranslated text title="Business Workspace" you use title="{i18n>OTX_WSP_TITLE}"

- The curly brackets in double quotes tell the framework to expect a binding path
- The prefix i18n> tells the framework to bind from the i18n localization model
- The rest is the unique text key

The i18n.properties files define the default texts replacing the text keys if no specific localization can be found for the current logon language.

Go to **9.5.4.6 ADDING FURTHER LOCALIZATIONS TO THE PROJECT AND PREVIEWING THEM** to learn how to add a specific localization, for example German, and how you can check this on Business Application Studio.

Now change to the SAPUI5 Visual editor to see the outcome.

After you changed files in your project the page of the Visual Editor may show a warning that project files were modified.

Then Click on Reload editor.

When in doubt whether the Visual Editor reloaded correctly, then close it and open it again.

You start again at the list view. Therefore, browse again to the details page.

There is a new tab named Business Workspace in the object page.

9.5.4.6 Adding further localizations to the project and previewing them

During creation of the adaptation project an i18n.properties file was already created in sub folder

i18n/ObjectPage/C_ObjPgMaintOrder/i18n.properties

1. Right-click this file and select **Copy**.
2. Select the parent folder.
3. Press **CTRL+V** (or select **Edit menu > Paste**)
4. A new file i18n copy.properties was created.
5. Right-click this file and select **Rename**. The cursor changes to the name of the file and rename it .In our example for the German i18n properties file which follows the name pattern i18n_de.properties.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

6. Press Enter to finish the renaming.
7. Double click on the new file and change the texts, for example “Business Workspace Deutsch” (This is just an addition to indicate a different language. We do not usually translate “Business Workspace” to a German term like “Geschäftsbereich”).

Execute the following steps for the preview with language change.

1. Right-click on the file manifest.appdescr_variant and select **Adaptation Project > Open Preview**
2. This starts a new service and opens the preview in a new browser tab with the selection page of the app.
3. In the URL of the browser insert a **sap-language=DE&** like …/AppVariant_index_preview.html?sap-language=DE&sap-ui-layer=CUSTOMER_BASE…



4. Then press **Enter**.
5. This reopens the app with German log on language.
6. Click **Go**.
7. In the result list select and click an item.
8. The section and sub section titles then should display the localized title for the business workspace section.

9.5.4.7 Inserting a Component Container Control

In the XML file otx.fragment.xml replace the placeholder text and insert a sap.ui.core.ComponentContainer control

```
<!-- Use stable and unique id's!-->
<core:FragmentDefinition xmlns:core='sap.ui.core'>
        <ObjectPageSection xmlns="sap.uxap" id="otx_xecm_sec" titleUppercase="false"
title="{i18n>OTX_WSP_TITLE}">
                <ObjectPageSubSection xmlns="sap.uxap" id="otx_xecm_subsec">
                        <core:ComponentContainer id="otx_xecm_compcont" name="
otx.ecmlink.businessworkspace_cc" url="/sap/bc/ui5_ui5/otcmfnd/csbws" settings='{AppMode: false}'
                componentCreated=".extension.customer.zxecm.eam.ordmans1.otx.onComponentCreated">
</core:ComponentContainer>
                </ObjectPageSubSection>
        </ObjectPageSection>
</core:FragmentDefinition>
```

Code Snippet 22 XML code with component container for business workspace

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

In Code Snippet 22 XML code with component container for business workspace the lines with the blue background color indicate the new control.

The content of Code Snippet 22 XML code with component container for business workspace is identical with Code Snippet 23 XML code of business workspace for otx.fragment.xml. We reached the final state of the XML code in our example.

Here is an explanation of the used elements/controls:

- The localized text OTX_WSP_TITLE needs to be defined in the i18n.properties file
- The sap.m.VBox control is used to make sure, that the controls are vertically aligned
- The sap.ui.core.HTML control will be filled with a style class definition for style class otx_bws_div. Note, that the ID of the HTML control, “otx_cust_html”, is the first parameter to call in function _setDivCssHeight().
- The sap.ui.core.ComponentContainer control is calling the SAPUI5 component for the OpenText Business Workspace widget.

Property of sap.ui.core.ComponentContainer	**Description**
id="otx_xecm_compcont"	Unique ID
name="otx.ecmlink.businessworkspace_cc "	This is the name/id of the OpenText component to show business workspace. See *9.2.2.1.1 Properties*
url="/sap/bc/ui5_ui5/otcmfnd/csbws "	This is the relative ICF path to the SAPUI5 app to show OpenText business workspace. See *9.2.2.1.1 Properties*
settings='{AppMode: false, WidgetStyleClass: "otx_bws_div"}'	This is part of the API. With this AppMode you tell the component, that it does not run as a stand-alone app, but rather in a component controller. Furthermore, the setting WidgetStyleClass defines the CSS style class otx_bws_div to be used for the DIV-Element into which the business workspace integration widget will be

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Property of sap.ui.core.ComponentContainer	Description
	rendered. “otx_bws_div” is used as second parameter value to call function _setDivCssHeight(). See *9.2.2.1.4 Component properties*

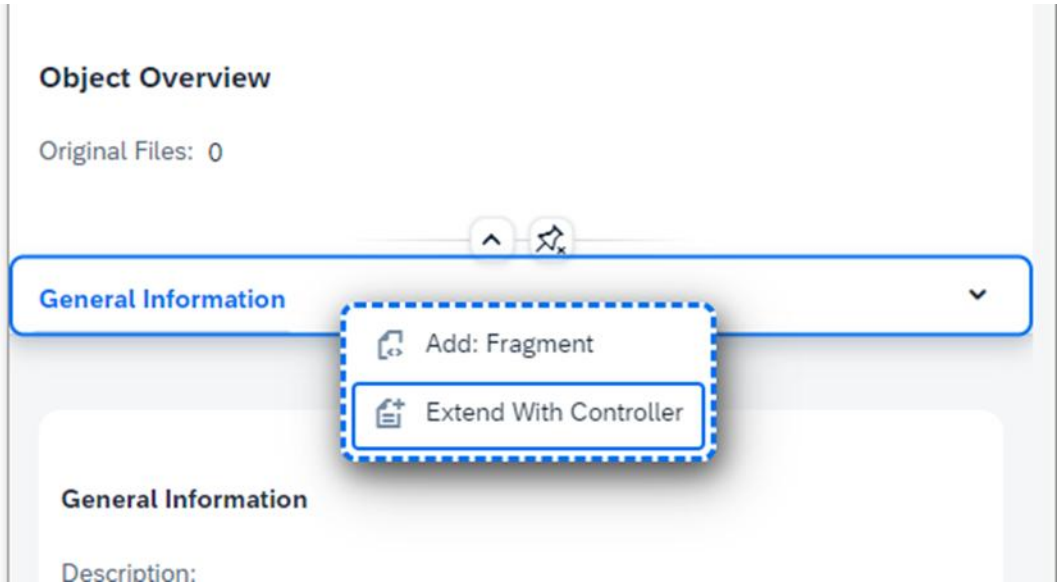
Event of sap.ui.core.ComponentContainer	Description
componentCreated= ".extension.customer. zxecm.eam.ordmans1. otx. onComponentCreated"	This event is fired when the component instance has been created by the ComponentContainer. This way we have a hook to trigger initialization and calculation of the SAP BOR object type (SapObject) and SAP BOR object ID (ObjectId). Please note, that the first part “.extension.customer.zxecm.eam.ordmans1” refers to the “id” in file manifest.appdescr_variant The part “.otx.onComponentCreated” refers to a new function onComponentCreated in the custom controller otx. See next section.

9.5.4.8 Adding the custom controller

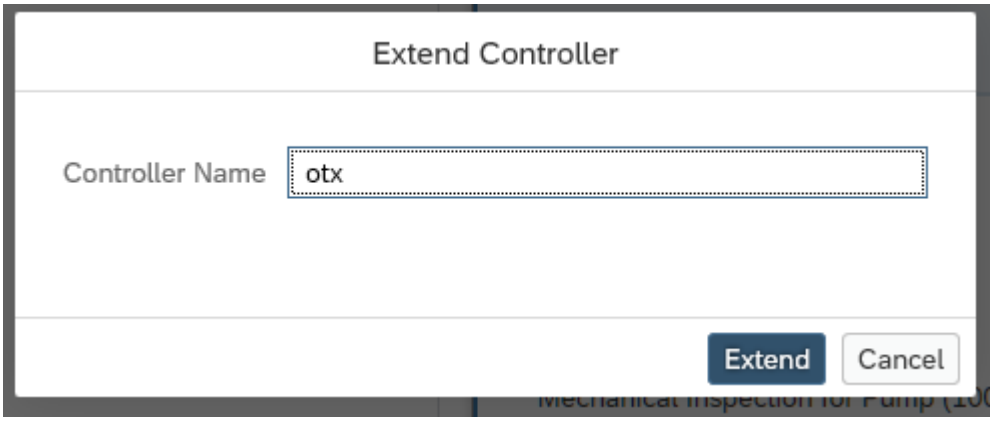
1. Open again the SAPUI5 Visual Editor and browse in preview mode to the object page.

2. In the edit mode right-click in the object page. Select the **Extend with Controller**.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.



3. In field the Controller Name enter otx.



4. Click Extend.

As a result, a new change file was created, for example id_1560524708154_313_codeExt.change.
Additionally, a JavaScript file named otx.js was created in subfolder webapp/changes/coding.
The content of the new custom controller is prefilled. In the next step add the new function code.

9.5.4.9 Inserting the callback function in the custom controller

In file coding/otx.js

- Comment out the metadata and methods area,
- Add the function definitions of the public callback function onComponentCreated() and of the private functions. Do this by

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

inserting the code snippets of section **9.5.9.2 JAVASCRIPT CODE SNIPPETS FOR CUSTOM CONTROLLER**.

- The code example of section **9.5.9.2 JAVASCRIPT CODE SNIPPETS FOR CUSTOM CONTROLLER** can be used for the business workspace control in the integration to the app F2175 = Find Maintenance Order
- The component of the OpenText Business Workspace app has an API function *refresh()*, which is described in **9.2.2.1.5 PUBLIC COMPONENT FUNCTIONS**. It is used to refresh the control with the SAP BOR object type and ID, when the data binding of the original SAP app is providing the runtime data.
- The parts indicated with light blue background need to be adjusted to your needs:
    - In function onComponentCreated() the call of _setDivCssHeight() defines a specific height. If you keep the names of the CSS class and custom HTML control as suggested for the XML code, the third parameter defining the height as a text string is the one you can modify to your needs.
    - In function _getSettingsObject() make sure to define the settings parameter SystemOrigin. See section **9.2.2.1.3 STARTUP PARAMETERS** for further details. But here are some important descriptions
        - Define a specific system alias to read the OData services from.
        - If you do not set the SystemOrigin at all, the OData services will be called from the default system alias
        - If you need to take over from the parameter sap-system, then you need to implement this specifically
    - In function _getObjectWithKey() you must modify the code to fit to the original app you are adapting:
        - Set the right SAP BOR object type for value oObject.SapObject
        - Make sure, that oObject.ObjectId is filled with the correct key/object ID.
        - See also the comment of the function in the code snippet: In the original app check for a controller file Attachments.controller.js, probably, in a sub folder ext/controller. Then look at the code in the onInit function. This should give you an idea, which runtime data is used to initialize the DMS related attachments control of SAP. The name of the object type is most

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

often not the same as the SAP BOR object type, but the ID or key value is often the same.

- As already mentioned before, the example code snippet does the calculation for a maintenance order in application F2175 = Find Maintenance Order.

- this._setDivCssHeight("otx_cust_html", "otx_bws_div", "650px");
  - "otx_cust_html" = ID of the custom HTML control in the XML fragment
  - "otx_bws_div" = name of the CSS class to set the custom height of the business workspace widget
  - "650px" = value of the custom height; use a CSS compatible value

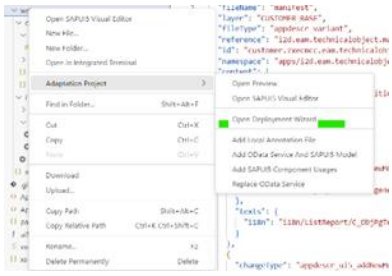
9.5.5 Making Original files or attachment sections invisible

This is not explicitly described here. But of course, other sections like Original files could be set to invisible, if you only want to display the business workspace. Please check SAP online documentation on how to change properties of existing parts in the object page.

9.5.6 Deploying the project to the SAP Gateway on premise

The steps and screenshots in this section are done with the example project for business workspace integration to the Find Maintenance Orders app. For other apps and adaptation projects you can process accordingly.

Right-click on the project name and select **Deploy > Deploy to SAP NW Application Server ABAP**:



In the wizard process the following steps.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

1. Select the system. The source system of your adaptation project is selected by default.
2. Click **Next**.
3. Enter the name of the development package to which the change shall be deployed. (Please create a new development package in Z-Namespace in the target SAP system if necessary).
4. Click Next.
5. Enter a transport request number. (Create a transport request on the target SAP system beforehand).
6. Click **Finish**.

After you have made further modifications on your project in Business Application Studio, you may deploy the modified project again. Then the data on the target system will be updated accordingly.

If the release of the target SAP system is sufficiently new, then you can browse to the deployed artifacts in transaction SE80.

1. Start transaction SE80
2. Browse to your package in Z-Namespace.
3. The new artifacts are listed under Application variants and Layered repository contents.

For older releases, check older SDK guides of Extended ECM with hints how to browse for the Layered repository contents

The transport request may be released and imported to your productive environment if tested OK.

9.5.7 Testing display of business workspaces

You have the following options to test the adaptation project:

- Testing the changes in preview mode of the SAPUI5 visual editor. This is described in this section and sub sections.
- Testing the resulting app on your on-premise environment after deployment. This is what you should always do at the end. You also can do this iteratively during development, but then you need to repeat the deployment of the app before testing again. See section **9.5.6** about deployment of the app and section **9.5.8** about accessing the custom app on your on-premise Fiori Launchpad.

9.5.7.1 Requirements for Business Application Studio tests

You need to do the obvious preparations for OpenText Content Management, like:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- You have a working OpenText Content Management configuration to access your business workspace and you have created a business workspace, which matches the data record you can browse to in the original Fiori app (In our example a business workspace for a maintenance order, business object type BUS2007).
- The SAPUI5 app (component) for OpenText business workspaces is installed and active. The related oData service is active.

Furthermore, you will need the following settings:

- **Content Server administration > Security Parameters**
  In the area for trusted relationships make sure, that addresses of the cloud platform domain (*.applicationstudio.cloud.sap) under which the Web IDE runs is a trusted referrer.
- The Content Server can be accessed with a secure HTTPS-URL and the SAP backend, which connects to the Content Server use this HTTPS-URL in its connection settings
- If you use the most recent version of the SAPUI5 component for business workspace (otx.ecmlink.businessworkspace_cc) it has the startup parameter UseClassicConfig. The example code presented in **CODE SNIPPET 28 FUNCTION _GETSETTINGSOBJECT()** already takes this into account. And depending on whether the app runs in the Web IDE test environment (that means the origin is in *.dispatcher.hana.ondemand.com) or not, the startup parameter is calculated.

9.5.7.2 Testing in preview mode

You start the SAPUI5 Visual Editor.

The business workspace widget is displayed in the custom object page (sub) section as implemented.

9.5.7.3 Testing as web application

This is documented in the SAP Help at Preview the Adaptation Project:

Right-click on the **manifest.appdescr_variant** file and execute **Adaptation Project > Open Preview**.

This starts the related test page in a new tab in the browser.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.5.8 Accessing the adapted Fiori app in on-premise Fiori Launchpad

Follow the guidance of SAP Help Portal at Deploy the Adaptation Project to the ABAP Repository, it gives you reference to configure the target mapping for the adaptation (Configuring Target Mappings with the Launchpad Designer).

You then call the adapted app with the URL of the new target mapping.

9.5.9 Reusable Code snippets

9.5.9.1 XML fragment code for OpenText Business Workspace

```
<!-- Use stable and unique id's!-->
<core:FragmentDefinition xmlns:core='sap.ui.core'>
        <ObjectPageSection xmlns="sap.uxap" id="otx_xecm_sec" titleUppercase="false"
title="{i18n>OTX_WSP_TITLE}">
                <ObjectPageSubSection xmlns="sap.uxap" id="otx_xecm_subsec">
                        <VBox id="otx_xecm_vbox" xmlns="sap.m">
                                <core:HTML id="otx_cust_html" content=""/>
                                <core:ComponentContainer id="otx_xecm_compcont"
name="otx.ecmlink.businessworkspace_cc" url="/sap/bc/ui5_ui5/otcmfnd/csbws"
                                                settings='{AppMode: false, WidgetStyleClass: "otx_bws_div"}'
        componentCreated=".extension.customer.zxecm.eam.ordmans1.otx.onComponentCreated">
                                </core:ComponentContainer>
                        </VBox>
                </ObjectPageSubSection>
        </ObjectPageSection>
</core:FragmentDefinition>
```

Code Snippet 23 XML code of business workspace for otx.fragment.xml

This code is reusable for different adaptations. But replace the value of componentCreated in light blue background color by the correct function name. Note the extra prefix ".extension.", which needs to be added before the name of the custom controller.

Hint: When adding a new custom controller ("otx.js") with the SAPUI5 Visual Editor it prefills the newly created JavaScript file with code. Search for a line like:

```
return ControllerExtension.extend("customer.zxecm.eam.ordmans1.otx", {
```

Code Snippet 24 Example for name of business workspace custom controller

9.5.9.2 JavaScript code snippets for custom controller

After you added a new custom controller file to the adaptation project, this file is already prefilled with JavaScript code.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
                metadata: {
                        //          // extension can declare the public methods
                        //          // in general methods that starts with "_" are private
                        methods: {
                                //                  publicMethod: {
                                //                          public: true /*default*/ ,
                                //                          final: false /*default*/ ,
                                //                          overrideExecution:
OverrideExecution.Instead /*default*/
                                //                  },
                                //                  finalPublicMethod: {
                                //                          final: true
                                //                  },
                                //                  onMyHook: {
                                //                          public: true /*default*/ ,
                                //                          final: false /*default*/ ,
                                //                          overrideExecution:
OverrideExecution.After
                                //                  },
                                //                  couldBePrivate: {
                                //                          public: false
                                //                  }
                                onComponentCreated: {
                                        public: true
                                },

                                _setDivCssHeight: {
                                        public: false
                                },

                                _getSettingsObject: {
                                        public: false
                                },

                                _getObjectWithKey: {
                                        public: false
                                }
                        }
                },
```

Code Snippet 25 declaration of new methods

In Code Snippet 25 declaration of new methods there are lines with blue background color. Find these lines in the already existing code and uncomment them. Then insert the four new method declarations (the lines with green background color.)

After that insert the implementation code of function onComponentCreated() as represented in Code Snippet 26 Function onComponentCreated()

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
/**
 * Callback for event componentCreated
 * @param {object} oEvent  event context when component is created
 */
onComponentCreated: function (oEvent) {
	/*
	 * Inserts style class inline in HTML control to set height
	 * of DIV element which contains the business workspace widget
	 * from OpenText Content Server
	 *
	 * This function does no harm for business documents
	 */
	this._setDivCssHeight("otx_cust_html", "otx_bws_div", "650px");

	/*
	 * Define SystemOrigin (that is the system alias)
	 * Set UseClassicConfig flag (only relevant for business workspace, will be ignored by business documents)
	 */
	var oSettings = this._getSettingsObject();

	/*
	 *  Use refresh function of OpenText component.
	 *  Trigger it when the bound runtime data of the original app is available
	 */

	var oRefreshComponent = oEvent.getParameter("component");
	var that = this;

	this.getView().getParent().getModel().attachRequestCompleted(function () {
		var oObject = that._getObjectWithKey();
		if (oObject && oObject.SapObject && oObject.ObjectId) {
			oRefreshComponent.refresh(oObject.SapObject, oObject.ObjectId, oSettings);
		}
	});
},
```

Code Snippet 26 Function onComponentCreated()

After that insert the code of the 3 other functions as represented in Code Snippet 27 Function _setDivCssHeight(), Code Snippet 28 Function _getSettingsObject() and Code Snippet 29 Function _getObjectWithKey().

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
/**
 * Sets style class definition in a custom HTML control once
 * --- relevant for Extended ECM business workspace ---
 * @param {string} sHTMLId  ID of the custom HTML control in context of the XML
fragment and view
 * @param {string} sCSSClassName name of the CSS class as used in component
setting WidgetStyleClass
 * @param {string} sHeight  text string representation for the height
 */
_setDivCssHeight: function (sHTMLId, sCSSClassName, sHeight) {
        var oHTML = this.byId(sHTMLId);
        if (oHTML && oHTML.getContent) {
                if (oHTML.getContent() === "") {
                        oHTML.setContent("<style type='text/css'>." +
                                sCSSClassName + " { height:" +
                                sHeight + " !important; }</style>");
                }
        }
},
```

Code Snippet 27 Function _setDivCssHeight()

```
/**
 *  Calculates settings for refresh of OpenText UI component
 *    UseClassicConfig : relevant for calculation of OpenText Content Server URL and
will be
 *                       ignored by refresh of business documents
 *    SystemOrigin:      system alias; modify the calculation if you need a specific value
 *                       Important: business documents and business workspace can only be
retrieved
 *                       from 1 system alias at a time
 */
_getSettingsObject: function () {
        var oSettingsObject = {};

        var oRegExp = RegExp(".dispatcher.hana.ondemand.com");
        if (oRegExp.test(window.location.origin)) {
                oSettingsObject.UseClassicConfig = true;
        }

        /*
         * system alias; use a different one if necessary
         */
        oSettingsObject.SystemOrigin = "LOCAL";
        return oSettingsObject;
},
```

Code Snippet 28 Function _getSettingsObject()

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
                        /**
                         * Calculates the SAP BOR object type (SapObject) and SAP BOR key (ObjectId)
                         *   THIS CODE IS DEPENDENT on adapted original app:
                         *
                         *   This code needs to be adjusted to the original app which is adapted.
                         *   In the original app check for a controller file Attachments.controller.js
                         *   probably in a sub folder ext/controller
                         *   and then look at the code in the onInit function.
                         *   This should give you an idea, which runtime data is used to
                         *   initialize the DMS related attachments control of SAP
                         *   The name of the object type is most often not the same as the SAP BOR object
type
                         *   but the ID or key value is often the same
                         */
                        _getObjectWithKey: function () {
                                var oObject = {};

                                /*
                                 * SAP BOR object type for Maintenance Orders
                                 */
                                oObject.SapObject = "BUS2007";

                                /*
                                 * Now do a runtime data retrieval similar to that of the original app
                                 */
                                var sProperty1 = "MaintOrderWithLeadingZeros";
                                var sProperty2 = "ActiveMaintenanceOrder";

                                if (this && this.getView && this.getView().getBindingContext) {
                                        oObject.ObjectId =
this.getView().getBindingContext().getProperty(sProperty1);
                                        if (!oObject.ObjectId) {
                                                oObject.ObjectId =
this.getView().getBindingContext().getProperty(sProperty2);
                                        }
                                }

                                return oObject;
                        }
```

Code Snippet 29 Function _getObjectWithKey()

The code lines with blue background color need to be adjusted as described in the comments or as explained in more details in the previous sections.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.6 Extending own SAP Fiori Elements-based app with app extensions during creation

9.6.1 General Information and Requirements

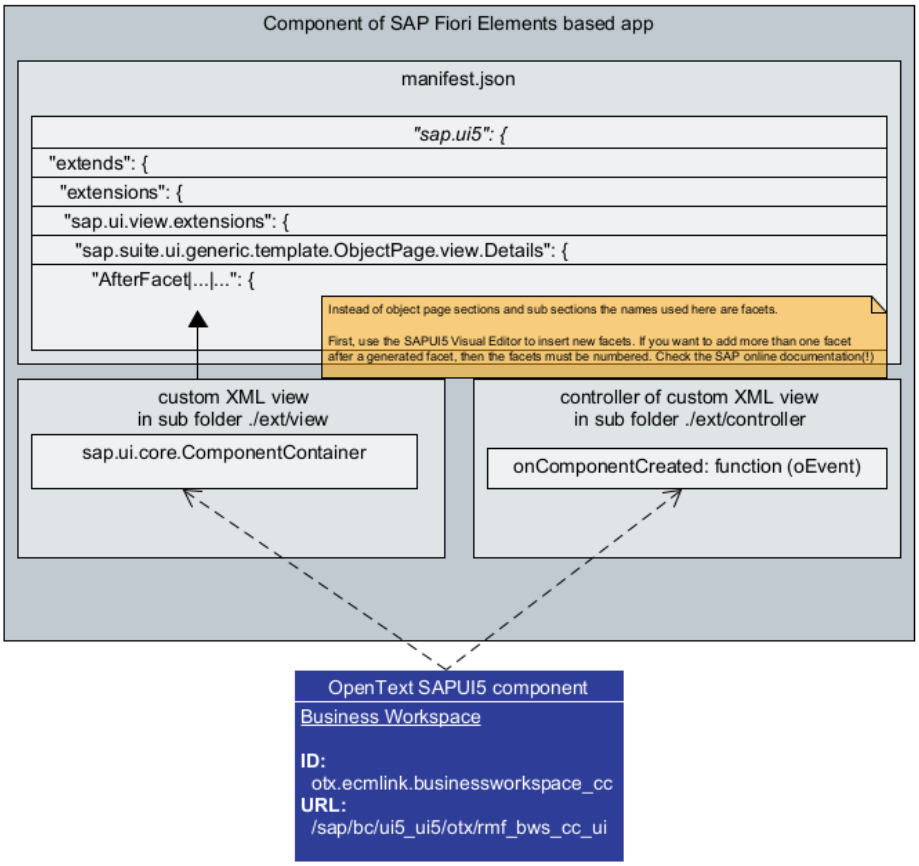


Figure 54 Own SAP Fiori Elements based app extended during creation

SAP Fiori Elements based apps are SAPUI5 apps, which are created with the SAP Fiori Elements framework. The SAP Web IDE allows you to create your own SAP Fiori Elements based apps by using the corresponding templates.

See SAP online documentation

> Introduction to SAP Fiori Elements
> or Developing Apps with SAP Fiori Elements

App extensions is one of two options to extend SAP Fiori Elements-based apps. They are done by developers during the creation of an SAP Fiori elements-based app, using framework extension points provided by SAP Fiori elements.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

OpenText rather recommends the option presented in **EXTENDING SAP FIORI ELEMENTS-BASED APP WITH ADAPTATION**, if you want to integrate an OpenText control into an SAP Fiori Elements-based app.

Note the following:

- This description is still based on Web IDE but should be applicable with Business Application Studio, accordingly.
- Use a recent version of the SAP Web IDE Full-Stack to create and extend an SAP Fiori Elements based app
  See also section **9.12 BASIC STEPS USING THE SAP CLOUD CONNECTOR AND THE SAP WEB IDE ON THE SAP CLOUD PLATFORM**.
- OpenText used a co-deployment on an SAP S/4HANA 1809 server with SAPUI5 1.60.12
  which behaved stable regarding the general SAPUI5 behavior and the SAPUI5 features specifically used by the OpenText components
- Your own custom app is deployed as a new app on your SAP Gateway system
  - Your custom app has an own BSP node
  - The app extensions are part of your custom app
- In SAP Web IDE you can create new extensions which are compatible with your original app

Important: Please refer to the most recent SAP online documentation like the SAPUI5 SDK regarding restrictions.

Here is a list of advantages and disadvantages of the procedure.

Advantages:

+ For the same kind of integration, you can reuse the same implementation steps and code snippets
+ The wizard of the SAP Web IDE restricts the areas in which extensions are allowed
+ No code replacement or overrides are required to insert new controls
+ Controller code to hook into the original app is short and no invasive code is required

Disadvantages:

- The extensions are part of your app. In the case of error, you cannot easily switch between an app version with or without the OpenText extensions.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- You require minimum SAPUI5 version 1.63 to add multiple sections at extension points; then you must modify manually to add extra keys

9.6.2 Quick procedure

Follow the steps in this section for a quick procedure.
If you are reading this the first time, skip this section and continue reading with the next section, **9.6.3 CREATING A NEW CUSTOM APP BASED ON FIORI ELEMENTS**.

1. In SAP Web IDE, you create your own SAP Fiori Elements-based app with a template containing an object page, typically List Report Application. You may select the option to use flexible column layout.
2. Create a new Object Page Extension and extend the Facet
    a. Select the Facet after which you want to insert a new facet
    b. Choose view as view type. This also will create a related controller file.
3. In the XML view replace the placeholder text control by the component container control
    a. For a facet (section) of the object page (in case of flexible columns it is the first object page column or the second overall column). See **CODE SNIPPET 32 XML FOR BUSINESS WORKSPACE IN FIRST OBJECT PAGE COLUMN**
    b. For a facet (section) of the second object page column in the case of a flexible column layout. See **CODE SNIPPET 33 XML CODE FOR BUSINESS WORKSPACE IN SECOND OBJECT PAGE COLUMN**
4. Insert new JavaScript code in the JS controller(s):
    a. Corresponding to 3.a see section **9.6.9.2 JAVASCRIPT CODE FOR CONTROLLER FOR FIRST COLUMN OBJECT PAGE**.
    b. Corresponding to 3.b see section **9.6.9.3 JAVASCRIPT CODE FOR CONTROLLER OF SECOND COLUMN OBJECT PAGE**.
    But modify the SAP BOR object type and key calculation according to the runtime data of your app. See next steps.
5. In the custom controller in function onComponentCreated() make sure, that function _setDivCssHeight() sets the right height and that the name of the HTML control and CSS style class are ok.
6. In function _getSettingsObject() make sure, that the SystemOrigin (that is the system alias) is ok.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

7. Adjust the code in function _getObjectWithKey() according to your app:
    a. Set the right SAP BOR object type for value oObject.SapObject
    b. Make sure, that oObject.ObjectId is filled with the correct key/object ID.
8. Deploy the project to your SAP on-premise system

Define a target mapping and tile, which call the app variant of your original app.

9.6.3 Creating a new custom app based on Fiori Elements

This section and the following sections give a detailed description of how to extend your own custom app based on SAP Fiori Elements at creation time. Please note, that this document does not focus on how to create your own CDS views or how to reuse already existing CDS views. For this, please refer to the SAP online documentation.

Nevertheless, the example code presented here will be based on two similar example apps, which are based on OData service SD_F1814_SO_FS_SRV. This OData service is part of the deployment of Sales Order (S/4HANA) – App ID: F1814 (see also SAP Fiori Apps Library)

If you or an administrator has done the setup for your subaccount and services in the subaccount, you may start in your browser from

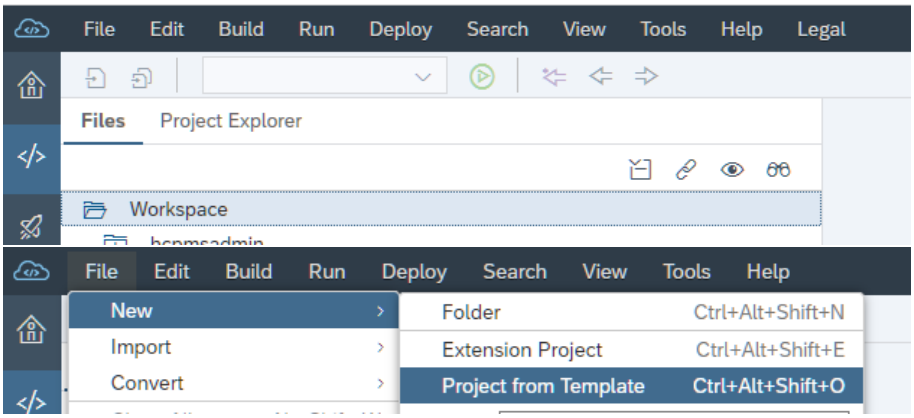
http://hana.ondemand.com

Then click on the tile of your global account. Then click on the navigation node for subaccounts and click on the tile of your subaccount. Then click on the navigation node Services. Then click on the tile SAP Web IDE Full-Stack. Finally, you click on the link Go to Service in the area *Take Action*.

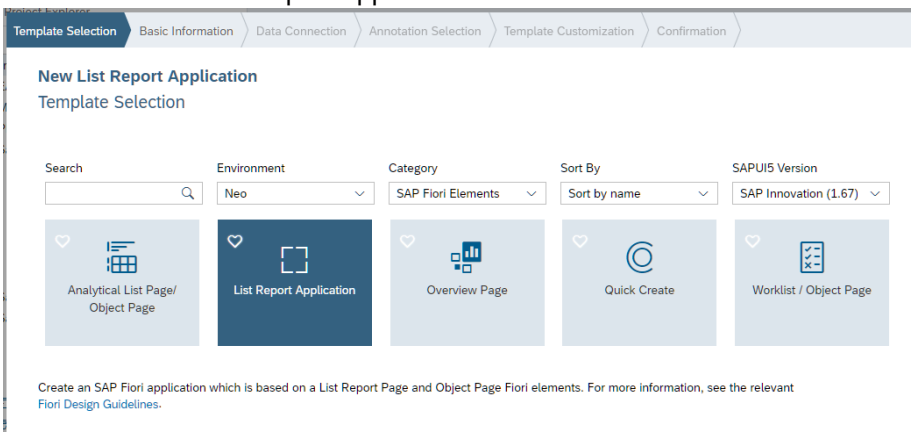
You can create a shortcut to the Full Stack Web IDE of your subaccount. If the technical name of the subaccount is, for example abcde12345, it will be https://webidecp-abcde12345.dispatcher.hana.ondemand.com.

1. At your project workspace of the Web IDE create a new project from the menu **File > New > Project From Template.**

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.



2. In tab Template Selection filter for the Category “SAP Fiori Elements”. And then select “List Report Application”.



You can also select other templates and most of them also contain an object page in their pattern. But the focus of this description is a list app with one or more object pages for details.

3. Click on **Next**. In the tab Basic Information enter reasonable data: Please note, that in the case of this template, the ID of the custom app is merged from the value of the namespace followed by a dot and the value of the project name. In our example the values are:

    a. Project Name: SalesOrderBrowser

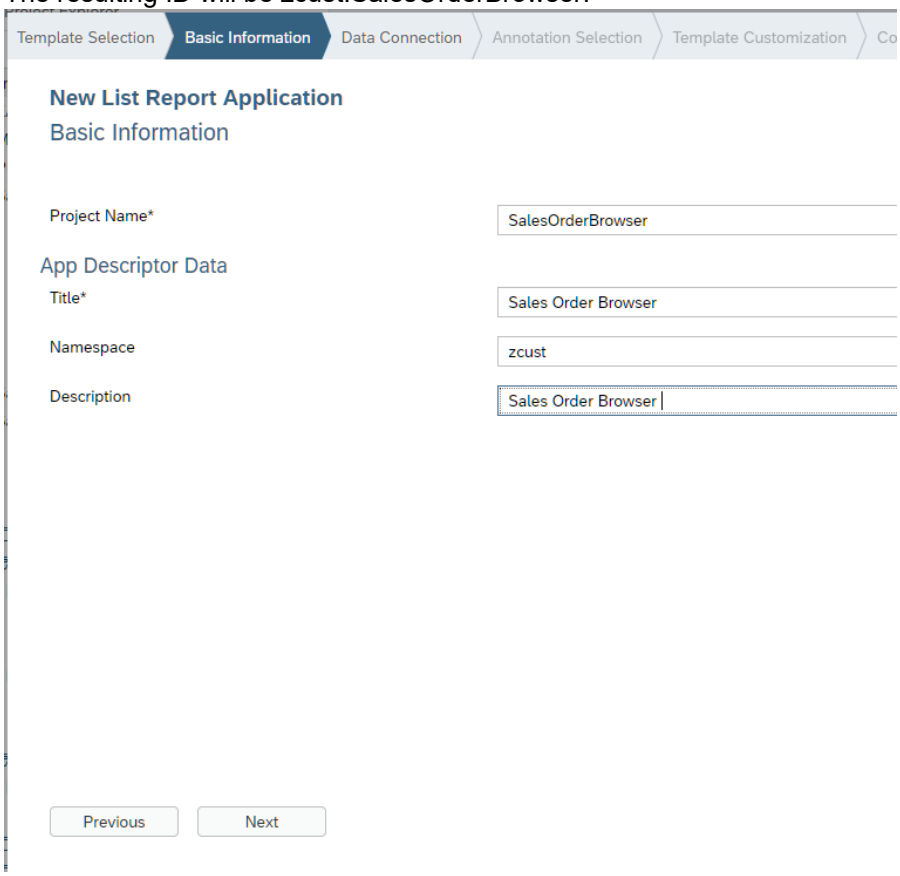
    b. Title: Sales Order Browser

    c. Namespace: zcust

    d. Description: Sales Order Browser

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

e. The resulting ID will be zcust.SalesOrderBrowser.



4. Click **Next**.

5. In the data connection select the connection to the remote SAP on-premise Gateway/Front-End server and select the OData service. In this example, the OData service SD_F1814_SO_FS_SRV is selected.

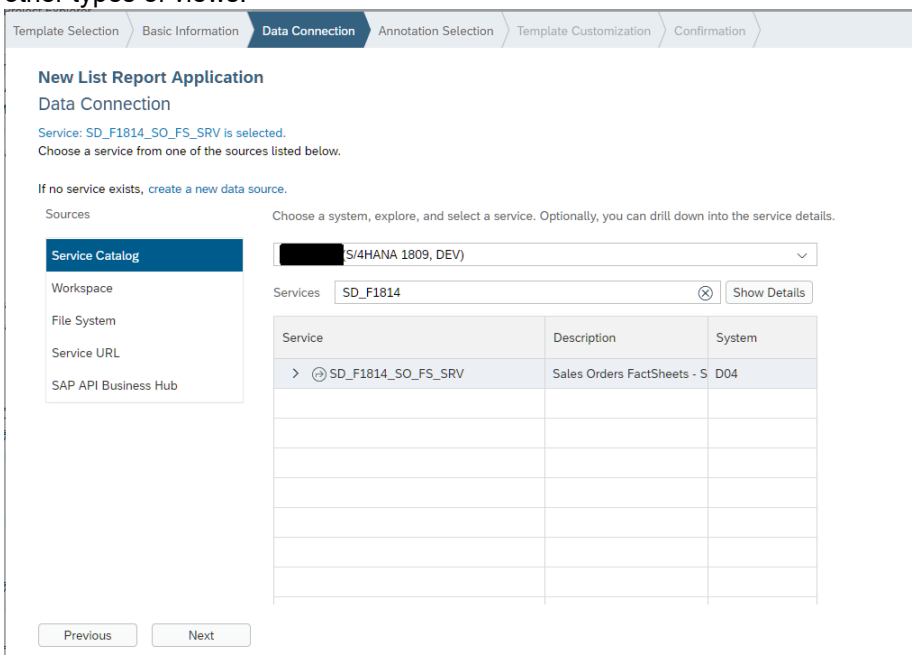
   Important: This OData service is only taken for simplicity and demonstration purposes, because the Fiori App with ID = F1814 is also SAP Fiori Elements based. Consider the following:

   a. If you use CDS views delivered from SAP, use released views, if possible

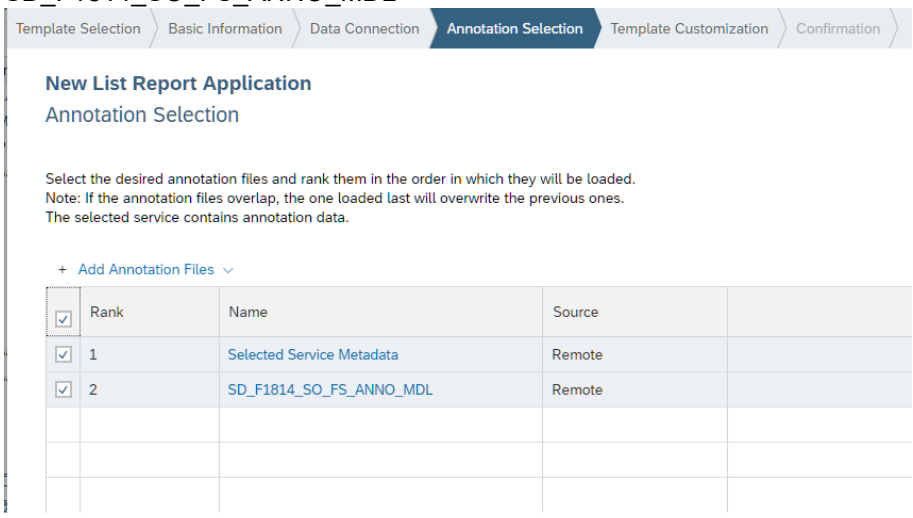
   b. In the case of consumption views (CDS views with annotation `@VDM.viewType: #CONSUMPTION`) note, that they are often adapted quite specifically and may change more often than

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

other types of views.



6. Click **Next**.
7. In tab Annotation Selection select the annotations. In our example select
    a. Selected Service Metadata
    b. SD_F1814_SO_FS_ANNO_MDL



8. Click **Next**.
9. In tab Template Customization select the data binding. It depends on the Entity sets and underlying CDS views within your OData service.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

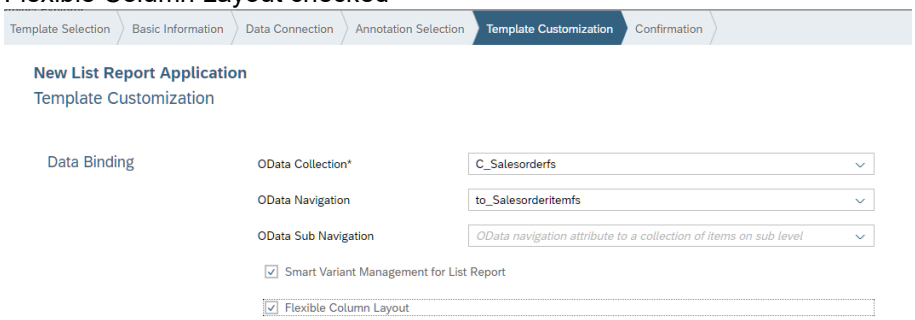
Within the app you can choose one navigation and even a level of sub navigation. In our example the following selections are done:

a. OData Collection: C_Salesorderfs

b. OData Navigation: to_Salesorderitemfs

c. Smart Variant Management for List Report: checked

d. Flexible Column Layout checked



10. Click **Next**.
11. Click **Finish**.

Things to note about the values in tab Template Customization

In this example, the OData collection on top level is C_Salesorderfs, which corresponds to a CDS view of the consumption type. The OData Navigation follows the path to_Salesorderitemfs which corresponds to the association _Salesorderitemfs from CDS view C_Salesorderfs to CDS view C_Salesorderitemfs. As the names of the CDS views already suggests, the main list of the app will select from the entities of sales orders and for each sales order a navigation to the list of related sales order items will be offered.

In this example the checkbox “Flexible Column Layout” is checked. This way the app will be created using the Flexible Column Layout. Specifically, in this example, if the user clicks on a sales order in the list, a new column is opened on the right side which contains the object page with details of the sales order. If the user then clicks on a sales order item, a third column is opened at the right side, which then shows the details of the sales order item.

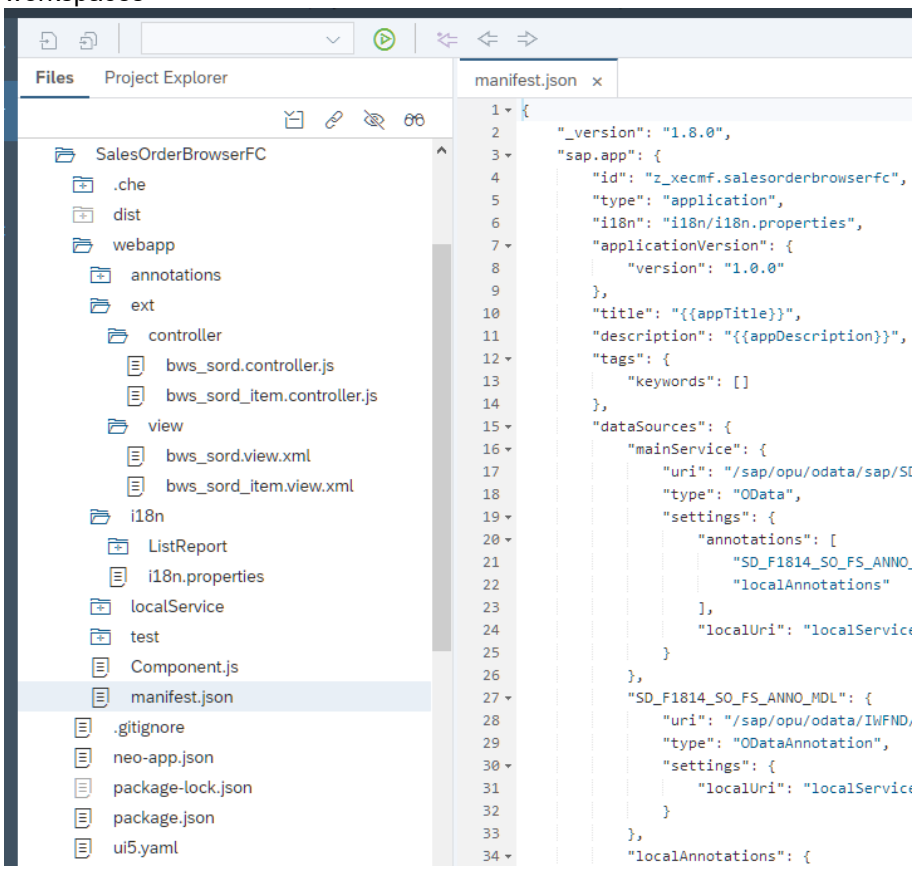
At the time being, the SAPUI5 SDK indicates, that the SAPUI5 Visual Editor does not support the flexible column layout. (This information may change in future SAPUI5 versions). Therefore, this special flavor is also an interesting example to use extension at creation time in your own app.

Important: The current restriction is not to use more than 1 business workspace control in one app, even if the flexible column layout would invite to do so.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Now suppose that for this example you created a new project:

- SalesOrderBrowserFC
    - The sales order browser based on SD_F1814_SO_FS_SRV which will be extended to also show OpenText business workspaces



The Web IDE has created all initial files and folders, for example the webapp/manifest.json.

Some of the files you already can see in the screenshots above, will be created in steps described in the next sections.

9.6.4 Adding relative path to ui5_ui5 node into neo-app.json file

It is not clear whether this is a bug or is missing on purpose. But to test the integrated OpenText components in the SAP Web IDE, add the settings of the ui5_ui5 ICF node into the neo-app.json file.

In your new project folder browse to neo-app.json and add the content of Code Snippet 30 New route for ui5_ui5 as the first entry of routes.

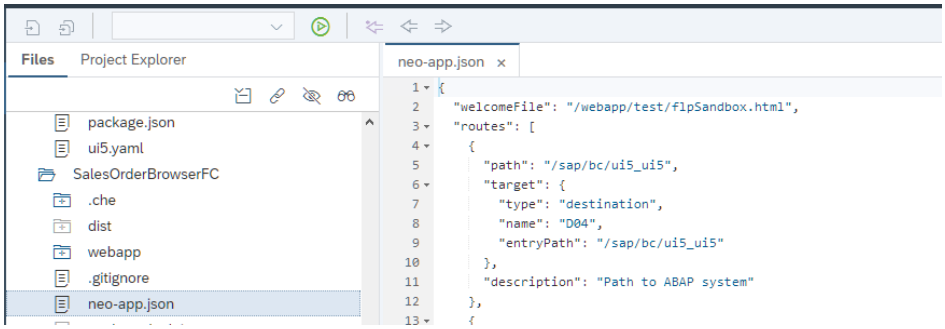
Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Change the name of the destination (indicated here with green background color) to the destination you are using to connect to your SAP frontend.

```
{
  "path": "/sap/bc/ui5_ui5",
  "target": {
    "type": "destination",
    "name": "D04",
    "entryPath": "/sap/bc/ui5_ui5"
  },
  "description": "Path to ABAP system"
},
```

Code Snippet 30 New route for ui5_ui5

The resulting file may then look like:

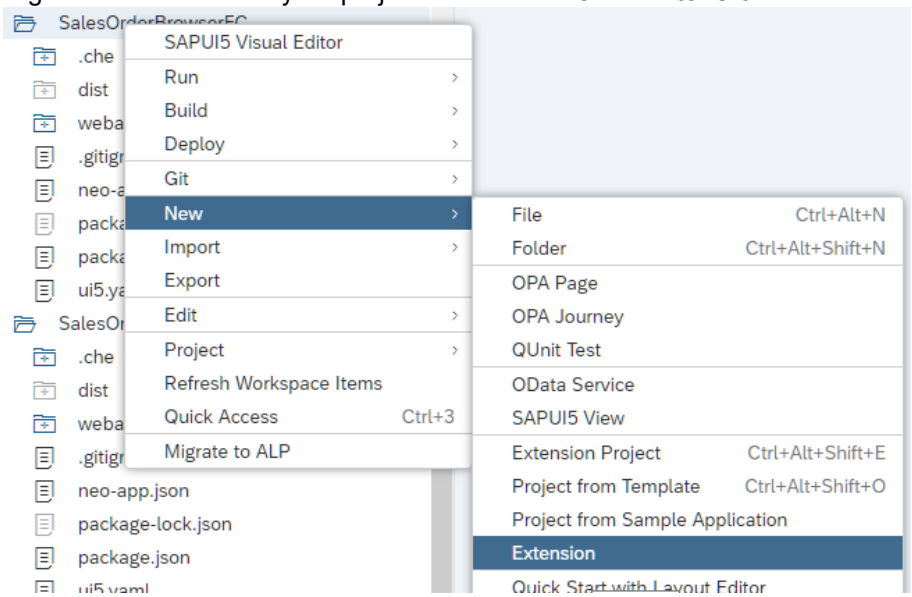


9.6.5 Integrating the OpenText controls

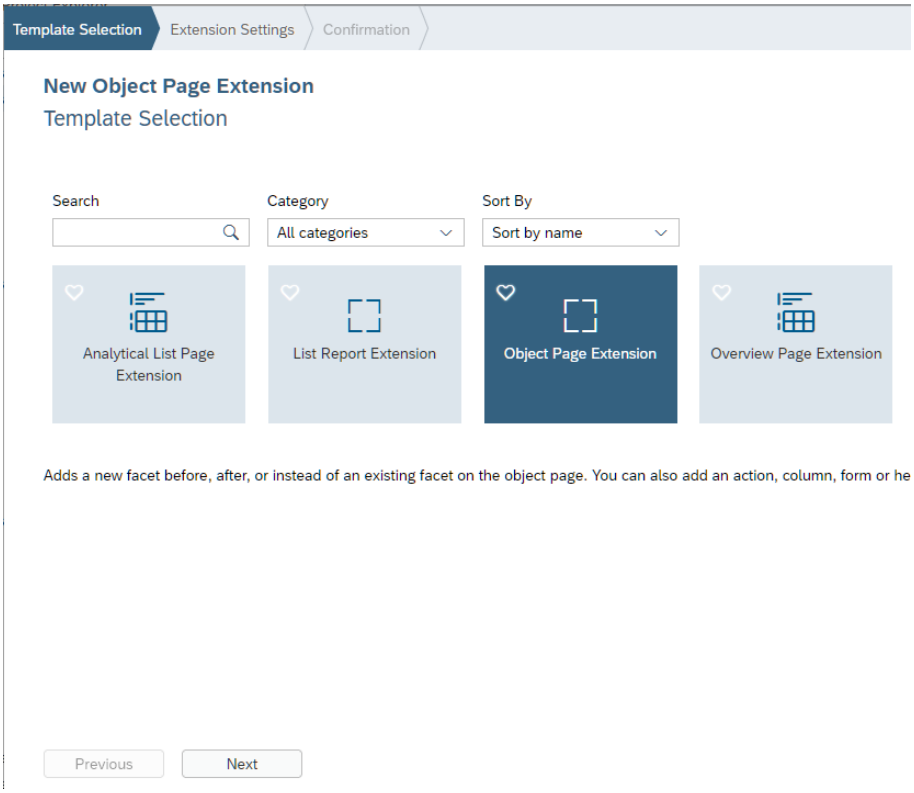
9.6.5.1 Using the extension templates to add a new XML view and controller

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

1. Right-click the name of your project and select **New > Extension**.



2. In tab Template Selection select the template for Object Page Extension.



Note: Based on the previous descriptions, the example was created using a list report template. Therefore, you can either select the List Report Extension or the Object Page extension as compatible choices.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

3. Click **Next**.
4. In the tab Extension Settings select one of the following two settings

    a. Business Workspace for Sales Order

        i. What do you want to extend? Select Radio button Facet.

        ii. Page (Entity Set): C_Salesorderfs

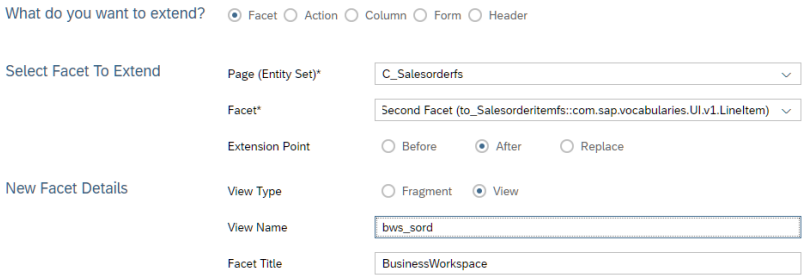
        iii. Facet: Second Facet (to_Salesorderitemfs::com.sap.vocabularies.UI.v1.LineItem)

        iv. Extension Point: After

        v. View Type: View

        vi. View Name bws_sord

        vii. Facet Title BusinessWorkspace



    b. Business Workspace for Sales Order Item

        i. What do you want to extend? Select Radio button Facet.

        ii. Page (Entity Set): C_Salesorderitemfs

        iii. Facet: General Information (GeneralInformation)

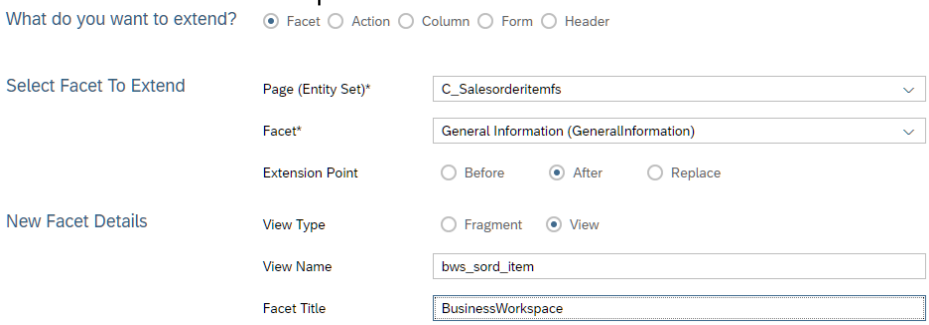
        iv. Extension Point: After

        v. View Type: View

        vi. View Name bws_sord_item

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

vii. Facet Title BusinessWorkspace



5. Only use one setting at a time.
6. Click **Next**.
7. Click **Finish**.

The result is that the SAP Web IDE has added two new files in your project, for example

- webapp/ext/controller/bws_sord.controller.js
- webapp/ext/view/bws_sord.view.xml

Remember, that you are restricted to use only one control for each type at a time. Hence, the four (equals two times two) options for extension settings in the steps above.

9.6.5.2 Using the code editor to add the control into XML

Your app for business workspace contains one new XML file:

- Either: bws_sord.view.xml
- Or: bws_sord_item.view.xml

Execute the following steps for one of the files

1. Double-click on the file. This opens the file in the code editor
2. Remove the XML-Tag

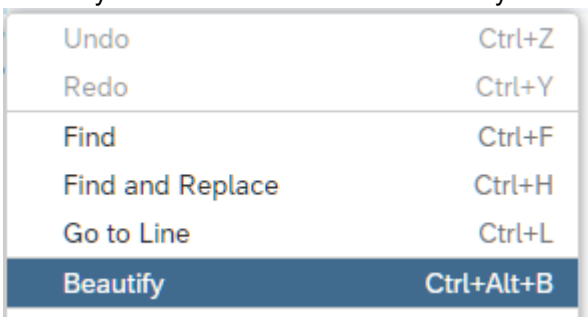
```
<Text text="I'm a view added as a dynamic facet extensionpoint"/>
```

3. Replace it by the following new XML code
   a. bws_sord.view.xml: **CODE SNIPPET 32 XML FOR BUSINESS WORKSPACE IN FIRST OBJECT PAGE COLUMN**

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

b. bws_sord_item.view.xml: **CODE SNIPPET 33 XML CODE FOR BUSINESS WORKSPACE IN SECOND OBJECT PAGE COLUMN**

4. You may use context menu action Beautify



5. Save the changed file.

Continue with the next section to apply the corresponding controller code.

9.6.5.3 Editing the controller code

The business workspace app also has one new controller file

- Either: bws_sord.controller.js
- Or: bws_sord_item.controller.js

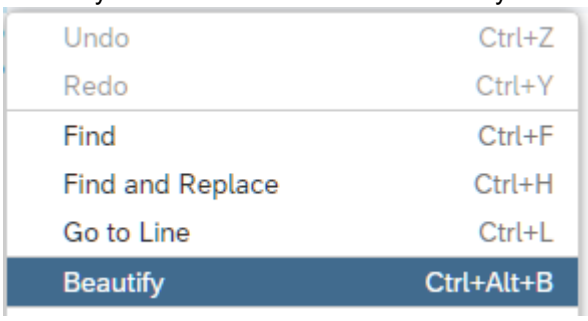
The SAP Web IDE has prefilled the file with definition code. Execute the following steps for one of the files:

1. Double-click on the file. This opens the file in the code editor
2. Insert the JavaScript code

   a. bws_sord.controller.js: Follow instructions of **SECTION 9.6.9.2 JAVASCRIPT CODE FOR CONTROLLER FOR FIRST COLUMN OBJECT PAGE**

   b. bws_sord_item.controller.js: Follow instructions of section **9.6.9.3 JAVASCRIPT CODE FOR CONTROLLER OF SECOND COLUMN OBJECT PAGE**

3. You may use context menu action Beautify



Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

4. Save the changed file.

9.6.5.4 Editing the i18n.properties file

Your custom app for business workspace has a manifest.json file, which defines a couple of i18n resources in the model section. As the example app is created with the flexible column layout, the only relevant i18n.properties file is ./webapp/i18n/i18n.properties or i18n/i18n.properties

For the app with with business workspaces add *Code Snippet 31 i18n text properties for business workspace* to the content of the i18n.properties file.

```
#XTIT: Application name
appTitle=Sales Order Browser

#YDES: Application description
appDescription=list template of SAP Fiori Elements; flexible column layout

#Texts used in local/annotations.xml

# XTIT: Title of a form or form group. Remove if not needed in local/annotations.xml
@GeneralInfoFacetLabel=General Information

# XTIT: Title of a facet within an object page if not needed in local/annotations.xml
@SecondFacetLabel=Sales Order Items

# XTIT: Title of OpenText business workspace facet
BusinessWorkspace=Business Workspace
```

Code Snippet 31 i18n text properties for business workspace

This has the effect that the second facet with the sales order items gets a proper title “Sales Order Items”.

And the titles of the custom sections are also defined correctly.

Remember again: only one control of each type is supported.

9.6.5.5 Adding further localizations to the project and previewing them

It is pretty the same procedure as described in section **9.5.4.6 ADDING FURTHER LOCALIZATIONS TO THE PROJECT AND PREVIEWING THEM**

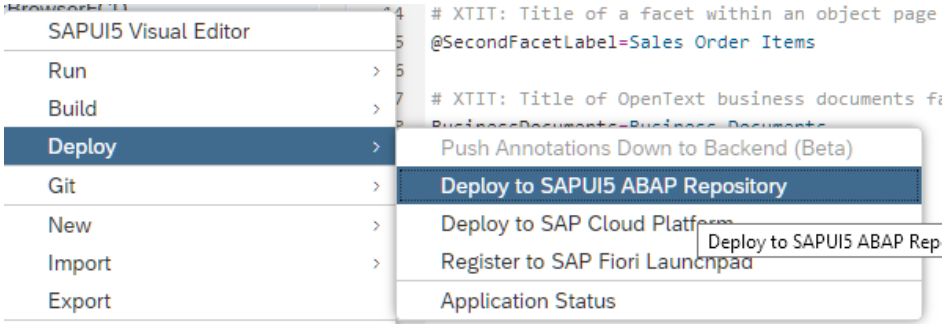
You make a copy of the i18n.properties file you modified in the previous section and the name of the copy is changed to the file name for the specific target language, for example i18n_de.properties for German. Then you translate the text labels into the new file.

9.6.6 Deploying the project to the SAP Gateway on premise

The steps and screenshots in this section can be processed accordingly with other projects as well.

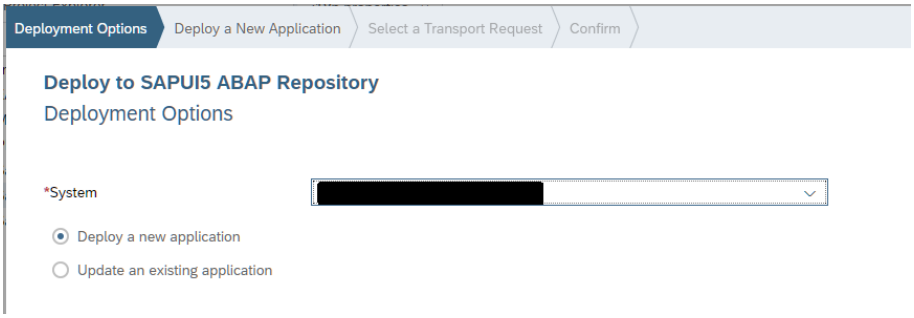
Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Right-click the project name and select **Deploy > Deploy to SAPUI5 ABAP Repository**:



If this is done the first time for the project, you must go through the following steps:

1. Select the target SAP Server to deploy to, this should be a development system. Also, select deploy to a new application. Then click **Next**.

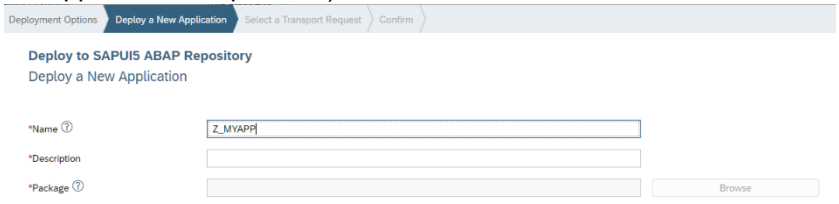


2. In the tab Deploy a New Application you define the name of the app, description, and the package.

    a. Name is the name of the Internet Connection Framework (ICF) node. If it is in the Z-/Y-namespace this node will be deployed under node /sap/bc/ui5_ui5/sap/z…

    b. Description: Enter a useful description of the app

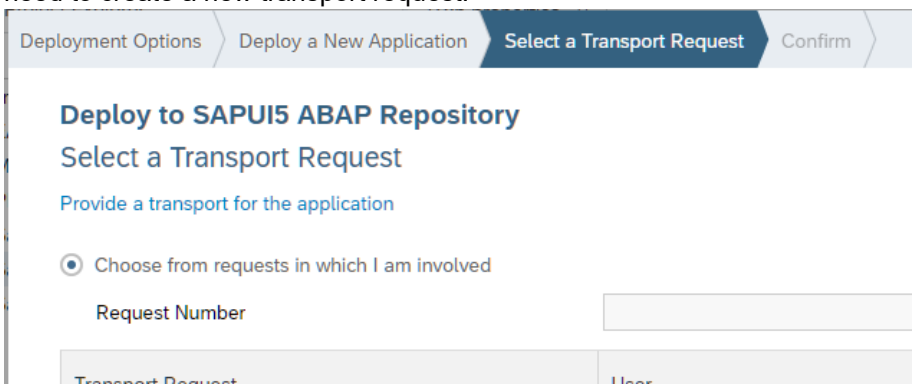
    c. Package needs to be an existing package with matching namespace for the BSP app or the $TMP package (but then the app is not transportable).



3. Click **Next**.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

4. The next tab is Select a Transport Request if you selected a transportable package. Select an existing transport request or you need to create a new transport request.



5. Click **Next**.
6. Click **Finish**.

Here are the steps in the case of updating an existing app.

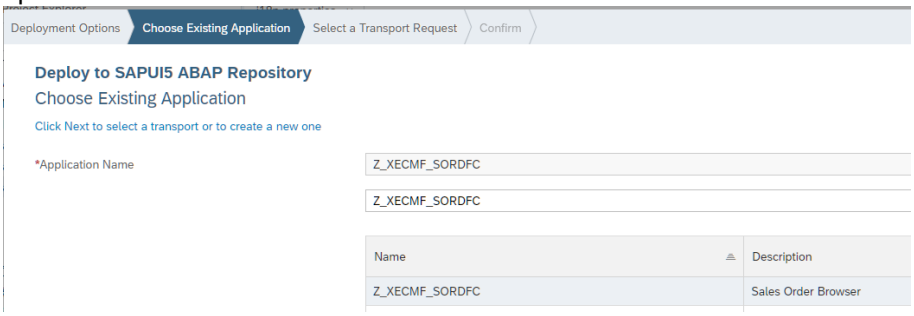
1. You select the deployment option Update an existing application.



2. Click **Next**.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

3. In tab Choose Existing Application select an existing application to update.



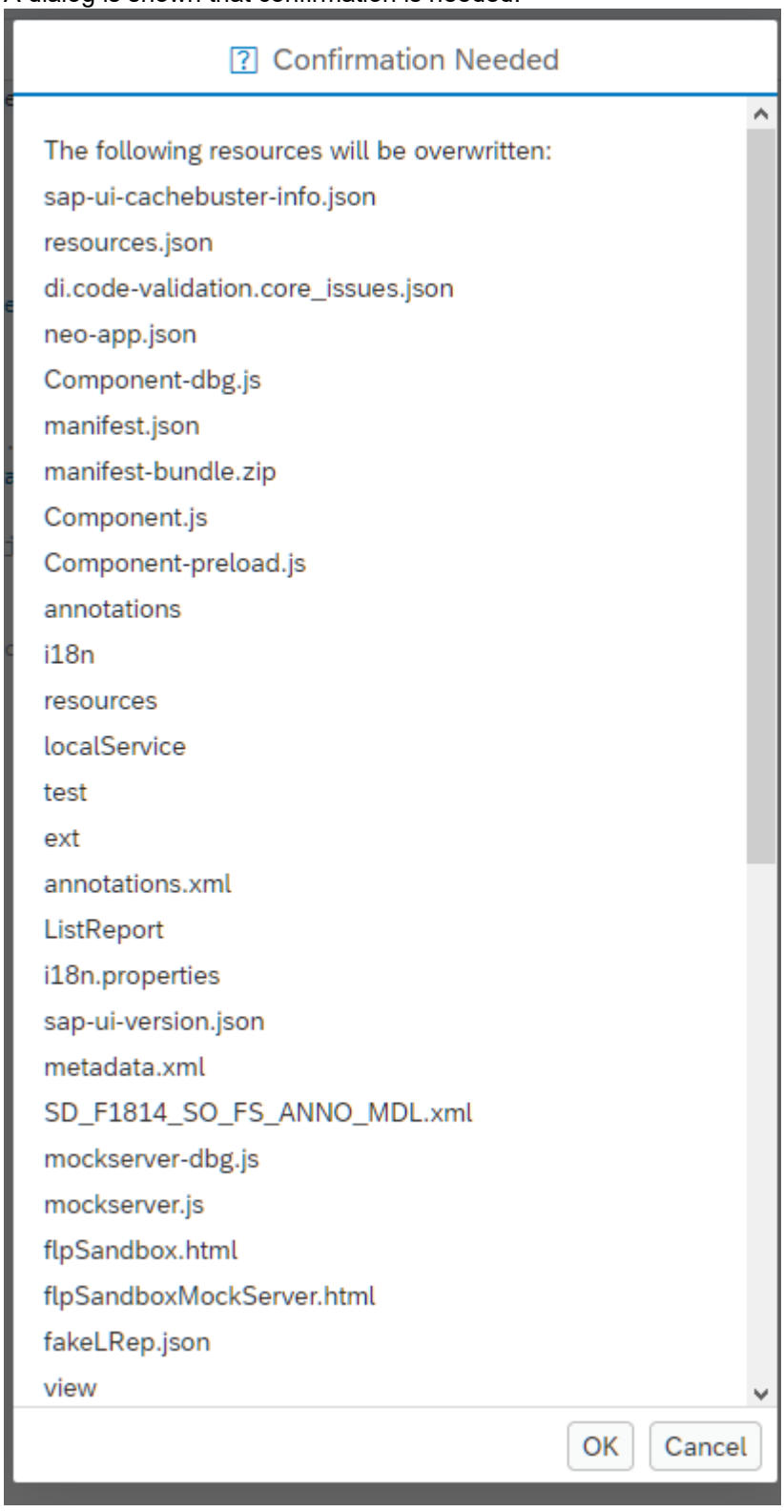
4. Click **Next**.
5. In tab Select a Transport Request choose an existing transport or create a new one.



6. Click **Next**.
7. Click **Finish**.
8. After Confirmation, the deploy action may take a while.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9. A dialog is shown that confirmation is needed.



Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

10. Click **OK**.
11. Wait some time until the SAP Web IDE confirms that the deployment is processed.

9.6.7 Testing display of business workspaces

You have the following options to test the adaptation project:

- Testing the changes in the Web IDE Full Stack either in preview mode of the SAPUI5 visual editor or in a test run as a web application. This is described in this section and sub sections
- Testing the resulting app on your on-premise environment after deployment. This is what you should always do at the end. You also can do this iteratively during development, but then you need to repeat the deployment of the app before testing again. See section **9.6.6** about deployment of the app and section **9.6.8** about accessing the custom app on your on-premise Fiori Launchpad.

9.6.7.1 Requirements for Web IDE tests

You need to do the obvious preparations for OpenText Content Management, like:

- You have a working OpenText Content Management configuration to access your business workspace and you have created a business workspace, which matches the data record you can browse to in the original Fiori app (In our example a business workspace for a sales order, business object type BUS2032, and for a material, business object type BUS1001006).
- The SAPUI5 app (component) for OpenText business workspaces is installed and active. The related oData service is active.

Furthermore, you will need these settings:

- Content Server administration > Security Parameters
  In the area for trusted relationships make sure, that addresses of the cloud platform domain (*.dispatcher.hana.ondemand.com) under which the Web IDE runs is a trusted referrer.
- The Content Server can be accessed with a secure HTTPS-URL and the SAP backend, which connects to the Content Server use this HTTPS-URL in its connection settings
- If you use the most recent version of the SAPUI5 component for business workspace (otx.ecmlink.businessworkspace_cc) it has the startup parameter UseClassicConfig. The example code presented in

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

the code snippets already take this into account. And depending on whether the app runs in the Web IDE test environment (that means the origin is in *.dispatcher.hana.ondemand.com) or not, the startup parameter is calculated.

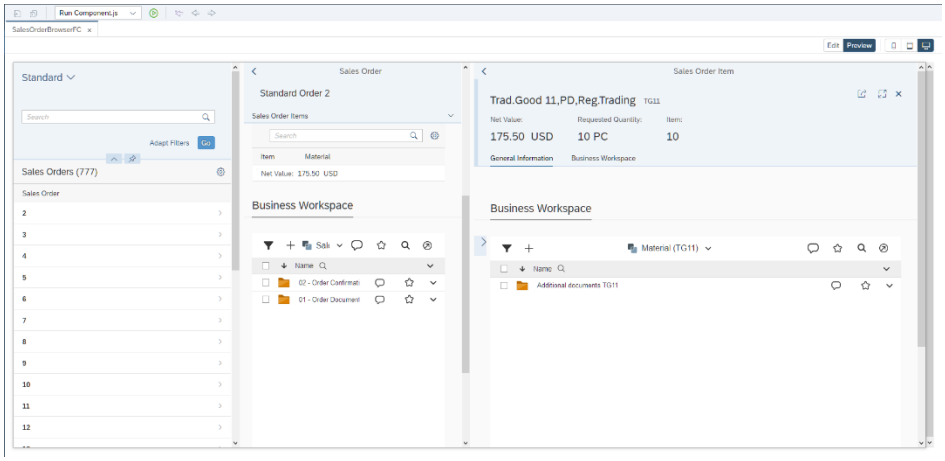
The test pages of the Web IDE do not enforce any special CORS or same origin policies. But they run under an HTTPS-URL. Therefore, other scripts like the scripts on the Content Server must be called via a secure URL. Otherwise, the browser may block unsecure content from loading.

But loading from the original on-premise HTTPS-URL of the Content Server is not an issue, if your browser can access this URL.

9.6.7.2 Testing in preview mode

You start the SAPUI5 Visual Editor.

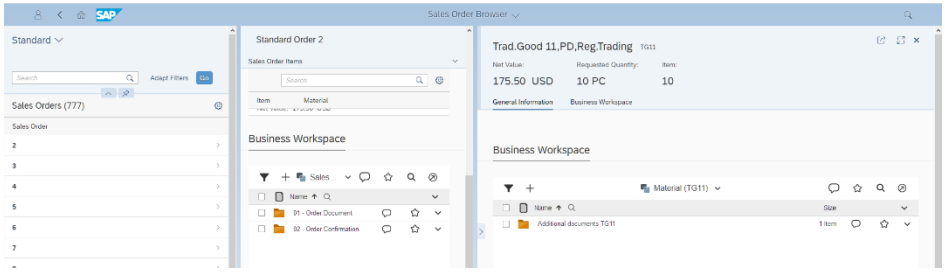
The business workspace widget is displayed in the custom object page (sub) section as implemented:



9.6.7.3 Testing as web application

Right-click on the project node and select **Run > Run as SAP Fiori Launchpad Sandbox**.

This starts the related test page in a new tab in the browser.



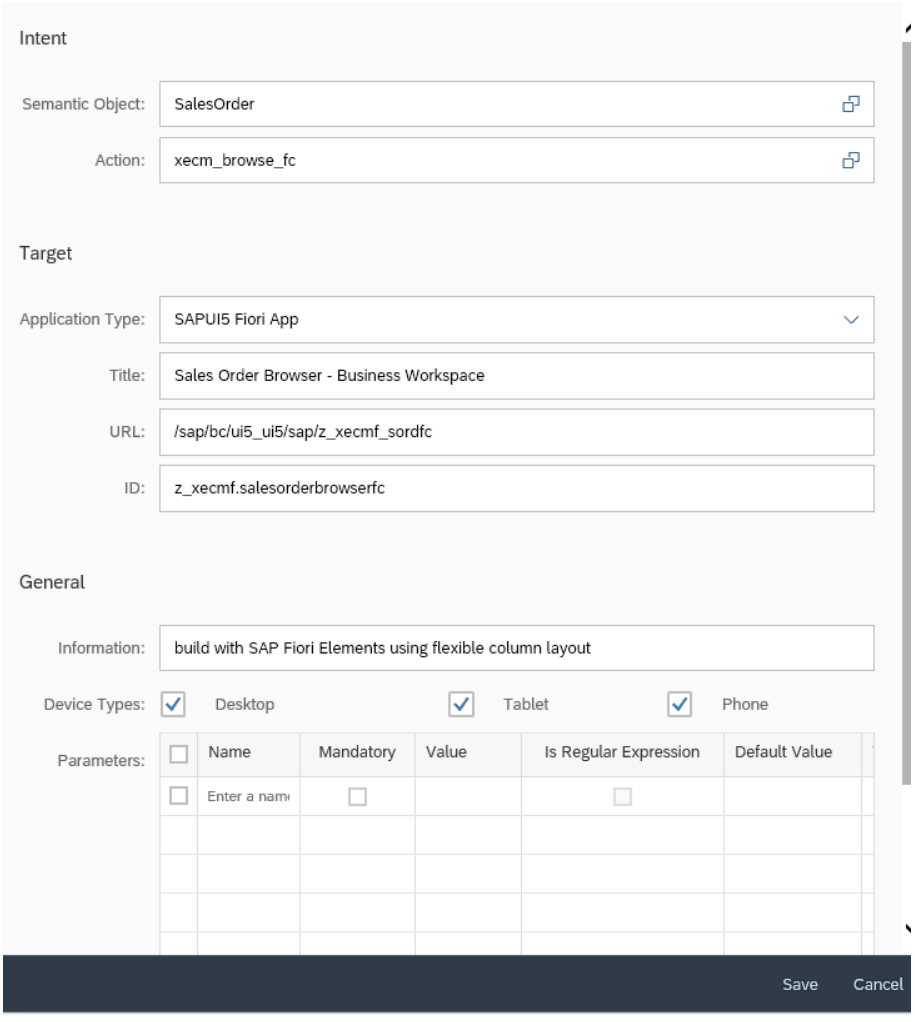
Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Please note the known issue, that changes to i18n.properties may not work when you launch with Adaptation_index.html or flpSandbox.html but in the preview of the SAPUI5 Visual Editor.

9.6.8 Accessing the custom app in on-premise Fiori Launchpad

As soon as you deployed the project to your SAP system it can be made accessible to users with the configuration of the Fiori Launchpad content.

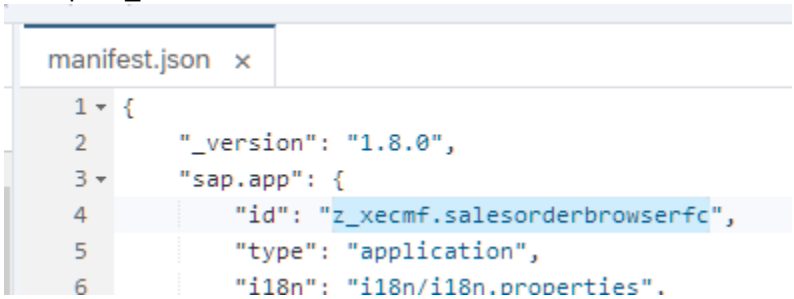
1. Start the Fiori Launchpad designer (cross-client or client-specific: transactions **`/UI2/FLPD_CONF`** or **`/UI2/FPLD_CUST`**)
2. In the designer create or modify a Fiori catalog.
3. Define a target mapping and a tile. Let us have a closer look at the definition of the target mapping.



Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

a. For Semantic Object and Action, a unique combination is required. As the example custom app is tightly related to standard apps for Sales Order the already existing Semantic Object SalesOrder is used. For Action the arbitrary value = xecm_browse_fc shall make sure, that the target mapping is unique.

b. Now at the area Target the relative URL of the custom app is used: /sap/bc/ui5_ui5/sap/z_xecm_sordfc
And for the ID the ID in the manifest.json of the app is used, for example z_xecmf.salesorderbrowserfc



Compare with the description at section **9.6.3 CREATING A NEW CUSTOM APP BASED ON FIORI ELEMENTS**. The ID is merged from the Namespace and the Project Name.

4. Then create a tile using the target mapping.

5. At the end do not forget to create a role in transaction PFCG, which contains the new SAP Fiori catalog in tab “Menu”, which is then assigned to the users, who shall have access to the app.

It may be necessary to delete the Fiori tile cache to make the new tile available.

Of course, you may also create a Fiori group which includes the new tile and is then assigned to users with roles containing the Fiori group.

9.6.9 Reusable Code snippets

9.6.9.1 XML code for OpenText Business Workspace

```
        <VBox id="otx_xecm_vbox">
                <core:HTML id="otx_cust_html" content=""/>
                <core:ComponentContainer id="otx_xecm_compcont"
name="otx.ecmlink.businessworkspace_cc" url="/sap/bc/ui5_ui5/otcmfnd/csbws"
                        settings='{AppMode: false, WidgetStyleClass: "otx_bws_div_so"}'
componentCreated="onComponentCreated"></core:ComponentContainer>
        </VBox>
```

Code Snippet 32 XML for business workspace in first object page column

This code is reusable in different projects. It needs to be inserted into the XML view automatically created for a “custom facet” (that is a custom object page

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

section) to replace the placeholder text box. If you use a flexible column layout this corresponds to the XML code for the first object page column. If you do not use flexible column layout, this is the XML code for every object page.

The text with blue background color is the name of the CSS style class, which defines the height of the integration widget. It is recommended to use a specific name for each widget instance. In this example the “_so” stands for Sales Order.

```
<VBox id="otx_xecm_vbox">
        <core:HTML id="otx_cust_html" content=""/>
        <core:ComponentContainer id="otx_xecm_compcont"
name="otx.ecmlink.businessworkspace_cc" url="/sap/bc/ui5_ui5/otcmfnd/csbws"
                settings='{AppMode: false, WidgetStyleClass: "otx_bws_div_soi"}'
componentCreated="onComponentCreated"
                modelContextChange="onModelContextChange"></core:ComponentContainer>
</VBox>
```

Code Snippet 33 XML code for business workspace in second object page column

This code is reusable in different projects. It needs to be inserted into the XML view automatically created for a “custom facet” (that is a custom object page section) to replace the placeholder text box. It corresponds to the XML code for the second object page column of the flexible column layout (that is the column, which renders the details of the navigation from the first details page).

The text with blue background color is the name of the CSS style class, which defines the height of the integration widget. It is recommended to use a specific name for each widget instance. In this example the “_soi” stands for Sales Order Item.

9.6.9.2 Javascript code for controller for first column object page

After the SAP Web IDE has created the custom controller file it contains the initial Controller.extend statement.

It looks like:

```
sap.ui.define([
    "sap/ui/core/mvc/Controller"
], function (Controller) {
    "use strict";

    return Controller.extend("[redacted]", {

    });

});
```

The name of the controller is blacked out, because it is the respective controller you are modifying.

The area in green between the curly brackets needs to be filled with the following code snippets:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

1. **CODE SNIPPET 34 FUNCTION ONCOMPONENTCREATED() FOR FIRST OBJECT PAGE COLUMNCODE** Snippet 34 Function onComponentCreated() for first object page column

    a. The code in blue background (“otx_bws_div_so”) needs to correspond with the name of WidgetStyleClass in the corresponding XML code (for example *Code Snippet 32 XML for business workspace in first object page column*)

    b. The code in the green background (“650px”) defines the height of the DIV element, which contains the business workspace widget.

2. **CODE SNIPPET 35 FUNCTION _SETDIVCSSHEIGHT()**

3. **CODE SNIPPET 36 FUNCTION _GETSETTINGSOBJECT()**

    a. The code in the green background is the name of the SystemOrigin or (System Alias) of the SAP backend server.

4. **CODE SNIPPET 37 FUNCTION _GETOBJECTWITHKEY()**

    a. The code in the blue background needs to be modified to match your actual business object type and data model of your app.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
		/**
		 * Callback for event componentCreated
		 * @param {object} oEvent  event context when component is created
		 */
		onComponentCreated: function (oEvent) {
			/*
			 * Inserts style class inline in HTML control to set height
			 * of DIV element which contains the business workspace widget
			 * from OpenText Content Server
			 *
			 * This function does no harm for business documents
			 */
			this._setDivCssHeight("otx_cust_html", "otx_bws_div_so", "650px");

			/*
			 * Define SystemOrigin (that is the system alias)
			 * Set UseClassicConfig flag (only relevant for business workspace, will be ignored by
business documents)
			 */
			var oSettings = this._getSettingsObject();

			/*
			 *  Use refresh function of OpenText component.
			 *  Trigger it when the bound runtime data of the original app is available
			 */

			var oRefreshComponent = oEvent.getParameter("component");
			var that = this;

			this.getView().getParent().getModel().attachRequestCompleted(function () {
				var oObject = that._getObjectWithKey();
				if (oObject && oObject.SapObject && oObject.ObjectId) {
					oRefreshComponent.refresh(oObject.SapObject, oObject.ObjectId,
oSettings);
				}
			});
		},
```

Code Snippet 34 Function onComponentCreated() for first object page column

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
/**
 * Sets style class definition in a custom HTML control once
 * --- relevant for Extended ECM business workspace ---
 * @param {string} sHTMLId  ID of the custom HTML control in context of the XML fragment and
view
 * @param {string} sCSSClassName name of the CSS class as used in component setting
WidgetStyleClass
 * @param {string} sHeight  text string representation for the height
 */
_setDivCssHeight: function (sHTMLId, sCSSClassName, sHeight) {
        var oHTML = this.byId(sHTMLId);
        if (oHTML && oHTML.getContent) {
                if (oHTML.getContent() === "") {
                        oHTML.setContent("<style type='text/css'>." +
                                sCSSClassName + " { height:" +
                                sHeight + " !important; }</style>");
                }
        }
},
```

Code Snippet 35 Function _setDivCssHeight()

```
/**
 *  Calculates settings for refresh of OpenText UI component
 *    UseClassicConfig : relevant for calculation of OpenText Content Server URL and will be
 *                       ignored by refresh of business documents
 *    SystemOrigin:      system alias; modify the calculation if you need a specific value
 *                       Important: business documents and business workspace can only be retrieved
 *                       from 1 system alias at a time
 */
_getSettingsObject: function () {
        var oSettingsObject = {};

        var oRegExp = RegExp(".dispatcher.hana.ondemand.com");
        if (oRegExp.test(window.location.origin)) {
                oSettingsObject.UseClassicConfig = true;
        }

        /*
         * system alias; use a different one if necessary
         */
        oSettingsObject.SystemOrigin = "LOCAL";
        return oSettingsObject;
},
```

Code Snippet 36 Function _getSettingsObject()

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
/**
 * Calculates the SAP BOR object type (SapObject) and SAP BOR key (ObjectId)
 *   THIS CODE IS DEPENDENT on adapted original app:
 *
 *   This code needs to be adjusted to the original app which is adapted.
 *   In the original app check for a controller file Attachments.controller.js
 *   probably in a sub folder ext/controller
 *   and then look at the code in the onInit function.
 *   This should give you an idea, which runtime data is used to
 *   initialize the DMS related attachments control of SAP
 *   The name of the object type is most often not the same as the SAP BOR object type
 *   but the ID or key value is often the same
 */
_getObjectWithKey: function () {
        var oObject = {};

        /*
         * SAP BOR object type for Sales Orders
         */
        oObject.SapObject = "BUS2032";

        /*
         * Now do a runtime data retrieval similar to that of the original app
         */
        var sProperty1 = "SalesOrder";

        if (this && this.getView && this.getView().getBindingContext &&
this.getView().getBindingContext()) {
                var sVal = this.getView().getBindingContext().getProperty(sProperty1);
                // leading zeros with length 10
                if (sVal) {
                        while (sVal.length < 10)
                                sVal = "0" + sVal;
                        oObject.ObjectId = sVal;
                }
        }

        return oObject;
}
```

Code Snippet 37 Function _getObjectWithKey()

The code snippets of this section can be applied to the first object page column of a flexible column layout (that is the second columns; that is the “details” of “master-details”).

9.6.9.3 JavaScript code for controller of second column object page

After the SAP Web IDE has created the custom controller file it contains the initial Controller.extend statement.

It looks like:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
sap.ui.define([
    "sap/ui/core/mvc/Controller"
], function (Controller) {
    "use strict";

    return Controller.extend("[redacted]", {

    });

});
```

The name of the controller is blacked out, because it is the respective controller you are modifying.

The area in green between the curly brackets needs to be filled with the following code snippets:

1. **Code Snippet 38 Function onModelContextChange()**
    a. **It needs to correspond with the XML code, which is**
        i. *Code Snippet 33 XML code for business workspace in second object page column*
2. **Code Snippet 39 onComponentCreated() for second object page column**
    a. The code in blue background (“otx_bws_div_soi”) needs to correspond with the name of WidgetStyleClass in the corresponding XML code (for example *Code Snippet 33 XML code for business workspace in second object page column*)
    b. The code in the green background (“600px”) defines the height of the DIV element, which contains the business workspace widget.
3. **Code Snippet 35 Function _setDivCssHeight()**
4. **Code Snippet 36 Function _getSettingsObject()**
    a. The code in the green background is the name of the SystemOrigin or (System Alias) of the SAP backend server.
5. **Code Snippet 40 Function _getObjectWithKey()**
    a. The code in the blue background needs to be modified to match your actual business object type and data model of your app.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
		/**
		 * Callback for event modelContextChange on component container
		 * @param {object} oEvent event context which contains the componentContainer as source
		 */
		onModelContextChange: function (oEvent) {
			/*
			 * Define SystemOrigin (that is the system alias)
			 * Set UseClassicConfig flag (only relevant for business workspace, will be ignored by
business documents)
			 */
			var oSettings = this._getSettingsObject();

			/*
			 * Get the component container from the event source
			 * Get the OpenText component as component instance embedded in the component
container
			 */
			var oRefreshComponent = {};
			if (oEvent.getSource && oEvent.getSource() &&
oEvent.getSource().getComponentInstance) {
				oRefreshComponent = oEvent.getSource().getComponentInstance();
			}

			/*
			 * Calculate the object instance from the current data binding
			 * if it is something valid
			 * and if the component has a refresh function
			 * execute a refresh
			 */
			var oObject = this._getObjectWithKey();
			if (oObject && oObject.SapObject && oObject.ObjectId &&
oRefreshComponent.refresh) {
				oRefreshComponent.refresh(oObject.SapObject, oObject.ObjectId,
oSettings);
			}
		},
```

Code Snippet 38 Function onModelContextChange()

```
		/**
		 * Callback for event componentCreated
		 * @param {object} oEvent  event context when component is created
		 */
		onComponentCreated: function (oEvent) {
			/*
			 * Inserts style class inline in HTML control to set height
			 * of DIV element which contains the business workspace widget
			 * from OpenText Content Server
			 *
			 * This function does no harm for business documents
			 */
			this._setDivCssHeight("otx_cust_html", "otx_bws_div_soi", "600px");

		},
```

Code Snippet 39 onComponentCreated() for second object page column

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
		/**
		 * Calculates the SAP BOR object type (SapObject) and SAP BOR key (ObjectId)
		 *   THIS CODE IS DEPENDENT on adapted original app:
		 *
		 *   This code needs to be adjusted to the original app which is adapted.
		 *   In the original app check for a controller file Attachments.controller.js
		 *   probably in a sub folder ext/controller
		 *   and then look at the code in the onInit function.
		 *   This should give you an idea, which runtime data is used to
		 *   initialize the DMS related attachments control of SAP
		 *   The name of the object type is most often not the same as the SAP BOR object type
		 *   but the ID or key value is often the same
		 */
		_getObjectWithKey: function () {
			var oObject = {};

			/*
			 * SAP BOR object type for Material
			 */
			oObject.SapObject = "BUS1001006";

			/*
			 * Now do a runtime data retrieval similar to that of the original app
			 */
			var sProperty1 = "Material";

			if (this && this.getView && this.getView().getBindingContext &&
this.getView().getBindingContext()) {
				var sVal = this.getView().getBindingContext().getProperty(sProperty1);
				// leading zeros with length 18 if Material only contains numbers
				var oRegex = RegExp("^[ 0123456789]*$");
				if (sVal && oRegex.test(sVal)) {
					while (sVal.length < 18) {
						sVal = "0" + sVal;
					}
					oObject.ObjectId = sVal;
				} else if (sVal) {
					oObject.ObjectId = sVal;
				}
			}

			return oObject;
		}
```

Code Snippet 40 Function _getObjectWithKey()

The code snippets of this section can be applied to the second object page column of a flexible column layout (that is the third of all columns; that is the second “details” of “master-details-details”).

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.7 Extending freestyle SAPUI5 apps with Adaptation

9.7.1 General Information and Requirements

See also **FIGURE 53 APP VARIANT RESULTING FROM EXTENDING WITH ADAPTATION PROJECT**

The term freestyle app refers to plain SAPUI5 apps using regular controls, which are not created automatically or semi-automatically from any framework like SAP Fiori Elements templates. But also mixed apps, which may use some smart controls can be seen in this group. This can be an SAP delivered app from the SAP Fiori apps library or your own custom app. See also Developing Apps.

A typical SAPUI5 application consists of bootstrap files, views, view controllers, and internationalization files.

Like for SAP Fiori Elements-based apps, SAP offers Adaptation extensions for SAPUI5 freestyle apps. You can introduce your own functionality as part of an adaptation project.

OpenText recommends adaptation of SAPUI5 freestyle apps instead of extensions using component configuration.

Note the following:

- A SAPUI5 freestyle app requires
  - Either property flexEnabled not defined or better property flexEnabled=true in the manifest.json
  - stable IDs
- You require a recent version of the SAP Web IDE Full-Stack to create and develop an adaptation project
  See also section **9.12 BASIC STEPS USING THE SAP CLOUD CONNECTOR AND THE SAP WEB IDE ON THE SAP CLOUD PLATFORM**.
- Currently supports on-premise only.
- Each adaptation project is deployed to the SAP Gateway server as an app variant in the customer layer of the SAPUI5 Flexibility framework
  - The variant ID starts with prefix `customer.`
  - The flexibility layer is actually `CUSTOMER_BASE`
  - The app variant has no own BSP node
  - The original app is not changed

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- If you want to adapt your own custom app, enable it for UI Adaptation. Search for topic SAPUI5 Flexibility: Enable Your App for UI Adaptation in the SAPUI5 SDK.

  .
- In SAP Web IDE the SAPUI5 Visual Editor lets you add changes
- The basic flow and steps to create an Adaptation project and to add changes uses the same principal flow of steps like adaptation of an SAP Fiori Elements based app.

Important: Please refer to the most recent SAP online documentation like the SAPUI5 SDK regarding restrictions. You can also search for key word Adaptation in the guide of SAP Web IDE Full-Stack in the SAP Help Portal.

Here is a list of advantages and disadvantages of the procedure.

Advantages:

+ The original app is not changed
+ For the same kind of integration, you can reuse the same implementation steps and code snippets
+ The SAPUI5 Visual Editor restricts the areas in which extensions are allowed
+ No code replacement or overrides are required to insert new controls
+ Controller code to hook into the original app is short and no invasive code is required
+ The changes are added as single change files and can be monitored quite easily in the adaptation project
+ Calling the deployed app variant in the SAP Fiori Launchpad is straightforward and easy

Disadvantages:

- After deployment, the code of the app variant is “hidden” in the database tables of the SAPUI5 Flexibility framework
- Tools to browse over the list of app variants deployed to an SAP system are not officially released by SAP yet (But this will change over time. SAP offers a function to remove an app variant from an SAP server)

9.7.2 Quick procedure

Follow the steps in this section for a quick procedure.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

If you are reading this the first time, skip this section and continue reading with the next section, **9.7.3 CREATING A NEW ADAPTATION PROJECT**

1. Create new adaptation project and modify in the next steps.
2. With the SAPUI5 Visual Editor add a new XML fragment named “otx_bws” (otx_bws.fragment.xml) or "otx_doc" (here at extension point extInIconTabBar)
3. With the SAPUI5 Visual Editor you may hide unwanted controls like the original attachments.
4. With the SAPUI5 Visual Editor extend the details view with a new custom controller named “otx” (otx.js).
5. Insert new XML code in the XML fragment. Use *Code Snippet 41 XML for IconTabFilter with Business Workspace (adaptation)*

   Make sure, that you set the right name of the function for event componentCreated.
6. Insert new JavaScript code in the JS controller: see code snippets listed in section **9.7.6 MODIFYING THE CUSTOM CONTROLLER**. But modify the SAP BOR object type and key calculation according to the runtime data of your original leading app. See next steps.
7. In the custom controller in function onComponentCreated() make sure, that function _setDivCssHeight() sets the right height and that the name of the HTML control and CSS style class are ok.
8. In function _getSettingsObject() make sure, that the SystemOrigin (that is the system alias) is ok.
9. Adjust the code in function _getObjectWithKey() according to your original app:
   a. Set the right SAP BOR object type for value oObject.SapObject
   b. Make sure, that oObject.ObjectId is filled with the correct key/object ID.
10. Deploy the Adaptation project to your SAP on-premise system
11. Define a target mapping and tile, which call the app variant of your original app.

9.7.3 Creating a new Adaptation project

This section and the following sections give a detailed description of how to extend an SAPUI5 freestyle app with extension points with an Adaptation project. As an example, this original app is used:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Request Maintenance - for Employee - Maintenance Info – App ID: F1511 (see also SAP Fiori Apps Library)

In this example, the ID of the BSP application is EAM_NTF_CRES1

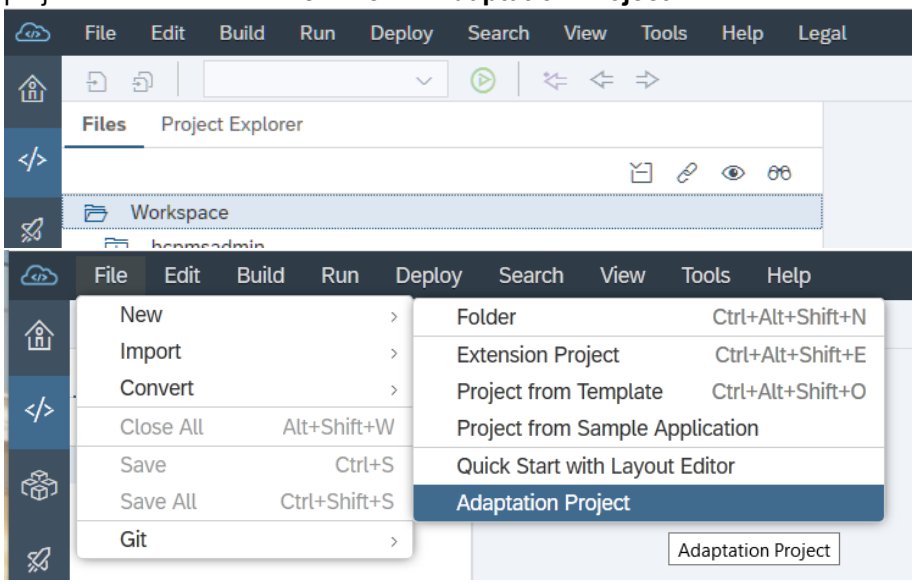
If you or an administrator has done the setup for your subaccount and services in the subaccount, you may start in your browser from

http://hana.ondemand.com

Then click on the tile of your global account. Then click on the navigation node for subaccounts and click on the tile of your subaccount. Then click on the navigation node Services. Then click on the tile SAP Web IDE Full-Stack. Finally, you click on the link Go to Service in the area *Take Action*.

You can create a shortcut to the Full Stack Web IDE of your subaccount. If the technical name of the subaccount is, for example abcde12345, it will be https://webidecp-abcde12345.dispatcher.hana.ondemand.com.

1. At your project workspace of the Web IDE create a new adaptation project from the menu **File > New > Adaptation Project**.



2. Fill in the tab Basic Information.

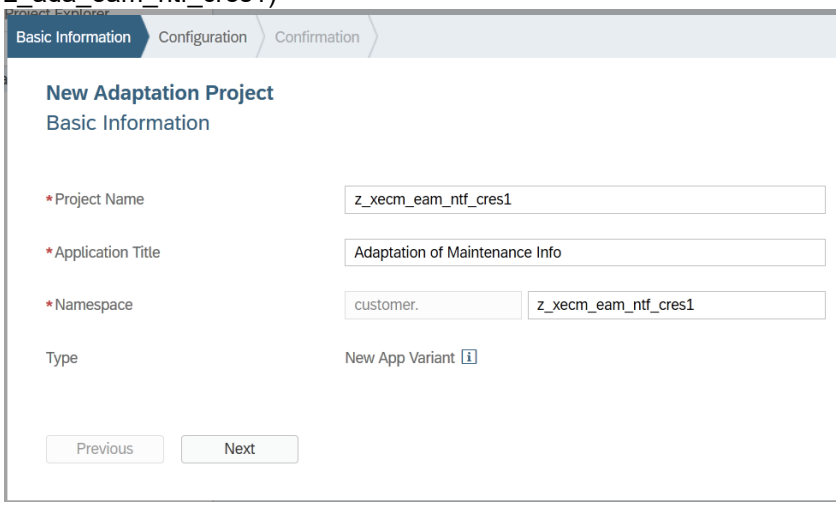
    a. In field Project Name, define a previously unused project name in the customer's name space, for example, z_xecm_eam_ntf_cres1 (or: z_ada_eam_ntf_cres1).

    b. Enter a title of your choice in the field Application Title.

    c. Fill in field Namespace. Although the project name and the namespace after "customer." do not need to be identical, we keep it that way. Then it is easier to map the project to the app. (here: customer. z_xecm_eam_ntf_cres1 or: customer.

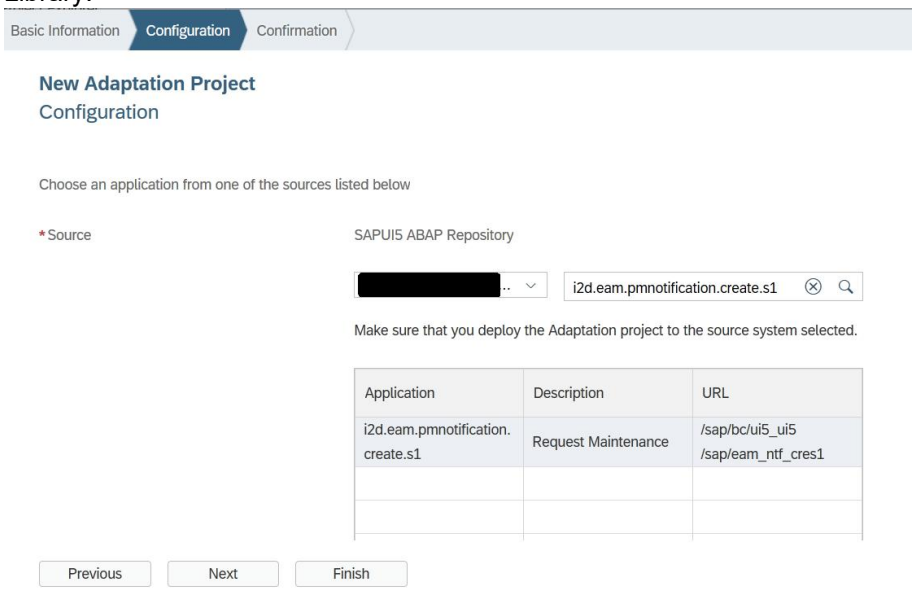
Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

z_ada_eam_ntf_cres1)



3. In the tab Configuration

   a. Select the connection to the remote SAP on-premise Gateway/Front-End server and select the Fiori application you want to extend. The URL/ID of the app, in this example: i2d.eam.pmnotification.create.s1, can be found from the configuration description on the web site of the Fiori Apps Library.

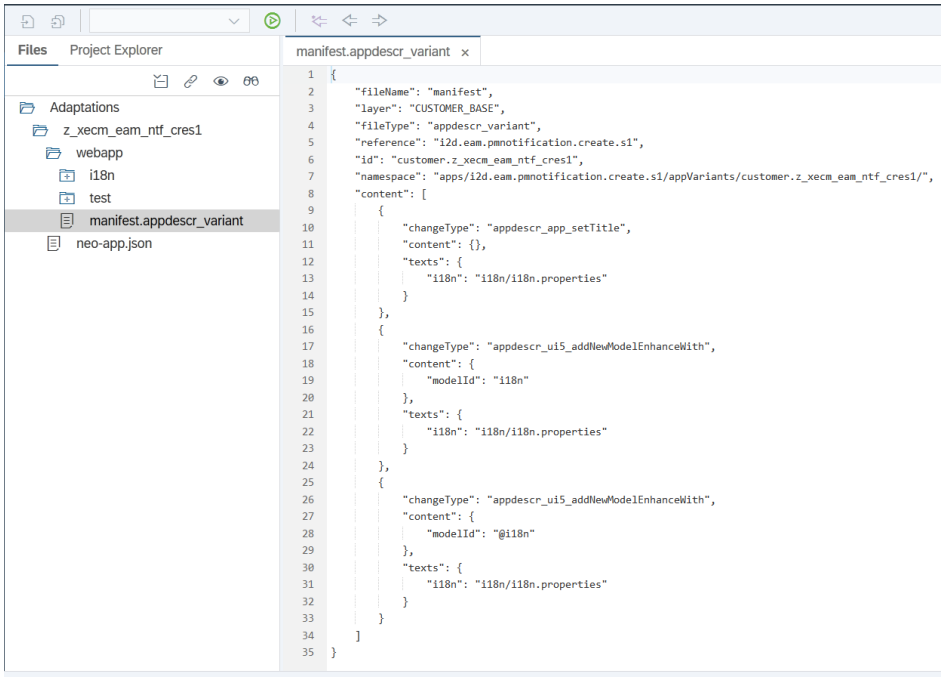


   b. Scroll down and uncheck the checkbox **Enable Safe Mode**.

4. Click **Next**.
5. Click **Finish**.

After these steps you will find the new project in your workspace:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

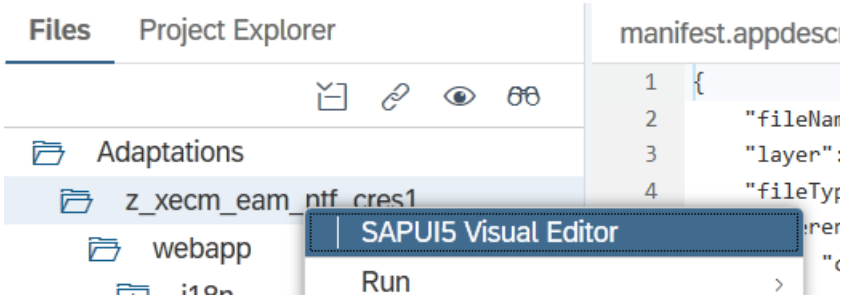


The Web IDE has created all initial files and folders, for example the file webapp/manifest.appdescr_variant.

9.7.4 Using the SAPUI5 Visual Editor to add changes

9.7.4.1 Starting the SAPUI5 Visual Editor

Right-click on the node of the Adaptation project and execute the action **SAPUI5 Visual Editor**.



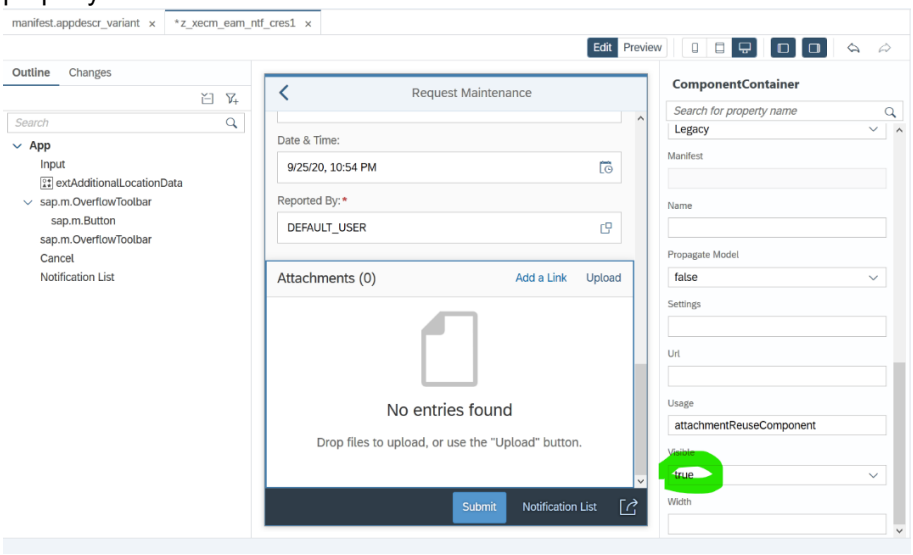
In the SAPUI5 Visual Editor you can browse within the app in preview mode. You can switch to edit mode to modify the existing app.

9.7.4.2 Hiding controls

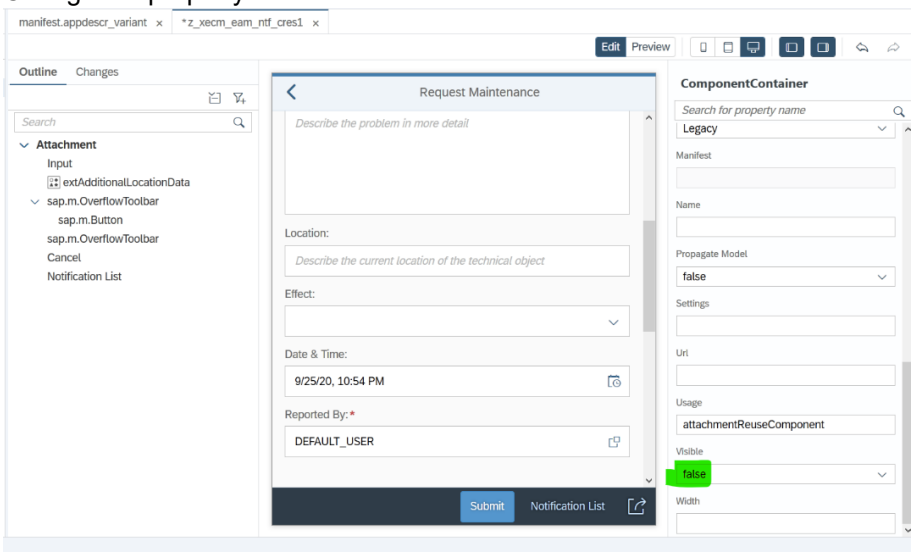
In our example app, the initial page shows an attachments control to add attachments during creation of a new maintenance request.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

1. To hide this control, switch to edit mode.
2. Select the control in the preview window. On the right side go to the property Visible.



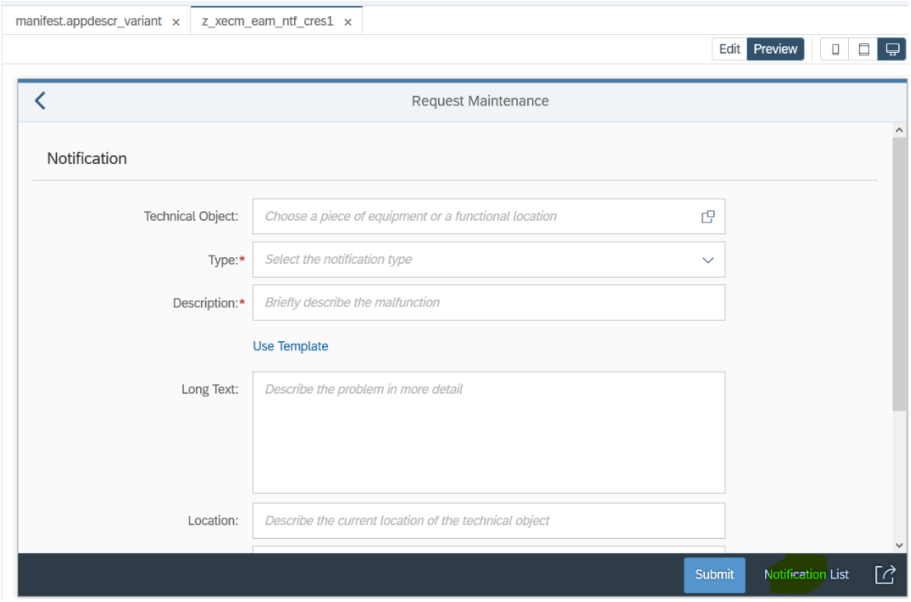
3. Change the property Visible to value false.



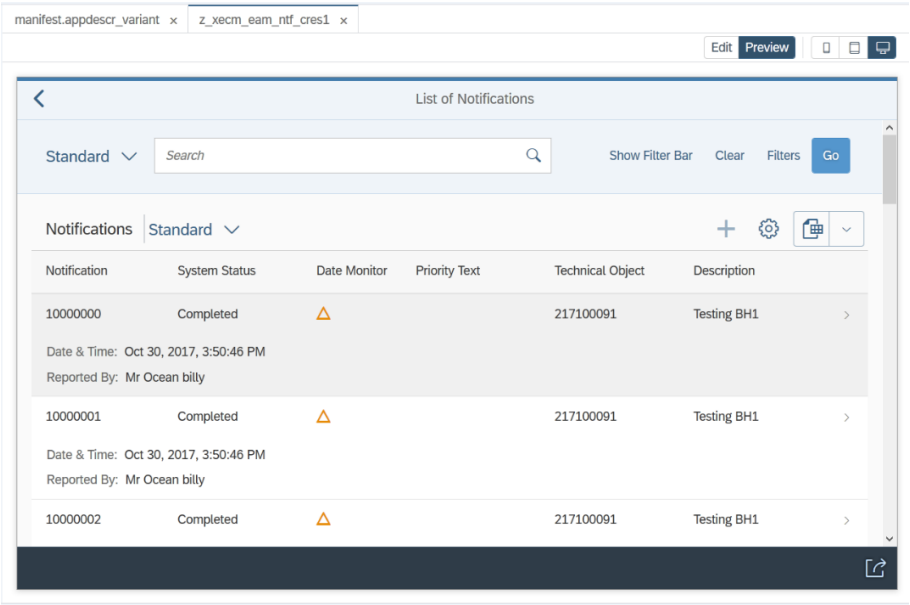
4. Save the change.
5. This automatically creates a change file like id_1601067560298_61_propertyChange.change.
6. Continue in SAPUI5 Visual Editor in preview mode.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

7. In the app preview click on **Notification List**.



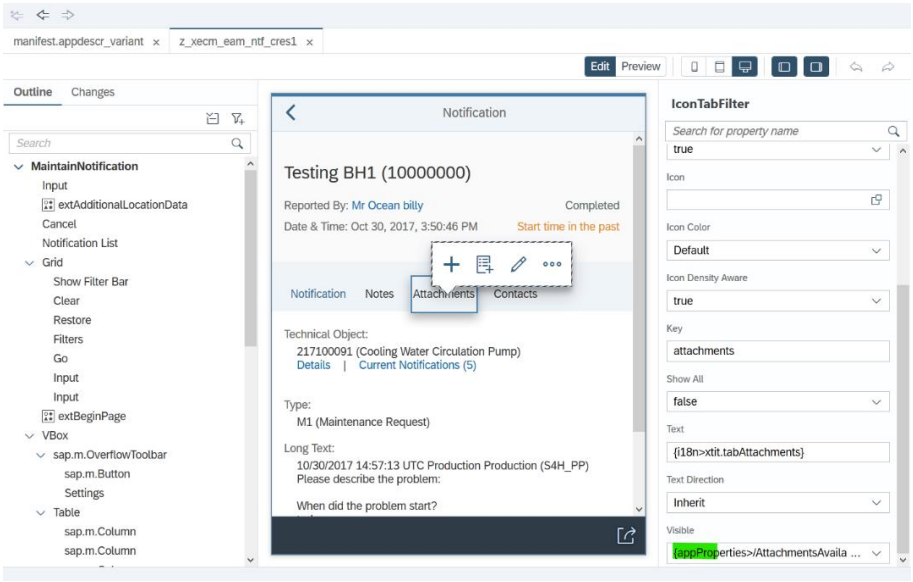
8. Click on the first row.



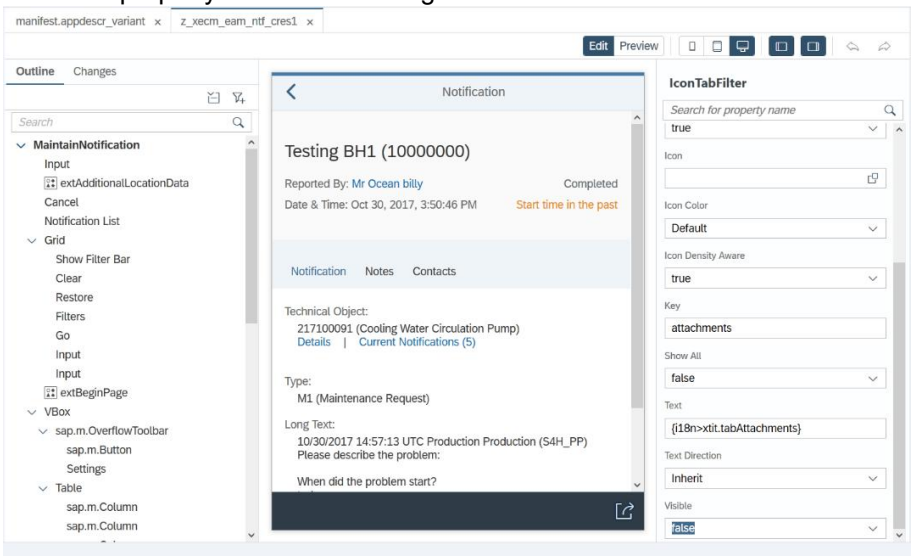
9. In the details of the maintenance notification click **Edit** to switch to edit mode.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

10. Select the tab Attachments.



11. Go to the property Visible and change the value to false.



12. Save the change.

13. This automatically creates a change file like id_1601068267511_133_propertyChange.change.

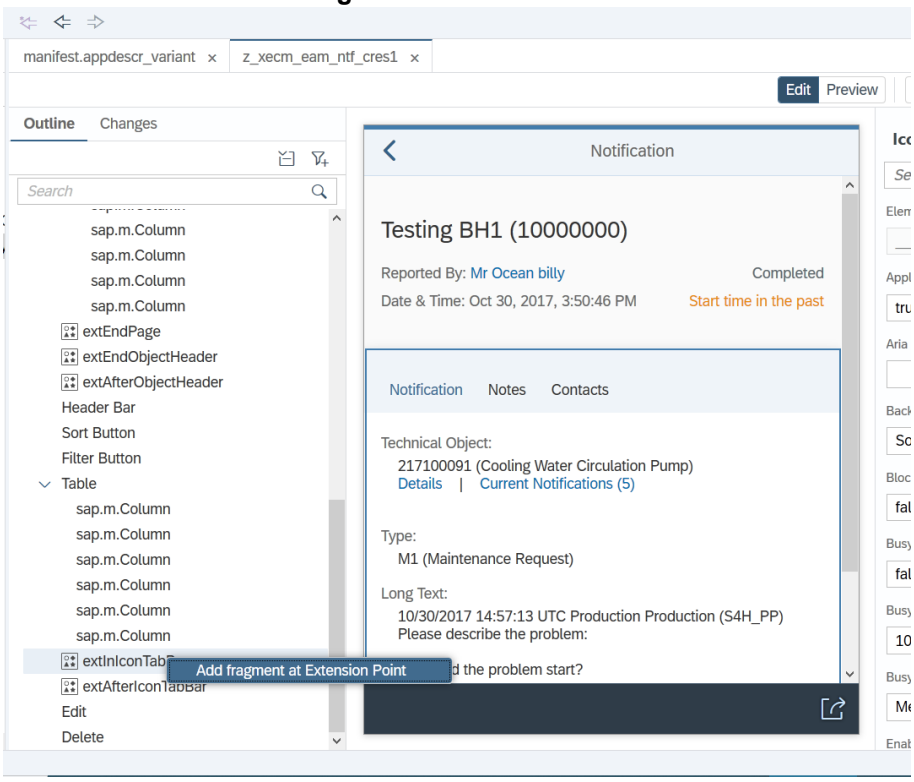
9.7.4.3 Adding an XML fragment (at extension point)

Our example app has extension points for adding XML fragments. Please note, that you also can add XML fragments within aggregations of other controls, if supported.

1. In the SAPUI5 Visual Editor in preview mode browse to the details of a maintenance notification.

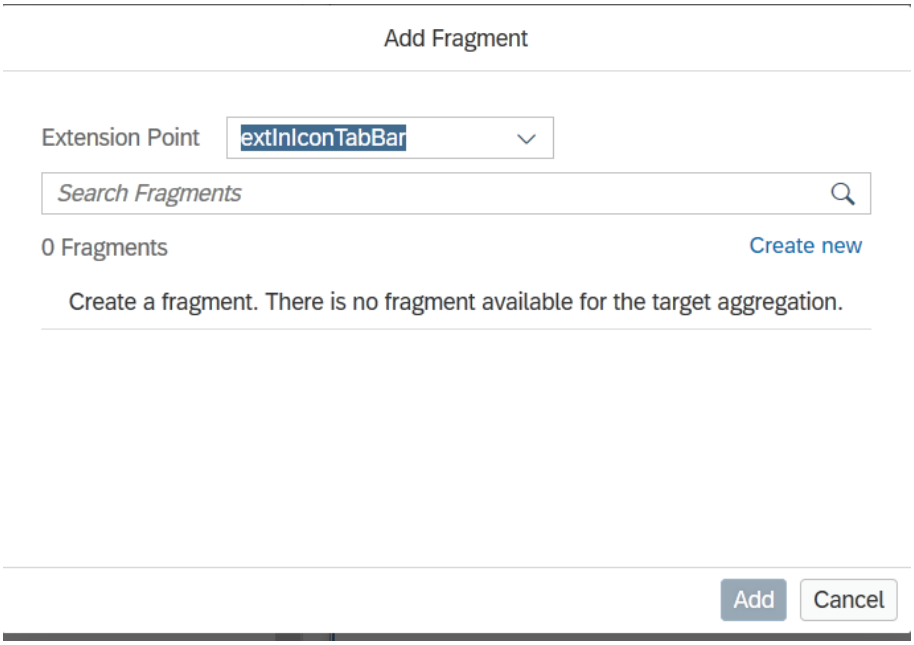
Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

2. Switch to Edit mode.
3. On the left side in the tree Outline go to the node extInIconTabBar. It can be recognized by the node icon for an extension point. Right-click the node and select **Add fragment at Extension Point**.

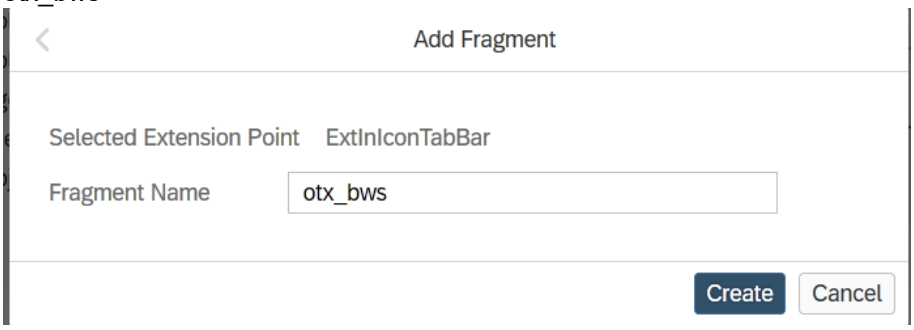


Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

4. In dialog Add fragment at field Extension Point select value extInIconTabBar.



5. Click **Create new**.
6. In the dialog Add Fragment enter in field Fragment Name enter value otx_bws



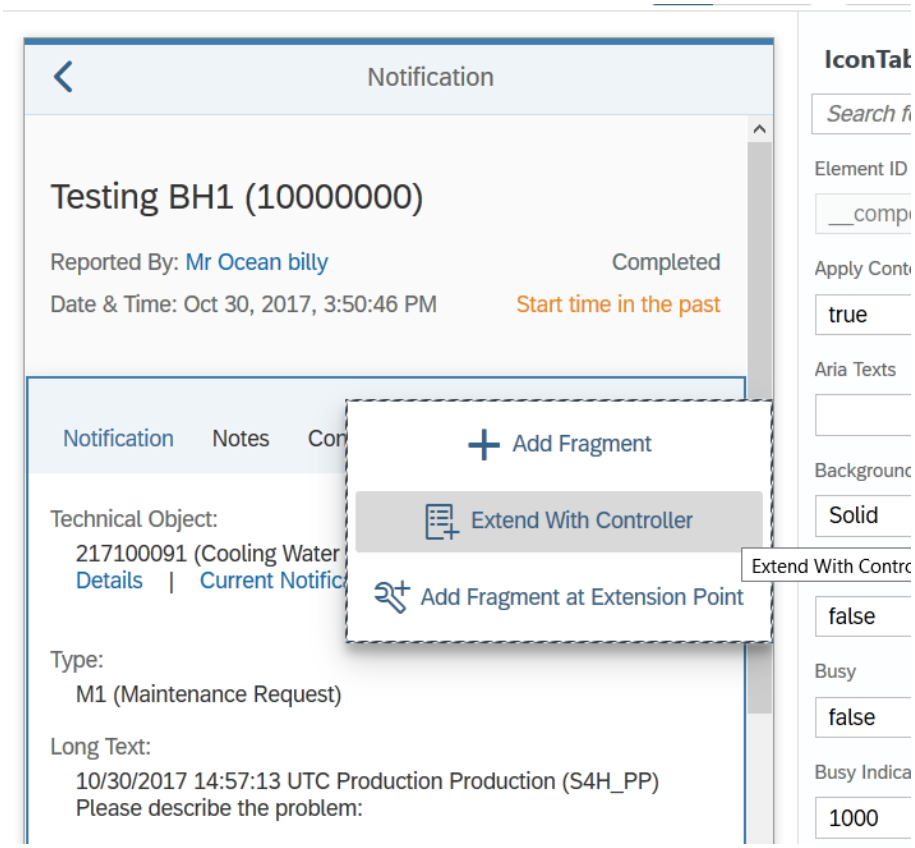
7. Click Create.
8. Save the changes.
9. This automatically creates a change file and the XML file for the fragment: webapp/changes/fragments/otx_bws.fragment.xml

9.7.4.4 Extending with controller

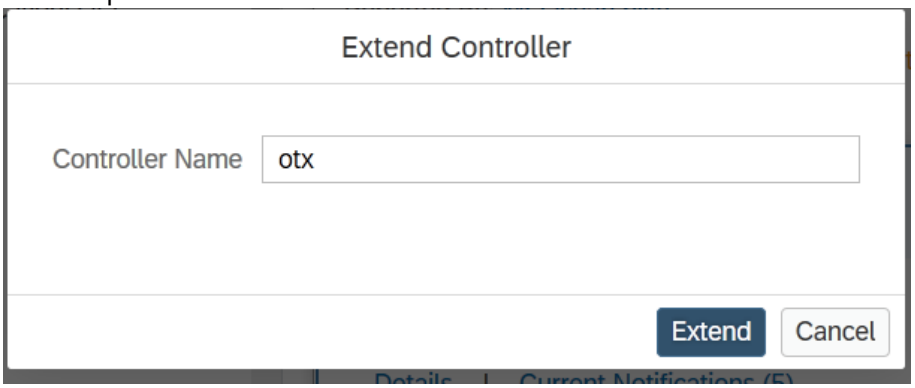
1. In the SAPUI5 Visual Editor in preview mode browse to the details of the maintenance notification.
2. Switch to edit mode.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

3. Select the IconTabBar control, right-click and execute **Extend With Controller**.



4. In the dialog Extend Controller insert the name of the new controller, for example otx.



5. Click Extend.
6. Save the changes.
7. This automatically creates a change file and the JavaScipt file webapp/changes/coding/otx.js.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.7.4.5 Removing changes

If you see that you did a change in the wrong area of the original app you can easily correct this, by removing the corresponding change file.

In the SAPUI5 Visual Editor in Edit mode you can use the Changes list to quickly remove changes using the trashcan icon.

Alternatively, you can delete the respective change file from the sub folder changes.

XML fragments and extension Javascript code, which are not needed anymore, can also be removed from the changes/coding and changes/fragments sub folders.

9.7.5 Modifying the XML fragment

In our example the XML fragment was added at extension point extInIconTabBar. Therefore, the new XML code must define a new sap.m.IconTabFilter control.

It will contain a sap.ui.core.ComponentContainer with the OpenText component.

1. To integrate the control for the business workspace, open the file webapp/changes/fragments/otx_bws.fragment.xml.
2. Insert *Code Snippet 41 XML for IconTabFilter with Business Workspace (adaptation)* inside of the core:FragmentDefinition tag.
3. Save the changes.

```
<IconTabFilter xmlns="sap.m" id="otx_tabfilter" key="otxtab" text="{i18n>xtit.tabBusinessWorkspace}">
    <content>
        <VBox id="otx_xecm_vbox" xmlns="sap.m">
            <core:HTML id="otx_cust_html" content=""/>
            <core:ComponentContainer id="otx_xecm_compcont"
name="otx.ecmlink.businessworkspace_cc" url="/sap/bc/ui5_ui5/otcmfnd/csbws"
                settings='{AppMode: false, WidgetStyleClass: "otx_bws_div"}'

componentCreated=".extension.customer.z_xecm_eam_ntf_cres1.otx.onComponentCreated"></core:Co
mponentContainer>
        </VBox>
    </content>
</IconTabFilter>
```

Code Snippet 41 XML for IconTabFilter with Business Workspace (adaptation)

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

The title text in blue background color is referring to a variable in the i18n.properties file of the adaptation project.

The ID in the green background and the style class name in light orange background color are used for setting the height of the DIV-Element into which the business workspace widget is rendered. The HTML code will be filled with CSS style definitions for the CSS class.

The value of property componentCreated in light red color is the name of the method within the extension controller, which is called at the event after the component was created. Please note, the parts of the name:

- .extension always add this prefix
- customer.z_xecm_eam_ntf_cres1.otx this is the full name of the controller. Open the .js file of the extension controller and copy the name from ControllerExtension.extend(“ … “
- onComponentCreated name of the function/method implemented in the .js file of the extension controller

9.7.6 Modifying the custom controller

With the step described in section 9.7.4.4 Extending with controller you created a new file webapp/changes/coding/otx.js in the adaptation project. The framework already filled it with template code.

The template code needs to be modified to match the controls added in the XML fragment.

1. Insert the code for function onComponentCreated as defined in *Code Snippet 42 JavaScript function onComponentCreated for business workspace in maintenance notification*
2. Enter code for function _setDivCssHeight as defined in *Code Snippet 27 Function _setDivCssHeight()*
3. Enter code for function _getSettingsObject as described in *Code Snippet 28 Function _getSettingsObject()*
4. For function _getObjectWithKey enter the code of *Code Snippet 45 JavaScript method _getObjectWithKey( ) for Maintenance Notification*
5. Insert an override section, which defines an onInit function as defined in *Code Snippet 43 override section with JavaScript function onInit for business workspace*

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
                        /**
                         * Callback for event componentCreated
                         * @param {object} oEvent  event context when component is created
                         */
                        onComponentCreated: function (oEvent) {
                                /*
                                 * Inserts style class inline in HTML control to set height
                                 * of DIV element which contains the business workspace widget
                                 * from OpenText Content Server
                                 *
                                 * This function does no harm for business documents
                                 */
                                this._setDivCssHeight("otx_cust_html", "otx_bws_div", "650px");

                                /*
                                 * Specific for example with sap.m.IconTabBar
                                 *  Use count function of OpenText component.
                                 *  Trigger it when the bound runtime data of the original app is available
                                 */

                                var oCountComponent = oEvent.getParameter("component");
                                var oIconTabFilter =
this.getView().byId("customer.z_xecm_eam_ntf_cres1.otx_tabfilter");
                                var that = this;

                                /*
                                 *  this event handler is only processed if component is for business
documents
                                 *  and has API function/method countDocuments
                                 */
                                this.getView().getModel().attachRequestCompleted(function () {
                                        var oObject = that._getObjectWithKey();
                                        if (oObject && oObject.SapObject && oObject.ObjectId &&
oCountComponent && oCountComponent.countDocuments) {
                                                if (oCountComponent.countDocuments) {

        oCountComponent.countDocuments(oObject.SapObject, oObject.ObjectId, oIconTabFilter.setCount,
oIconTabFilter);
                                                }
                                        }
                                });
                        },
```

Code Snippet 42 JavaScript function onComponentCreated for business workspace in maintenance notification

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
// // this section allows to extend lifecycle hooks or override public methods of the base controller
override: {
	//		/**
	//		 * Called when a controller is instantiated and its View controls (if available)
are already created.
	//		 * Can be used to modify the View before it is displayed, to bind event
handlers and do other one-time initialization.
	//		 * @memberOf customer.z_xecm_eam_ntf_cres1.otx
	//		 */
	onInit: function () {
		var oIconTab = this.getView().byId("pmNotifIconTab");

		/*
		 * Set the header mode to inline as recommended by SAP Fiori UX
Guidelines
		 *   https://experience.sap.com/fiori-design-web/icontabbar/
		 *   requires SAPUI5 1.40 or higher
		 *   do not use "Inline" if tabs have icons
		 */
		oIconTab.setHeaderMode("Inline");
		var that = this;
		/*
		 * Define SystemOrigin (that is the system alias)
		 * Set UseClassicConfig flag (only relevant for business workspace, will be
ignored by business documents)
		 */
		var oSettings = this._getSettingsObject();
		oIconTab.attachSelect(function () {
			/*
			 * get selectwed key
			 */
			var sKey = oIconTab.getSelectedKey();
			if (sKey === "otxtab") {
				if (!oIconTab.getExpanded()) {
					return;
				}
				var oIconTabFilter =
that.getView().byId("customer.z_xecm_eam_ntf_cres1.otx_tabfilter");
				/*
				 * get the component instance depending on layout of
control
				 */
				var oRefreshComponent = null;
				if (oIconTabFilter.getContent()[0]) {
					if (oIconTabFilter.getContent()[0].getItems) {
						oRefreshComponent =
oIconTabFilter.getContent()[0].getItems()[1].getComponentInstance();
					} else {
						oRefreshComponent =
oIconTabFilter.getContent()[0].getComponentInstance();
					}
				}
				var oObject = that._getObjectWithKey();
				if (oObject && oObject.SapObject && oObject.ObjectId
&& oRefreshComponent && oRefreshComponent.refresh) {

		oRefreshComponent.refresh(oObject.SapObject, oObject.ObjectId, oSettings);
				}
			}
		});
	}
```

Code Snippet 43 override section with JavaScript function onInit for business workspace

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.7.7 Defining localized text labels

The adaptation project already includes an internationalization file webapp/i18n/i18n.properties for default language (English).

Maintain the title used in the XML fragment.

Add *Code Snippet 44 i18n short text for business workspace tab* to the i18n.properties file.

```
# XTIT: Title of the Business Workspace Icon Tab

xtit.tabBusinessWorkspace=Business Workspace
```

Code Snippet 44 i18n short text for business workspace tab

9.7.8 Deploying the project to the SAP Gateway on premise

See the description at **9.5.6 DEPLOYING THE PROJECT TO THE SAP GATEWAY ON PREMISE** .

9.7.9 Testing display of business workspaces

You have the following options to test the adaptation project:

- Testing the changes in the Web IDE Full Stack either in preview mode of the SAPUI5 visual editor or in a test run as a web application. This is described in this section and sub sections
- Testing the resulting app on your on-premise environment after deployment. This is what you should always do at the end. You also can do this iteratively during development, but then you need to repeat the deployment of the app before testing again. See section **9.7.8** (**9.5.6**) about deployment of the app and section **9.7.10** about accessing the custom app on your on-premise Fiori Launchpad.

9.7.9.1 Requirements for Web IDE tests

You need to do the obvious preparations for OpenText Content Management, like:

- You have a working OpenText Content Management configuration to access your business workspace and you have created a business workspace, which matches the data record you can browse to in the original Fiori app (In our example a business workspace for a maintenance notification, business object type BUS2038).

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- The SAPUI5 app (component) for OpenText business workspaces is installed and active. The related oData service is active.

Furthermore, you will need these settings:

- Content Server administration > Security Parameters
  In the area for trusted relationships make sure, that addresses of the cloud platform domain (*.dispatcher.hana.ondemand.com) under which the Web IDE runs is a trusted referrer.
- The Content Server can be accessed with a secure HTTPS-URL and the SAP backend, which connects to the Content Server use this HTTPS-URL in its connection settings
- If you use the most recent version of the SAPUI5 component for business workspace (otx.ecmlink.businessworkspace_cc) it has the startup parameter UseClassicConfig. The example code presented in **CODE SNIPPET 28 FUNCTION _GETSETTINGSOBJECT()** already takes this into account. And depending on whether the app runs in the Web IDE test environment (that means the origin is in *.dispatcher.hana.ondemand.com) or not, the startup parameter is calculated. Since OpenText Content Server 22.3 it sends the required response headers for cross-origin resource sharing (CORS). Then you can always set the value UseClassicConfig = true.
- If the original app uses reuse libraries and if you want to test the application using the test page flpSandbox.html, the references to these libraries need to be added to the routes of the file neo-app.json.

  See details in section **9.7.9.3** below.

The test pages of the Web IDE do not enforce any special CORS or same origin policies. But they run under an HTTPS-URL. Therefore, other scripts like the scripts on the Content Server must be called via a secure URL. Otherwise, the browser may block unsecure content from loading.

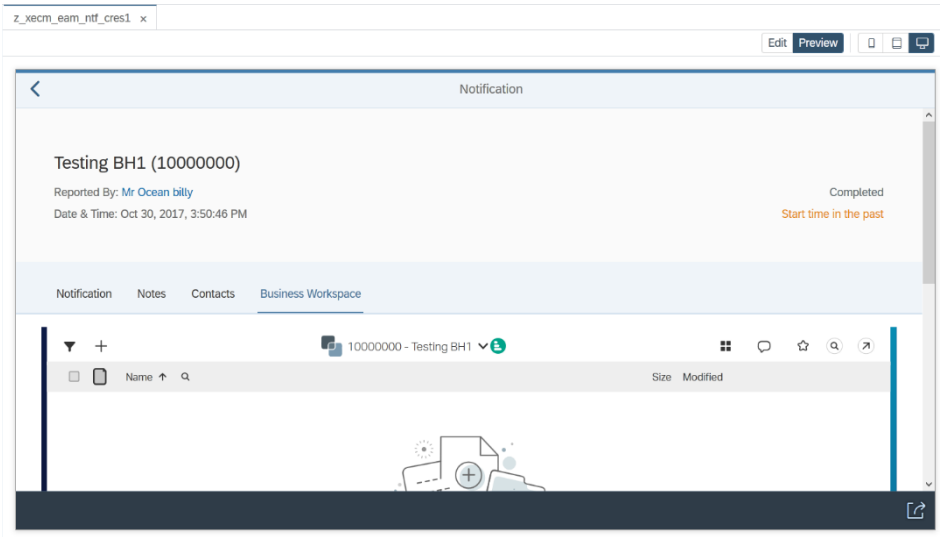
But loading from the original on-premise HTTPS-URL of the Content Server is not an issue if your browser can access this URL.

9.7.9.2 Testing in preview mode

You start the SAPUI5 Visual Editor.

The business workspace widget is displayed in the custom tab as implemented:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.



9.7.9.3 Testing as web application

You have two options to preview the app variant:

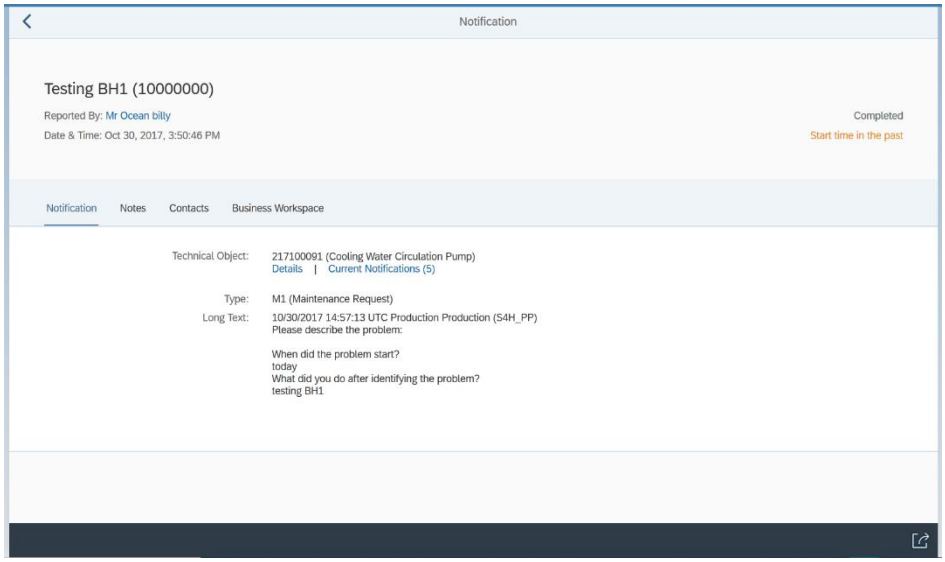
- Directly launch in the browser with test page **Adaptation_index.html**
- Launch the app variant as a tile in Fiori Launchpad via test page **flpSandbox.html**

This is documented in the SAP Help portal :

- search for the guide: SAP Web IDE Full-Stack
- Inside that guide search for: Preview the App Variant

Right-click on the project node and execute **Run > Run as Web Application**.
Or: Select the project node and execute from menu **Run > Run as Web Application**.
This starts the related test page in a new tab in the browser, for example:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.



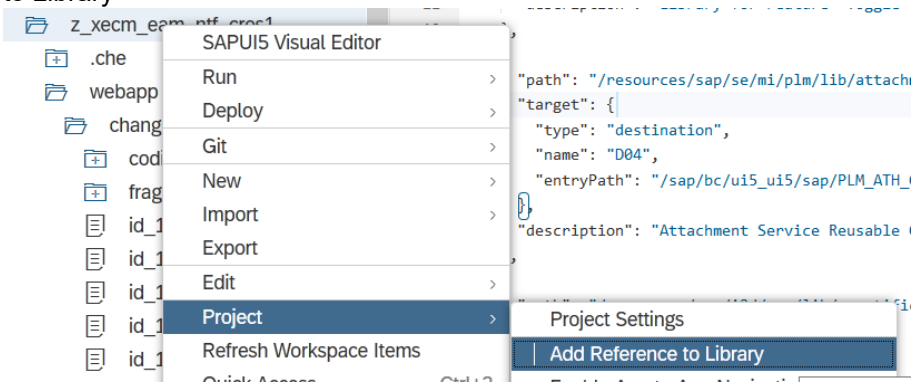
Please note the known issue, that changes to i18n.properties may not work when you launch with Adaptation_index.html or flpSandbox.html but in the preview of the SAPUI5 Visual Editor.

References to additional libraries:

If the original app uses reuse libraries and if you run the application in SAP Web IDE as a tile (flpSandbox.html) you must also reference these libraries in the neo-app.json file.

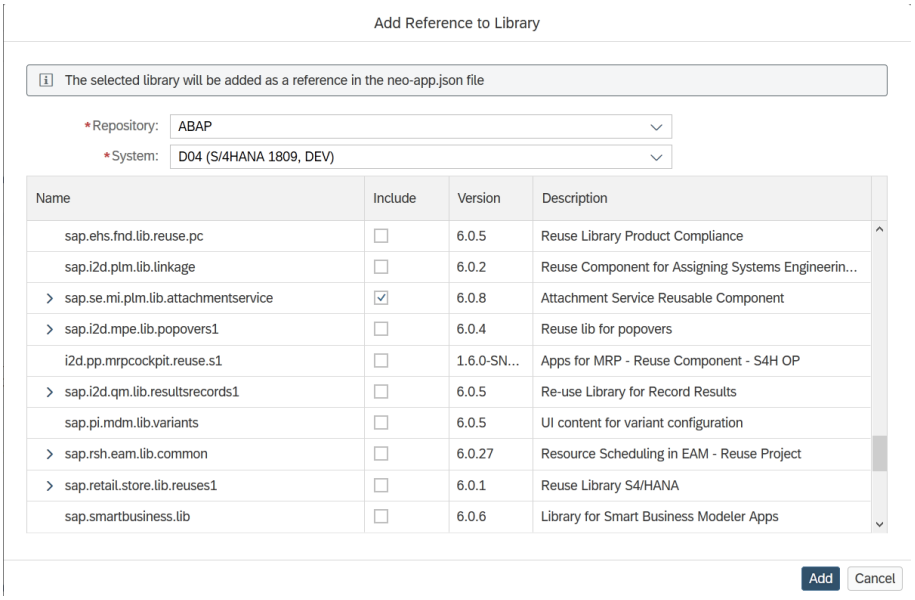
For example: The library sap.se.mi.plm.lib.attachmentservice is required.

1. Select the node of your project.
2. With the right mouse click execute the action Project > Add Reference to Library



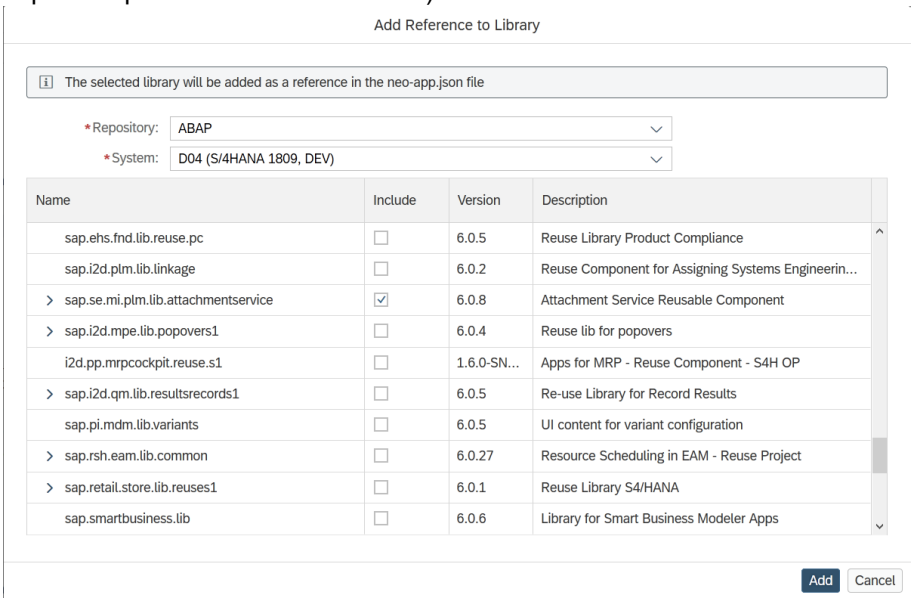
3. In the dialog Add Reference to Library select the Repository (ABAP) and the SAP System.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.



Then after some time the list of libraries is displayed.

4. Use the check box in column include to select the library (for example sap.se.mi.plm.lib.attachmentservice) and click on button Add:



5. The result is that a new entry is added to file **neo-app.json**.

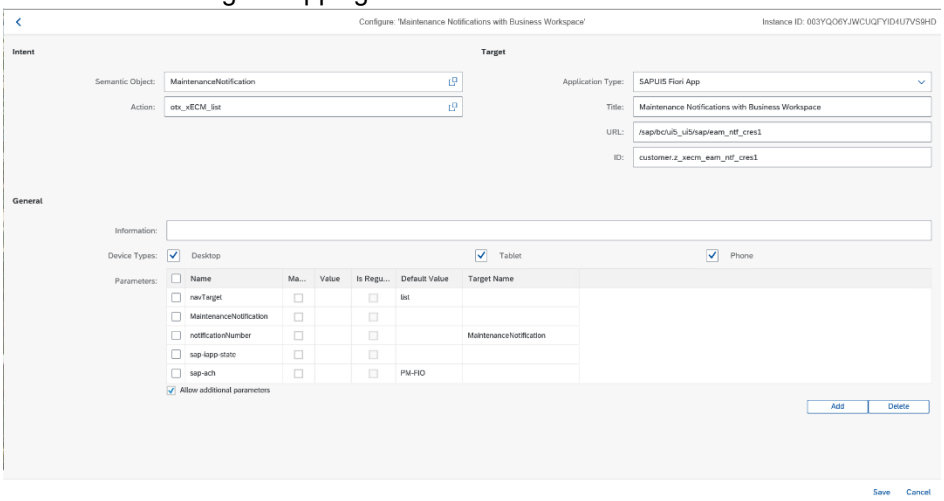
Repeat these steps for every required library reference.

9.7.10 Accessing the adapted Fiori app in on-premise Fiori Launchpad

As soon as you deployed the project to your SAP system it can be made accessible to users with the configuration of the Fiori Launchpad content.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

1. Start the Fiori Launchpad designer (cross-client or client-specific: transactions **`/UI2/FLPD_CONF`** or **`/UI2/FPLD_CUST`**)
2. In the designer create or modify a Fiori catalog.
3. Define a target mapping and a tile. Let us have a closer look at the definition of the target mapping.



   a. For Semantic Object and Action, a unique combination is required. In the example the already existing Semantic Object MaintenanceNotification is used. For Action the arbitrary value otx_xECM_list shall make sure, that the target mapping is unique.

   b. Now at the area Target the relative URL of the original app is used: /sap/bc/ui5_ui5/sap/eam_ntf_cres1
   And for the ID the ID in the manifest.json of the app is used.
   For example: customer.z_xecm_eam_ntf_cres1

```
manifest.appdescr_variant ×
1  {
2      "fileName": "manifest",
3      "layer": "CUSTOMER_BASE",
4      "fileType": "appdescr_variant",
5      "reference": "i2d.eam.pmnotification.create.s1",
6      "id": "customer.z_xecm_eam_ntf_cres1",
```

   Compare with the description at section **9.7.3 CREATING A NEW ADAPTATION PROJECT**. The ID is merged from the Namespace and the Project Name.

   c. The parameter settings in our example are the same as in the target mapping of #MaintenanceNotification-list_simple of Fiori app Monitor Maintenance Requests.

4. Create a tile using the target mapping.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

5. At the end do not forget to create a role in transaction **PFCG**, which contains the new SAP Fiori catalog in tab “Menu”, which is then assigned to the users, who shall have access to the app.

It may be necessary to delete the Fiori tile cache to make the new tile available.

Of course, you may also create a Fiori group which includes the new tile and is then assigned to users with roles containing the Fiori group.

9.7.11 Reusable Code Snippet

9.7.11.1 JavaScript code for calculation of business object and key of Maintenance Notification

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
                        /**
                         * Calculates the SAP BOR object type (SapObject) and SAP BOR key (ObjectId)
                         *   THIS CODE IS DEPENDENT on adapted original app:
                         *
                         *   This code needs to be adjusted to the original app which is adapted.
                         *   In the original app check for a controller file Attachments.controller.js
                         *   probably in a sub folder ext/controller
                         *   and then look at the code in the onInit function.
                         *   This should give you an idea, which runtime data is used to
                         *   initialize the DMS related attachments control of SAP
                         *   The name of the object type is most often not the same as the SAP BOR object
type
                         *   but the ID or key value is often the same
                         */
                        _getObjectWithKey: function () {
                                var oObject = {};
                                /*
                                 * SAP BOR object type for Maintenance Notification
                                 */
                                oObject.SapObject = "BUS2038";
                                /*
                                 * Now do a runtime data retrieval similar to that of the original app
                                 */
                                var sProperty1 = "NotificationNumber";
                                if (this && this.getView && this.getView().getBindingContext &&
this.getView().getBindingContext()) {
                                        var sVal =
this.getView().getBindingContext().getProperty(sProperty1);
                                        // leading zeros with length 12 if MaintenanceNotification only
contains numbers
                                        var oRegex2 = RegExp("^[ 0123456789]*$");
                                        if (sVal && oRegex2.test(sVal)) {
                                                while (sVal.length < 12) {
                                                        sVal = "0" + sVal;
                                                }
                                                oObject.ObjectId = sVal;
                                        } else if (sVal) {
                                                oObject.ObjectId = sVal;
                                        }
                                }
                                return oObject;
                        },
```

Code Snippet 45 JavaScript method _getObjectWithKey( ) for Maintenance Notification

This code snippet can be used for the calculation of the business object key for Maintenance Notifications.

9.8 Integrating own freestyle SAPUI5 apps during creation

9.8.1 General Information and Requirements

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

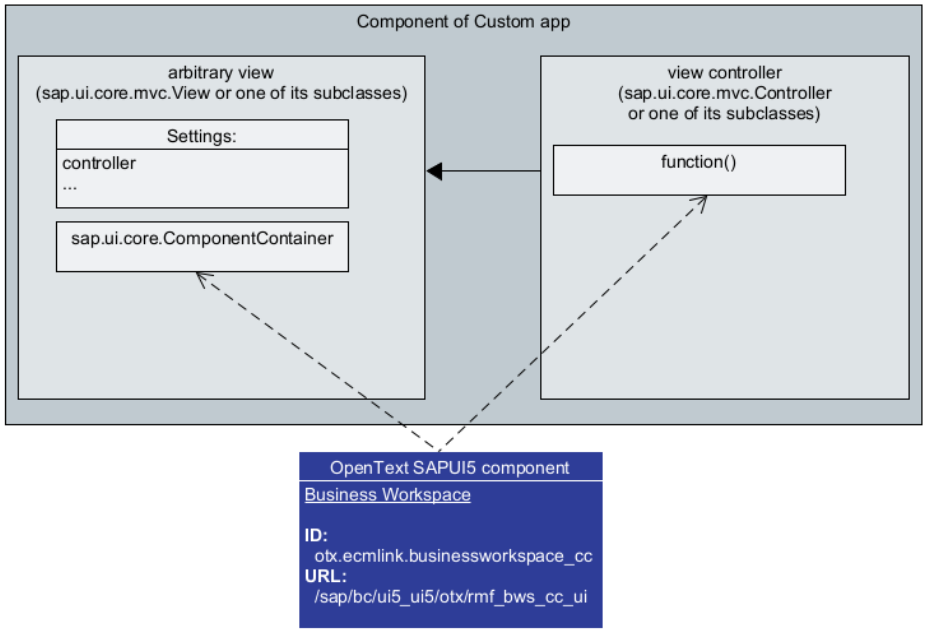


Figure 55 Own freestyle app integrating OpenText controls

The term freestyle app refers to plain SAPUI5 apps using regular controls, which are not created automatically or semi-automatically from any framework like SAP Fiori Elements templates. See also Developing Apps.

A typical SAPUI5 application consists of bootstrap files, views, view controllers, and internationalization files.

If you create your own freestyle app you may add custom code of your choice at creation time, that means, you do not need to use extension techniques to add new controls to your app.

Note the following:

- If you prefer to have an original app without any custom controls from OpenText or other vendors, then extension techniques may be more interesting, even if Using Component Configuration is the only choice.

Here is a list of advantages and disadvantages of the procedure.

Advantages:

+ No special extension techniques required. This way it still can be feasible to do your development without SAP Web IDE
(But SAP Web IDE in the Cloud has the advantage, that you do not need to setup your own local development tools and environment)

+ You can do what you want

Disadvantages:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- Your original app is changed
- You need to take full care of upgrades

This section does not contain any code snippets. Please compare with the previous chapter and look at the OpenText Test launcher app (BSP Node /OTX/RMF_LAUNCH). It is an example for a SAPUI5 freestyle app integrating OpenText business workspace (see customizing guide of OpenText Content Management for SAP Solutions).

The according basic steps are:

- Use sap.ui.core.ComponentContainer to insert one of the following components as a control:
  - Business Workspace Component – see section **9.2.2.1**
- You have the componentCreated event, which can be used to convey the reference to the actual component to be refreshed to a controller.
- Use the refresh() function of the component to initialize and change the parameters
  - Business Workspace – see section **9.2.2.1.5 PUBLIC COMPONENT FUNCTIONS**
- Check, whether you need the parameter UseClassicConfig to switch for development environments – see section **9.2.2.1 BUSINESS WORKSPACE COMPONENT**
- You may use the WidgetStyleClass parameter for a custom height of the wrapping DIV-Element in which the OpenText Business Workspace widget is rendered. See section **9.2.2.1.4 COMPONENT PROPERTIES** As it is your own app, you may include your own CSS file, in which you define the style class selector.

9.9 Integrating into SAP Cloud Platform with external access to on-premise landscape

9.9.1 General Information and requirements

This is a special use case of one or the other integration presented in the previous sections.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Search also for “External Access Point Landscape” in the SAP Fiori Cloud Landscape Configuration Guide at https://help.sap.com:

> *“Landscape scenario for SAP Fiori Cloud that exposes the business data for the SAP Fiori apps to the internet: OData is provisioned on the SAP Cloud Platform directly from back-end systems and all front-end server components are deployed on the SAP Cloud Platform.*
>
> *The architecture uses an Internet-based access point scenario that combines the benefits of a front-end server deployed on SAP Cloud Platform and an on-premise system that holds the business data and the OData services to provide the data to the SAP Fiori apps.”*

Furthermore, in https://help.sap.com in the SAP Cloud Platform Portal for the Neo Environment guide, the section Creating Apps that Access External Resources describes:

- SAP Fiori launchpad provides an automatic routing mechanism for accessing external resources
- It assumes that SAP Fiori launchpad is the sole entry point for running business applications.

There are additional requirements to be considered:

- The OpenText SAPUI5 app for Business Workspace must be activated and hosted on the system, which also hosts the OData services
- Access to further web resources on premise (like Archive Center, Content Suite Viewer, Brava Viewer) may be needed. Then OpenText recommends the use of a web proxy.

9.9.2 Access to other Content Resources

Please note, that the Content Suite Viewer of the OpenText Content Server requires the web browser to get direct access to the Brava Server Instance for document display. OpenText recommends that you have a web proxy that can be used for securing both internal and external access of the Brava Server.

9.9.3 Implementation hints

9.9.3.1 neo-app.json

As pointed out by SAP in their guides, sections are needed in the neo-app.json file for routes.

Use destinations to the SAP Cloud Connector to redirect to paths of the OData Service (/sap/opu/odata) and the OpenText components (for example sap/bc/ui5_ui5/otx) on your SAP Gateway System on premise. Also redirects via Cloud connector need to be defined to the scripting (for example /otcs) and support (for example /img) paths of the OpenText Content Server.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Additionally, the following section needs to be added:

```
"headerWhiteList": [
        "OTDSTicket",
        "otcsticket"
],
```

Code Snippet 46 headerWhiteList in neo-app.json

9.9.3.2 manifest.json

The relative paths for the OpenText components need to be adjusted. One approach to do this is to add the new relative paths into the resourceRoots section of manifest.json:

```
"resourceRoots": {
        "otx.ecmlink.businessworkspace_cc": "./sap/bc/ui5_ui5/otcmfnd/csbws"
}
```

Code Snippet 47 resourceRoots in manifest.json

In *Code Snippet 47 resourceRoots in manifest.json* the path for business workspace with create and complete integration widget (otx.ecmlink.businessworkspace_cc) is changed.

With this, in your custom app or custom extension, you only need to define the ID of the component without URL setting.

9.9.3.3 Initializing OpenText components

In code examples like Code Snippet 28 *Function _getSettingsObject*(), it was suggested to define the parameter UseClassicConfig with value “true”. This is only helpful in internal access if you develop with Web IDE or do not need to work with a reverse proxy. For external access do not set UseClassicConfig. In addition, your leading custom app needs to define the setting PrefixPath when initializing the component of the OpenText business workspace. In an XML view of your custom app, this may look like the following:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
<mvc:View controllerName="otxxecm4scp.xecm_launch.controller.View1" xmlns="sap.uxap"
xmlns:mvc="sap.ui.core.mvc"
        xmlns:layout="sap.ui.layout" xmlns:m="sap.m" xmlns:core="sap.ui.core"
xmlns:forms="sap.ui.layout.form" displayBlock="true">
…
                                                <core:ComponentContainer id="otx_xecm_cc_compcont"
name="otx.ecmlink.businessworkspace_cc"
                                                        settings="{ parts: ['launch>/PrefixPath'], formatter:
'._formatBWSCCSettings' }" componentCreated="onComponentCreated"></core:ComponentContainer>
…
```

Code Snippet 48 XML example for external access

In Code Snippet 48 XML *example for* external access the business workspace component otx.ecmlink.businessworkspace_cc is defined within a sap.ui.core.ComponentContainer. A function onComponentCreated within the controller is used to process steps after the component was created. The settings for the component are defined by a formatter function which uses the parts array as input. In this case the only item in the parts array is the data property PrefixPath in a data model named “launch”. Replace this by your own data model and property. The next code is an example for the formatter function:

```
        _formatBWSCCSettings: function (sPrefixPath) {
                return {
                        AppMode: false,
                        PrefixPath: sPrefixPath,
                        WidgetStyleClass: "otx_bws_cc_div",
                        WkspDisplayMode: 'fullPage'
                };
        },
```

Code Snippet 49 example of function _formatBWSCCSettings

The formatter returns a JSON object of the settings (see *9.2.2.1.4 Component properties*). AppMode is false because the component is not rendered standalone. The parameter PrefixPath is filled from the first item in the parts array in the XML view. Furthermore, a widget style class is initiated to define the height of the widget. And the widget is initiated in fullPage mode.

The next code example shows how your leading app can calculate the prefix path. Please note, that this example works if Component.js (or if built, Component-preload.js) runs under the root folder of the app. Depending on your build environment (for example, UI5 build or Grunt) this may need further

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

adjustments. You can also define the prefix path by a fixed value. Then you may also hardcode all settings directly into the XML view.

```
try {
    var sResolveUri = this.getOwnerComponent().getMetadata().getManifestObject().resolveUri("");
    if (sResolveUri === "./") {
        oLaunchModel.PrefixPath = "";
    } else {
        var rPattern = new RegExp('^(.*)\\/$');
        sResolveUri = sResolveUri.replace(rPattern, "$1");
        oLaunchModel.PrefixPath = '/' + sResolveUri;
    }
} catch (err) {
    oLaunchModel.PrefixPath = "";
}
//add model to view
this.getView().setModel(new JSONModel(oLaunchModel), "launch");
```

Code Snippet 50 Using resolveUri() to calculate external path

In *Code Snippet 50 Using resolveUri() to calculate external path* the method resolveUri() of sap.ui.core.Manifest is used to calculate the relative path. It is propagated as PrefixPath to the business workspace component. In this example, the JSON object oLaunchModel is later used to define a new data model named “launch” as referenced in our example of the XML view.

9.10 Integrating into SAP S/4HANA Cloud Essential Edition

9.10.1 General Information and Requirements

The SAP Cloud Essential Edition is a multi-tenant SaaS where one instance of a software stack serves multiple customers. That means configuration is restricted, but extensibility can be achieved by whitelisted APIs.
You can integrate the business workspace into Fiori apps of S/4HANA Essential Edition which have Smart Link navigation or Related Apps button.

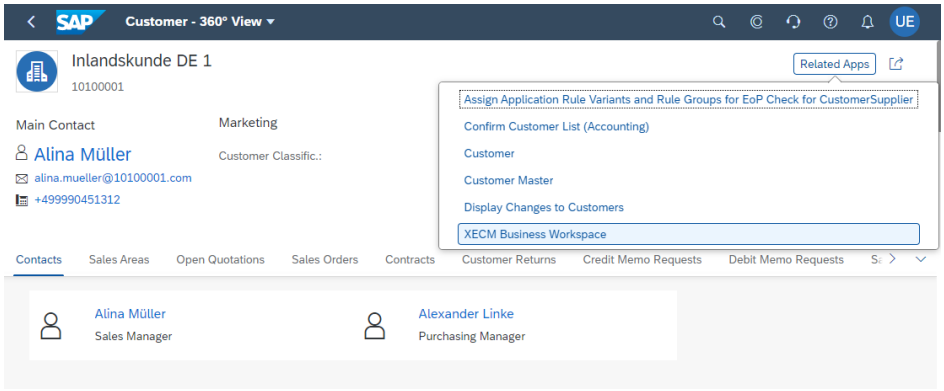
An SAP S/4HANA Cloud instance with an integrated development environment is required. (This document is based on Web IDE, but similar steps are required if you are using the Business Application Studio.). In addition, you

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

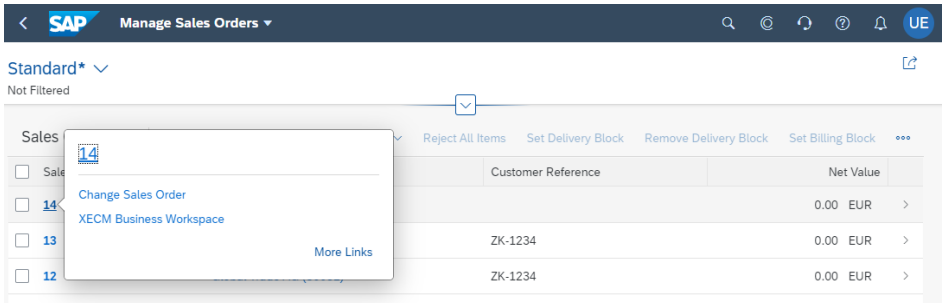
need the template UI application provided by OpenText (ZOTX_WKSP.zip), see customizing guide for location of this file. If SSO is required, an additional trust configuration via SAML based SSO is required between your IDP (identity provider) and the Open Text Directory Services. For further information about the authentication see the customizing guides of OpenText Content Management for SAP Solutions.

9.10.2 Examples for the UI integration

Customer – 360° App (F2187) is a Fact sheet app which has the Related Apps button:



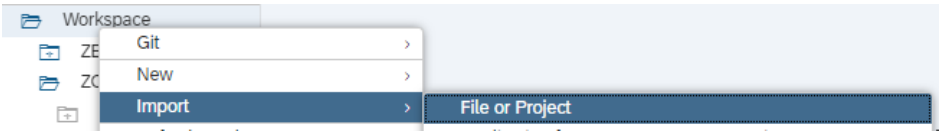
The Manage Sales Order app is a transactional app with Smart Link navigation:



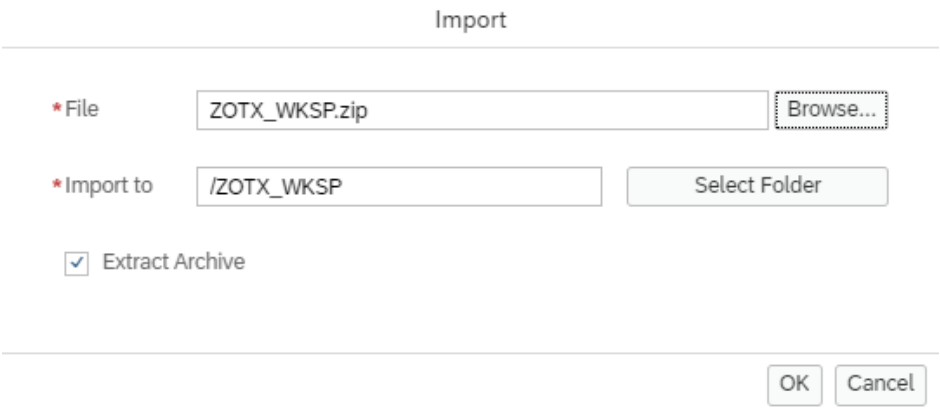
9.10.3 Implementation steps

9.10.3.1 Import the template application

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.



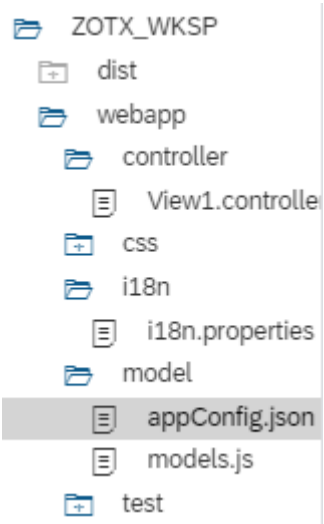
Use the Web IDE context menu to import the template application from the .zip file



Import the application into a new folder the let Web IDE extract the archive.

9.10.3.2 Adapt the template code

Navigate to the appConfig.json file and open it for editing. This file contains important technical information required for Open Text Content Server.



Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

appConfig.json

```
{
    "iframeService":{
        "schema": "https",
        "host":"<-- put the Content Server hostname here -->",
        "port":443,
        "service":"otcs/cs.exe?func=xecmpf.GetWspIntegration&bctx={BCONTEXT}"
    },
    "systemId":"<-- put unique system ID here -->",
    "contextMapping":{
        "Customer": {
            "businessObject": "BusinessPartner",
            "keyMapping": {"Customer":"BusinessPartner"}
        },
        "BusinessPartner":{
            "businessObject":"BusinessPartner",
            "keyMapping":{"Supplier":"BusinessPartner"}
        },
        "SalesOrder":{
            "businessObject":"SalesOrder",
            "keyMapping":{"SalesOrder":"SalesOrder"}
        },
        "Material":{
            "businessObject":"Product",
            "keyMapping":{"Product":"Product"}
        },
        "SalesInquiry":{
            "businessObject":"SalesInquiry",
            "keyMapping":{"SalesInquiry":"SalesInquiry"}
        },
        "SalesQuotation":{
            "businessObject":"SalesQuotation",
            "keyMapping":{"SalesQuotation":"SalesQuotation"}
        }

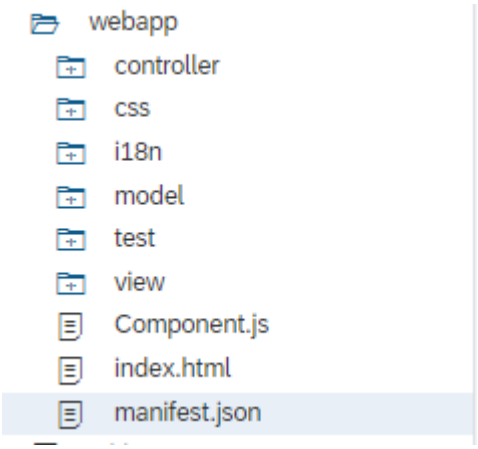
    }
}
```

Replace the placeholders with valid values. The “systemId” must match the connection name defined on Content Server side under “Connections to Business Applications (External Systems)”.

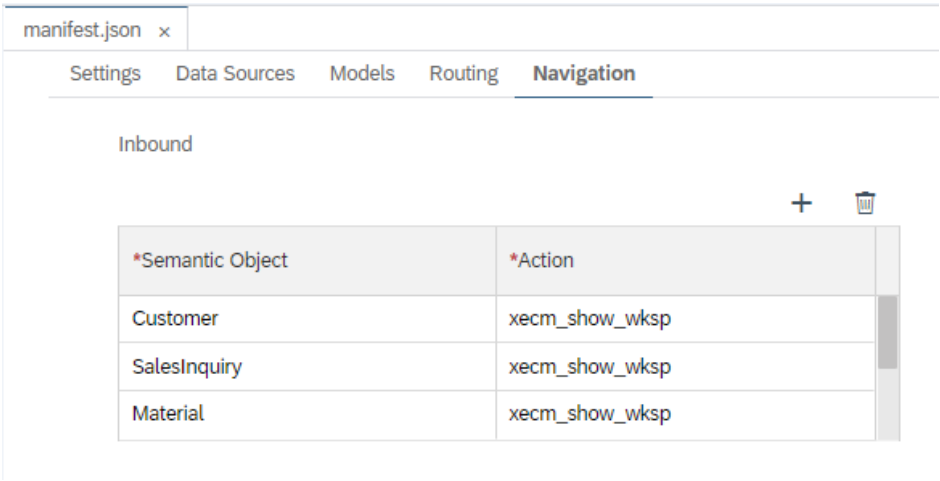
In the context mapping you see already some examples how the “Semantic Object” is mapped to business object type and business object key. For additional Fiori apps you must enhance the context mapping by extracting the corresponding values from the URL of your Fiori app.
On Content Server side a business object configuration must have been configured according to the customizing guides of OpenText Content Management for SAP Solutions.

Besides, additional Fiori apps must be leveraged in the manifest.json as well. Navigate to the manifest.json file:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

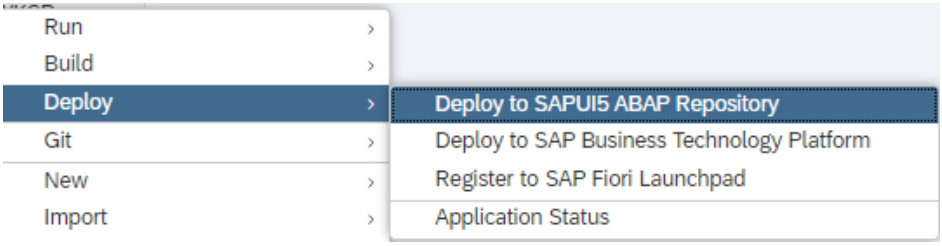


After double click on this file the “Descriptor Editor” opens. Go the “Navigation” tab and add additional “Semantic Objects”:

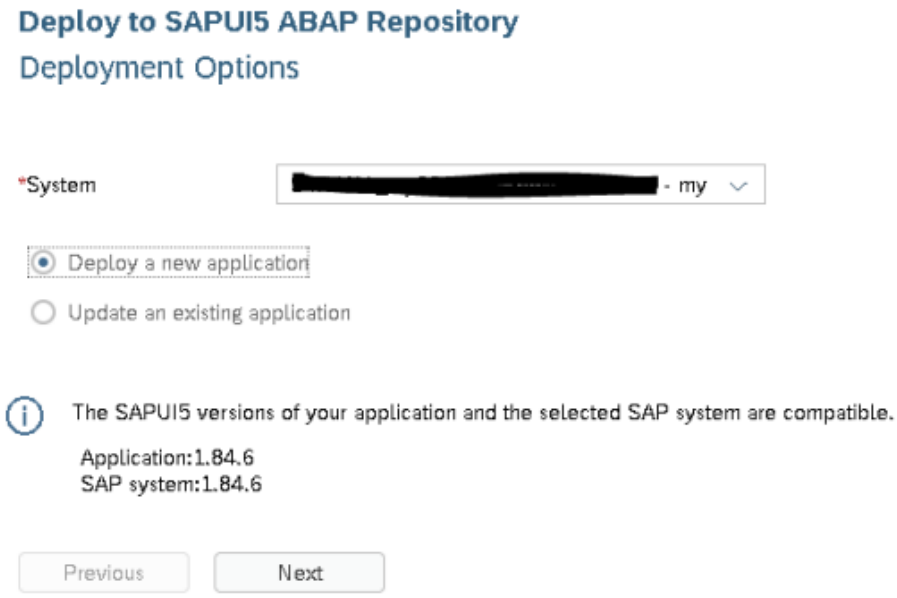


9.10.3.3 Deploy the custom UI5 application

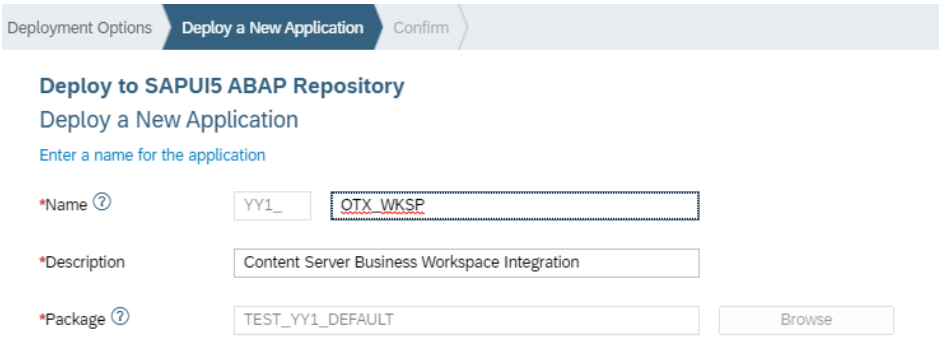
Right-click on the project folder and start application deployment



Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.



Choose your system.



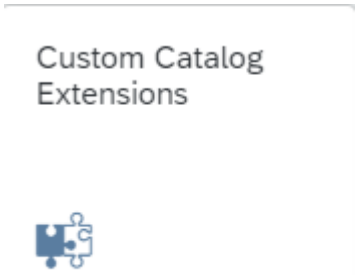
Enter the application name, finish the wizard, and wait until the application is deployed into the SAP S/4HANA Cloud system.

9.10.3.4 Business catalog assignment

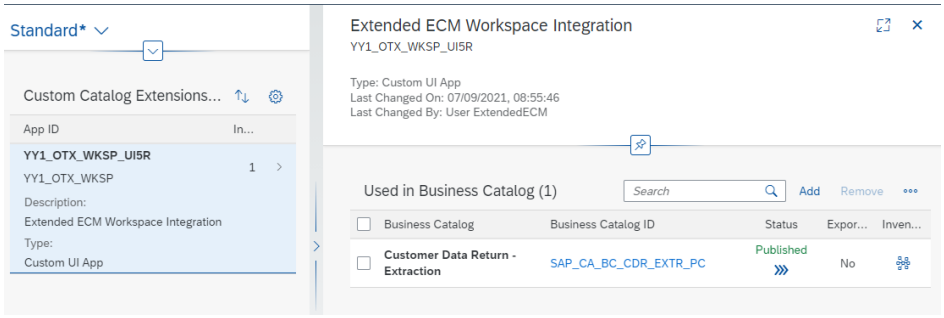
Once the application is deployed, you need to assign it to a business catalog to make it accessible to business users.

Use the custom catalog extension app to assign the new application to a catalog:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

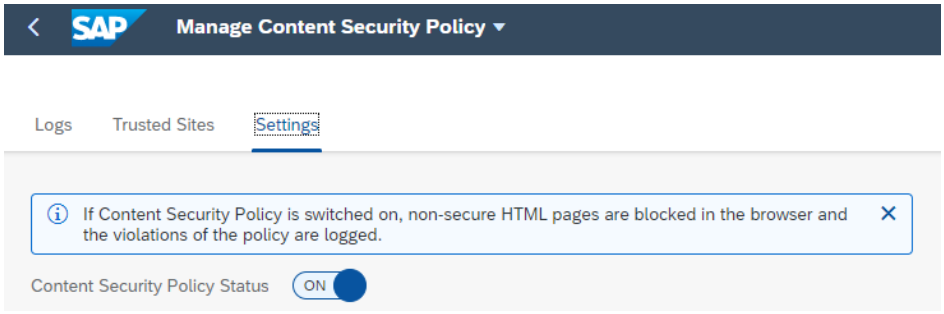


Search for the app. After you find it, click the list item:



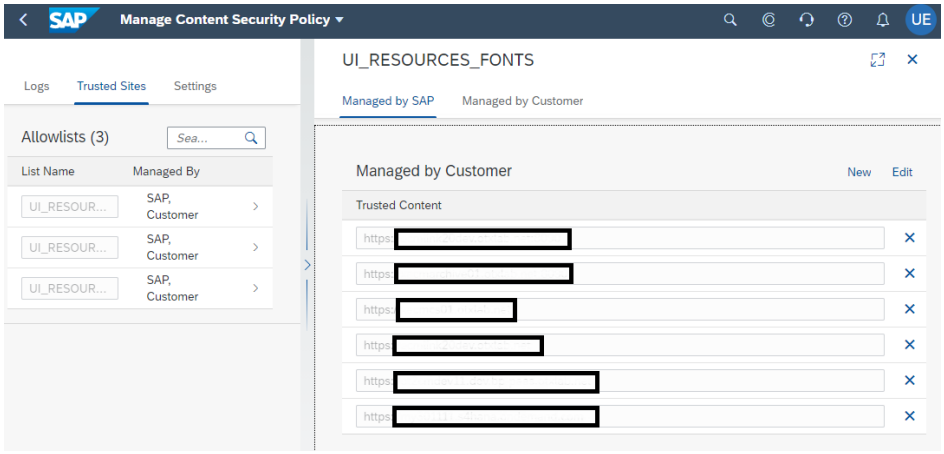
Click Add to assign a catalog which is included in the role(s) that need access to the custom app and publish the changes.

9.10.3.5 Security policy settings



If the security settings are active, you need to create custom rules to allow the integration.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.



Maintain your Content Server entry for each resource type in the list

9.10.3.6 Maintain Content Server Trusted Websites

Enter on Content Server in Administration -> Security -> Trusted Websites your SAP Cloud S/4HANA Url.

9.11 Resolving cross-origin resource sharing scenarios with Content Server

The SAPUI5 framework (at least when executed on-premise in an SAP Fiori Launchpad) rewrites httpRequest.send.
Starting with release 22.3 Content Server sends additional headers in the HTTP response to overcome cross-origin resource sharing (CORS) issues.

The following scenarios are possible:

1. You use a reverse proxy to redirect from the SAP Fiori Launchpad and the Content Server.
   This corresponds to the use of parameter UseClassicConfig=false or if the parameter UseClassicConfig is not set.
   Usage of UseClassicConfig=true should have no negative impact either.

2. You integrate into SAP Cloud platform with external access to on-promise landscape. You must use UseClassicConfig=false or do not set parameter UseClassicConfig.

3. No reverse proxy is used. Then you must make sure that parameter UseClassicConfig=true is set.
   No workarounds with rewrite engine are required anymore.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

The first option also satisfies the security requirement, to avoid that the web servers are directly exposed.

The second option makes use of the SAP cloud platform as proxy.

In former releases the third option required further workarounds like, for example the usage of a rewrite engine. That is not required anymore. It is required, that the host of the SAP Fiori Launchpad is matching the whitelist of trusted websites in the security parameters of the Content Server.

See section SAPUI5 app for Business Workspace at page 117 for further details about parameter UseClassicConfig.

9.11.1 Example for scenario 1: Using SAP Web Dispatcher in SAP Fiori scenarios

This chapter is not a complete description of how to set up and maintain an SAP Web Dispatcher. It points out important aspects of customizing in context of SAP Fiori scenarios with OpenText Content Management for SAP Solutions.

Please read SAP online documentation (for example at https://help.sap.com) to learn about architecture, functions, and administration of an SAP Web Dispatcher.

9.11.1.1 Customizing OTDS and Content Server

Add OpenText OTDS and Content Server as external systems to your SAP Web dispatcher profile, for example:

```
wdisp/system_1 =
SID=EX1,EXTSRV=http://deotds01.sapcustomer.com:8080,SRCSRV=*:*,SR
CURL=/otdsws,SSL_ENCRYPT=0
```

```
wdisp/system_2 =
SID=EX2,EXTSRV=http://deotcontent.sapcustomer.com:80,SRCSRV=*:*,S
RCURL=/otcs;/OTCS;/img;/IMG,SSL_ENCRYPT=0
```

9.11.1.2 Defining custom header parameters X-Forwarded-...

Important: Since version 20.4 and if UseClassicConfig parameter is false or not set, then the business workspace app now calls the widget script with crossOrigin=false matching to same origin.

9.11.2 Scenario 2 - SAP Cloud platform with external access to on-promise landscape

Read section *9.9 Integrating into SAP Cloud Platform with external access to on-premise landscape* for details.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.11.3 Scenario 3 – no reverse proxy

As the Content Server now adds additional headers to the HTTP response no extra configuration like using an IIS rewrite engine is required anymore.

Please make sure, that the integration widget is called with parameter UseClassicConfig=true. Then the URL of the support folder and the widget script are calculated from the connection data defined in the OpenText Content Management for SAP Solutions connection. The widget script is called with crossOrigin=true.

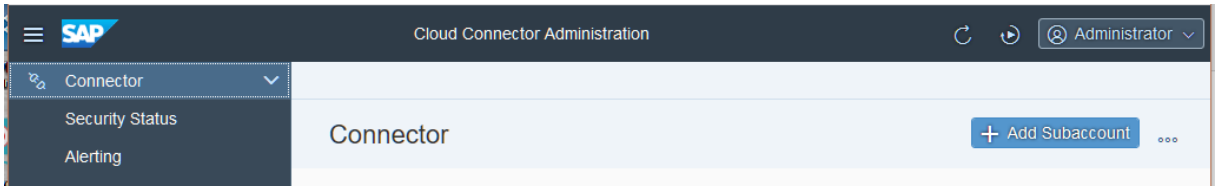
9.12 Basic Steps using the SAP Cloud Connector and the SAP Web IDE on the SAP Cloud Platform

Most of the integration techniques presented in this guide require the use of the SAP Web IDE (Full-Stack) hosted on the SAP HANA Cloud platform.
The descriptions are related to a full Cloud platform access, but many of the functions are also available to users with a trial access.

- Your company has a global account on http://hana.ondemand.com and below that you have access to a subaccount (Your s-user is member of that subaccount with Developer or even Administrator role)
- Typically, the subaccount has settings like

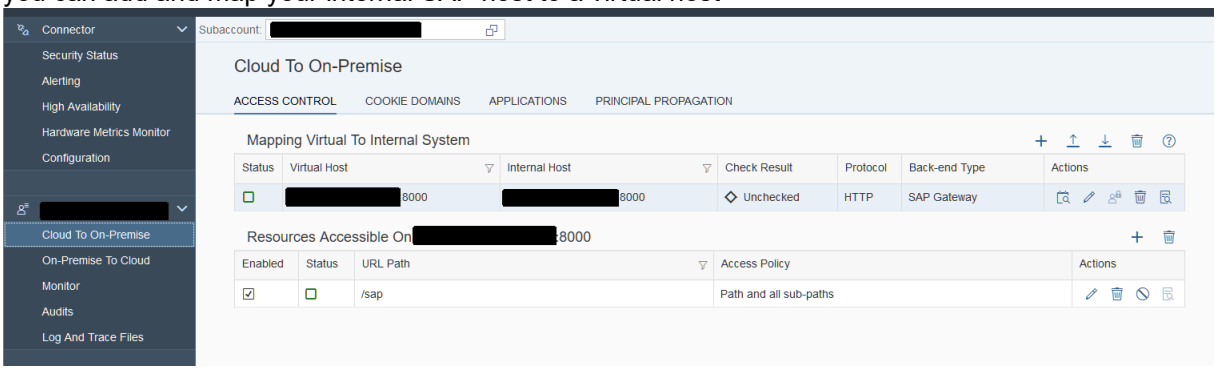
  Environment: Neo
  Provider: SAP
  Region: Europe (Rot)

  With a region with good access from your office.
- On the side of your on-premise landscape you host an SAP Cloud connector (see https://tools.hana.ondemand.com/#cloud).
- In the administration of the SAP Cloud connector, you have added the subaccount



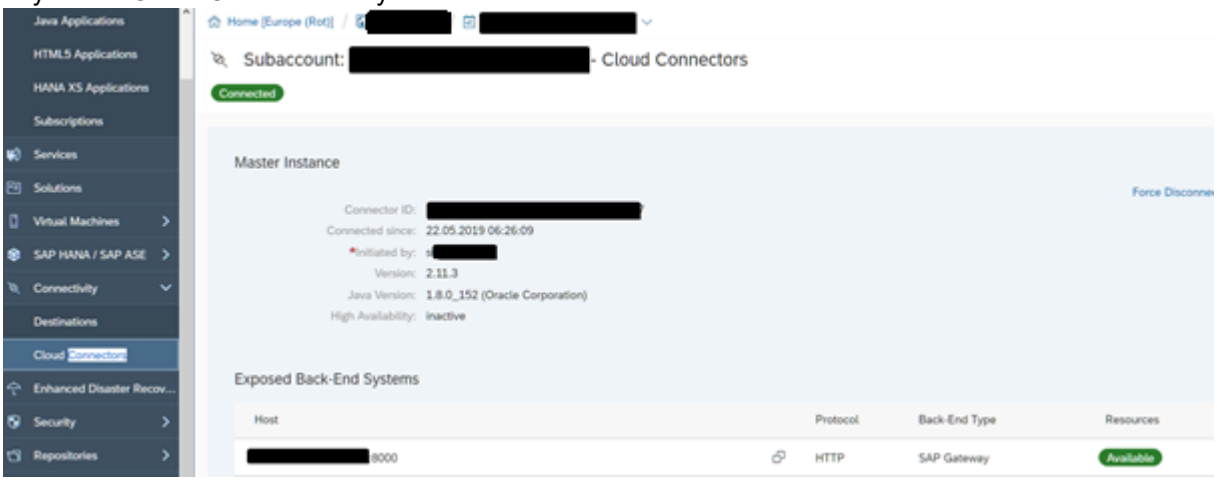
Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- Then select your subaccount and click on Cloud to On-Premise here you can add and map your internal SAP host to a virtual host



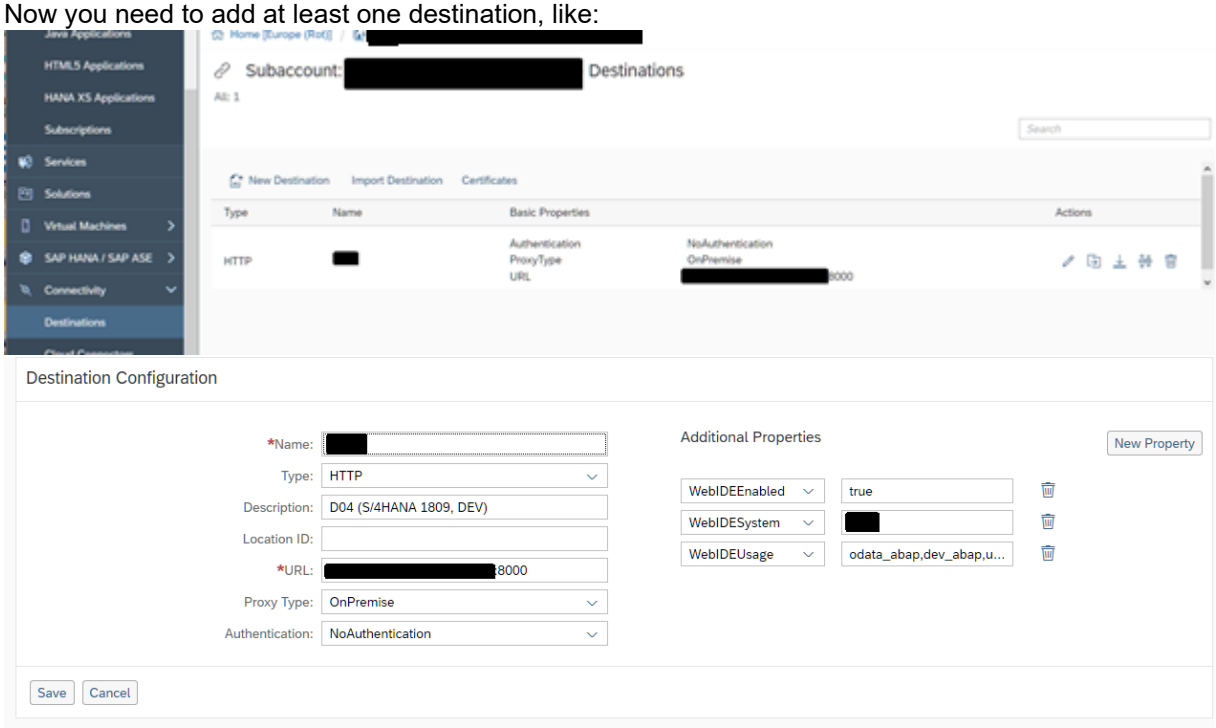
Typically, the Back-end Type will be SAP Gateway and the host will be your SAP Gateway server.
You also need to decide which paths and sub-paths shall be available via the tunnel.

- Back on the SAP Cloud platform under node “Connectivity” you find any active Cloud Connector for your subaccount:



Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

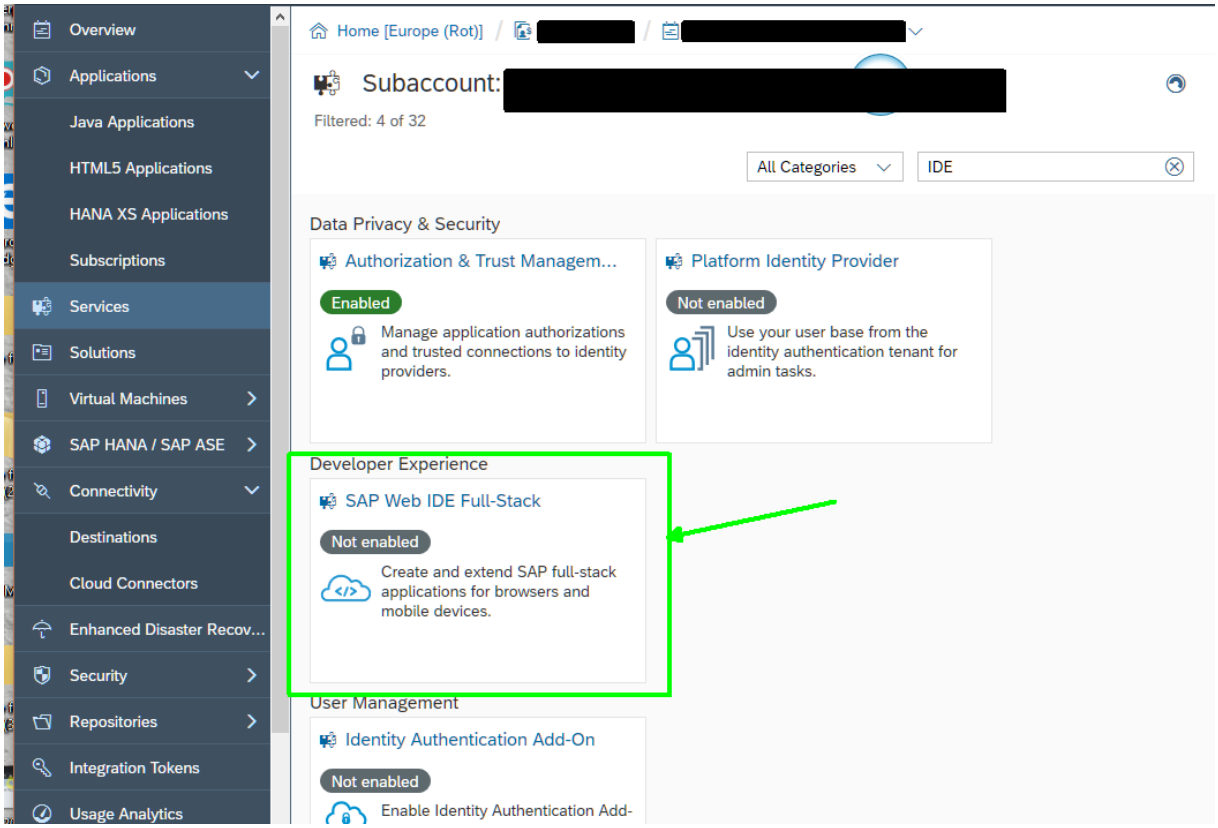
- Now you need to add at least one destination, like:



  - WebIDESystem = 3-digit system ID of your SAP system
  - Suggested values for WebIDEUsage:
    odata_abap,dev_abap,ui5_execute_abap

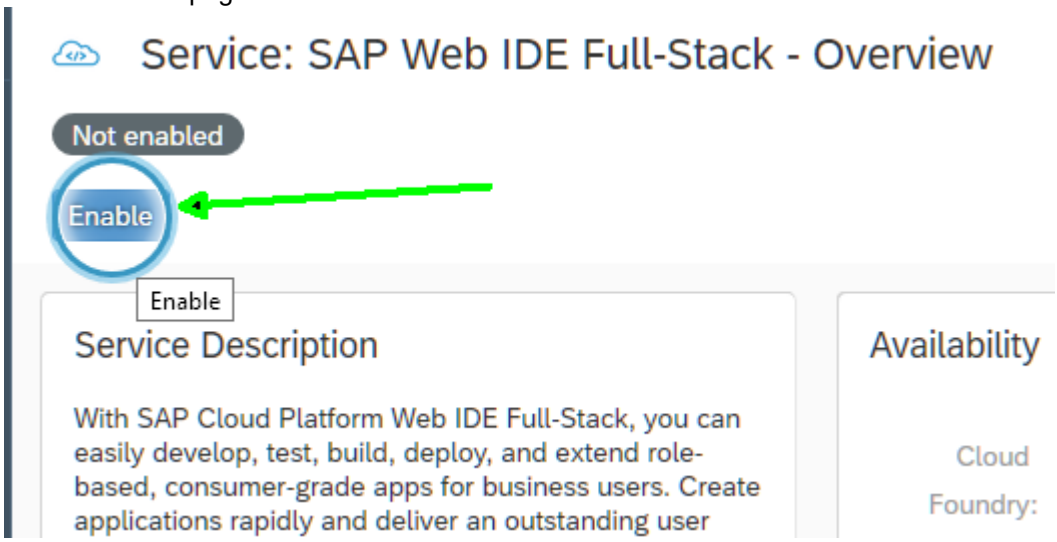
Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- Now at node Services



  - Search for SAP Web IDE Full-Stack
  - Click on the tile

- And in the next page click on Enable

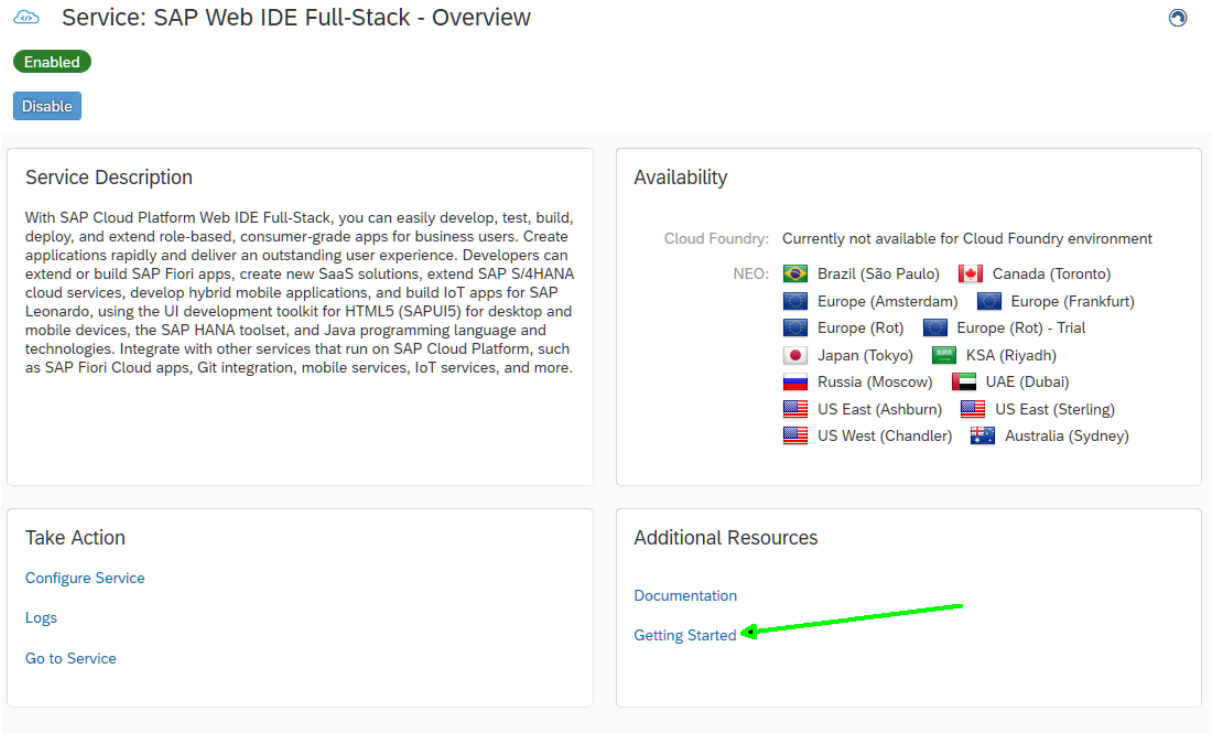


Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- After some time of processing the display changes to



- To learn how to configure and start working with the service follow the documentation at:



  - If everything is setup you can click on Go to Service to start a new browser page with the actual service

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

9.13 Behavior on mobile phones

Starting with version 22.4 we can discern between the following use cases.

9.13.1 Usage of Startup parameter mobileOpenDirectly=true

This is described in *Integrating with cross-app (intent-based) navigation*.

9.13.2 Other

This describes the behavior for all other cases. This means that the URL parameter mobileOpenDirectly is false or is not set at all.

This is also true if you migrate from a previous version without mobileOpenDirectly.

The SAPUI5 based component otx.ecmlink.businessworkspace_cc starts the Create and Complete integration widget.

The widget detects whether it runs on an Android phone or iPhone.

- If it is an Android phone
    - If the business workspace does not exist: The widget displays a message “No Business Workspace available.”
    - If the business workspace exists, the widget displays a message like “Open Business Workspace in CS mobile app” and a Continue button. When users click the button:
        - If the Content Server Mobile App is installed: The login page of the installed Content Server Mobile app is started. After successful login, the content of the business workspace is displayed in the app.
        - If the Content Server Mobile App is not installed: Users are redirected to Google Play to get the Content Server Mobile app
- If it is an iPhone
    - If the business workspace does not exist: The widget displays a message “No Business Workspace available.”
    - If the business workspace exists, the widget displays like “Open Business Workspace in CS mobile app” and a Continue button. When users click the button:
        - If the Content Server Mobile App is installed: The login page of the installed Content Server Mobile app is started. After successful login, the content of the business workspace is displayed in the app.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

- If the Content Server Mobile App is not installed: Users are redirected to App Store to download the Content Server Mobile app.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

10 OpenText™ Activator for SAP Solutions

This chapter describes the components of OpenText Activator for SAP Solutions and contains tutorials for its implementation.

10.1 About OpenText Activator for SAP Solutions

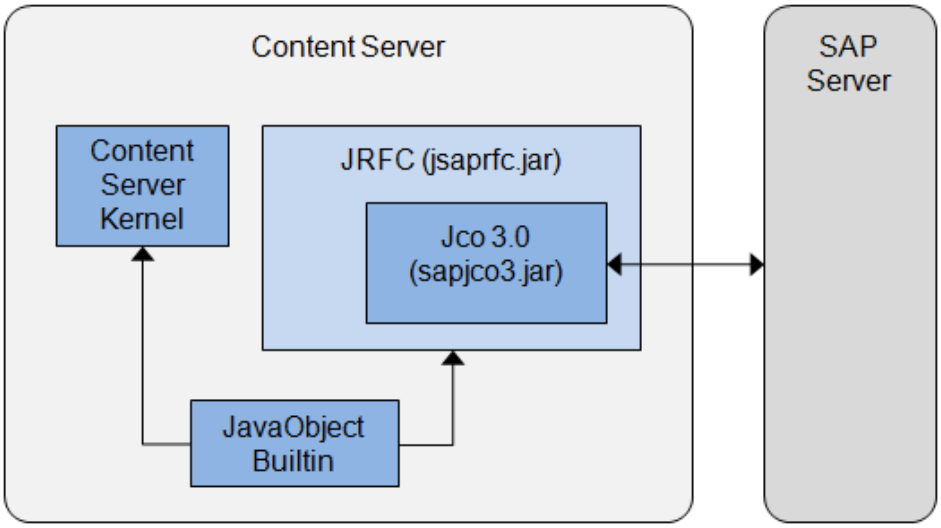
OpenText Activator for SAP Solutions (Activator) is a Content Server functionality that offers integration to SAP systems. The Activator API allows OScript programs to connect to SAP systems and call RFCs and BAPI methods.

Connecting to an SAP system requires specifying various connection parameters; these parameters can be passed via the OScript program. Activator can administer connection parameters of various SAP systems in the Content Server Administration and use just the logical SAP system name to connect from within OScript.

10.2 Components

The Content Server loads a Java library (jsaprfc.jar) that contains a Java class (JRFC) that acts as a wrapper of SAP's JCo library.

The communication between Content Server and an SAP server is done by the SAP JCo 3.0 library.



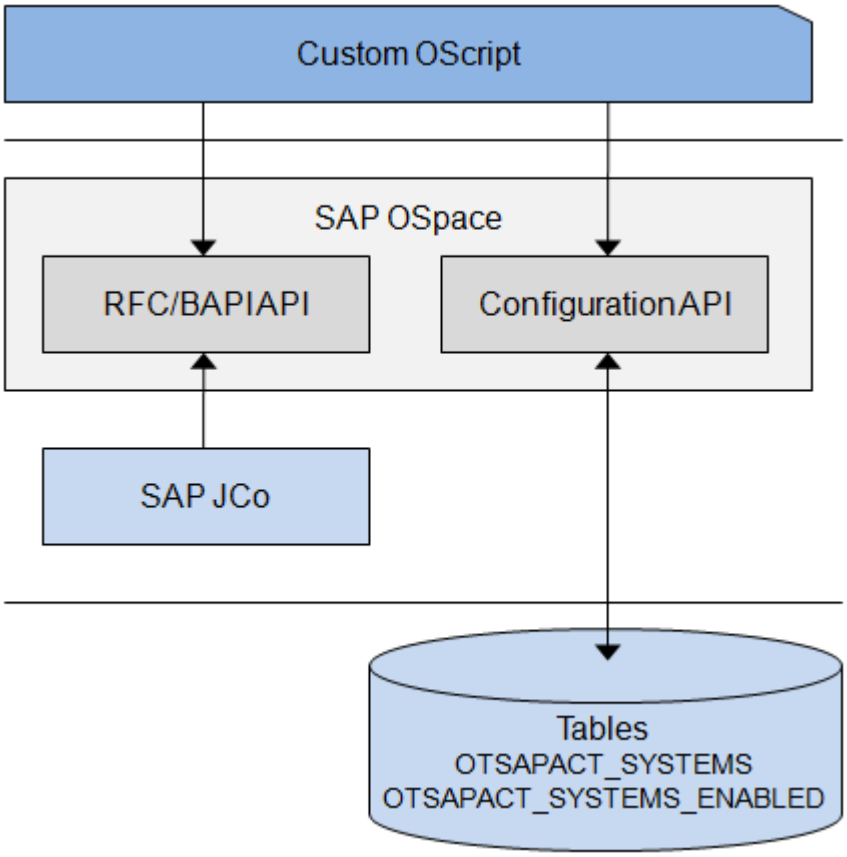
Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Activator implements storage for connection configuration data in the Content Server database. Applications can use the SAP Systems configuration pages provided by Activator, which eliminates the need of custom implementation of such configuration pages.

For detailed information on how to use the configuration pages, see OpenText Content Management for SAP Solutions - Customizing Guide.

All functions needed to call RFCs or BAPIs are included in the SAP OSpace; this SAP OSpace wraps the Java calls needed to perform RFCs. All database access regarding SAP system configuration data are hidden in the configuration API of the SAP OSpace.

Custom OScript can use the configuration API to retrieve the list of configured SAP systems and to connect to a specific SAP system. To call RFCs or BAPIs, the RFC/BAPI API of SAP OSpace provides the necessary functions.



10.3SAP OSpace

Activator encapsulates the SAP JCo Java classes within an OScript API.

The API is subdivided in two parts:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

RFC/BAPI API

This API creates new temp objects for RFCs or BAPI methods and provides importing, exporting and table parameters as features of these temp objects.

Configuration API

This API provides access to the SAP systems configuration data that is maintained by administrators in the Content Server administration pages.

10.3.1 RFC/BAPI API

10.3.1.1 $SAP.RFC ConnectEx function (String, String, String, String, String, String, String, String, String, String, String)

Connects to the specified SAP system and returns the initialized JCoDestination object.

Parameters

destination

Type: string

The SAP destination name; this parameter is case-sensitive.

type

Type: string

The RFC connection type. Use value A for the application server (parameter ashost must be specified), value B for load balancing using message server (parameters mshost and pool must be specified).

ashost

Type: string

The SAP ABAP application server hostname; mandatory for a direct connection. Specify empty string if RFC connection type is B.

mshost

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Type: string

The SAP message server hostname; mandatory for a logon balancing connection. Specify empty string if RFC connection type is A.

sysnr

Type: string

The system number of the SAP ABAP application server; mandatory for a direct connection.

client

Type: string

The SAP client; mandatory logon parameter. Client numbers are usually three-digit numbers.

user

Type: string

The SAP username to log in to the SAP system.

pass

Type: string

The password associated with the SAP username.

lang

Type: string

The code for the logon language; parameters are e.g. DE (German) or EN (English).

groupname

Type: string

The group of SAP application servers; mandatory for a logon balancing connection.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

sysid

Type: string
The system ID of the SAP system; mandatory for a logon balancing connection.

Return Value

Type: Assoc
.OK: true if call of $SAP.RFC Connect was successful, otherwise false.
.errMsg: error message (.OK is false).
.connection: JCoDestination object already connected to SAP system (.OK is true).
.exception: Assoc with exception data if .OK is false, and an exception occurred.

10.3.1.2 $SAP.RFC New function (Dynamic, String)

Creates a new temp of the base RFC object if it does not already exist for the given RFC name. The feature jcoConnection will be set with the given JCoDestination object; the features OSName and fRfcName will be set with the name of the RFC.

In $SAP.RFC New, the target SAP system gets called to retrieve the description of the function interface of the desired RFC; the function then uses this information to create features for each RFC parameter. In addition, an Assoc fImports is created with all import parameters and an Assoc fExports is created with all export parameters. Table parameters are added to the fImports and fExports.

To set values of importing or table parameters, just set the values of the features with the name of the parameter in the newly created RFC object.

Parameters

connection

Type: dynamic

The JCoDestination instance to be used for connecting to the SAP system.

rfcName

Type: string

Name of the RFC for which the RFC object should be created.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Return Value

Type: $SAP.RFC

A new temp of a $SAP.RFC object or undefined in case of an error.

10.3.1.3 $SAP.RFC Call function

Calls the RFC that was specified with the $SAP.RFC New function. The implementation of the $SAP.RFC Call function passes every importing or table parameter to the actual JCo call and executes the RFC. The features of the $SAP.RFC object that represent the export or table parameters are set with the values returned by the RFC.

Return Value

Type: Assoc

.OK: true if execution of $SAP.RFC Call was successful, otherwise false.

.message: error message (.OK is false).

.stacktrace: Java stacktrace (.OK is false).

10.3.1.4 $SAP.RFC Delete function ($SAP.RFC temp object)

Releases the temp object of $SAP.RFC; this also releases the internally stored JCoDestination object.

Parameters

rfcObject

Type: object

The $SAP.RFC temp object to be deleted.

10.3.1.5 $SAP.BAPI ConnectEx function (String, String, String, String, String, String, String, String, String, String, String)

Identical to the $SAP.RFC ConnectEx function; for details, see 10.3.1.1.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

10.3.1.6 $SAP.BAPI New function (Dynamic, String, String)

Creates a new temp of the base BAPI object if it does not already exist for the given BAPI name and BAPI method. Then it creates a new child object of the temp for the specified method.The feature jcoConnection will be set with the given JCoDestination object; the features OSName and fBapiName will be set with the name of the BAPI.

The implementation of the $SAP.BAPI New is calling SWO_BUSINESS_OBJECTS_GET to get the list of BAPIs and then RPY_OBJECTTYPE_READ to get keyfields, methods, methodParams, methodExceptions and documentation of the BAPI.

For each method a new child object of the new temp of $SAP.BAPI is created. The feature OSName is set with the method name. The feature fBapiMethodName is set with the uppercase method name. Features are created for each import, export and table parameter. In addition, an Assoc fImports is created with all import parameters and an Assoc fExports is created with all export parameters. Table parameters are added to the fImports and fExports, exceptions are added to the Assoc fExceptions. To set values of importing or table parameters, just set the values of the features with the name of the parameter in the newly created BAPI method object.

Note:

$SAP.BAPI behaves like $SAP.RFC but with the difference that for the BAPI methods new child objects are created and the parameters are added as features to this child object.

Parameters

connection

Type: dynamic

The JCoDestination instance to be used for connecting to the SAP system.

bapiName

Type: string

Name of the SAP business object type (BAPI).

bapiMethod

Type: string

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Name of the method of the specified SAP business object type.

Return Value

Type: $SAP.BAPI

A new temp of a $SAP.BAPI object or undefined in case of an error.

10.3.1.7 $SAP.BAPI Call function

Identical to the $SAP.RFC Call function; for details, see 10.3.1.3.

10.3.1.8 $SAP.BAPI Delete function ($SAP.BAPI temp object)

Identical to the $SAP.RFC Delete function; for details, see 10.3.1.4.

10.3.2 Configuration API

Activator has a database table (OTSAPACT_SYSTEMS_CFG) for storing configuration data, which is maintained by an administrator using the SAP Systems administration pages of the Content Server Administration. An OScript API allows retrieving the configuration data without needing to know the schema of the database table. Therefore, OpenText recommends using the following functions to access the SAP system configuration.

10.3.2.1 $SAP.SapSystems ConnectToDestination function (Object, String)

Connects to an SAP system that is configured with its connection parameters in the SAP Systems administration pages of the Content Server Administration. All connection parameters required, including username and password are taken from the SAP system's configuration settings that are stored in the Content Server database.

Parameters

prgCtx

Type: object

Program context

destination

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Type: string

Name of the logical SAP system. Must be configured in the SAP Systems section of the Content Server Administration.

Return Value

Type: Assoc

.OK: true if call of $SAP.RFC Connect was successful, otherwise false.

.errMsg: error message (.OK is false).

.connection: JCoDestination instance (.OK is true).

10.3.2.2 $SAP.SapSystems ExistsSapSystem function (Object, String)

Checks the Content Server database for an SAP system configuration for the specified logical SAP system name.

Parameters

prgCtx

Type: Object

Program context

destination

Type: String

Name of the logical SAP system.

Return Value

Type: Assoc

.OK: true if call of $SAP.RFC Connect was successful, otherwise false.

.errMsg: error message (.OK is false).

.found: number of configured SAP systems with the specified logical SAP system name.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

10.3.2.3 $SAP.SapSystems GetEnabledSapSystems function (Object)

Returns an Assoc with the configuration data of all configured SAP systems that are enabled.

Parameters

prgCtx

Type: object

Program context

Return Value

Type: Assoc

.OK: true if call of $SAP.RFC Connect was successful, otherwise false.

.errMsg: error message (.OK is false).

.sapsystems: Assoc with an Assoc item for each SAP system. The key in the .sapsystems Assoc is the logical SAP system name. Each of the SAP system items has the following elements:

- DEST: Logical SAP system name.
- ASHOST: Application server hostname.
- MSHOST: Message server hostname.
- GROUPNAME: Group of SAP application servers; mandatory for a logon balancing connection.
- RFCTYPE: RFC connection type.
- SYSNR: SAP system number.
- BASEURL: Configurable URL part that is permanent for this SAP system; can be used by applications to build URLs for accessing SAP transactions via the web browser.
- USERNAME: Username to be used for logging into the SAP system.
- PASSWORD: Password that belongs to the USERNAME; encrypted by the Content Server data encryption key.
- CLIENT: Client number to be used for logging into the SAP system.
- ENABLED: Enabled flag.
- SAPSYSTEMID: SAP system ID (SAP server and client number constitute the unique SAP system ID).

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

10.3.2.4 $SAP.SapSystems GetEnabledSapSystemsByClientAndSystemId function (Object, String, String)

Returns an Assoc with the configuration data for a SAP system specified by client and system ID.

Parameters

prgCtx

Type: object

Program context

client

Type: string

SAP client

sapSystemId

Type: string

SAP System ID

Return Value

Type: Assoc

.OK: true if no errors occurred in that function.

.errMsg: error message (.OK is false).

.found: 0 if no SAP system is configured and enabled with the specified parameters. 1 if one is found.

.sapsystem: if .found is 1, Assoc with configuration data for the SAP system.

For the description of the Assoc's elements, see 10.3.2.3.

10.3.2.5 $SAP.SapSystems GetSapSystem function (Object, String)

Returns an Assoc with the configuration data of the specified SAP system.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Parameters

prgCtx

Type: object

Program context

destination

Type: string

Name of the logical SAP system.

Return Value

Type: Assoc

.OK: true if call of $SAP.RFC Connect was successful, otherwise false.

.errMsg: error message (.OK is false).

.sapsystem: Assoc with configuration data for the SAP system.

For the description of the Assoc's elements, see 10.3.2.3.

10.3.2.6 $SAP.SapSystems GetSapSystems function (Object)

Returns an Assoc with the configuration data of all configured SAP systems independently whether they are enabled or not.

Parameters

prgCtx

Type: object

Program context

Return Value

Type: Assoc

.OK: true if call of $SAP.RFC Connect was successful, otherwise false.

.errMsg: error message (.OK is false).

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

.sapsystems: Assoc with an Assoc item for each SAP system.

For the description of the Assoc's elements, see 10.3.2.3.

10.4Tutorials

In the following you find tutorials for the practical usage of the different methods.

10.4.1 Using SAP OSpace

10.4.2 Calling RFC RFC_READ_TABLE

Use Content Server Builder and create a new empty script. Paste the following code into it, compile and run the script. If successfully performed, it displays the number of records of the requested table to the debug output.

Procedure – The OScript below does it by performing the following steps:

1. Connect to the SAP system by calling $SAP.Rfc.ConnectEx.
2. Create a new instance of the $SAP.RFC object for RFC_READ_TABLE.
3. Set an importing parameter and call the RFC.
4. Get the requested data from the returned data and echo the results to the debug output window.
5. Release the $SAP.RFC object.

Code –

```
Assoc result

Assoc retval

retval.ok = true

Boolean ok = true


String destination = "DPX"

String type = "A"

String ashost = "mucr3dpx"

String mshost = ""

String sysnr = "01"

String client = "800"
```

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
String user = "mysapuser"
String pass = "xxx"
String lang = "en"
String groupname = ""
String sysid = ""

Dynamic connection = undefined
Object rfcMethod = undefined
RecArray records = undefined

// 1) connect to the SAP system by using the given connection
parameters

result = $SAP.Rfc.ConnectEx( destination, type, ashost, mshost,
sysnr, client, user, pass, lang, groupname, sysid )
if result.OK
      connection = result.connection
else
      retval.ok = false
      retval.errMsg = result.errMsg
      ok =false
end

// 2) create a new instance of the $SAP.RFC object for
RFC_READ_TABLE

if ok
      rfcMethod = $SAP.RFC.New( connection, "RFC_READ_TABLE" )
      if IsUndefined( rfcMethod )
            retval.errMsg = "ERROR: can't create $SAP.Rfc object
for RFC_READ_TABLE - see thread log for more information"
            retval.ok = false
            ok = false
      end
end
```

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
// 3) set a importing parameter and call the RFC


if ok

      rfcMethod.QUERY_TABLE = "USR01"

      result = rfcMethod.Call()

      if result.ok == false

            ok = false

            retval.ok = false

            retval.errMsg = Str.Format( "ERROR: rfcErrorGroup:
%1, rfcErrorKey: %2, rfcErrorMessage: %3, rfcErrorCode: %4.",
result.rfcErrorGroup, result.rfcErrorKey, result.rfcErrorMessage,
result.rfcRetCode )

      else

            // retrieve the results

            records = rfcMethod.FIELDS

      end

end


// 4) display the results - just display the number of records
for simplicity or the error message


if ok

      echo ( Str.Format( "Table %1 has %2 records.",
rfcMethod.QUERY_TABLE, Length( records ) ) )

else

      echo ( retval.errMsg )

end


// 5) cleanup


if IsDefined( rfcMethod )

      $SAP.RFC.Delete( rfcMethod )

end
```

When you run the script, you will get the following output in the debug window:

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

Table USR01 has 18 records.

10.4.2.1 Calling BAPI method GetList of BAPI EmployeeAbstract

In the following example the method GetList of the BAPI EmployeeAbstract is called to retrieve the employee data for a specific person. The personnel number is specified to get only the data for one specific employee.

Procedure – The OScript below does it by performing the following steps:

1. Connect to the SAP system by calling $SAP.Rfc.ConnectEx.
2. Create a new instance of the $SAP.BAPI object for BAPI Employee and method GetList.
3. Set an importing parameter and call the BAPI method.
4. Check BAPI return value and retrieve the results.
5. Echo the results to the debug output window.
6. Release the $SAP.BAPI object.
7. Use Content Server Builder and create a new empty script. Paste the following code into it, compile and run the script. If successfully performed, it displays the name of the requested employee in the debug output.

Code:

```
Assoc result

Assoc retval

retval.ok = true

Boolean ok = true


String destination = "DPX"

String type = "A"

String ashost = "mucr3dpx"

String mshost = ""

String sysnr = "01"

String client = "800"

String user = "mysapuser"

String pass = "xxx"
```

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
String lang = "en"

String groupname = ""

String sysid = ""


Dynamic connection = undefined

Object bapiMethod = undefined

RecArray records = undefined


// 1) connect to the SAP system by using the given connection
parameters


result = $SAP.Rfc.ConnectEx( destination, type, ashost, mshost,
sysnr, client, user, pass, lang, groupname, sysid )

if result.OK

      connection = result.connection

else

      retval.ok = false

      retval.errMsg = result.errMsg

      ok =false

end


// 2) create a new instance of the $SAP.BAPI object for BAPI
Employee and method GetList


if ok

      bapiMethod = $SAP.BAPI.New( connection, "EmployeeAbstract",
"GetList" )

      if IsUndefined( bapiMethod )

            retval.errMsg = "ERROR: can't create $SAP.BAPI object
for BAPI Employee and method GetList - see thread log for more
information"

            retval.ok = false

            ok = false

      end

end


// 3) set a importing parameter and call the BAPI method
```

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
if ok

      bapiMethod.PersonnelNumber = "00001006"

      result = bapiMethod.Call()

      if result.ok == false

            ok = false

            retval.ok = false

            retval.errMsg = Str.Format( "ERROR: rfcErrorGroup:
%1, rfcErrorKey: %2, rfcErrorMessage: %3, rfcErrorCode: %4.",
result.rfcErrorGroup, result.rfcErrorKey, result.rfcErrorMessage,
result.rfcRetCode )

      else

            // 4) check BAPI Return value and retrieve the
results

            if Length( bapiMethod.("Return").Code ) != 0

                  retval.ok = false

                  retval.errMsg = Str.Format("BAPI returned the
message: %1.", bapiMethod.("Return").MESSAGE )

                  ok = false

            else

                  retval.message = Str.Format("Employee with
personnel number %1 is %2 %3.", bapiMethod.PersonnelNumber,
bapiMethod.PersonalData[ 1 ].FIRSTNAME, bapiMethod.PersonalData[
1 ].LAST_NAME )

            end

      end

end


// 5) display the results


if ok

      echo ( retval.message )

else

      echo ( retval.errMsg )

end


// 6) cleanup
```

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

```
if IsDefined( bapiMethod )
      $SAP.BAPI.Delete( bapiMethod )
end
```

When you run the script, you will get the following output in the debug window:

Employee with personnel number 00001006 is Yasmin Awad.

Copyright © 2026 Open Text. All rights reserved. Trademarks owned by Open Text.

About OpenText

OpenText enables the digital world, creating a better way for organizations to work with information, on-premises or in the cloud. For more information about OpenText (NASDAQ/TSX: OTEX), visit opentext.com.

Connect with us:

OpenText Executive blog

X | LinkedIn